AI and ML for Us

Lecture:

Searching

Pradumn K Pandey

CSE, Indian Institute of Technology Roorkee

Today…

- Today’s Session:
  - What is **Search**ing?
  - Real Life examples
  - Different searching strategies
    - Uninformed (Blind) searching
    - Informed searching (Heuristic-Based)
    - Local searching
    - Adversarial Search
    - Specialized Search Methods
    - Online and Real-Time Search
- Announcements:

harsh verma

Home | My Network | Jobs | Messaging | Notifications | Me | For Business

People | 1st | 2nd | 3rd+ | Locations | Current company | All filters

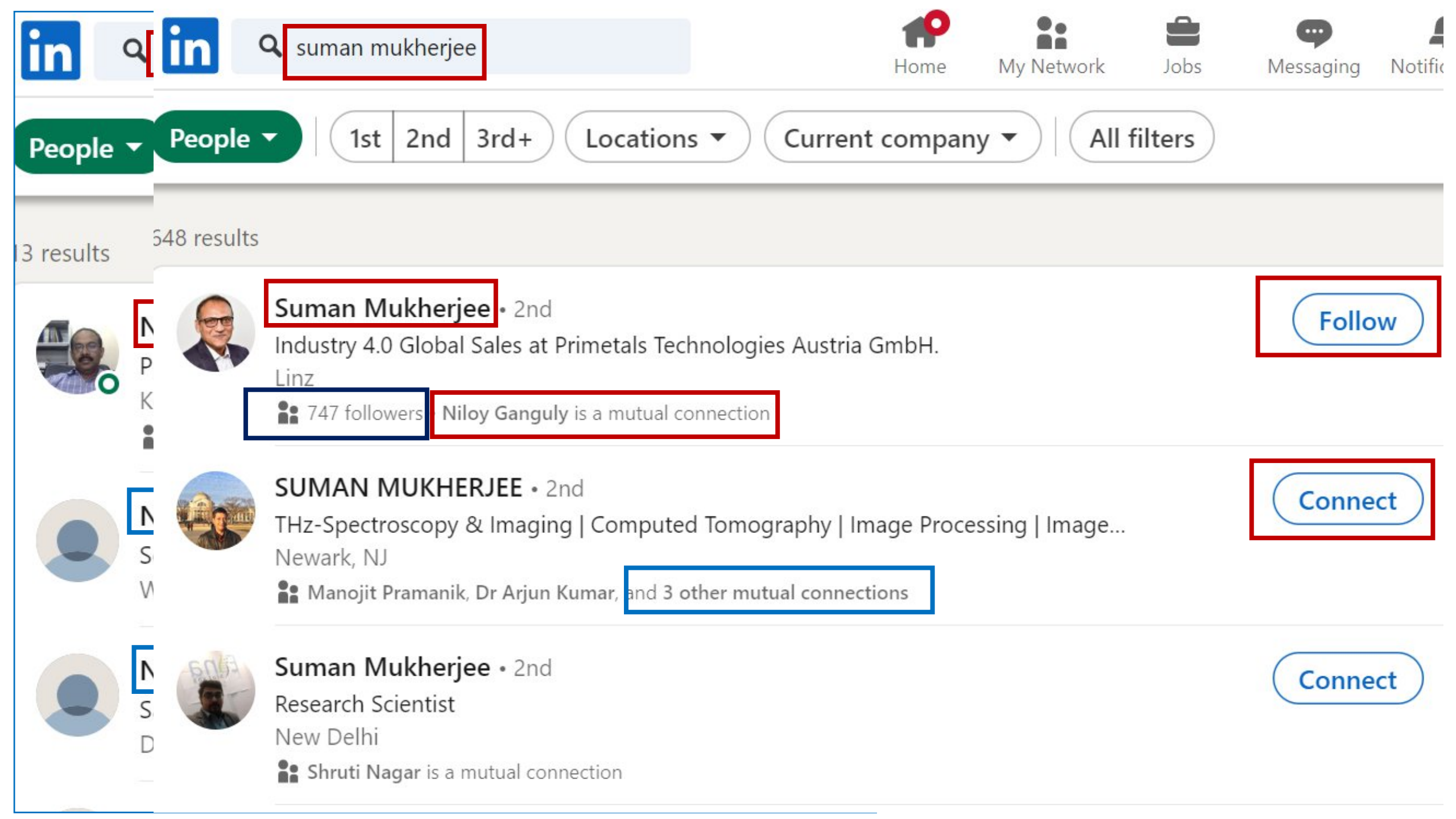
HARSH VERMA • 1st
SSTC || CSE[DS] Student
Bhilai
Chetan Arora, Dr. Krishna Kanth Avalur, and 24 other mutual connections
Message

Harsh Verma • 2nd
B.Tech, IIT Kanpur'21 | 10K+ Followers | Travelled CN KR
Kanpur
11K followers • Gaurav Raizada, Rajat Moona, and 55 other mutual connections
Follow

Harsh Verma • 2nd
Former R&D Intern @ISRO | Former Entrepreneur | Master's in Physics @NIT SURAT |...
Ahmedabad
9K followers • Sanjukta Roy, annima gupta, and 73 other mutual connections
Connect

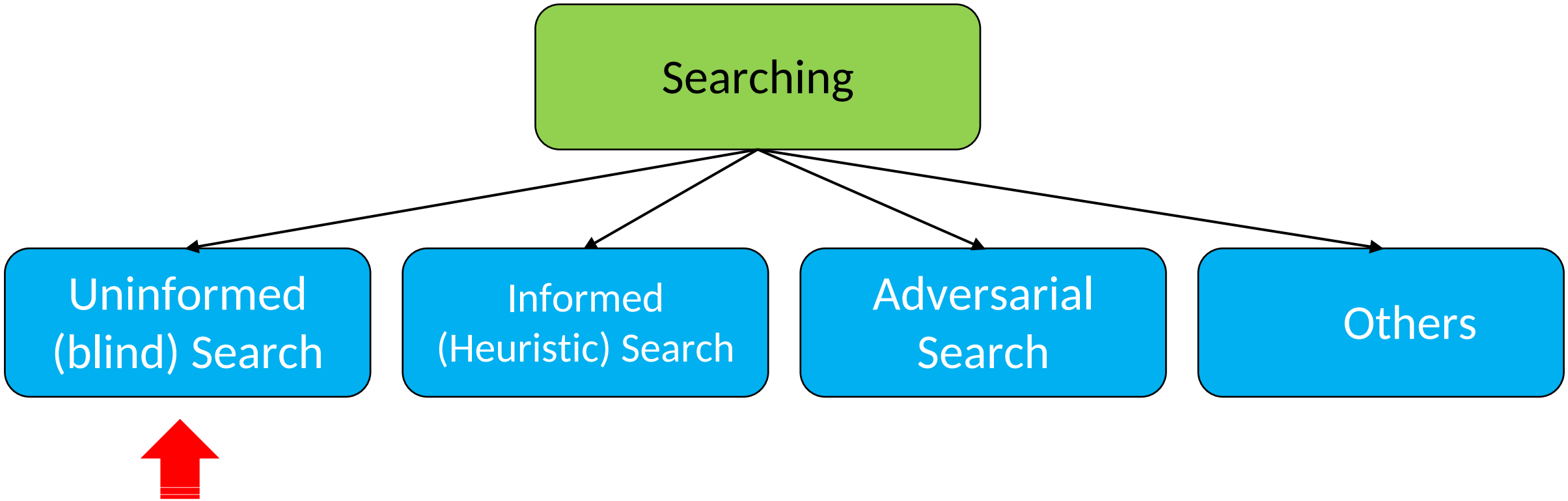
Harsh Verma • 2nd
Manager - Marketing at Godrej & Boyce | Tech M | IIM Kashipur & NIT Jameshedpur Alumnus
Uttarakhand, India
Indranee Pise is a mutual connection
Connect

Harsh Verma • 2nd
Analytics & BI at Appsilon - R, RShiny
Delhi, India
Dr. Munish Jindal, Mayank Gupta, and 10 other mutual connections
Connect

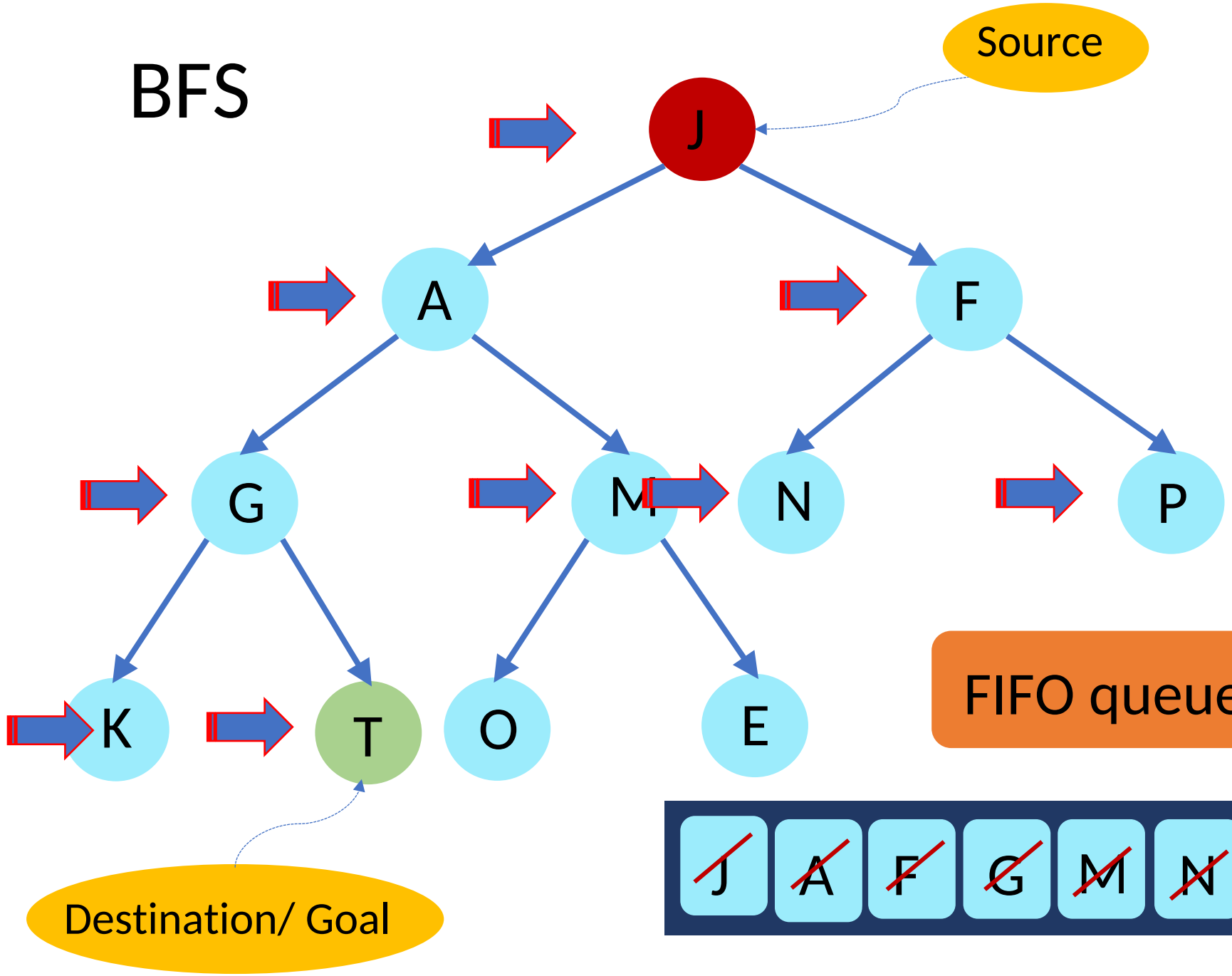

suman mukherjee
Home
My Network
Jobs
Messaging
People
People
1st
2nd
3rd+
Locations
Current company
All filters
3 results
548 results
Suman Mukherjee • 2nd
Industry 4.0 Global Sales at Primetals Technologies Austria GmbH.
Linz
747 followers
Niloy Ganguly is a mutual connection
Follow
SUMAN MUKHERJEE • 2nd
THz-Spectroscopy & Imaging | Computed Tomography | Image Processing | Image...
Newark, NJ
Manojit Pramanik, Dr Arjun Kumar, and 3 other mutual connections
Connect
Suman Mukherjee • 2nd
Research Scientist
New Delhi
Shruti Nagar is a mutual connection
Connect

Finding the shortest path on a map

Outline


Searching
Uninformed (blind) Search
Informed (Heuristic) Search
Adversarial Search
Others

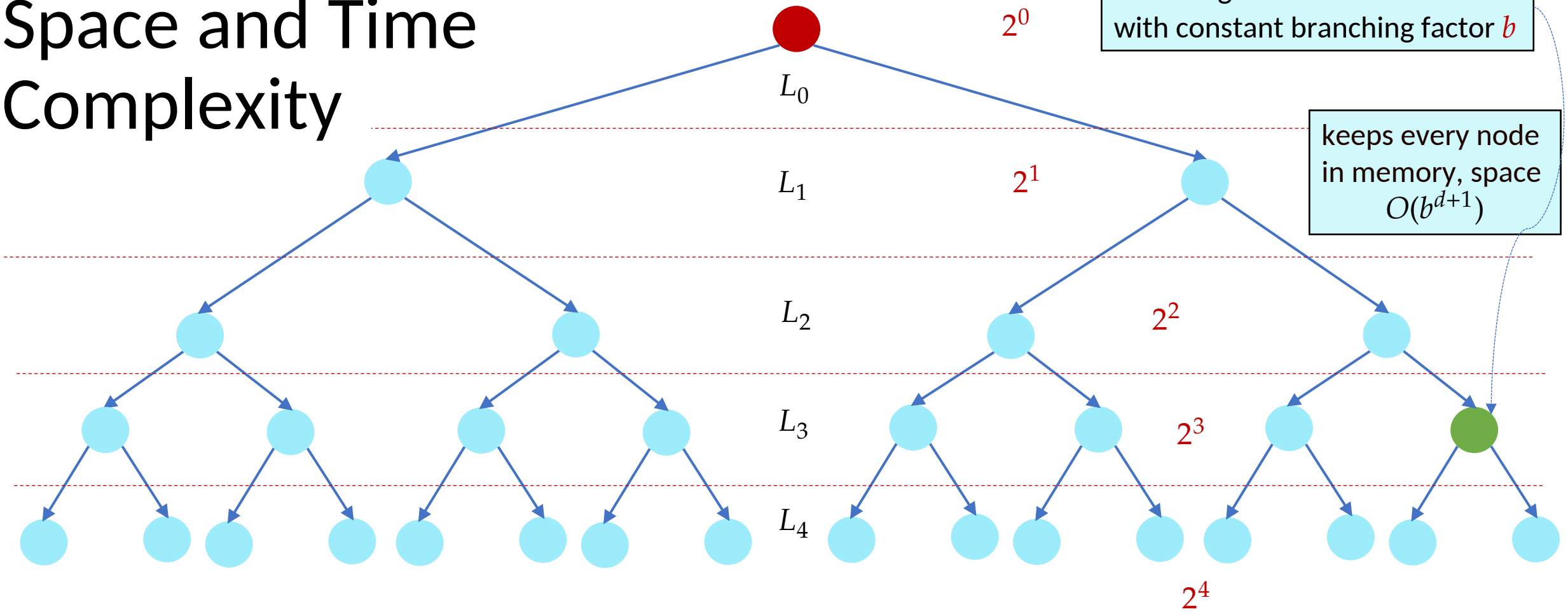
BFS



Main idea:
Expand all nodes at depth (i) before expanding nodes at depth ($i + 1$)

FIFO queue

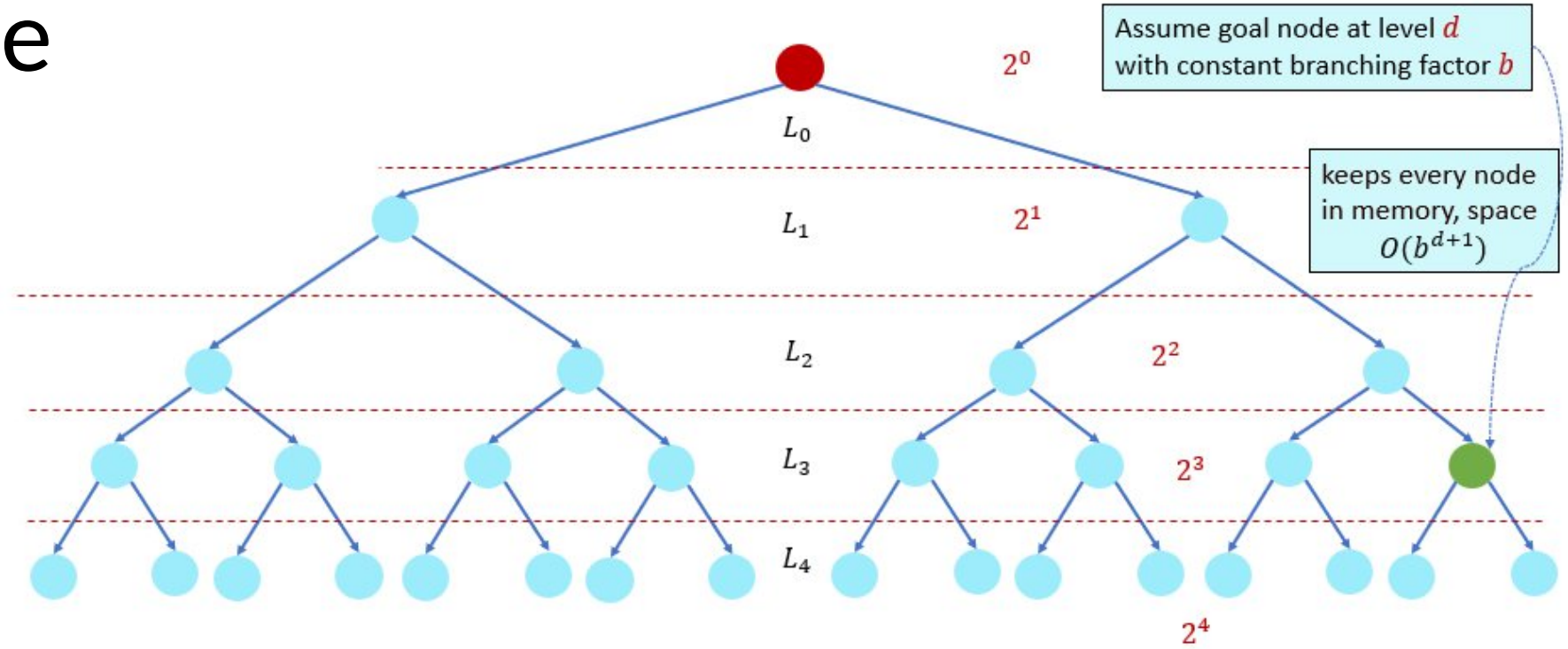
Space and Time Complexity



$$b^0 + b^1 + b^2 + \ldots + b^d + b^{d+1} = O(b^{d+1})$$

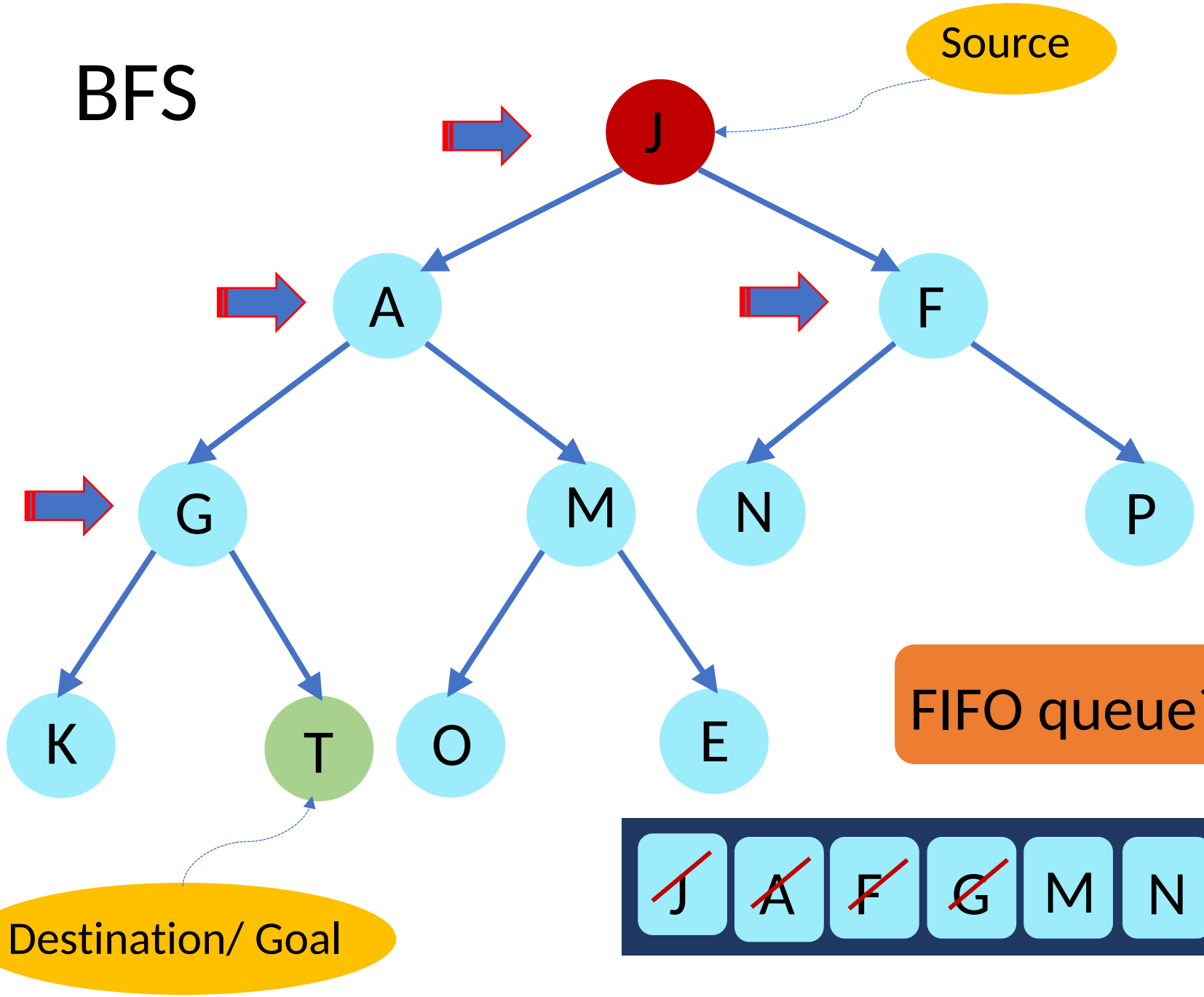
Space and Time Complexity

$$2(b-1) \geq b$$
$$\Rightarrow b \geq 2$$



$$b^0 + b^1 + b^2 + \ldots + b^d + b^{d+1}$$
$$= \frac{(b^{d+2}-1)}{(b-1)} = \frac{2(b^{d+2}-1)}{2(b-1)} \leq \frac{2\,b^{d+2}}{2\,(b-1)} \leq \frac{2\,b^{d+2}}{b} = 2\,b^{d+1}$$
$$= O(b^{d+1})$$

BFS



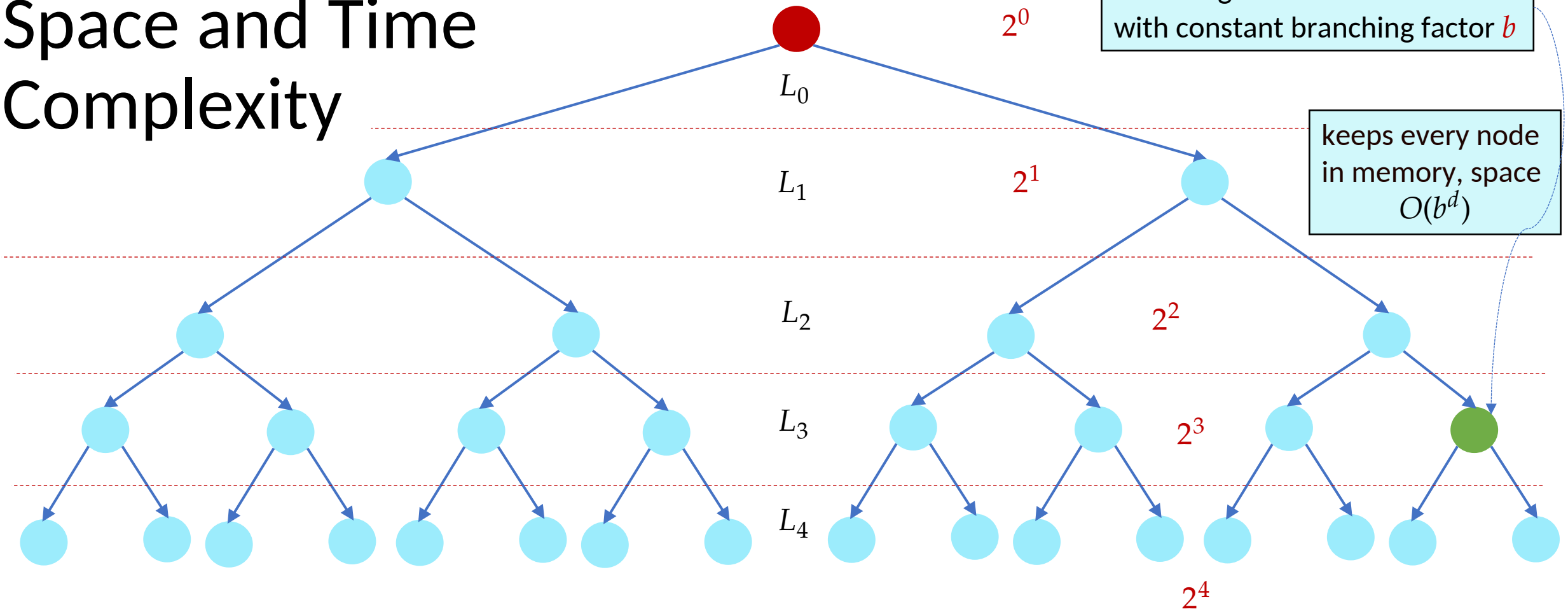
How to find T?

Main idea:
Expand all nodes at depth (i) before expanding nodes at depth ($i + 1$)

FIFO queue?

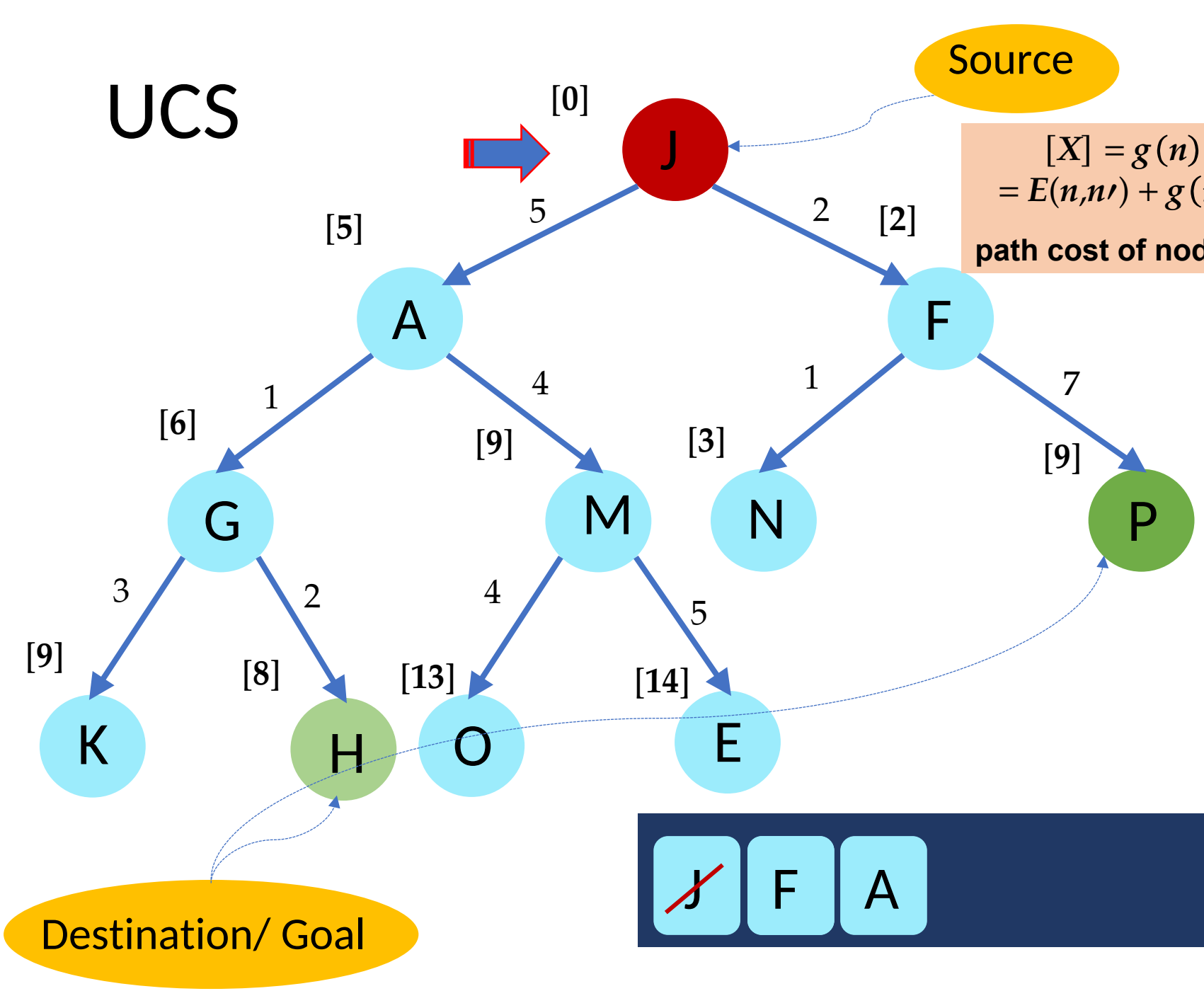
Array?

Space and Time Complexity



$$b^0 + b^1 + b^2 + \ldots + b^d = O(b^d)$$

UCS



$$[X] = g(n)$$
$$= E(n,n') + g(n')$$

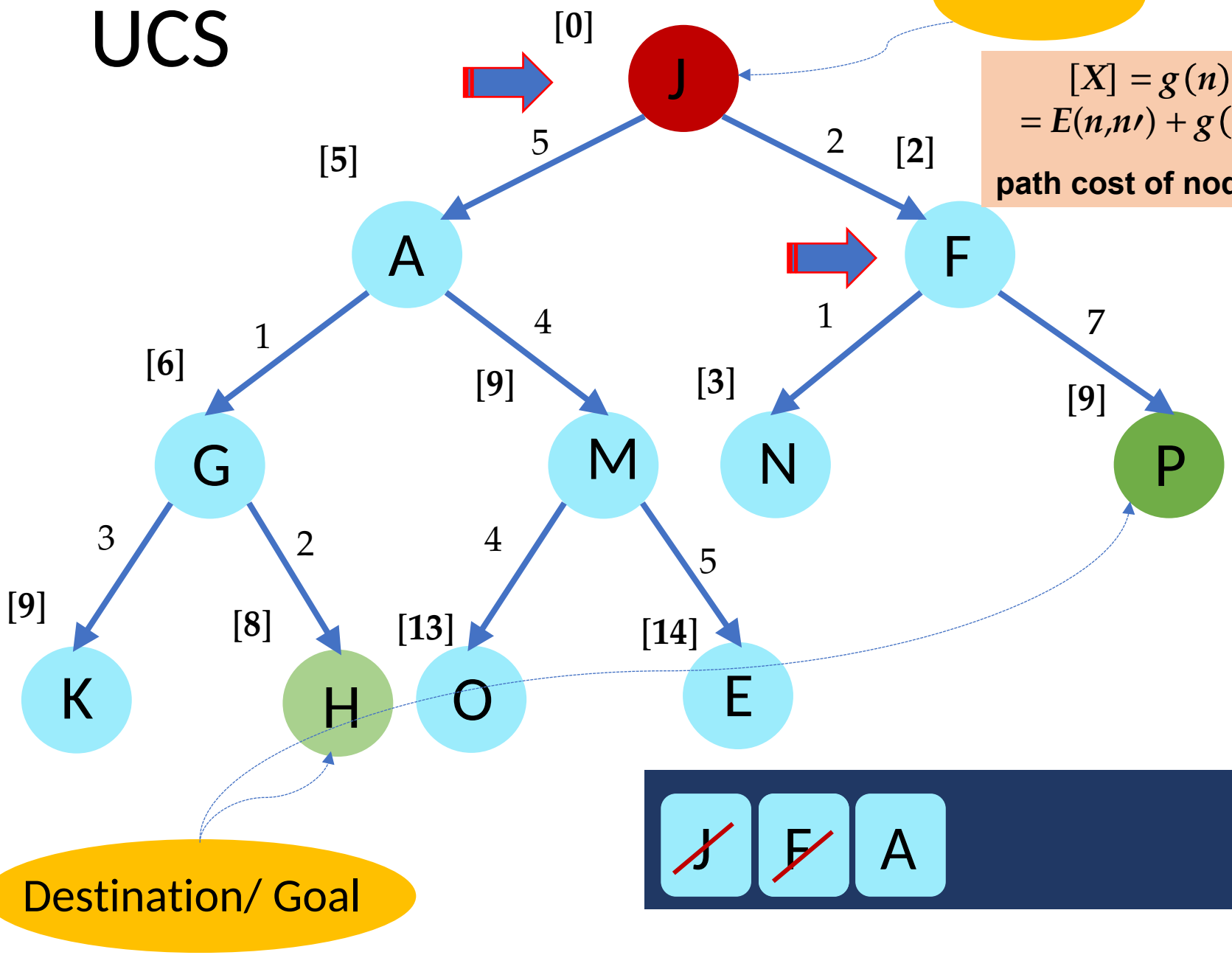
path cost of node n

How to find shortest path to his hole?

Main idea:
Uniform Cost Search expands the node n with the lowest path cost.

~~J~~ F A

UCS



$$[X] = g(n)$$
$$= E(n,n') + g(n')$$

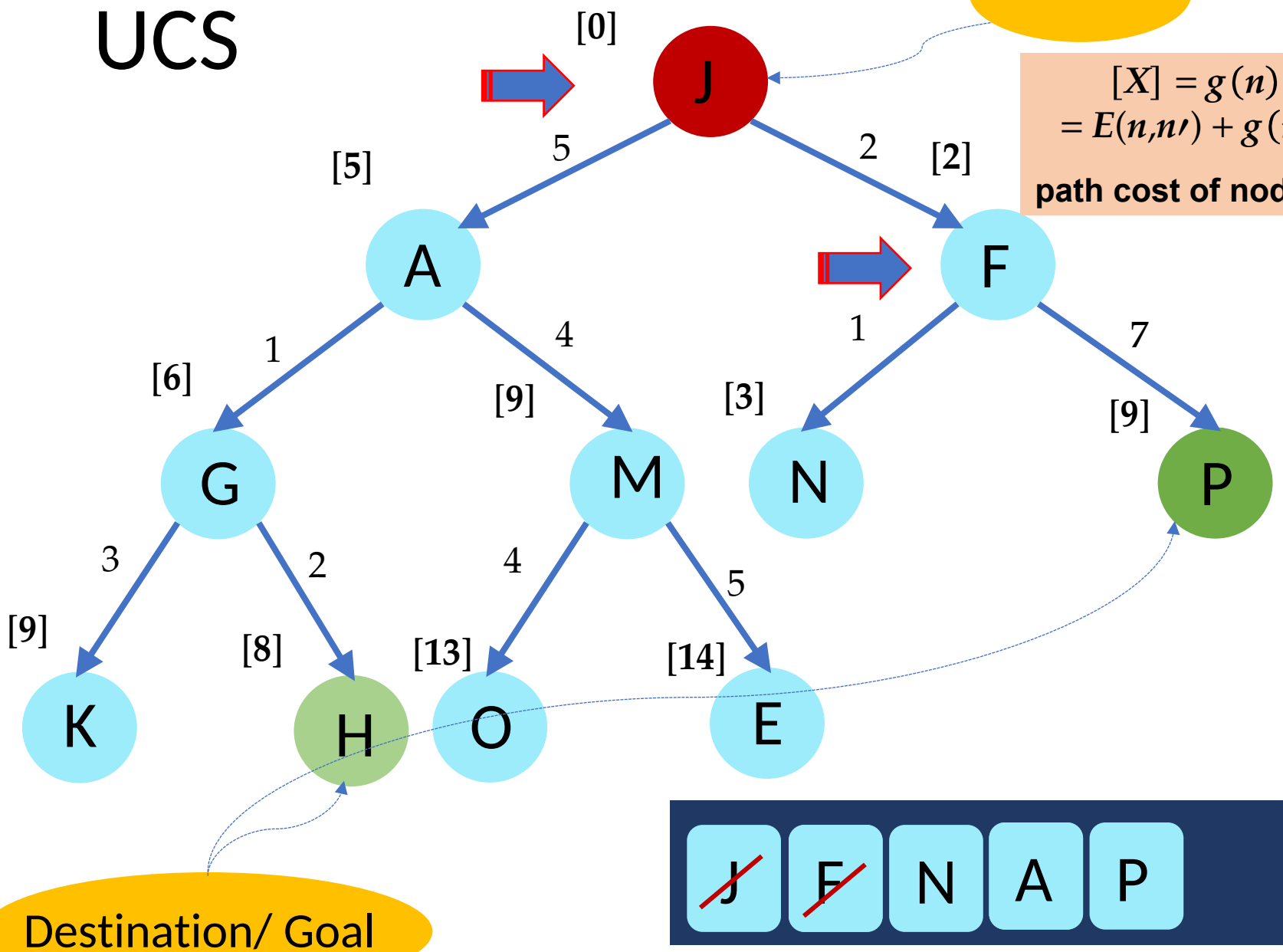
path cost of node n

How to find shortest path to his hole?

Main idea:
Uniform Cost Search expands the node n with the lowest path cost.

~~J~~ ~~F~~ A

UCS



$$[X] = g(n)$$
$$= E(n,n') + g(n')$$

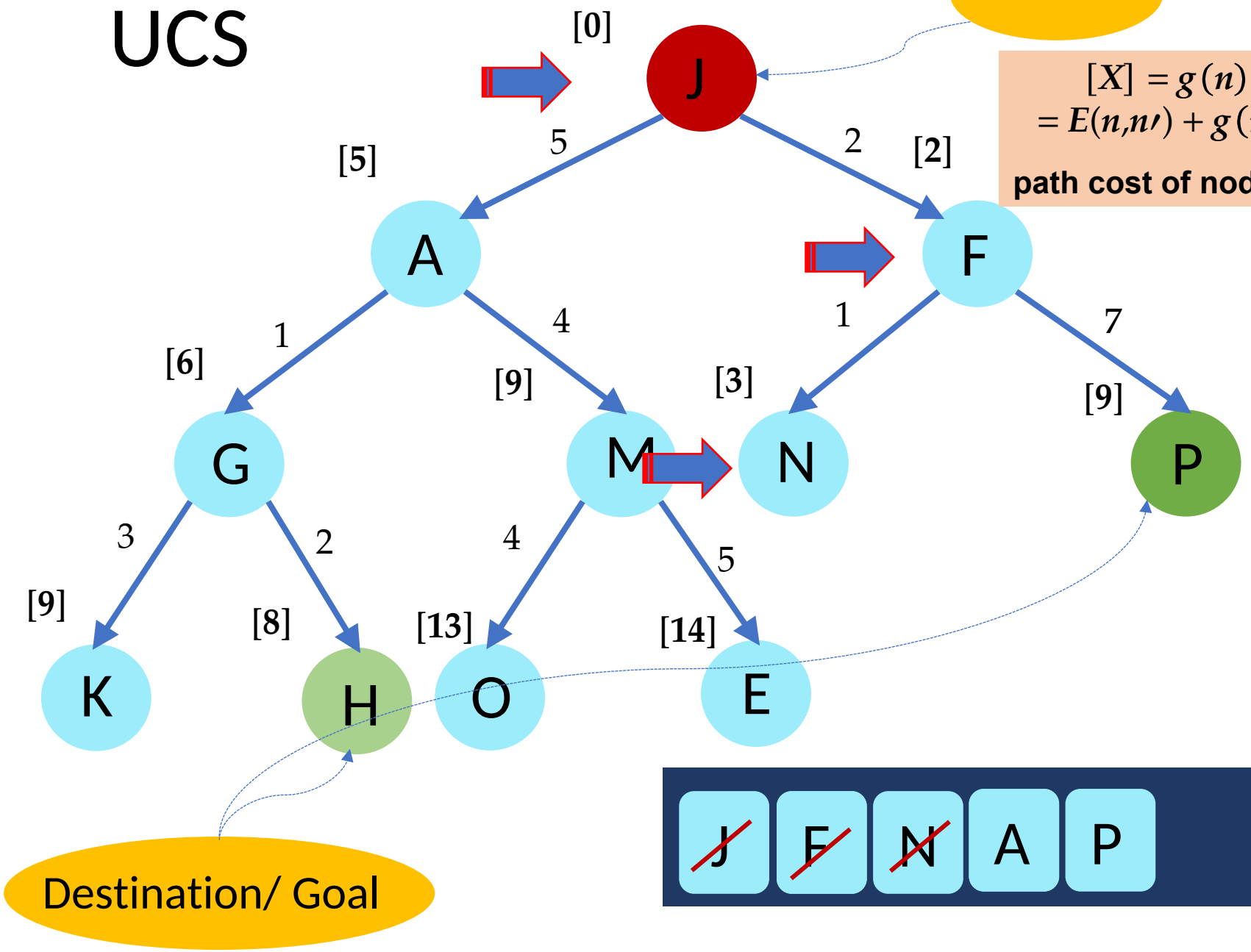
path cost of node n

How to find shortest path to his hole?

Main idea:
Uniform Cost Search expands the node n with the lowest path cost.

~~J~~ ~~F~~ N A P

UCS



$$[X] = g(n)$$
$$= E(n,n') + g(n')$$

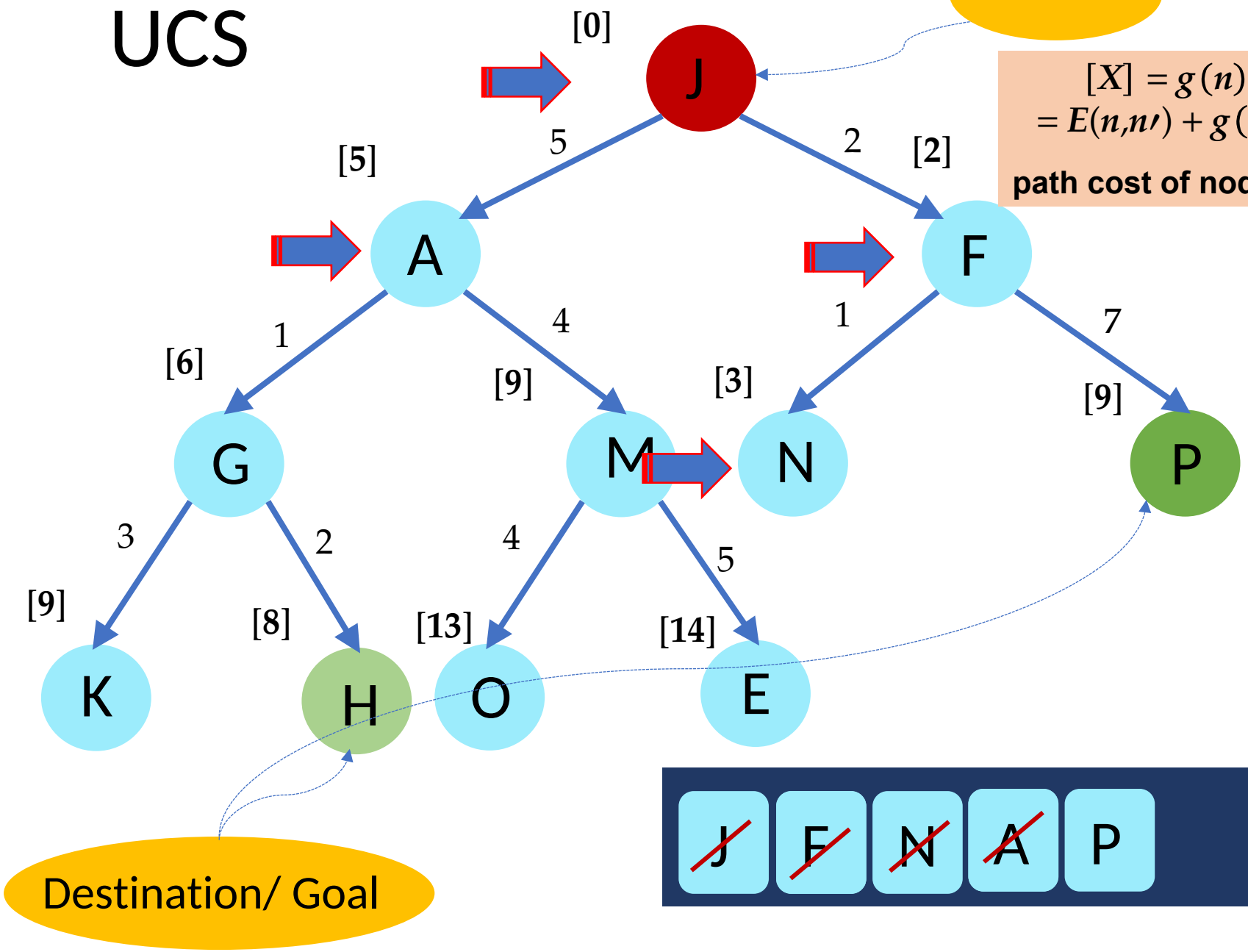
path cost of node n

How to find shortest path to his hole?

Main idea:
Uniform Cost Search expands the node n with the lowest path cost.

~~J~~	~~F~~	~~N~~	A	P

UCS



$$[X] = g(n)$$
$$= E(n,n') + g(n')$$

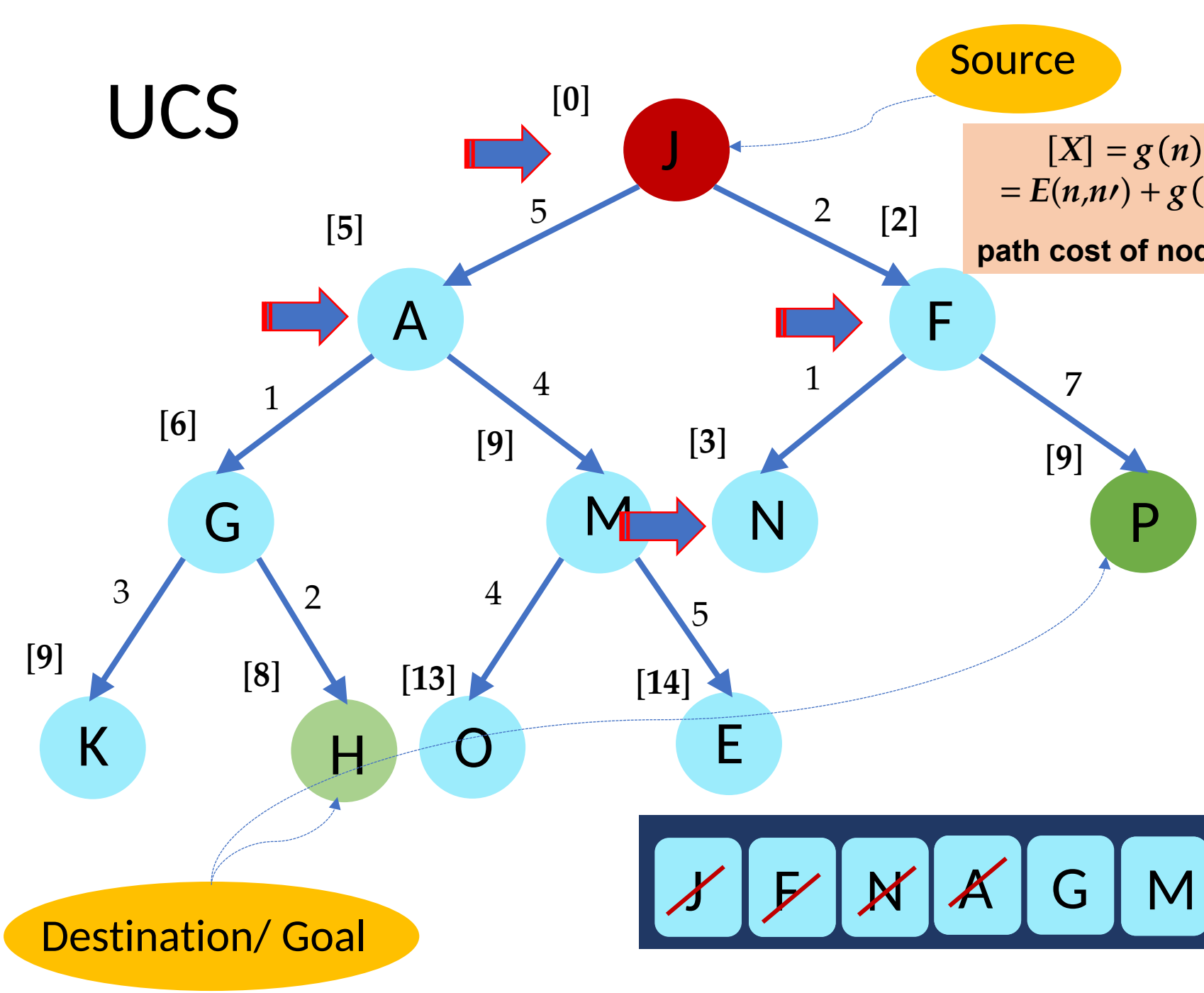
path cost of node n

How to find shortest path to his hole?

Main idea:
Uniform Cost Search expands the node n with the lowest path cost.

~~J~~ ~~F~~ ~~N~~ ~~A~~ P

UCS



$$[X] = g(n)$$
$$= E(n,n') + g(n')$$

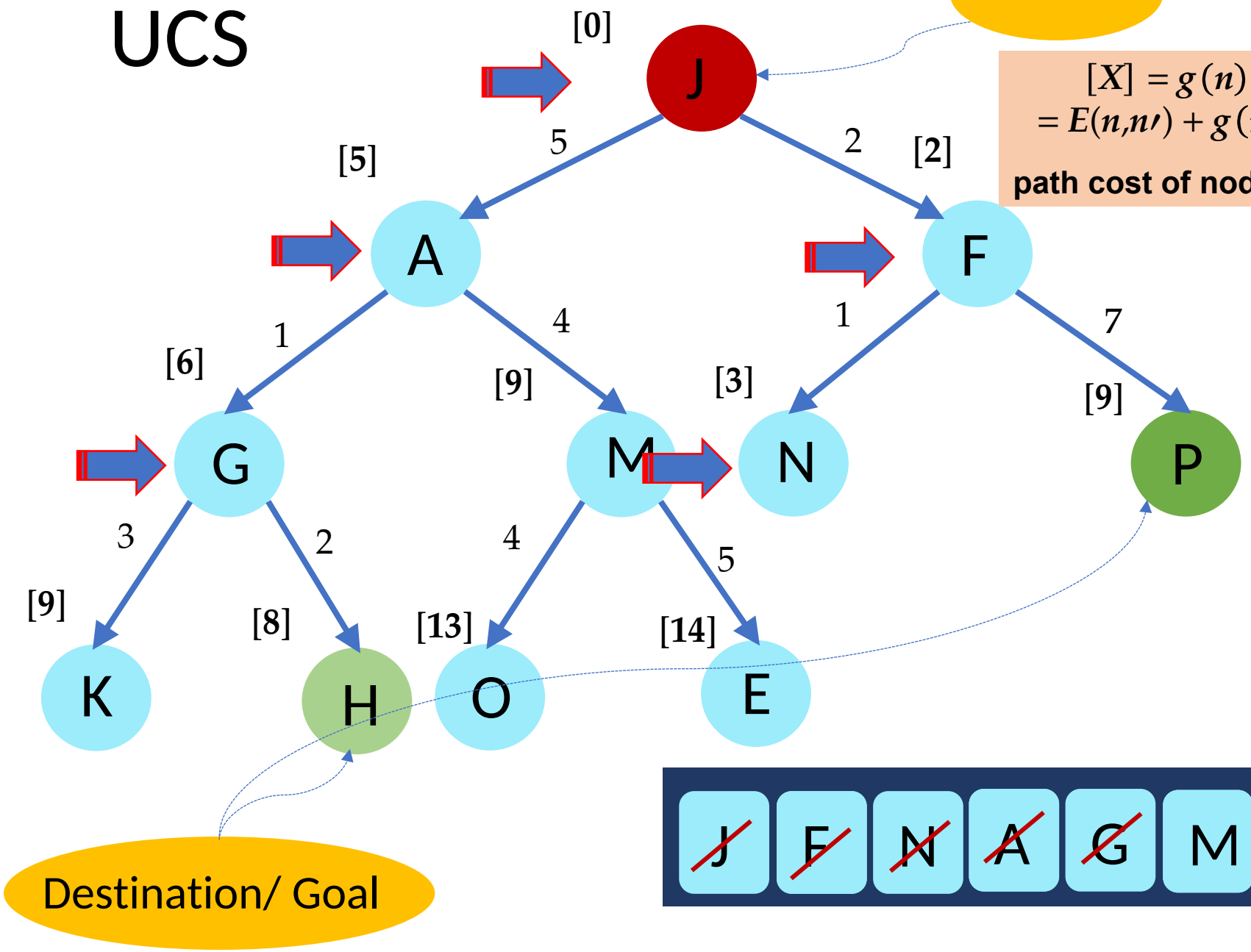
path cost of node n

How to find shortest path to his hole?

Main idea:
Uniform Cost Search expands the node n with the lowest path cost.

~~J~~	~~F~~	~~N~~	~~A~~	G	M	P

UCS



$$[X] = g(n)$$
$$= E(n,n') + g(n')$$

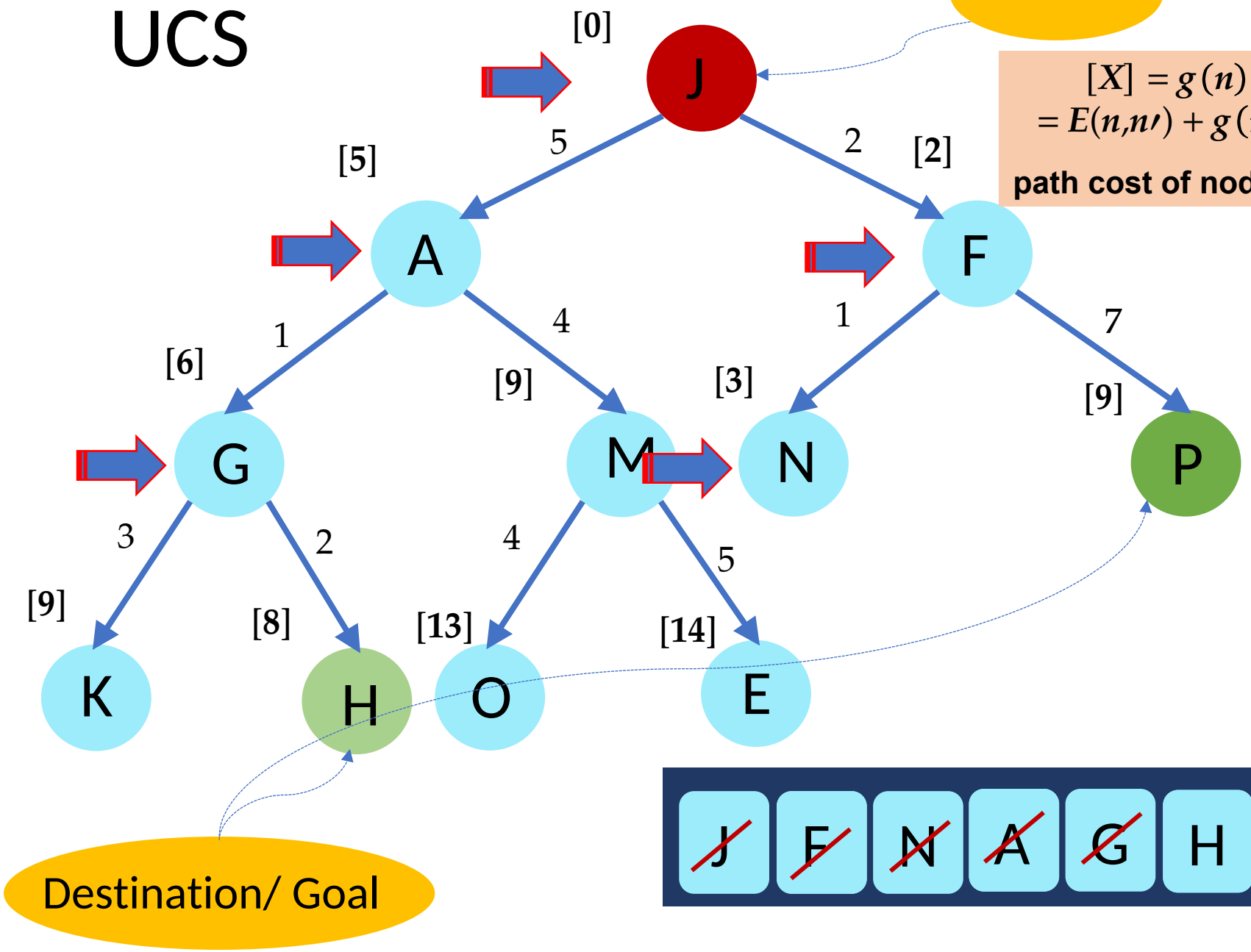
path cost of node n

How to find shortest path to his hole?

Main idea:
Uniform Cost Search expands the node n with the lowest path cost.

~~J~~	~~F~~	~~N~~	~~A~~	~~G~~	M	P

UCS



$$[X] = g(n)$$
$$= E(n,n') + g(n')$$

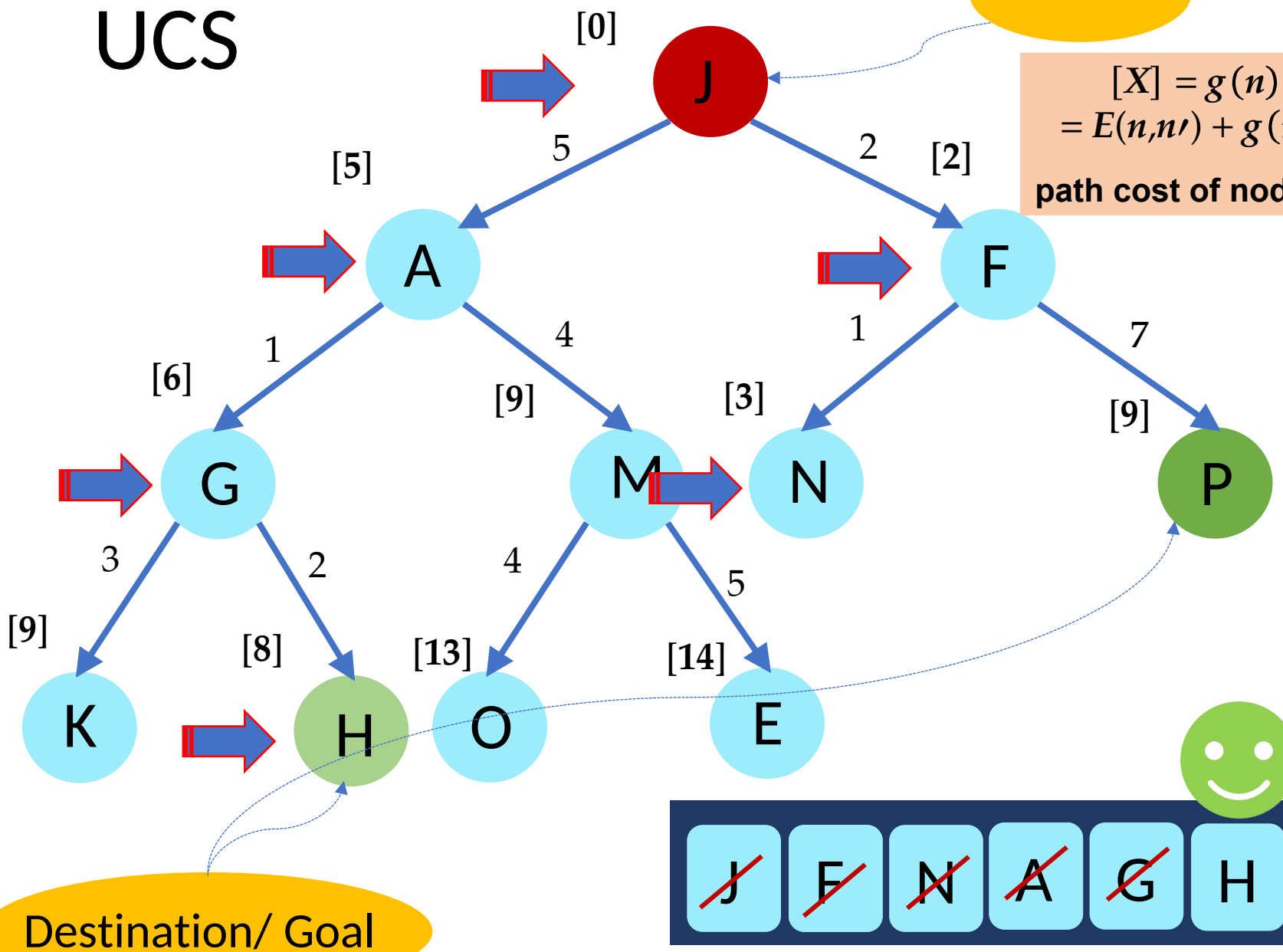
path cost of node n

How to find shortest path to his hole?

Main idea:
Uniform Cost Search expands the node n with the lowest path cost.

~~J~~	~~F~~	~~N~~	~~A~~	~~G~~	H	K	M	P

UCS



$$[X] = g(n)$$
$$= E(n,n') + g(n')$$

path cost of node n

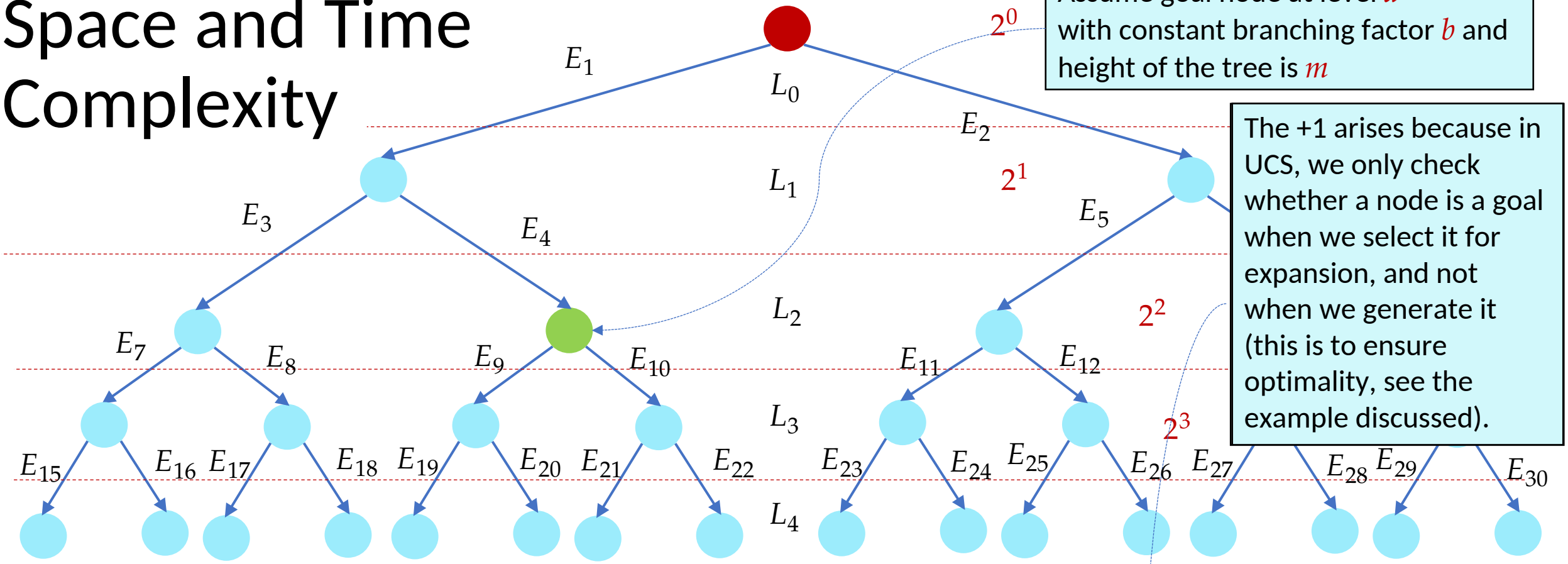
How to find shortest path to his hole?

Main idea:
Uniform Cost Search expands the node n with the lowest path cost.

Priority queue

~~J~~	~~F~~	~~N~~	~~A~~	~~G~~	H	K	M	P

Space and Time Complexity



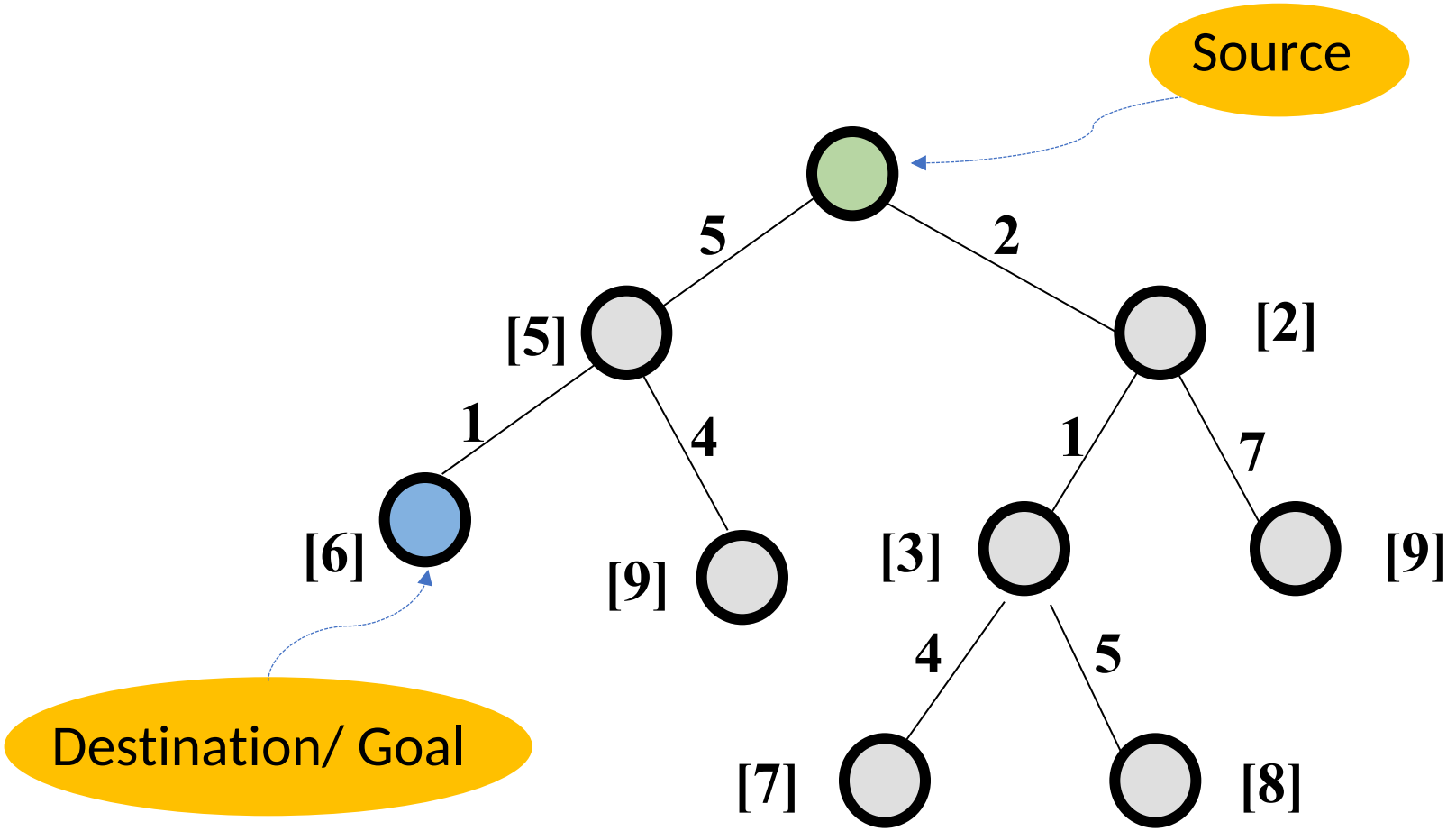
Assume goal node at level d with constant branching factor b and height of the tree is m

The +1 arises because in UCS, we only check whether a node is a goal when we select it for expansion, and not when we generate it (this is to ensure optimality, see the example discussed).

If we rearrange the nodes as per their cost and the cost (E_1+ E_4) sets goal node at lower level. $C(E_1 + E_4)$ is the cost of solution path and each step gets you ϵ closer to the goal.

$$0, \epsilon, 2\epsilon, 3\epsilon b, 4\epsilon, \ldots, \left(C/\epsilon\right)\epsilon, \left(up\ to\ C/\epsilon\ levels\right) = O\left(b^{\left(1+C/\epsilon\right)}\right) = O(b^m)$$

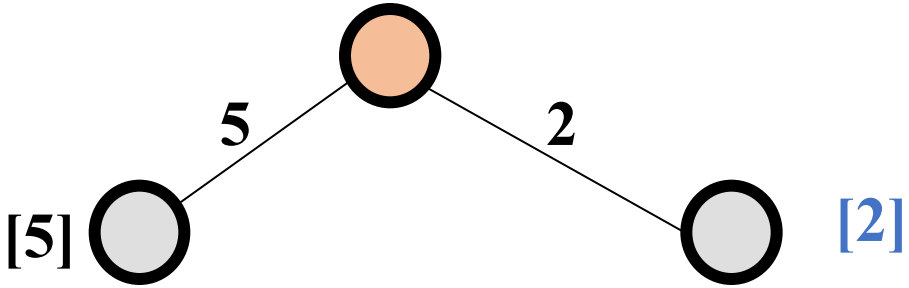
Uniform Cost Search (UCS)



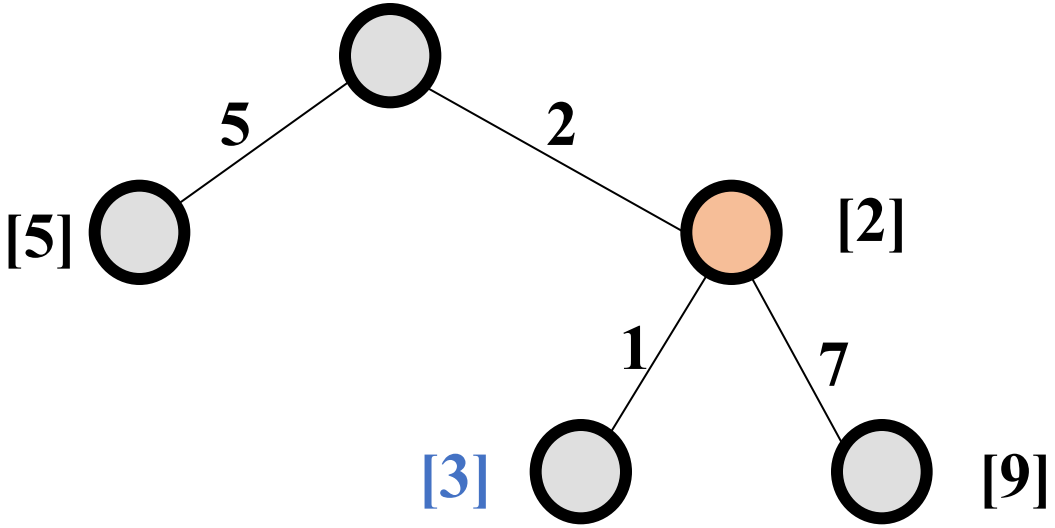
$[X] = g(n)$

path cost of node n

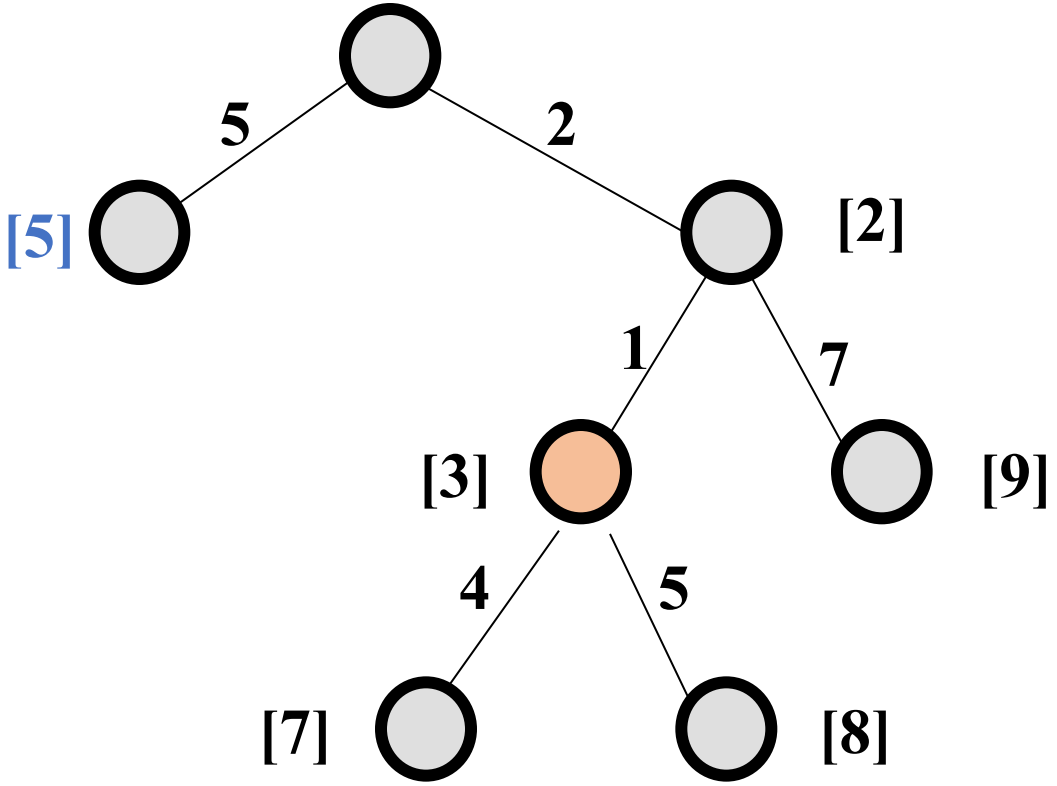
Uniform Cost Search (UCS)



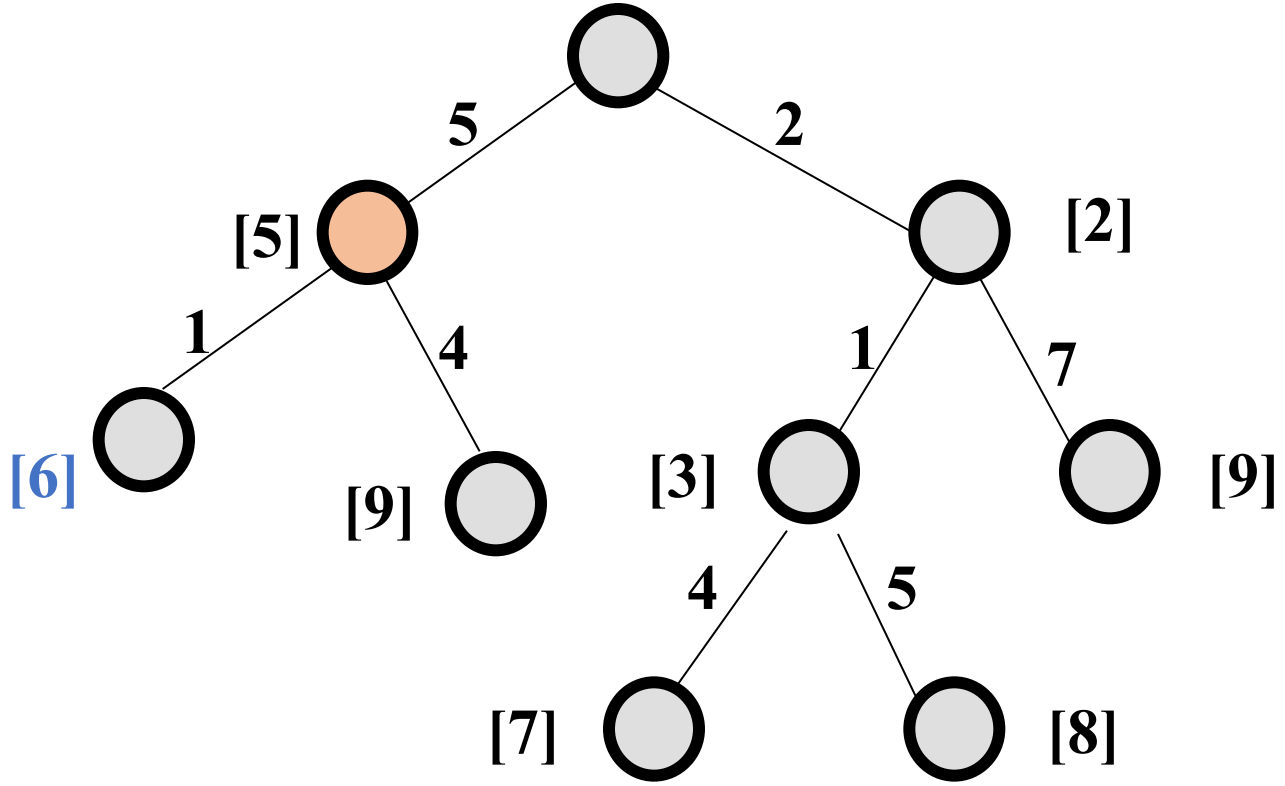
Uniform Cost Search (UCS)



Uniform Cost Search (UCS)



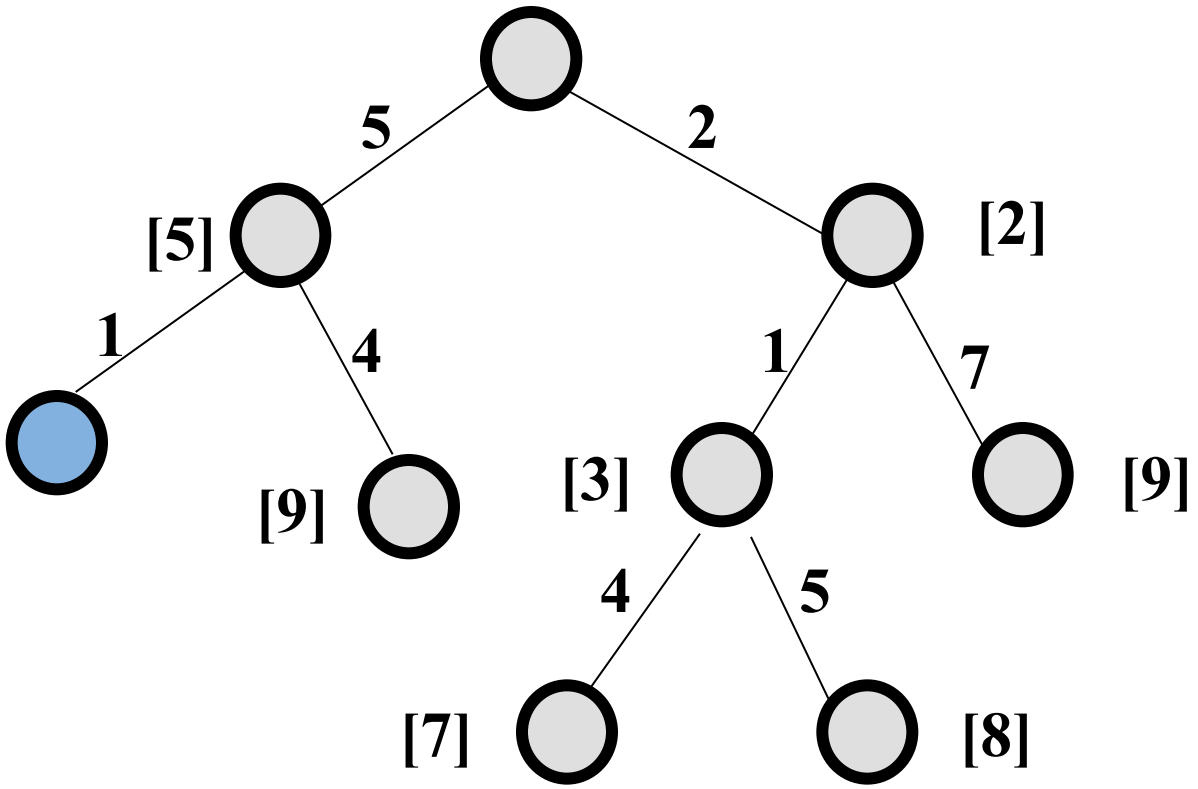
Uniform Cost Search (UCS)



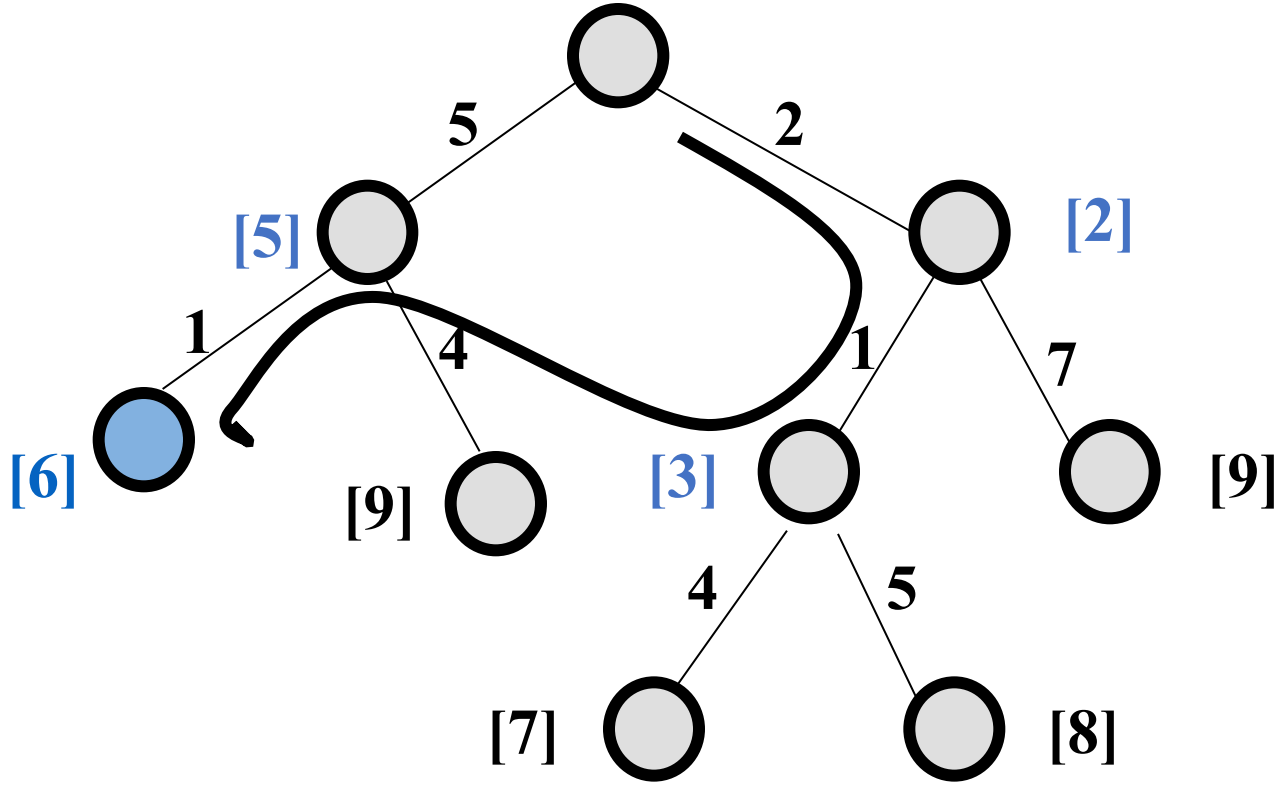
Uniform Cost Search (UCS)

Goal path cost

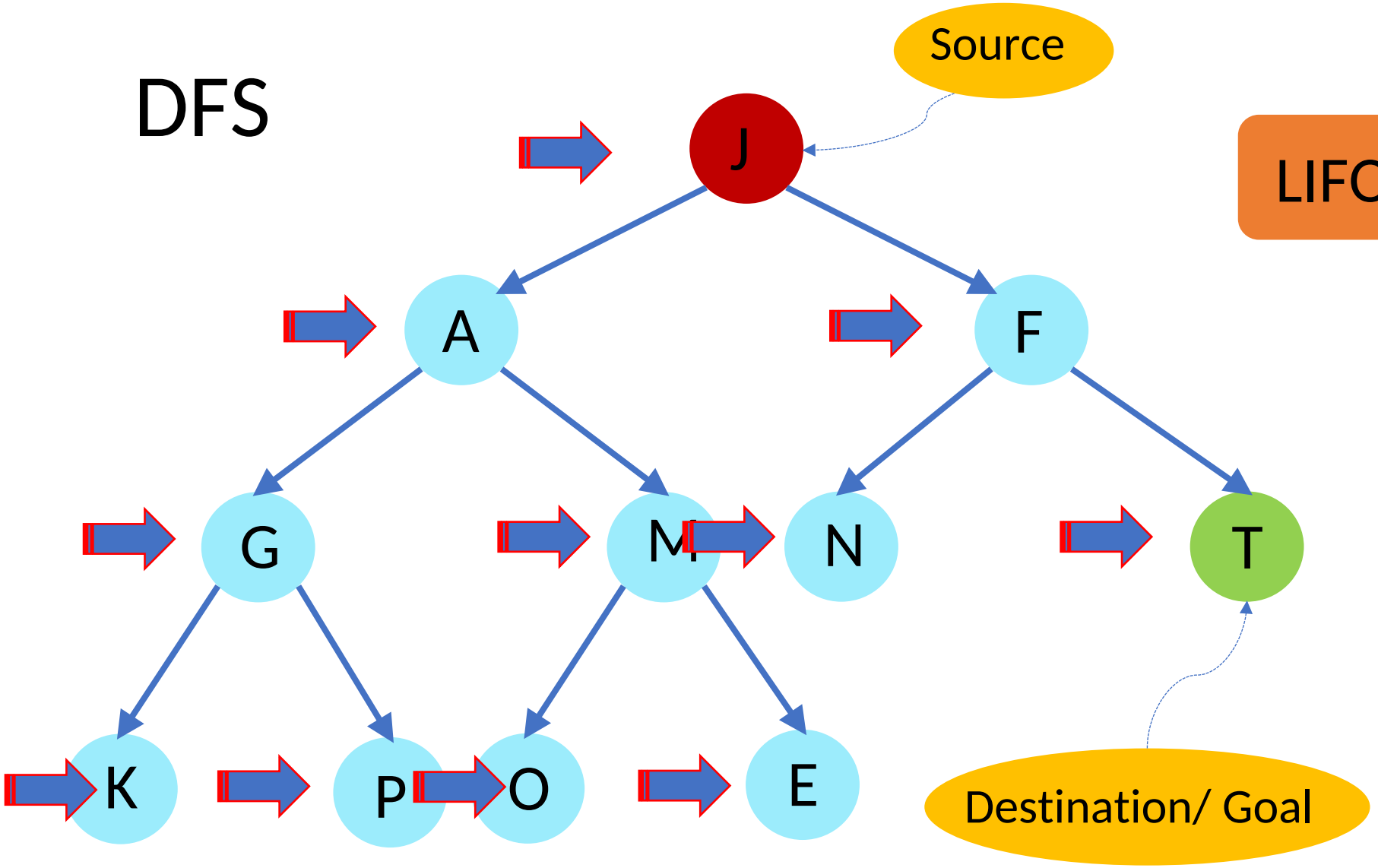
$g(n) = [6]$



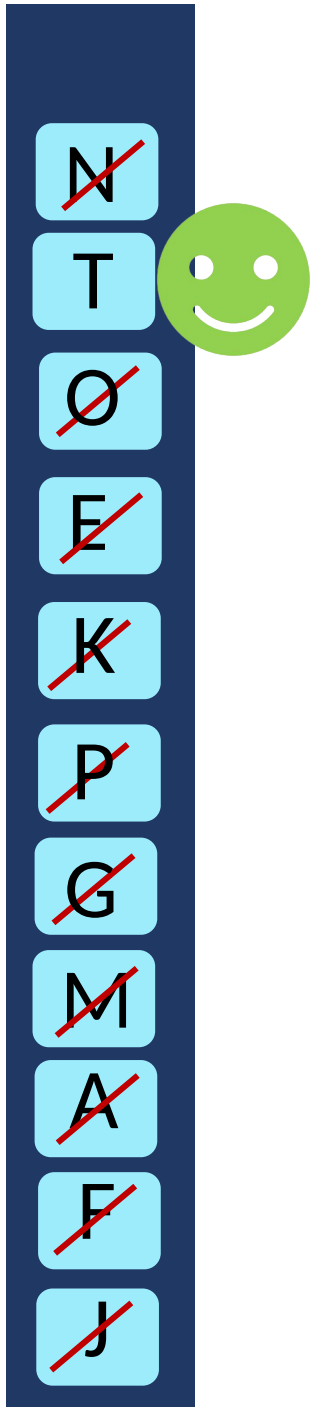
Uniform Cost Search (UCS)



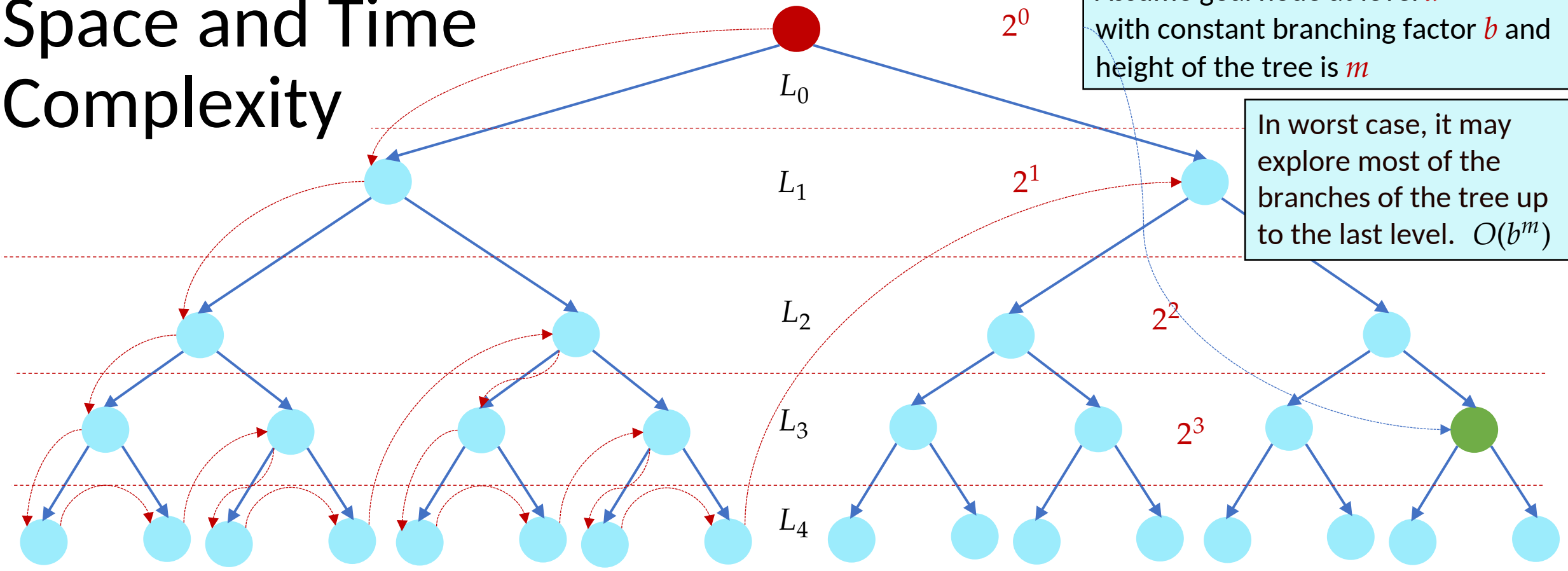
DFS



LIFO stack



Space and Time Complexity



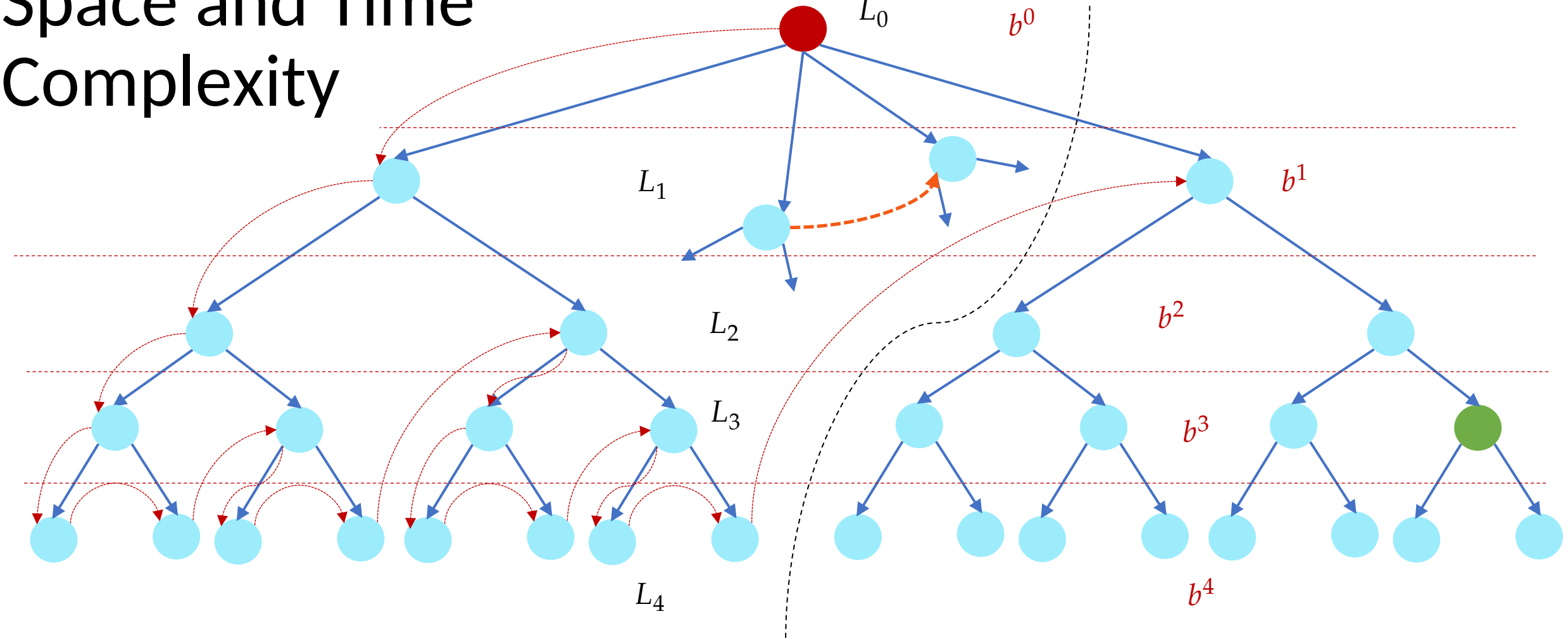
Assume goal node at level d with constant branching factor b and height of the tree is m

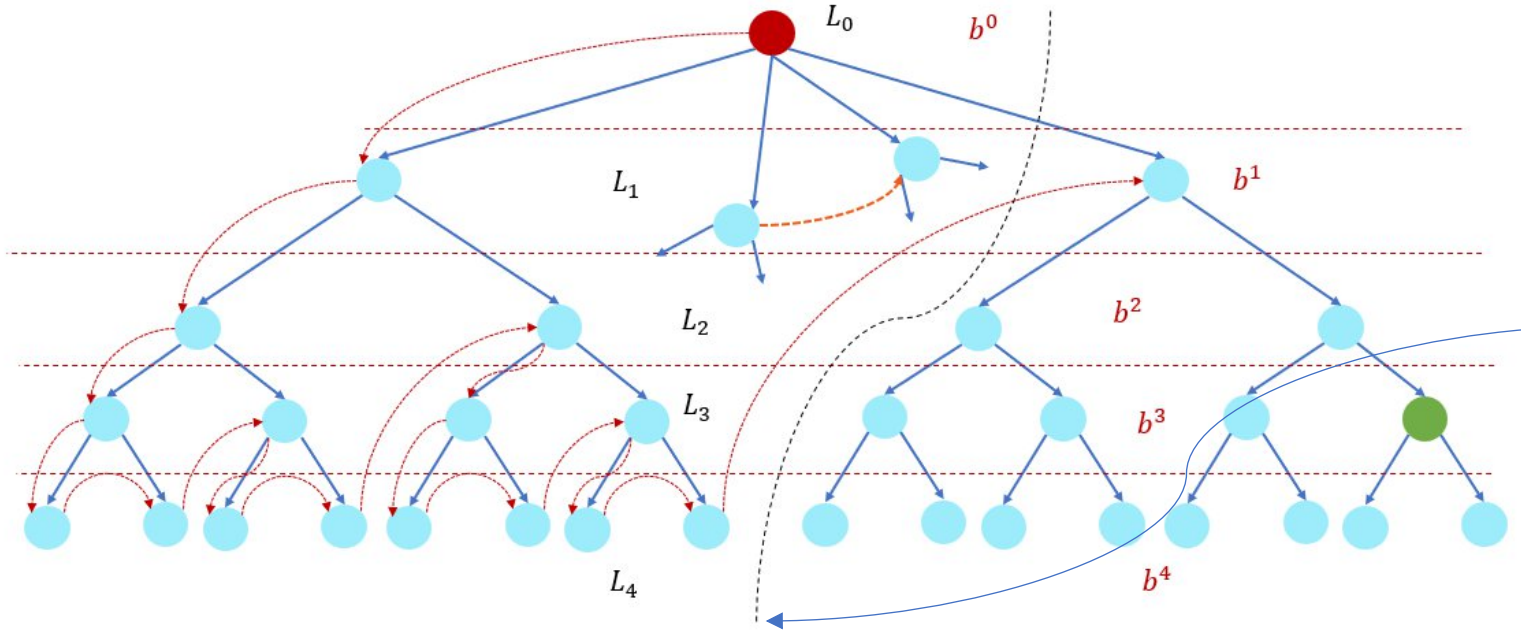
In worst case, it may explore most of the branches of the tree up to the last level. $O(b^m)$

Only need to save one set of children at each level

Space: $b + b + b + b + \ldots + b\,(up\ to\ m) = m\text{X}b = O(bm)$

Space and Time Complexity





Branching factor b and height of the tree is m

Let a goal node be chosen randomly. The probability of the goal node being on the right side of the boundary is

$$\approx \left(\frac{1}{b}\right)$$

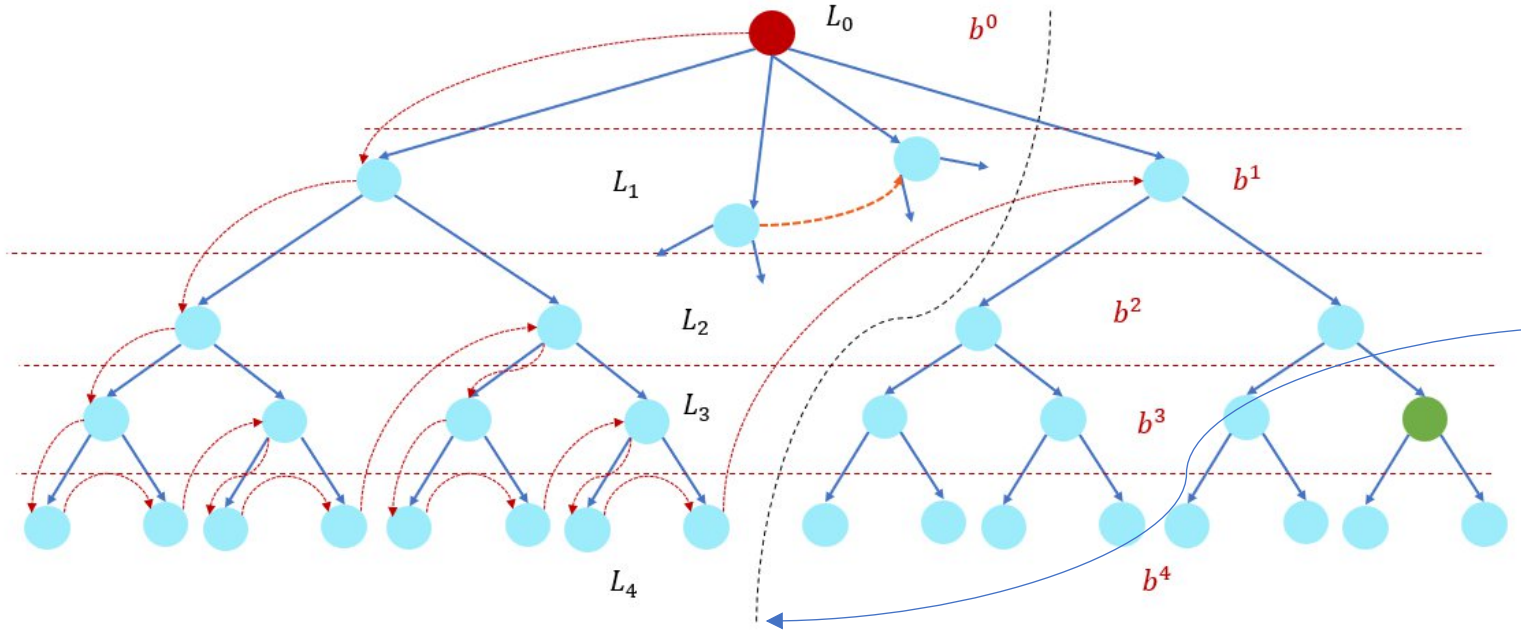
$$b^0 + b^1 + \ldots + b^m = b^0\left(\frac{b^{m+1}-1}{b-1}\right)$$

Number of nodes in the tree (except last subtree)

$$\left(\frac{b-1}{b}\right)\left(\frac{b^{m+1}-1}{(b-1)}\right) = \left(b^m - \frac{1}{b}\right) = O(b^m)$$

$$\approx \frac{1}{b}\left(b^m - \frac{1}{b}\right)$$

$$O(b^{m-1})$$



Branching factor b and height of the tree is m

Let a goal node be chosen randomly. The probability of the goal node being on the right side of the boundary is

$$= \left(\frac{1}{2}\right)$$

$$b^0 + b^1 + \ldots + b^m = b^0\left(\frac{b^{m+1}-1}{b-1}\right)$$

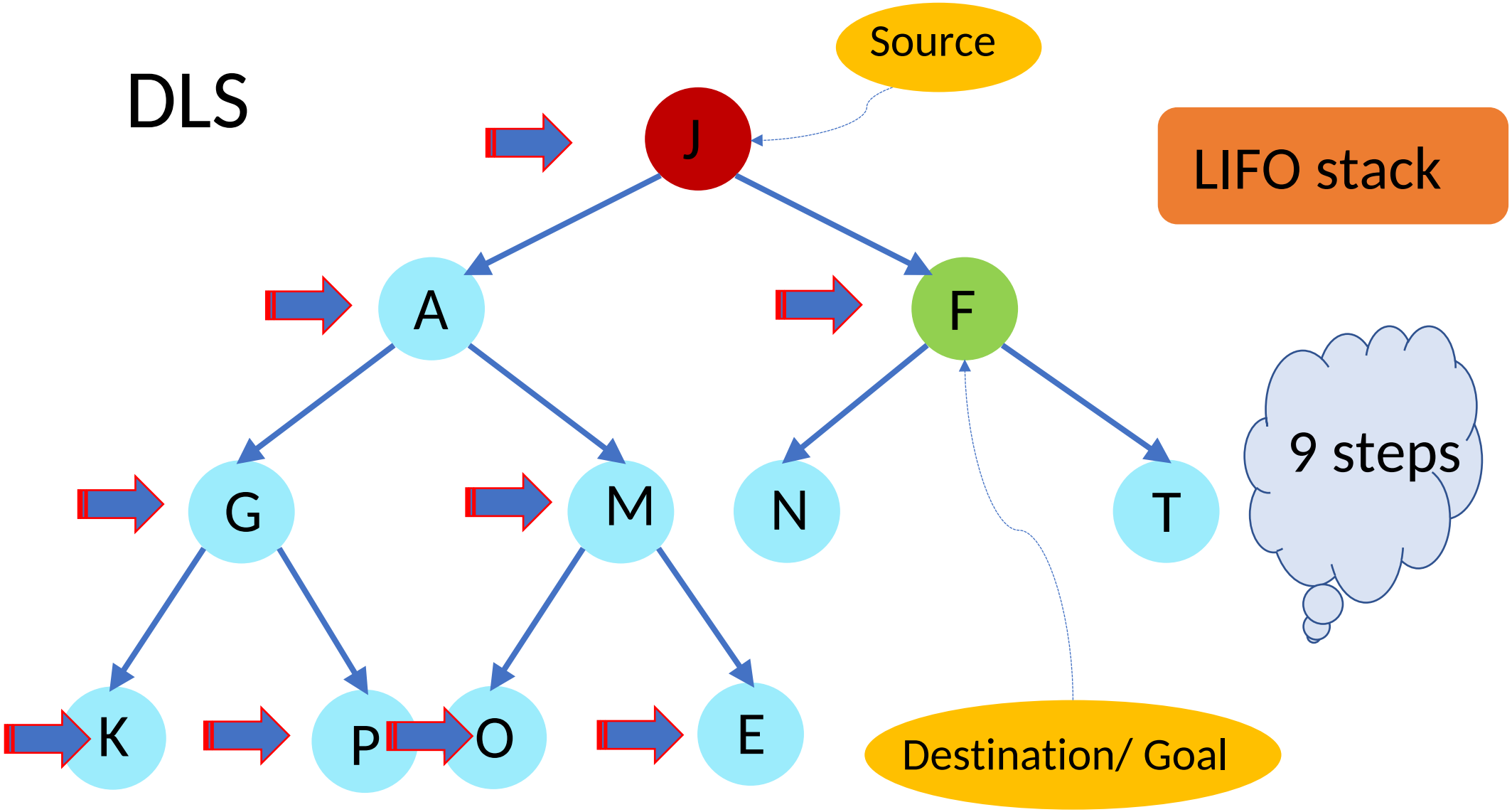
If at least half of the nodes need to be explored

$$\left(\frac{b^{m+1}-1}{2(b-1)}\right)$$

$$= \frac{b^{m+1}-1}{4(b-1)}$$

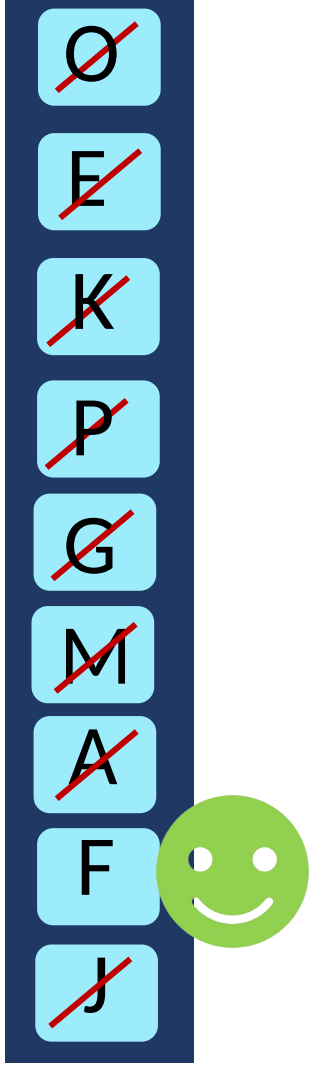
$$O(b^m)$$

DLS



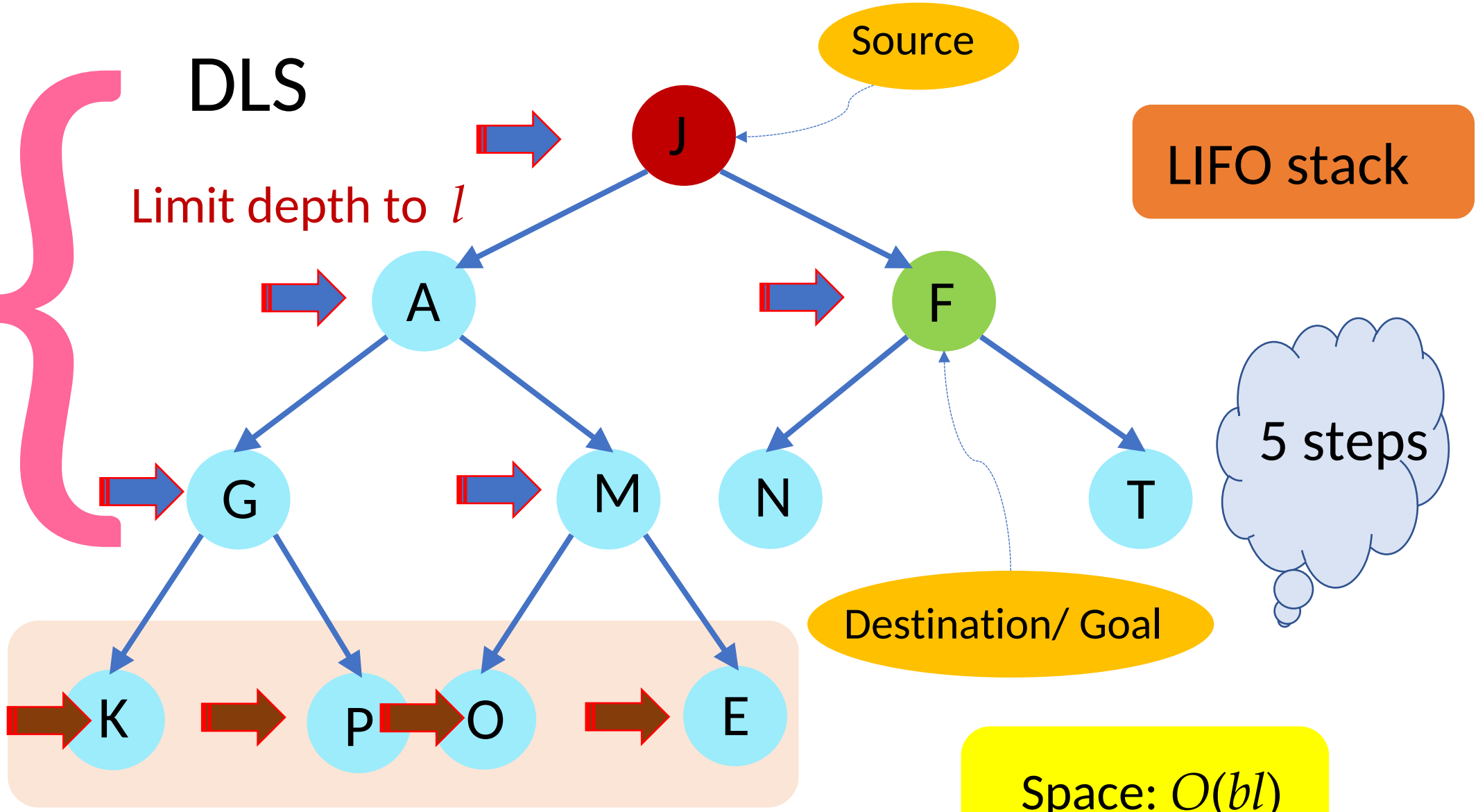
LIFO stack

9 steps



DLS

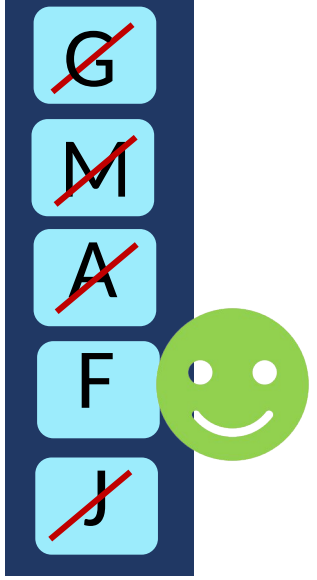
Limit depth to l



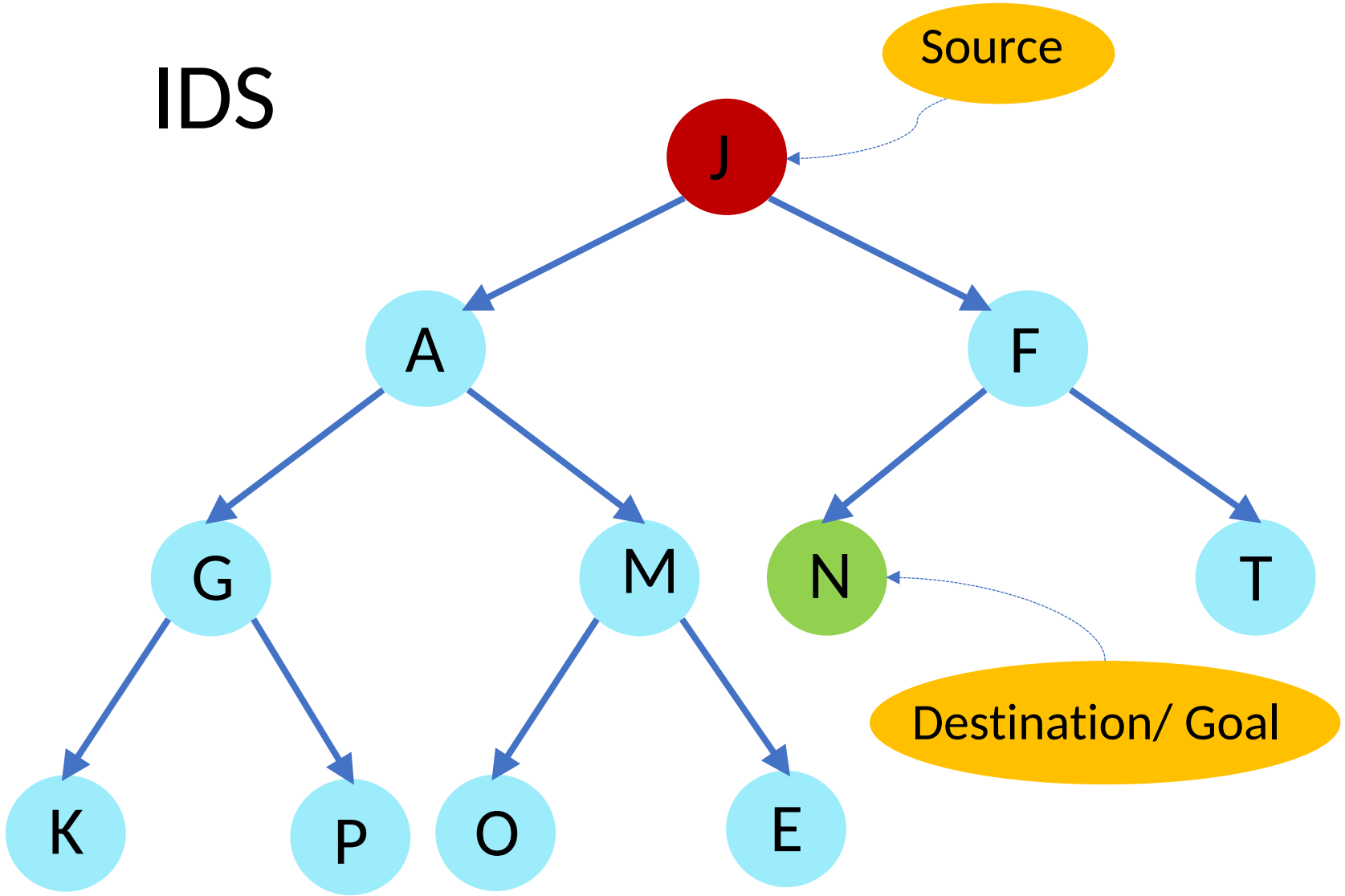
LIFO stack

Space: $O(bl)$
Time: $O(b^l)$

Depth-Limited Search (DLS)

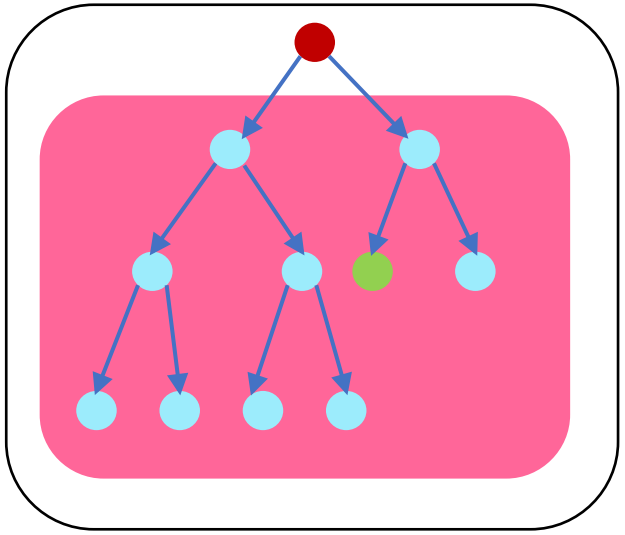


IDS

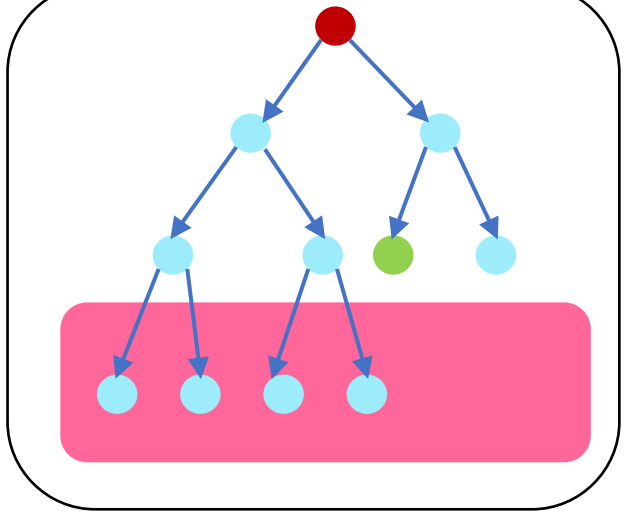


What is the depth of the solution or goal?

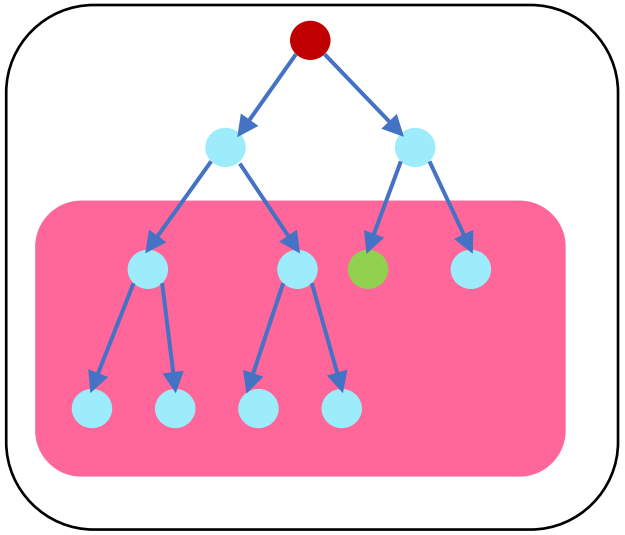
Iterative Deepening Search (IDS)



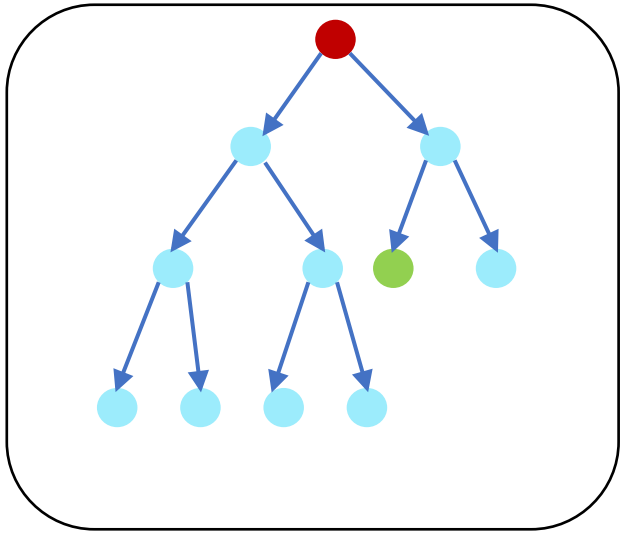
$l = 0$
$O(b^0)$



$l = 2$
$O(b^2)$



$l = 1$
$O(b^1)$



$l = 3$
$O(b^3)$

Iterative Deepening Search (IDS)

Key idea: Iterative deepening search (IDS) applies DLS repeatedly with increasing depth. It terminates when a solution is found or no solutions exists.

DLS

Space: $O(bl)$

Time: $O(b^l)$

IDS combines the benefits of BFS and DFS: Like DFS the memory requirements are very modest ($O(bd)$). Like BFS, it is complete when the branching factor is finite.

In general, iterative deepening is the preferred uninformed search method when there is a large search space and the depth of the solution is not known.

Iterative Deepening Search (IDS)

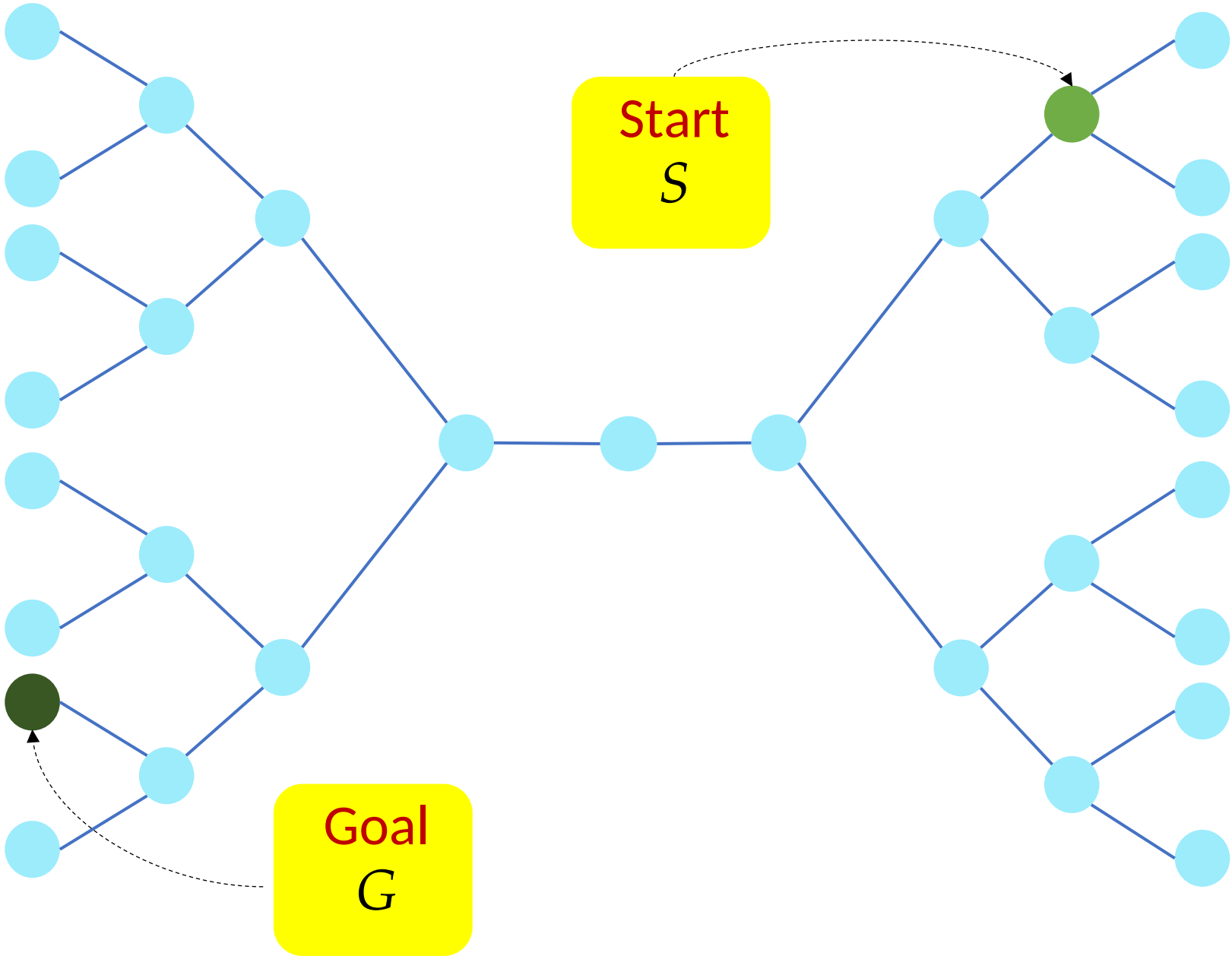
$$1 + [b] + [b + b^2] + [b + b^2 + b^3] + \ldots + [b + b^2 + \ldots + b^d]$$

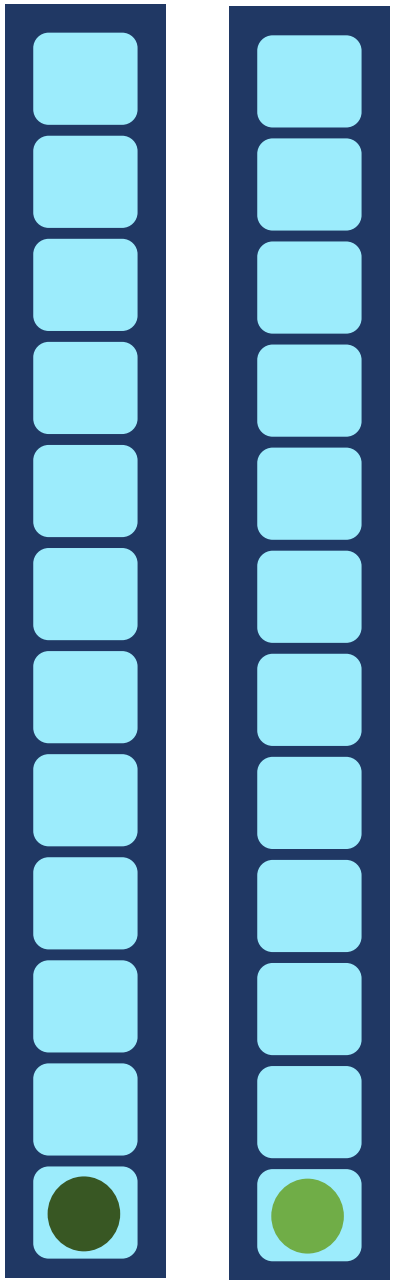
$$S = 1 + (d)\,b + (d-1)\,b^2 + \ldots + (2)\,b^{d-1} + (1)\,b^d$$

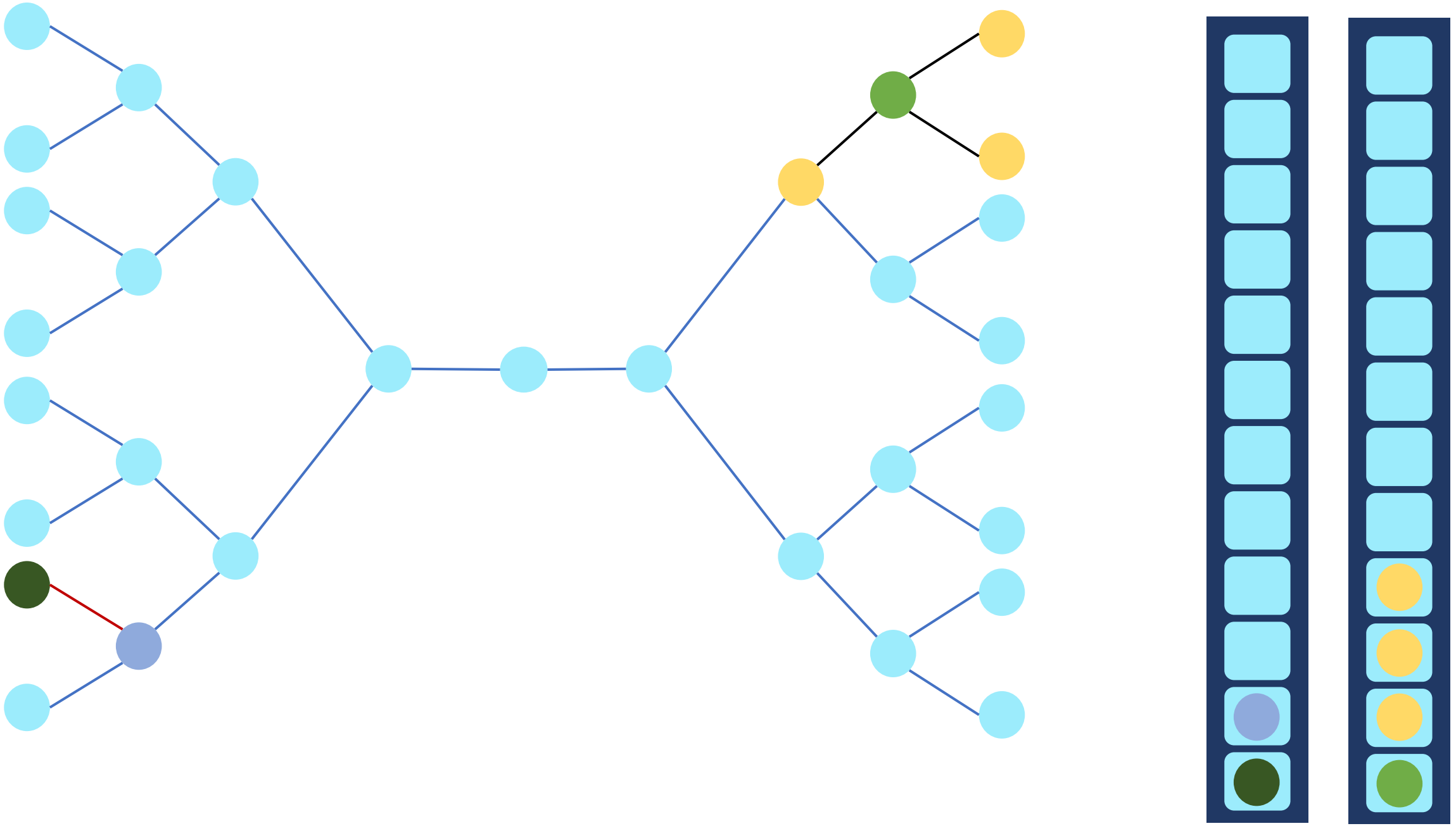
$$Sb = b + \quad (d)\,b^2 + (d-1)\,b^3 + \ldots + (2)\,b^d + (1)\,b^{d+1}$$

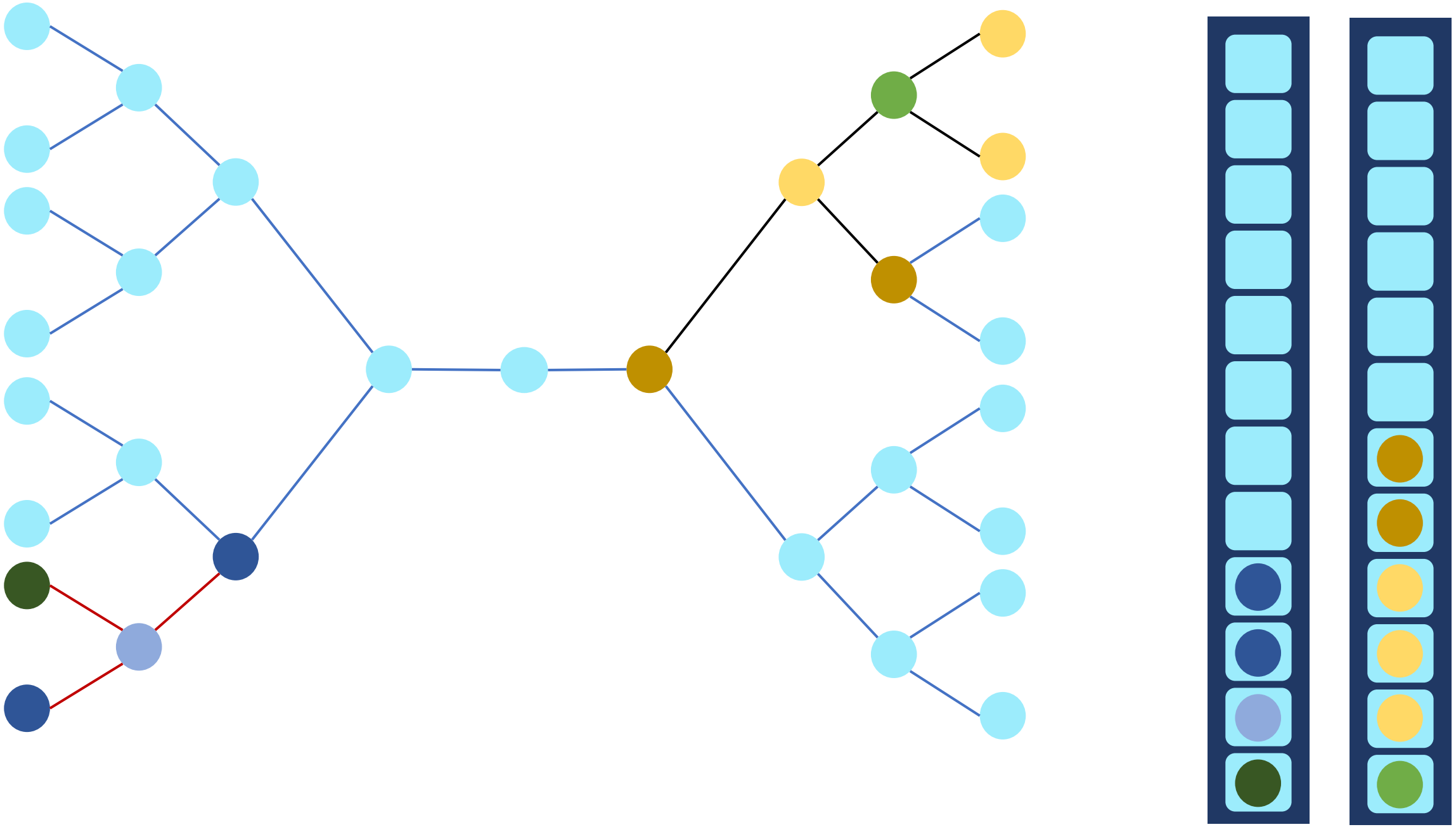
$$S\,(b-1) = [-\,(d-1)\,b - 1] + b^2 + b^3 + \ldots + b^{d+1}$$

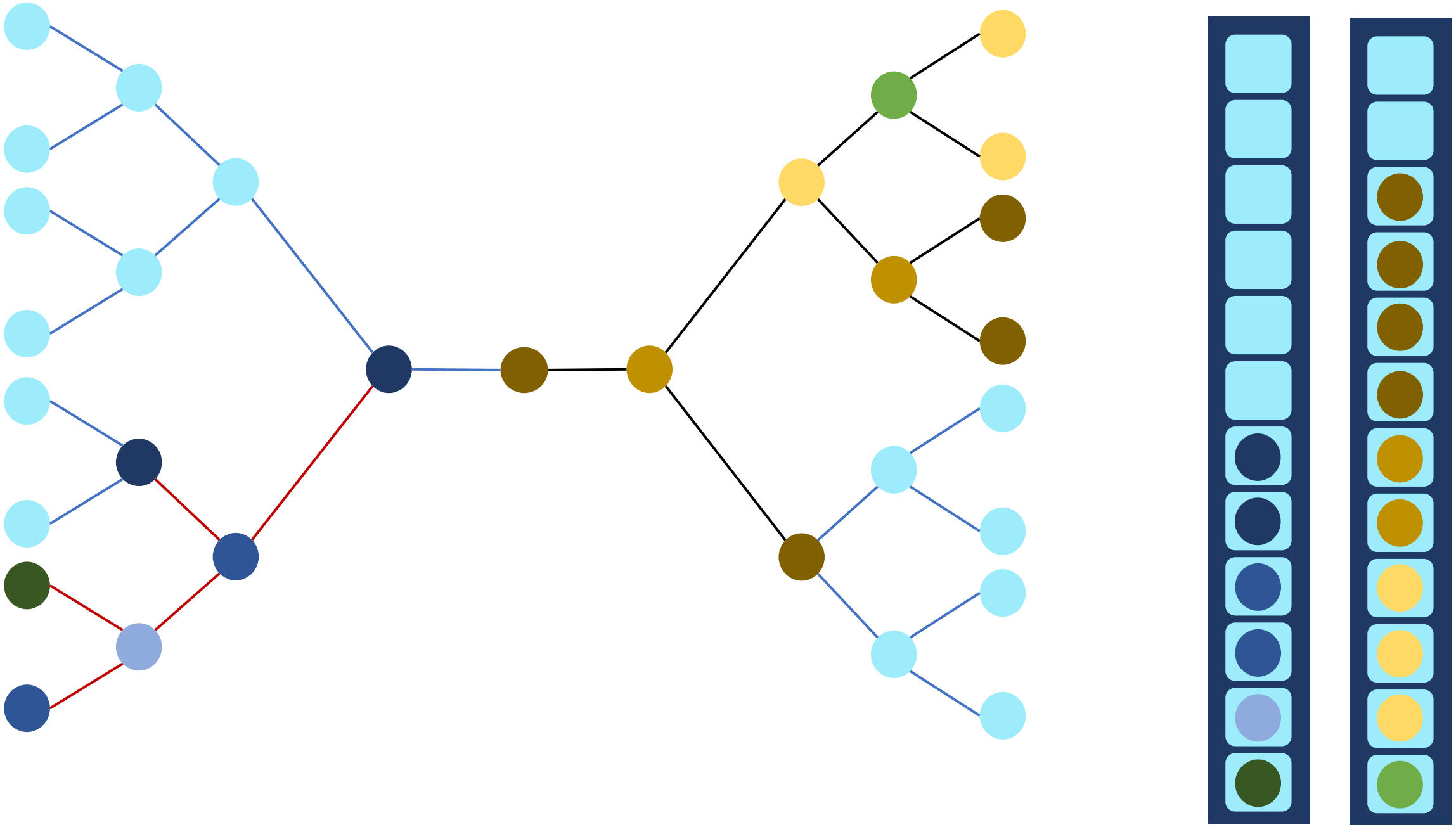
$$S \le \frac{b^2\left(b^d - 1\right)}{(b-1)^2} = \frac{4b^2\left(b^d - 1\right)}{4\,(b-1)^2} \le \frac{4b^2\left(b^d - 1\right)}{4b^2} = O(b^d)$$

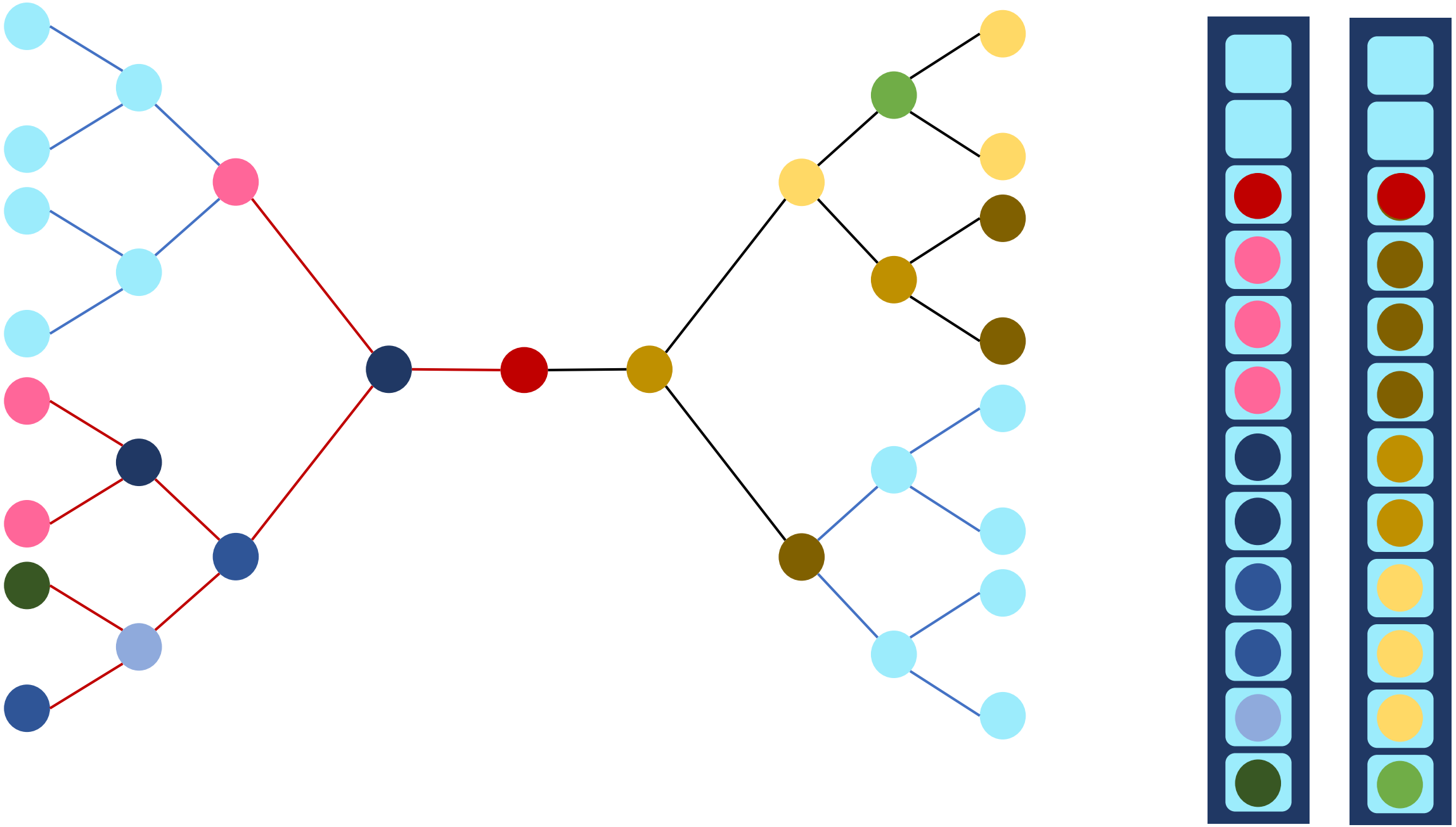

Start
S
Goal
G



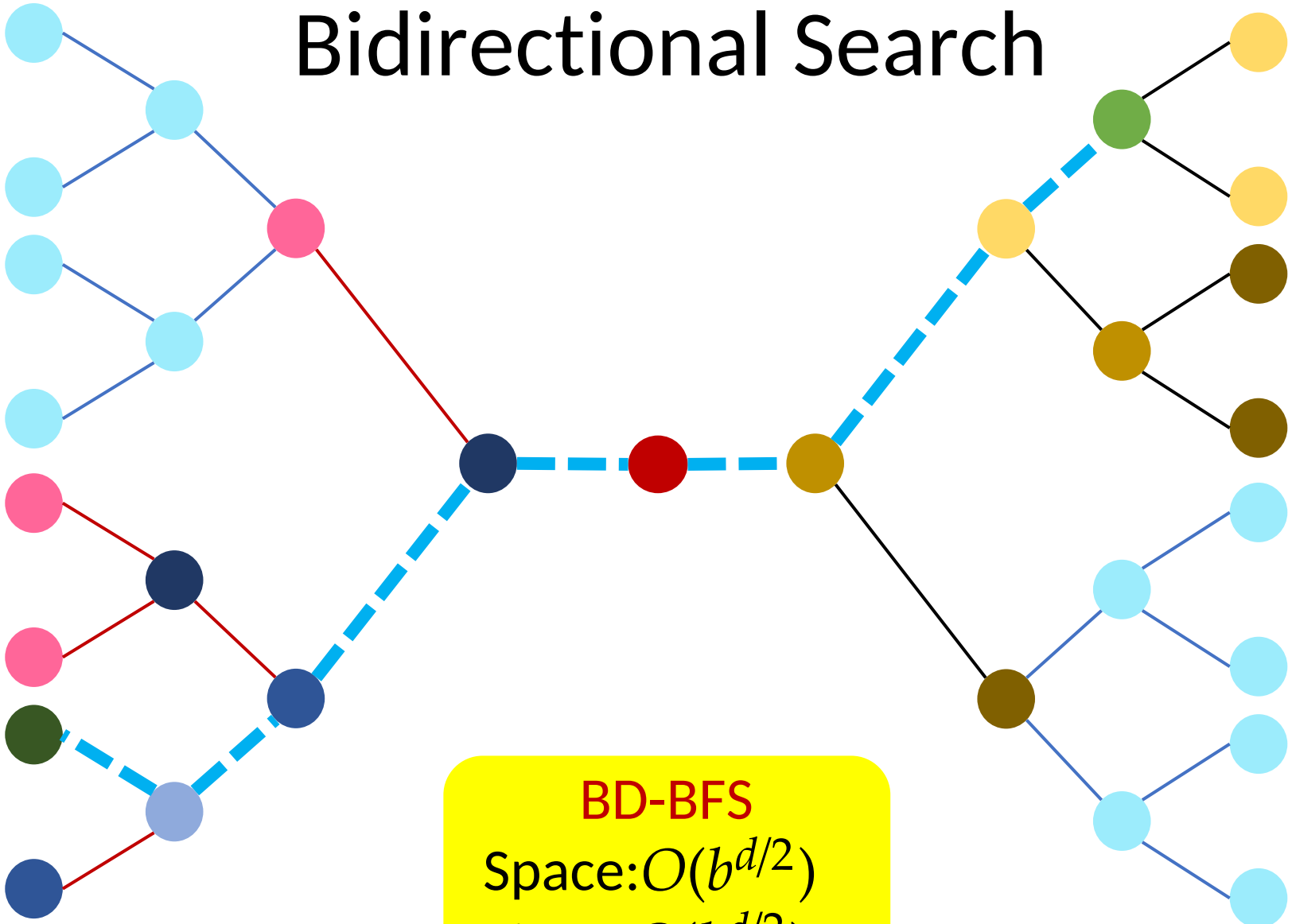








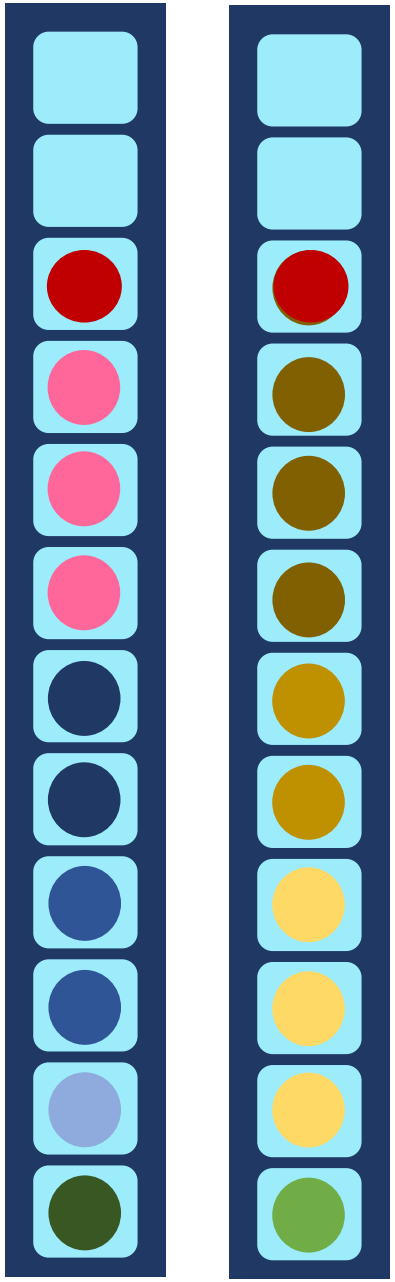
Bidirectional Search



BD-BFS

Space: $O(b^{d/2})$

Time: $O(b^{d/2})$



Search

- **Function** GENERAL-SEARCH (*problem*, *strategy)*
- **returns** a solution or failure
- Initialize the search tree using the initial state of problem
- **loop do**
- **if** there are no candidates for expansion, **then return** failure
- Choose a node for expansion according to *strategy*
- **if** node contains goal state **then return** *solution*
- **else** expand node and add resulting nodes to search tree.
- **end**

Uninformed (Blind) search

- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Uniform-Cost Search (UCS)
- Depth-Limited Search (DLS)
- Iterative Deepening Search (IDS)
- Bidirectional Search

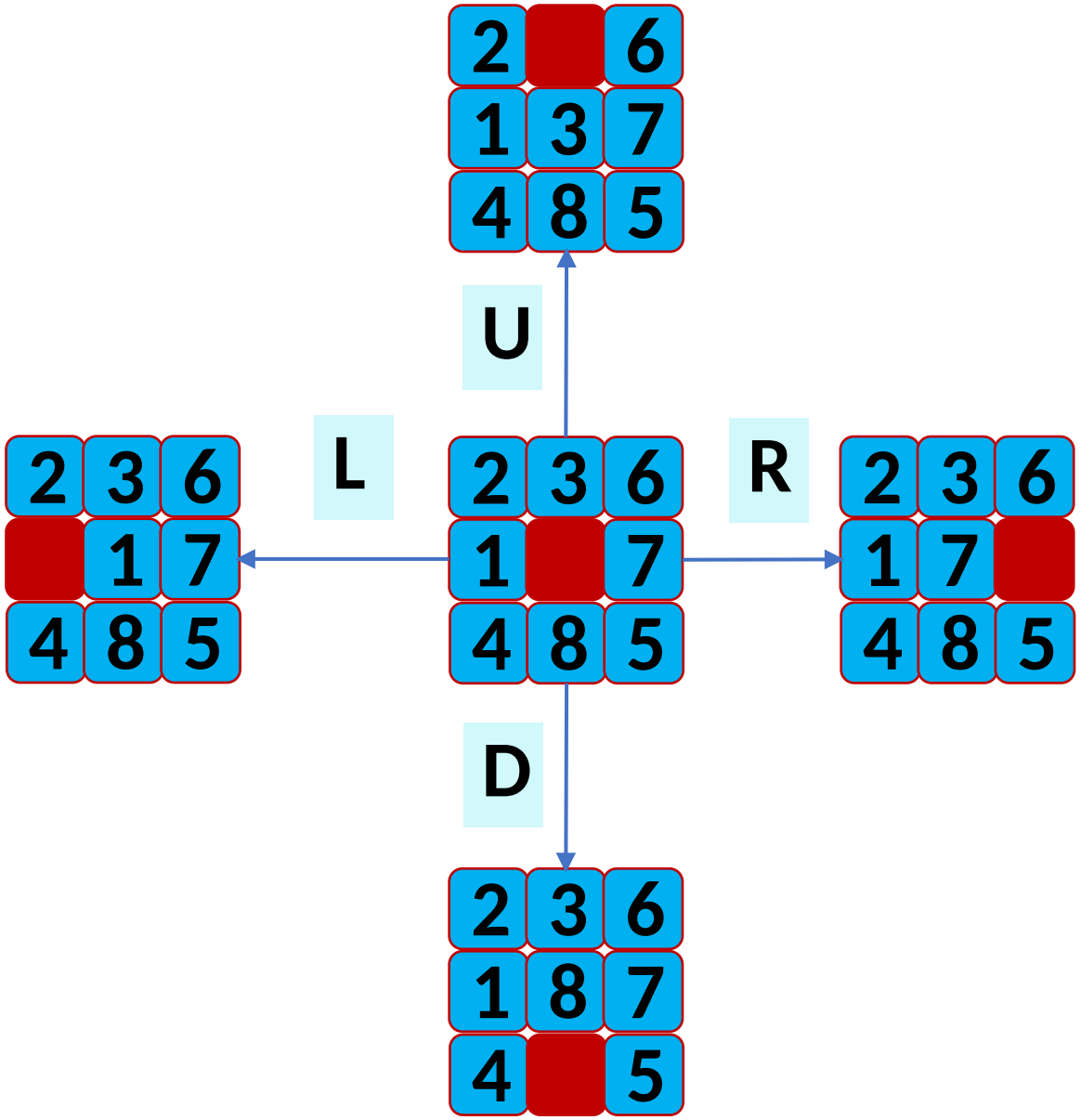
Breadth-First Search (BFS)

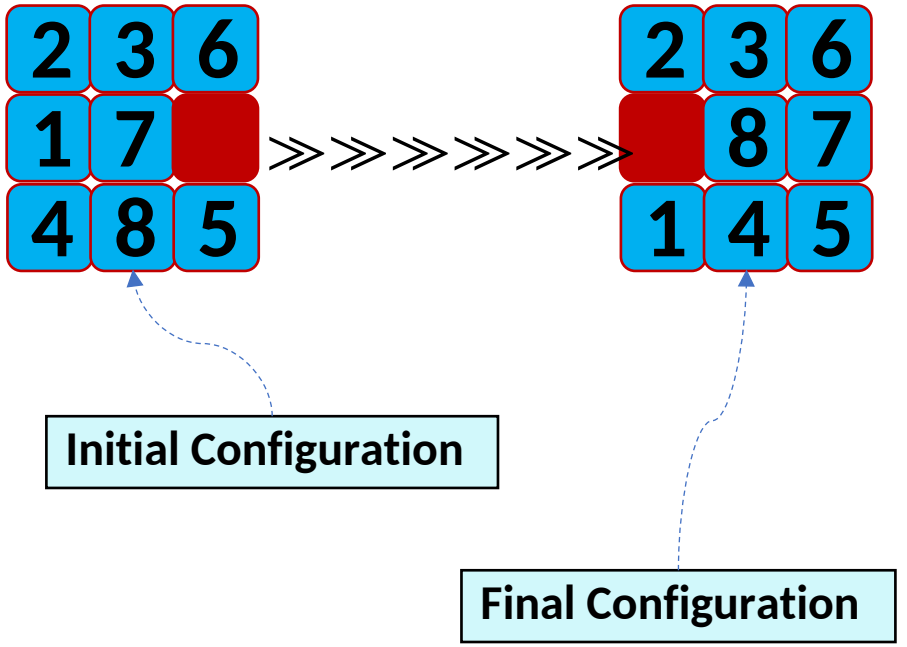
- Finding the shortest path
  - Explores all nodes at the current depth level before moving to the next level.
  - Guarantees the shortest path in an unweighted graph.
- Finding the shortest path on a map
- Finding the shortest path to a friend
- Finding a win position in chess
- Searching the web (Web crawlers)
- Finding friends on social media (Finding friend suggestions)

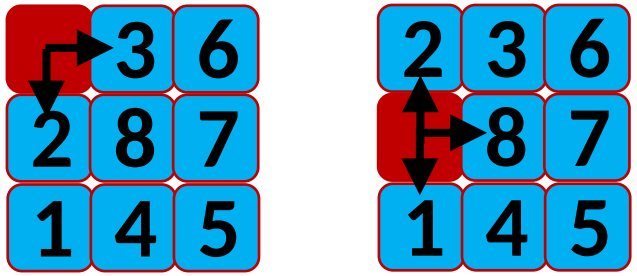
Informed Search Methods (Heuristic-Based)

- Best-First Search
- A^* Search
- *Iterative Deepening* A^* Search (*IDA*)
- Weighted A^*
- Memory-Bounded A^*
- Beam Search

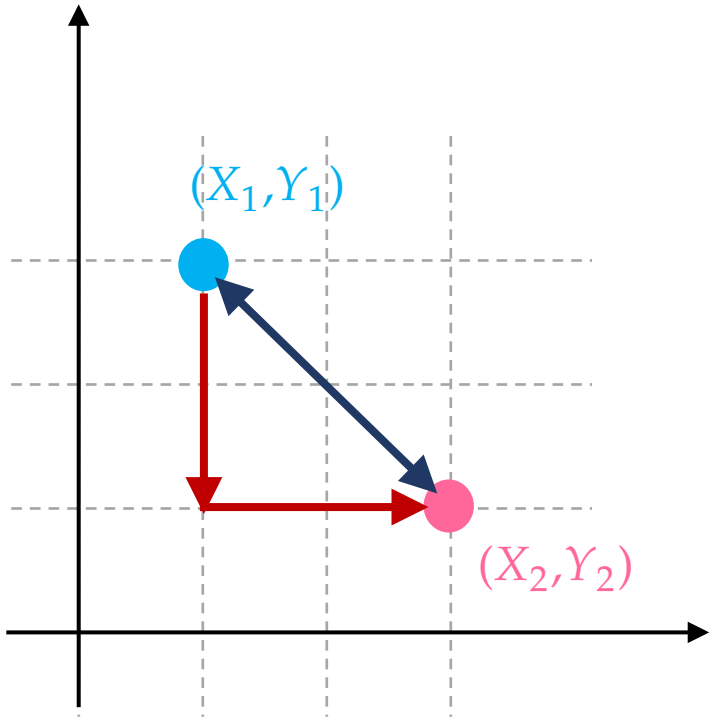
8 Puzzle Problem

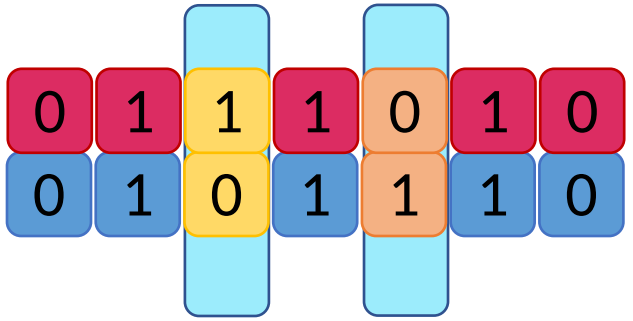






Distance: Hamming, Euclidean, Manhattan

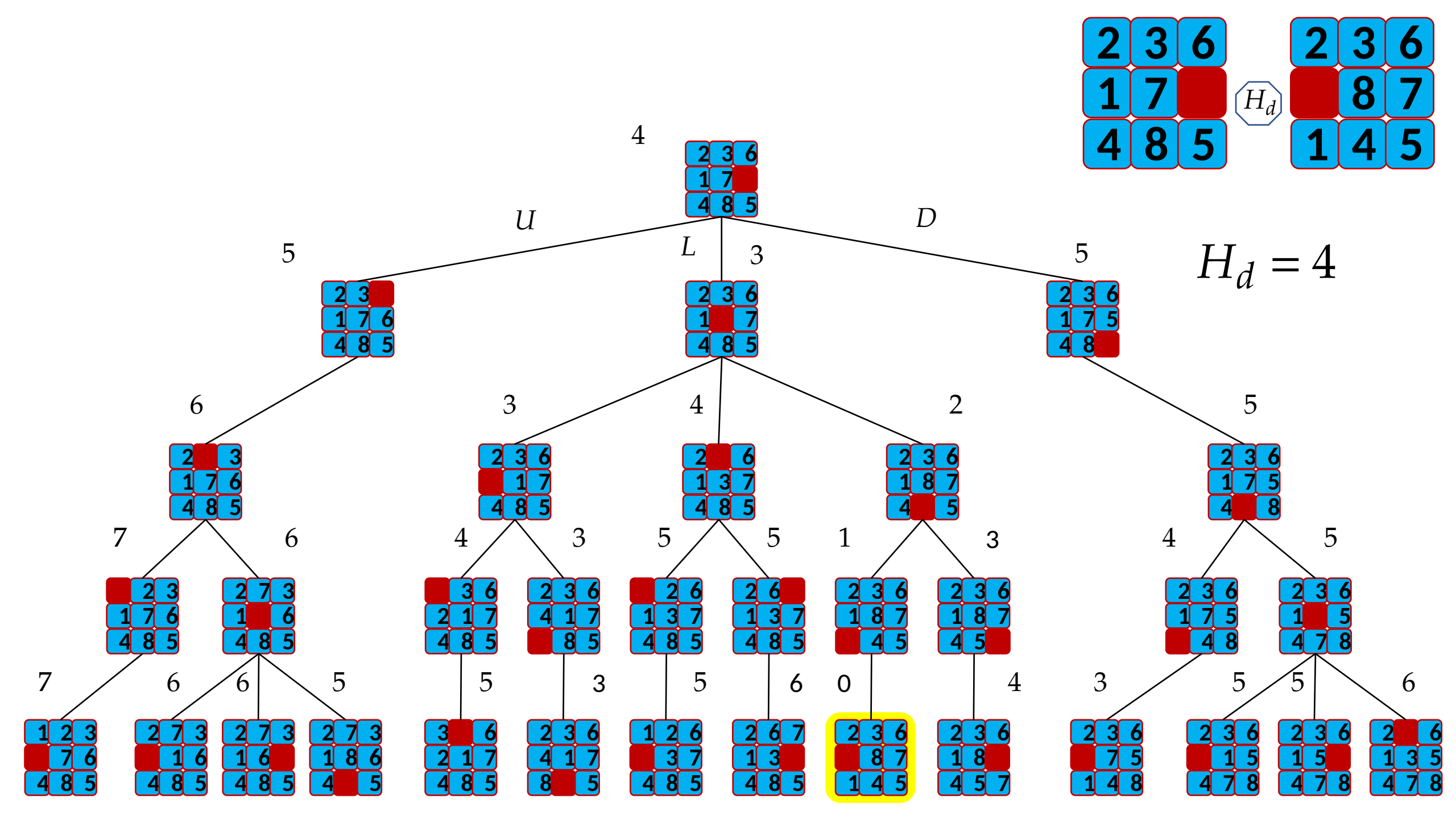


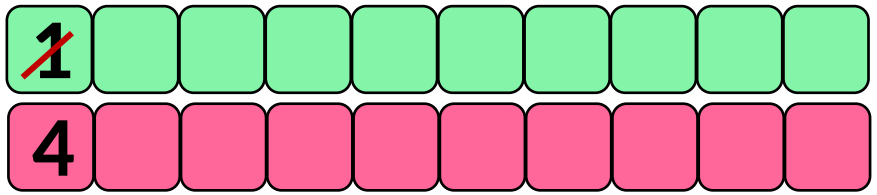


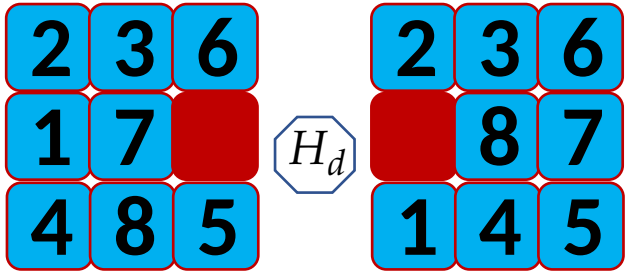
$$H_d = 2$$

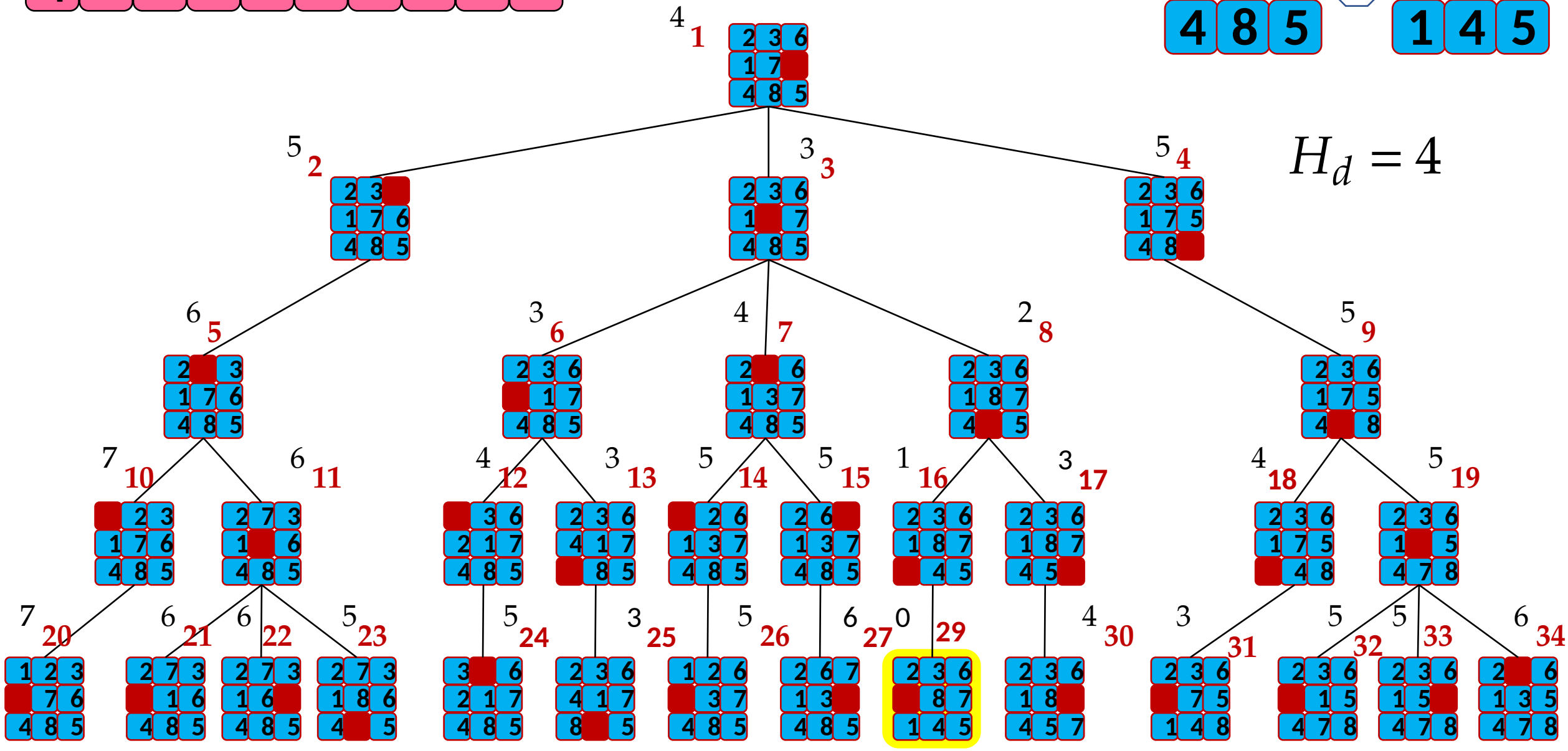
$$M_d = |X_1 - X_2| + |Y_1 - Y_2|$$

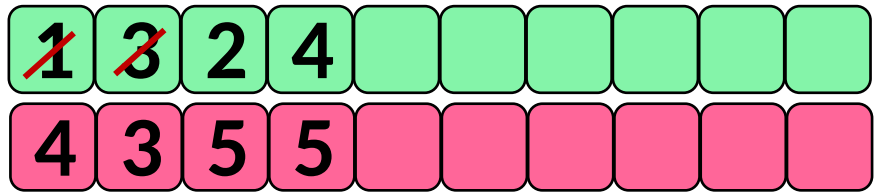
$$E_d = \sqrt{(X_1 - X_2)^2 + (Y_1 - Y_2)^2}$$

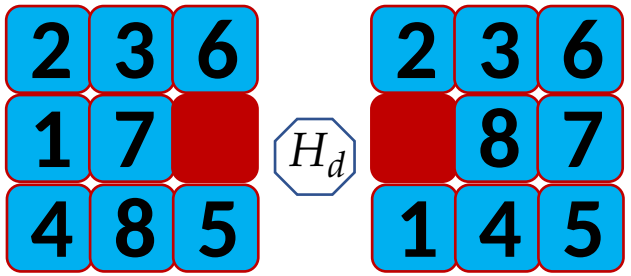

H_d
$H_d = 4$
U
L
D



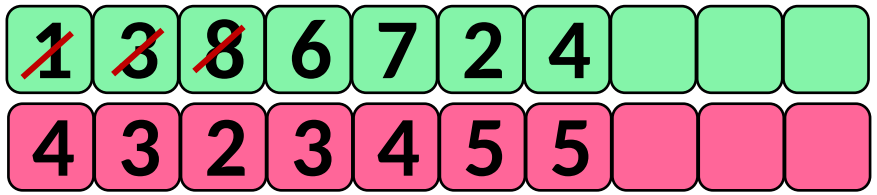

H_d

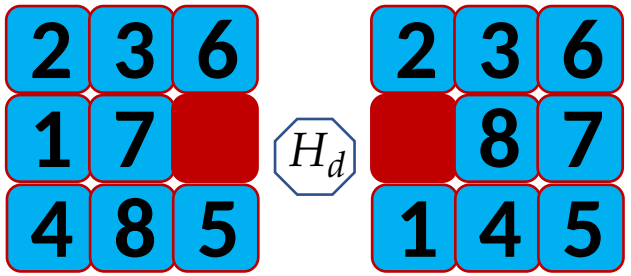

$H_d = 4$



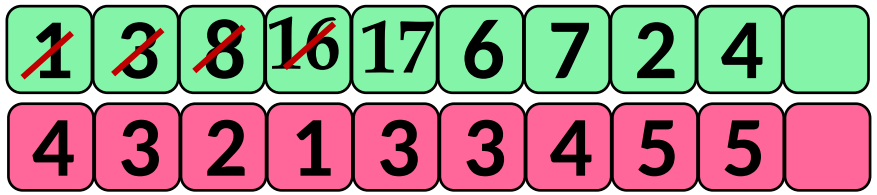


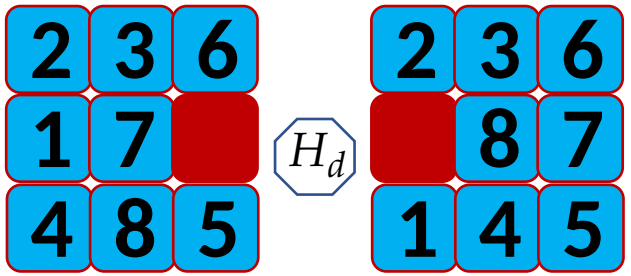
$H_d = 4$



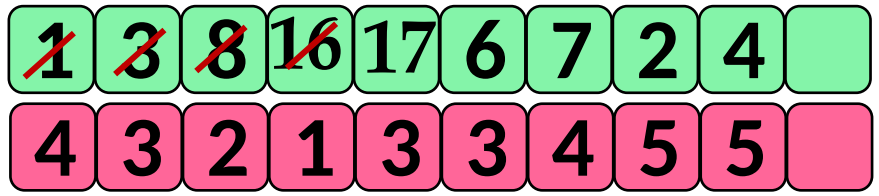


$$H_d = 4$$

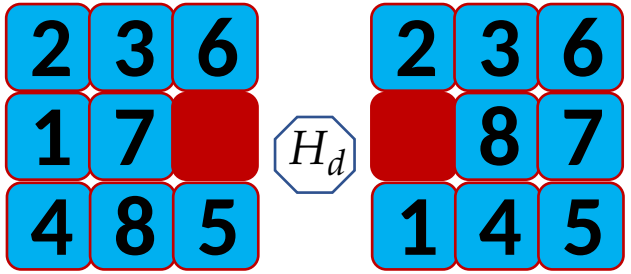


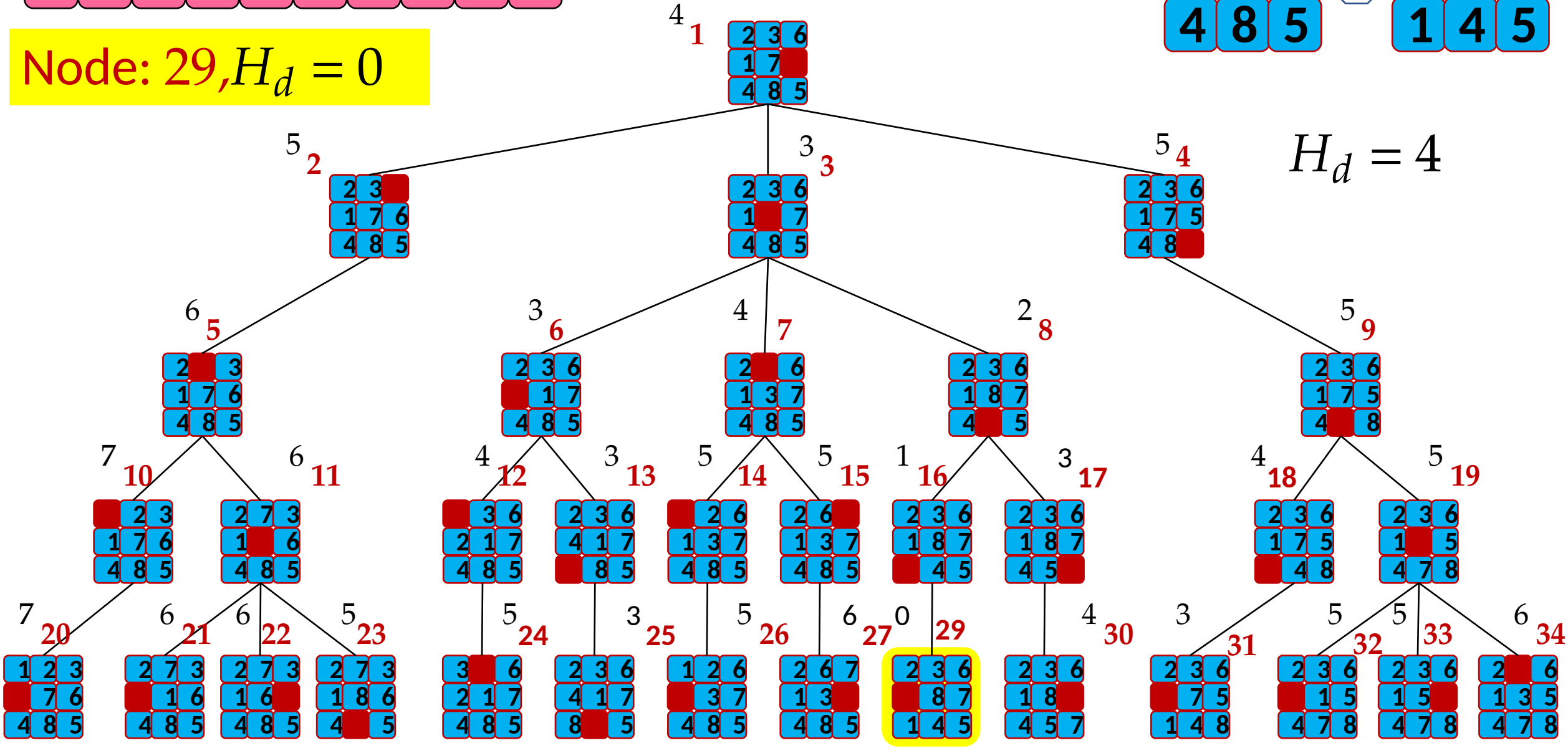


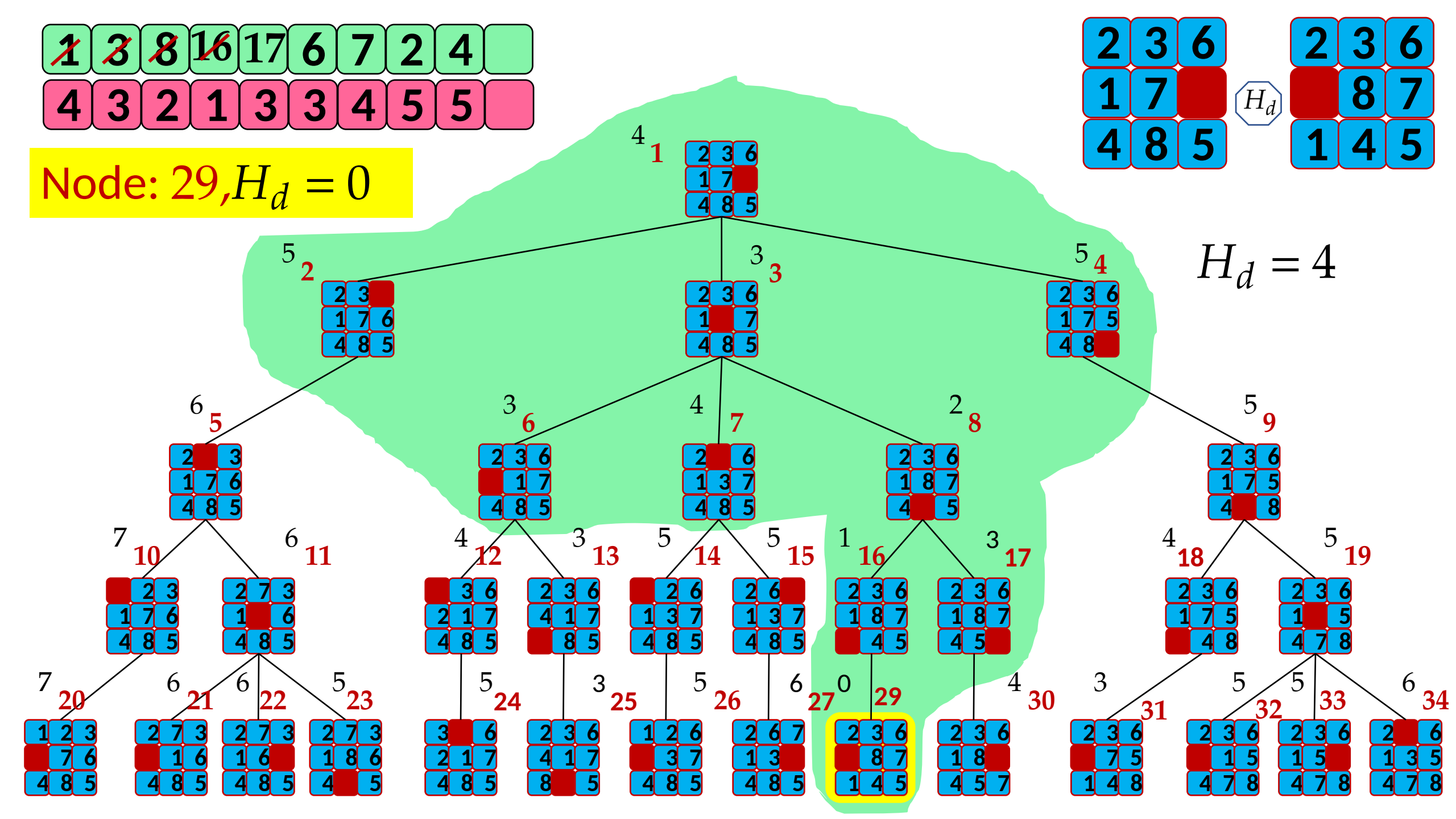
$H_d = 4$



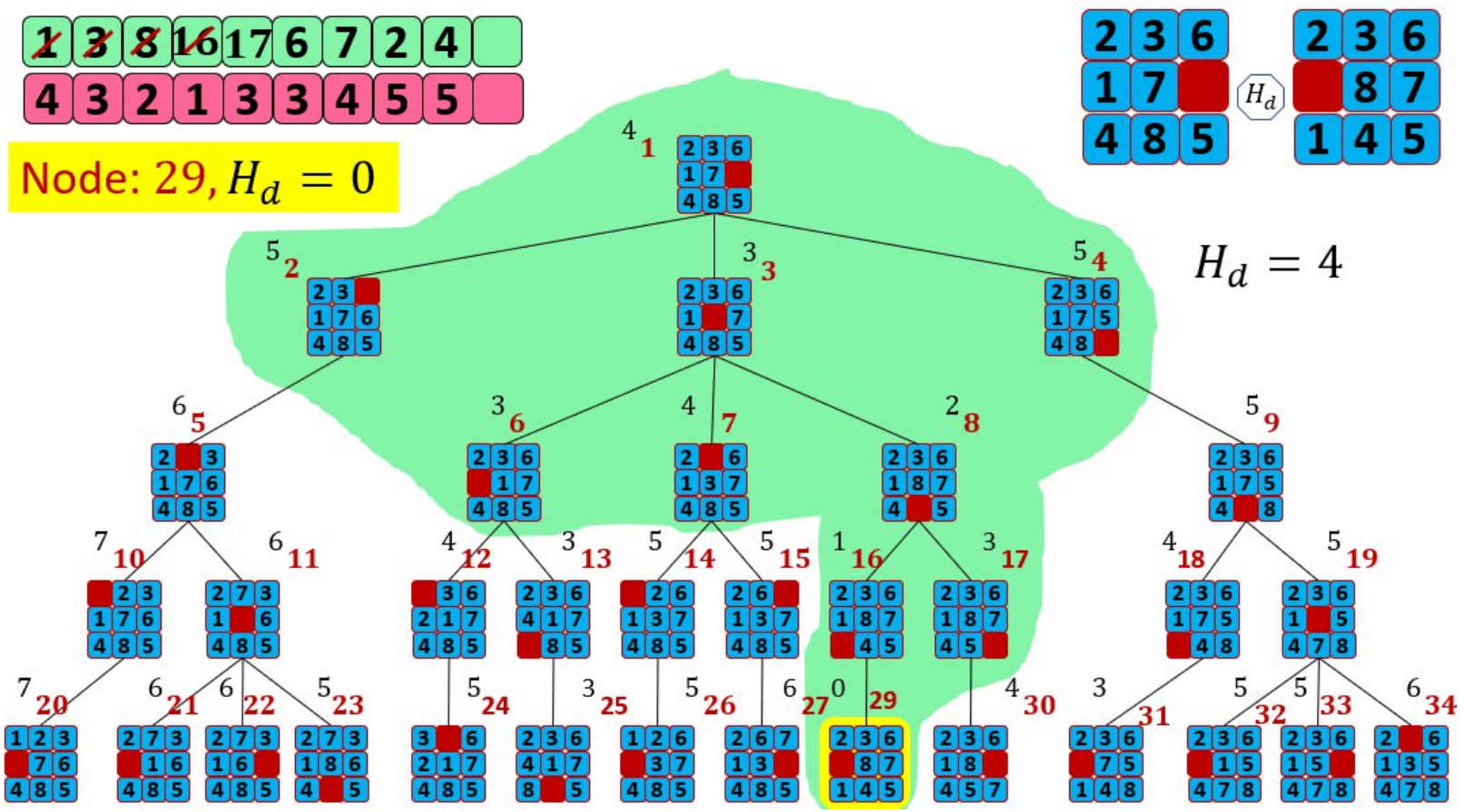
Node: 29, $H_d = 0$

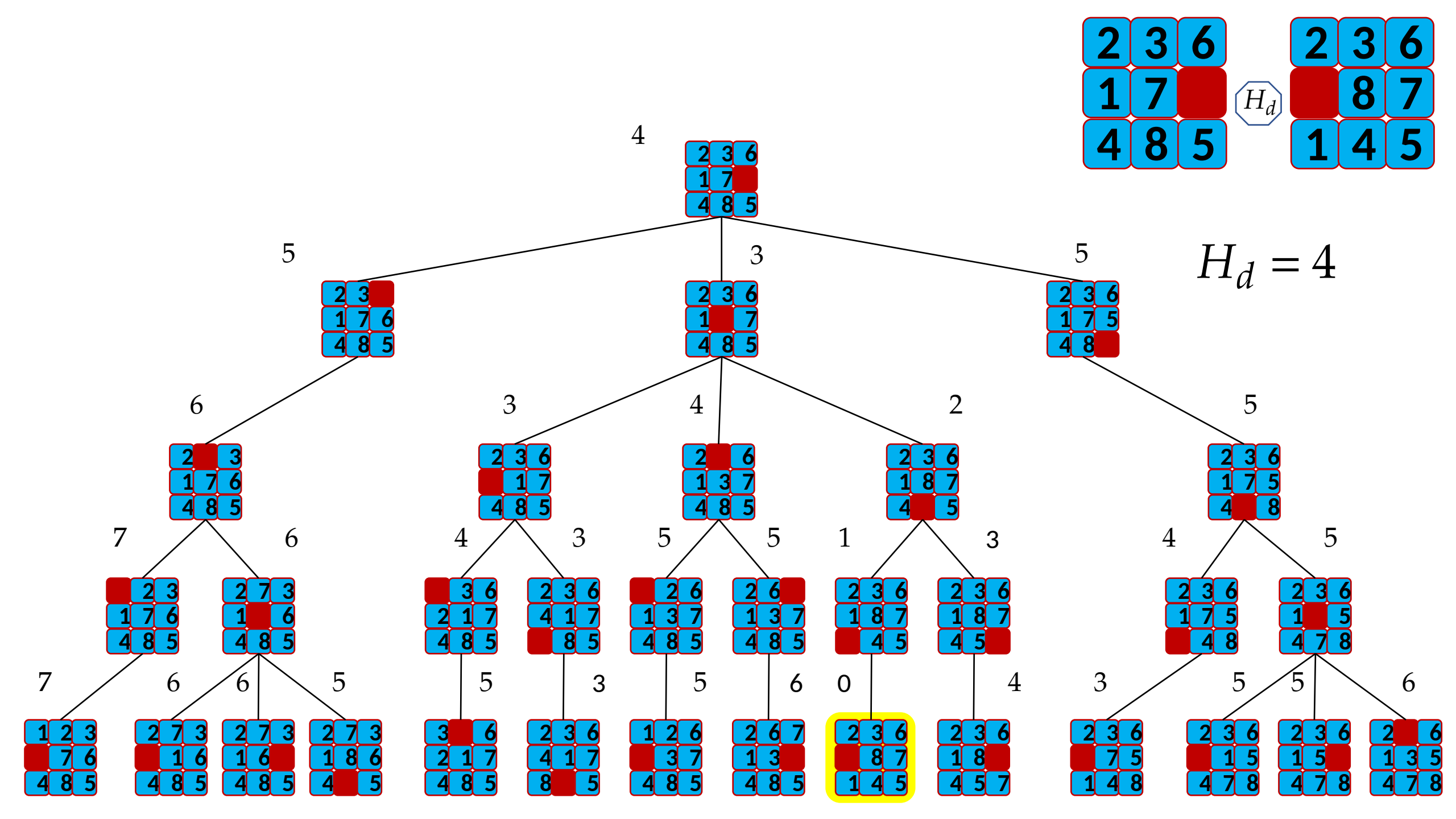


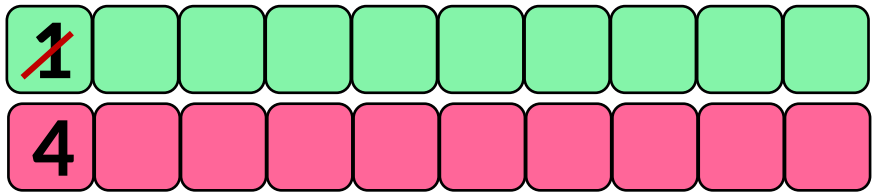


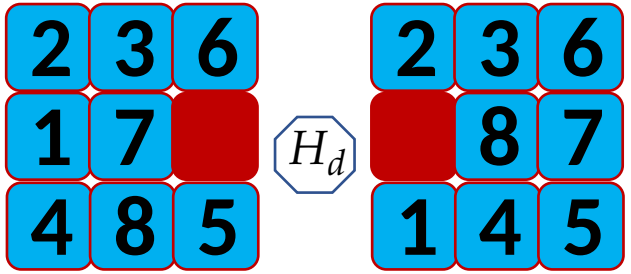

1 3 8 16 17 6 7 2 4
4 3 2 1 3 3 4 5 5
Node: 29, $H_d = 0$
H_d
$H_d = 4$

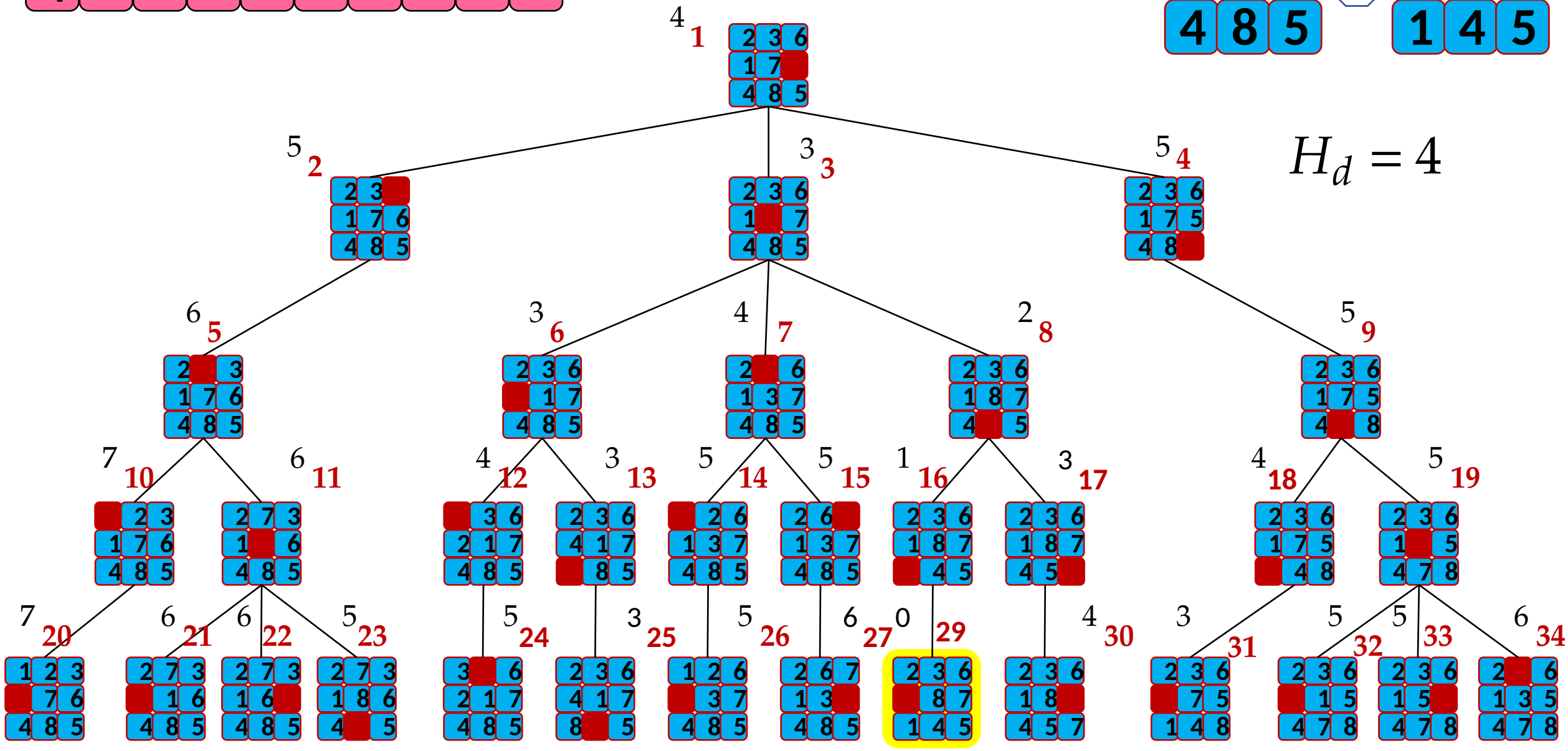
Best-First Search (BFS)

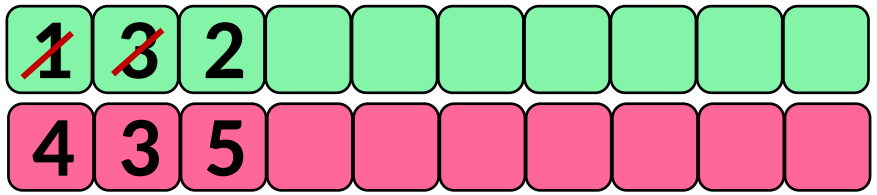


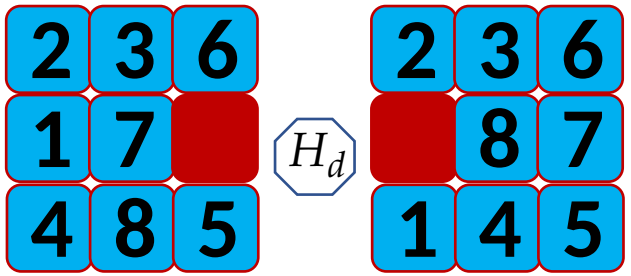

H_d
$H_d = 4$



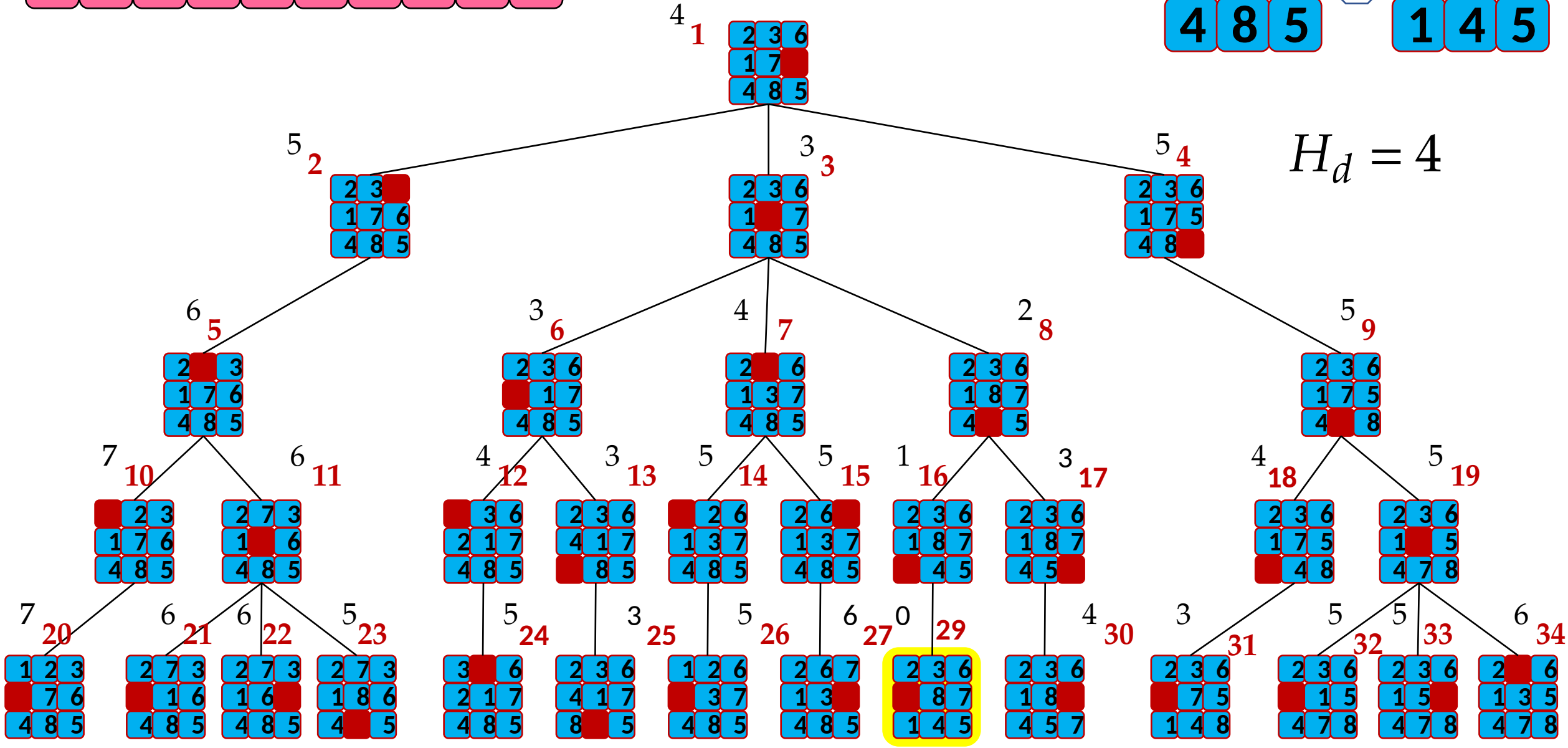

H_d

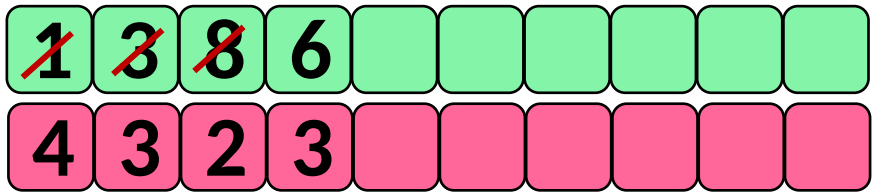

$H_d = 4$

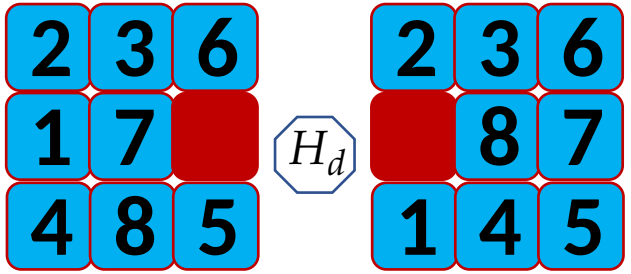

1 3 2
4 3 5


2 3 6
1 7
4 8 5
H_d
2 3 6
8 7
1 4 5

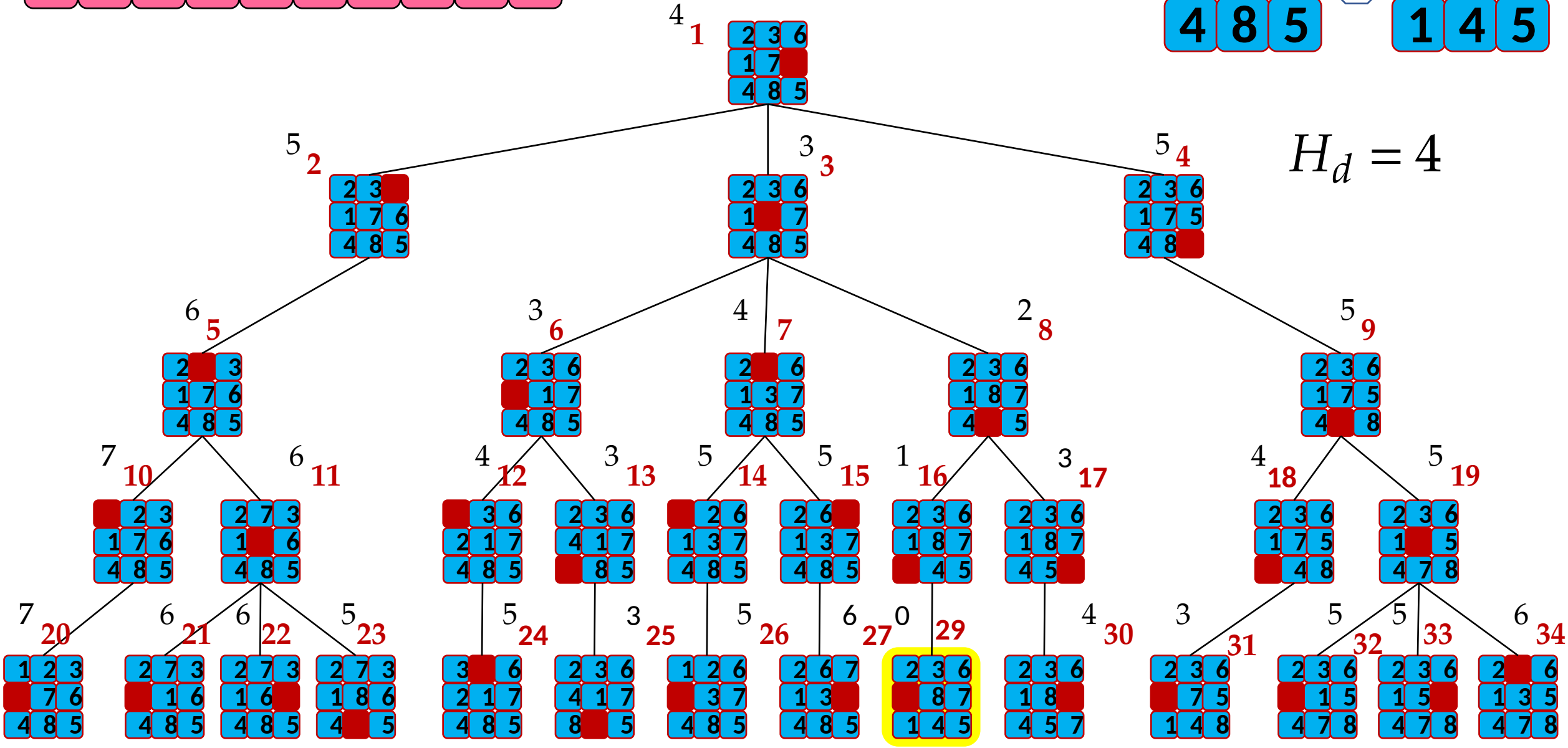
$H_d = 4$

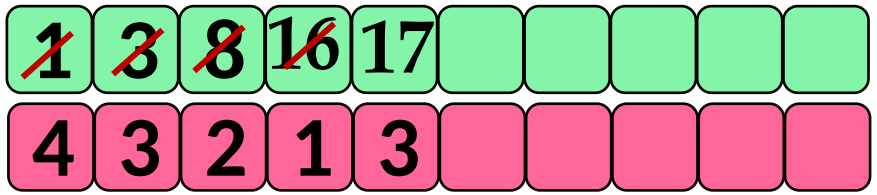

4 1
5 2
3 3
5 4
6 5
3 6
4 7
2 8
5 9
7 10
6 11
4 12
3 13
5 14
5 15
1 16
3 17
4 18
5 19
7 20
6 21
6 22
5 23
5 24
3 25
5 26
6 27
0 29
4 30
3 31
5 32
5 33
6 34

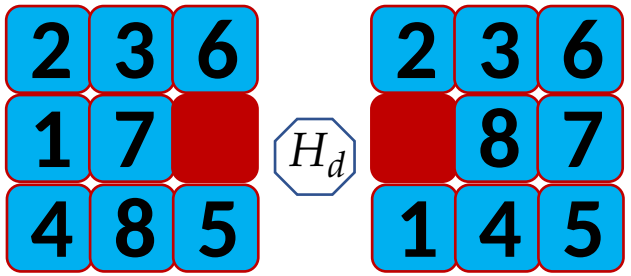

1 3 8 6
4 3 2 3


2 3 6
1 7
4 8 5
H_d
2 3 6
8 7
1 4 5

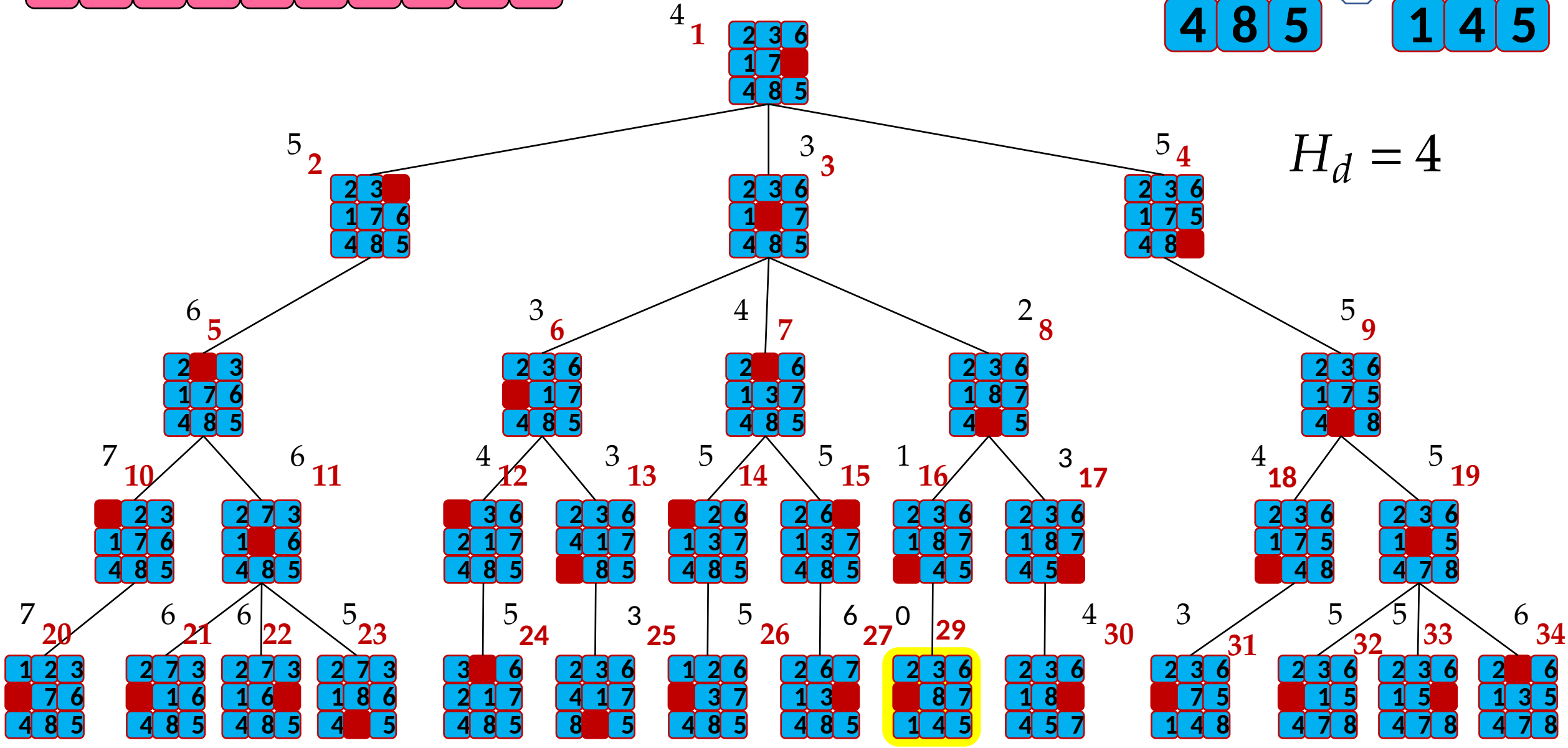
$H_d = 4$

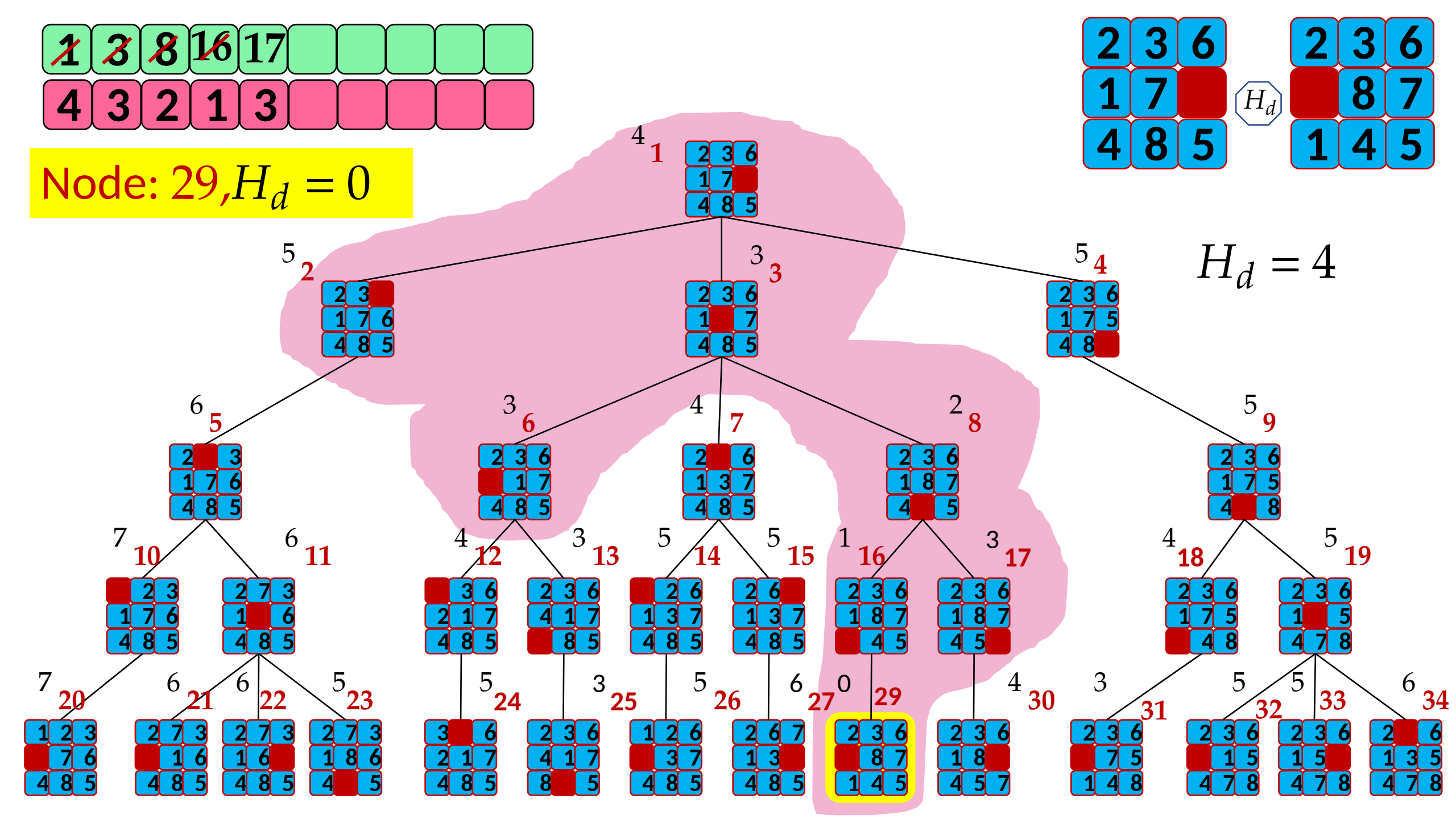

4 1
5 2
3 3
5 4
6 5
3 6
4 7
2 8
5 9
7 10
6 11
4 12
3 13
5 14
5 15
1 16
3 17
4 18
5 19
7 20
6 21
6 22
5 23
5 24
3 25
5 26
6 27
0 29
4 30
3 31
5 32
5 33
6 34


1 3 8 16 17
4 3 2 1 3

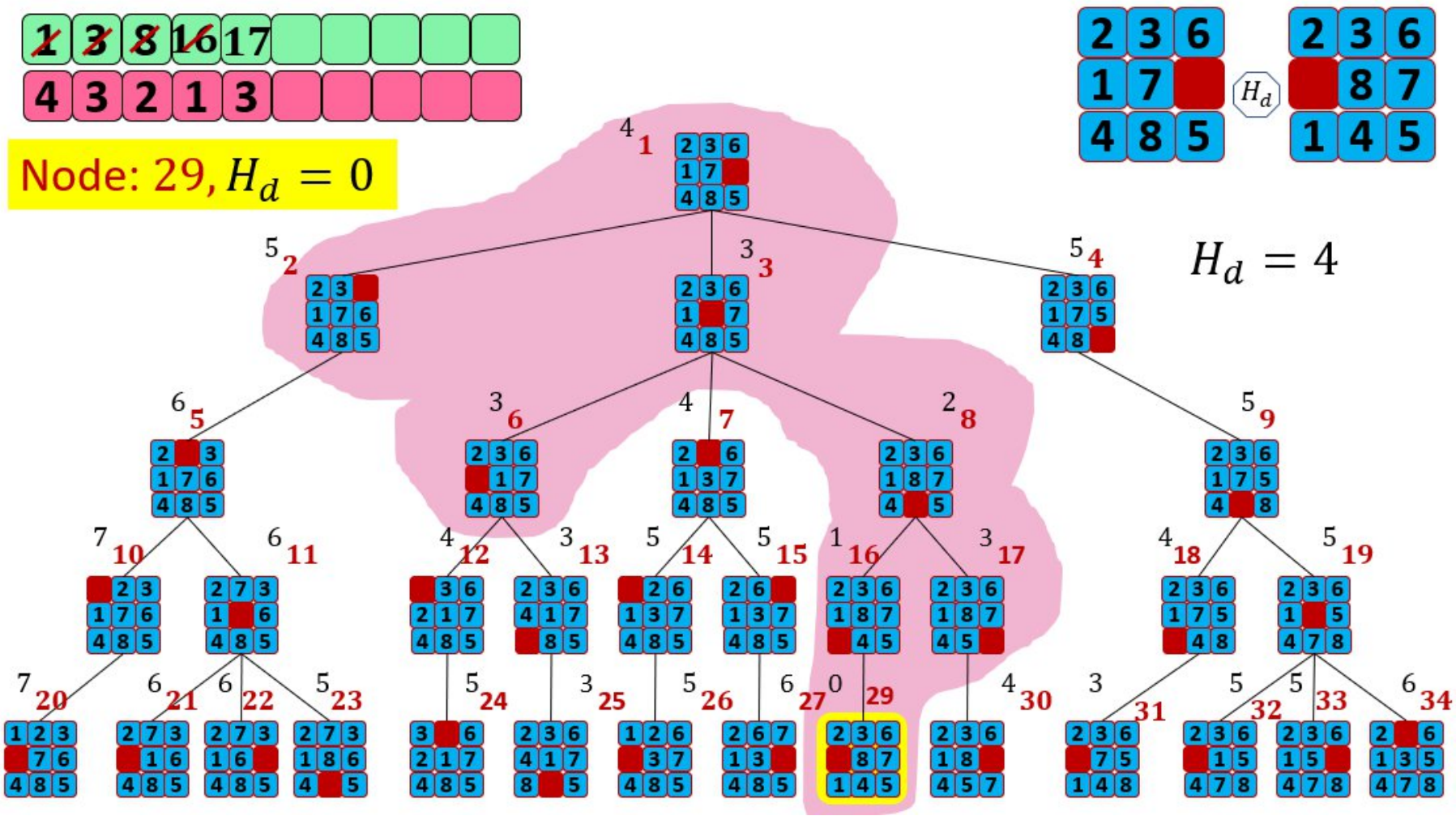

H_d

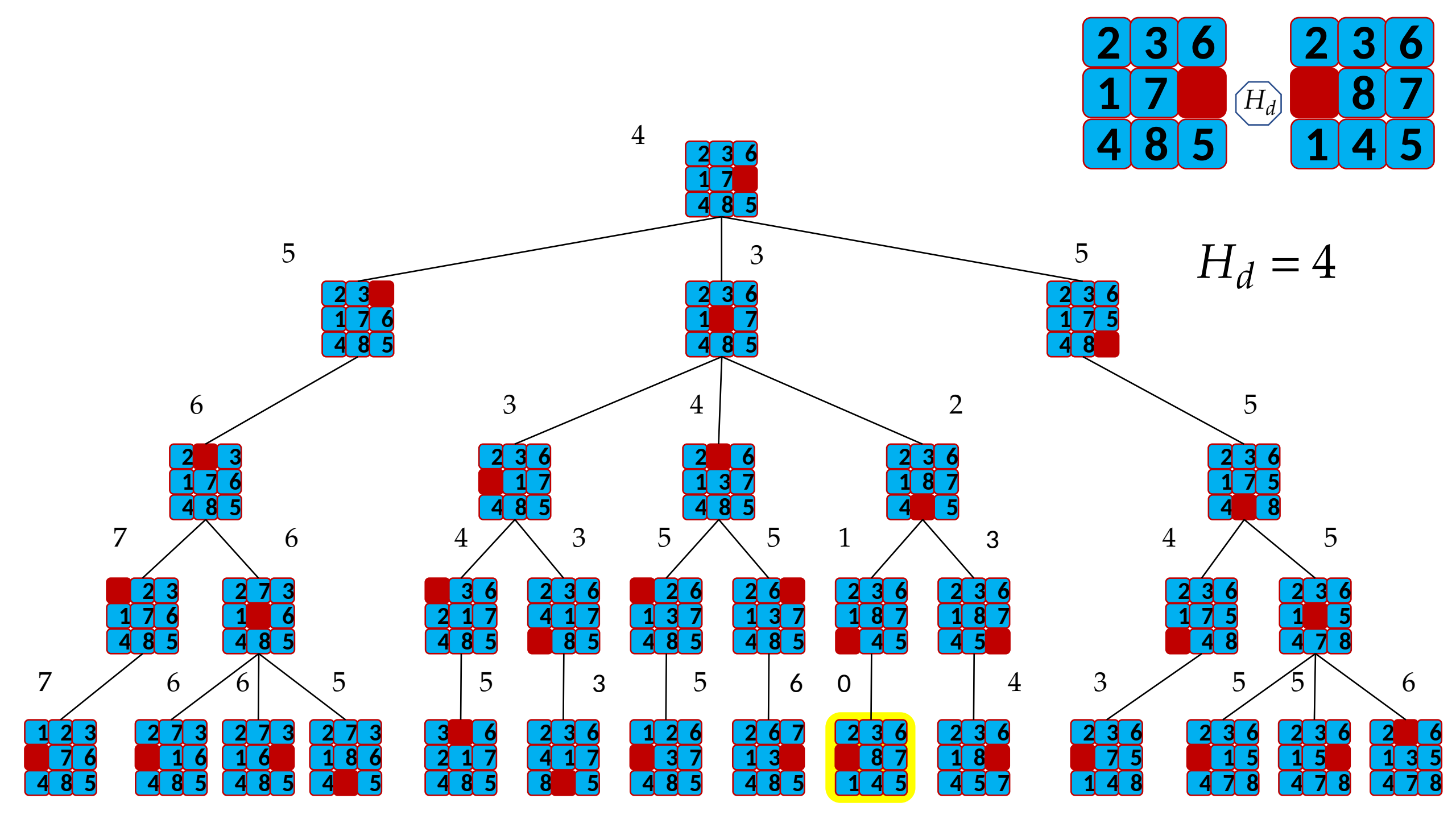
$H_d = 4$

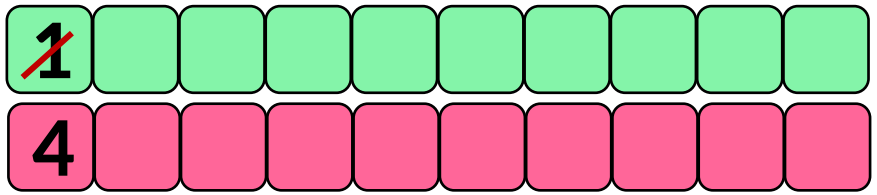

4 1
5 2
3 3
5 4
6 5
3 6
4 7
2 8
5 9
7 10
6 11
4 12
3 13
5 14
5 15
1 16
3 17
4 18
5 19
7 20
6 21
6 22
5 23
5 24
3 25
5 26
6 27
0 29
4 30
3 31
5 32
5 33
6 34

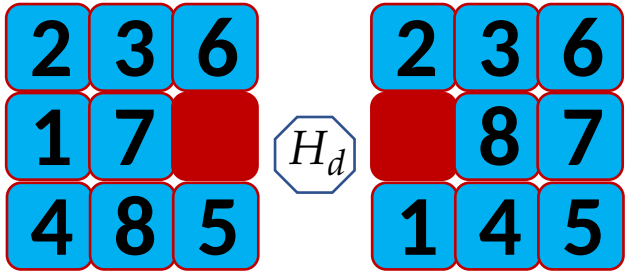

1 3 8 16 17
4 3 2 1 3
Node: 29, $H_d = 0$
H_d
$H_d = 4$

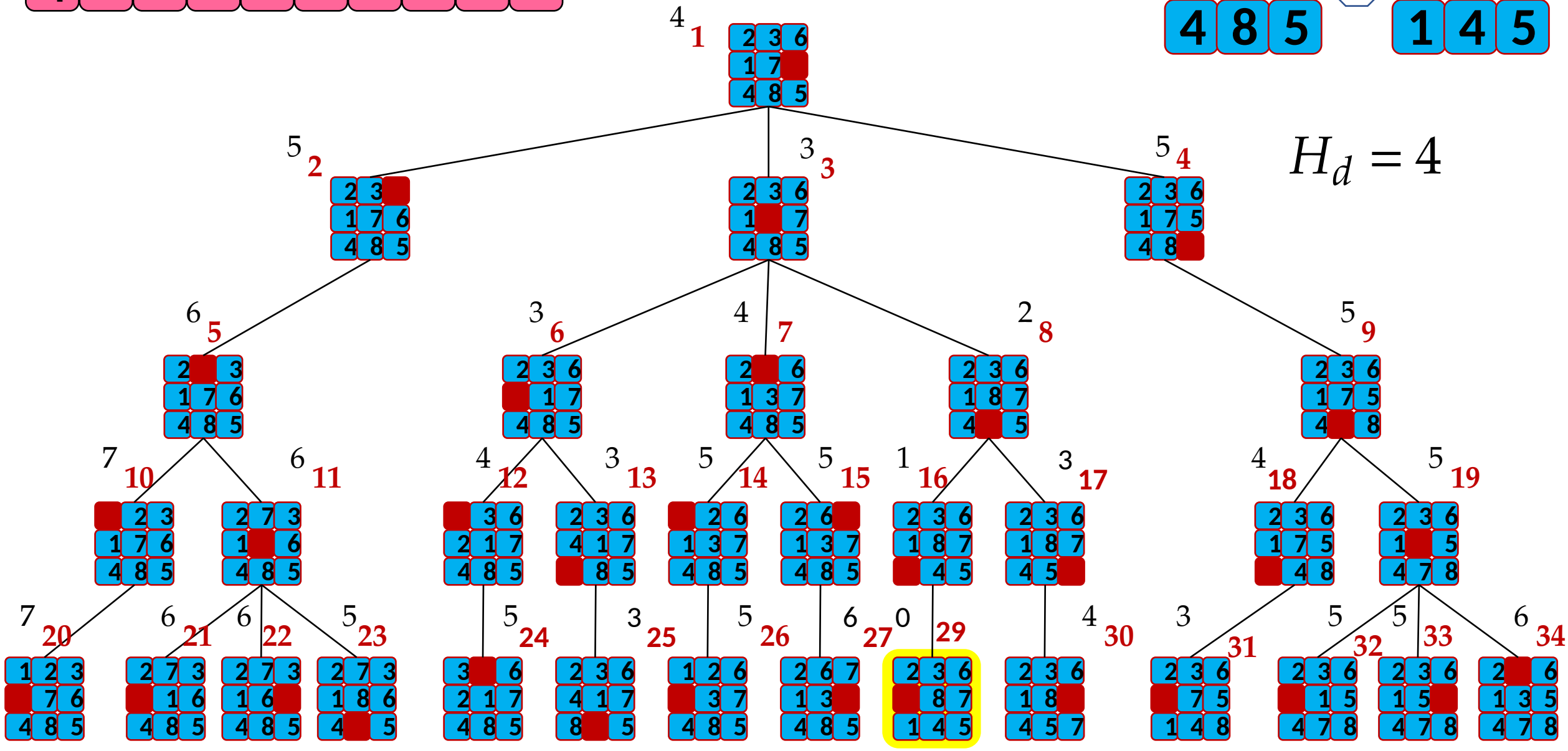
Beam Search

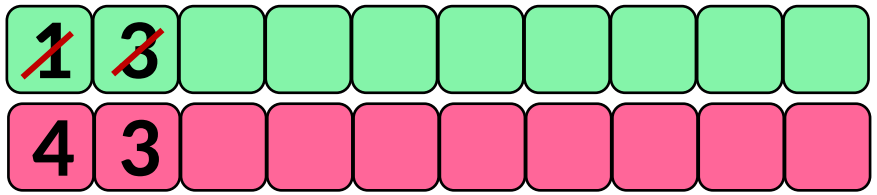


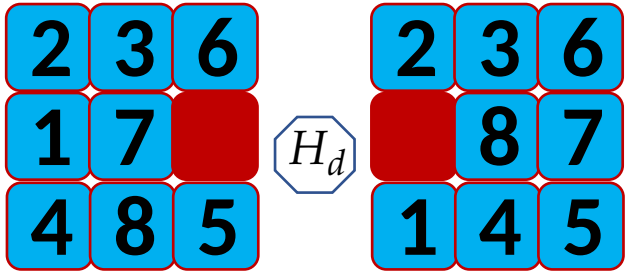

H_d
$H_d = 4$



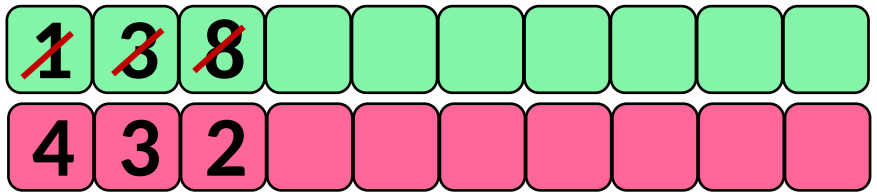

H_d

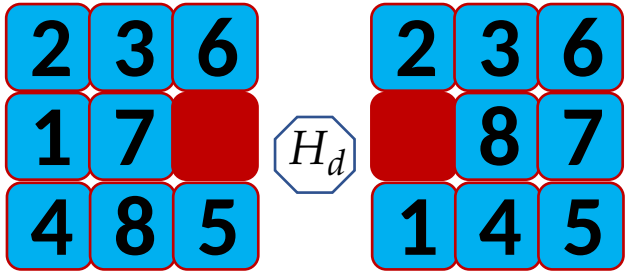

$H_d = 4$



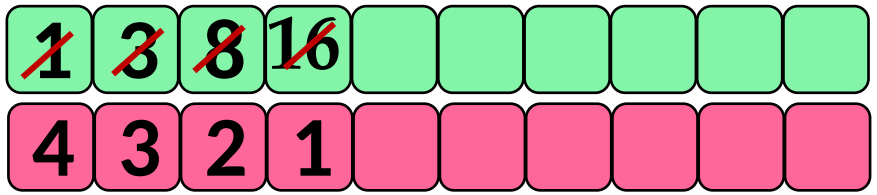


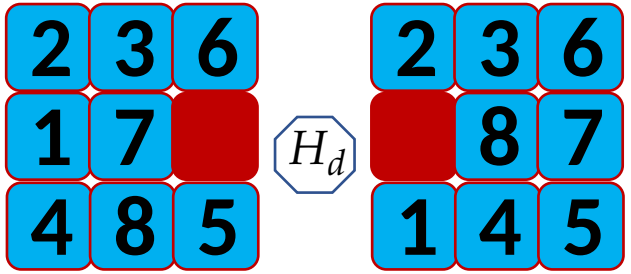
$H_d = 4$



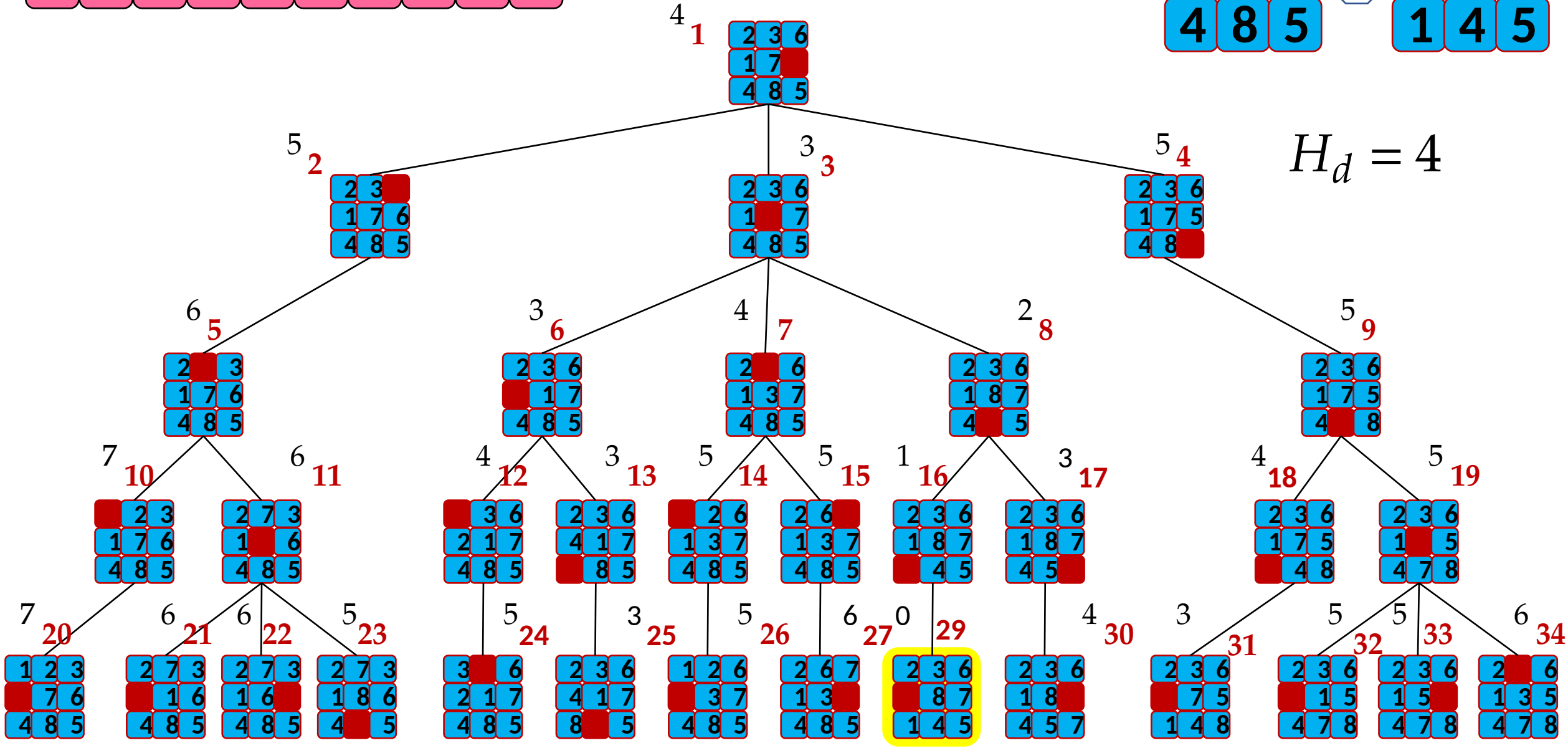


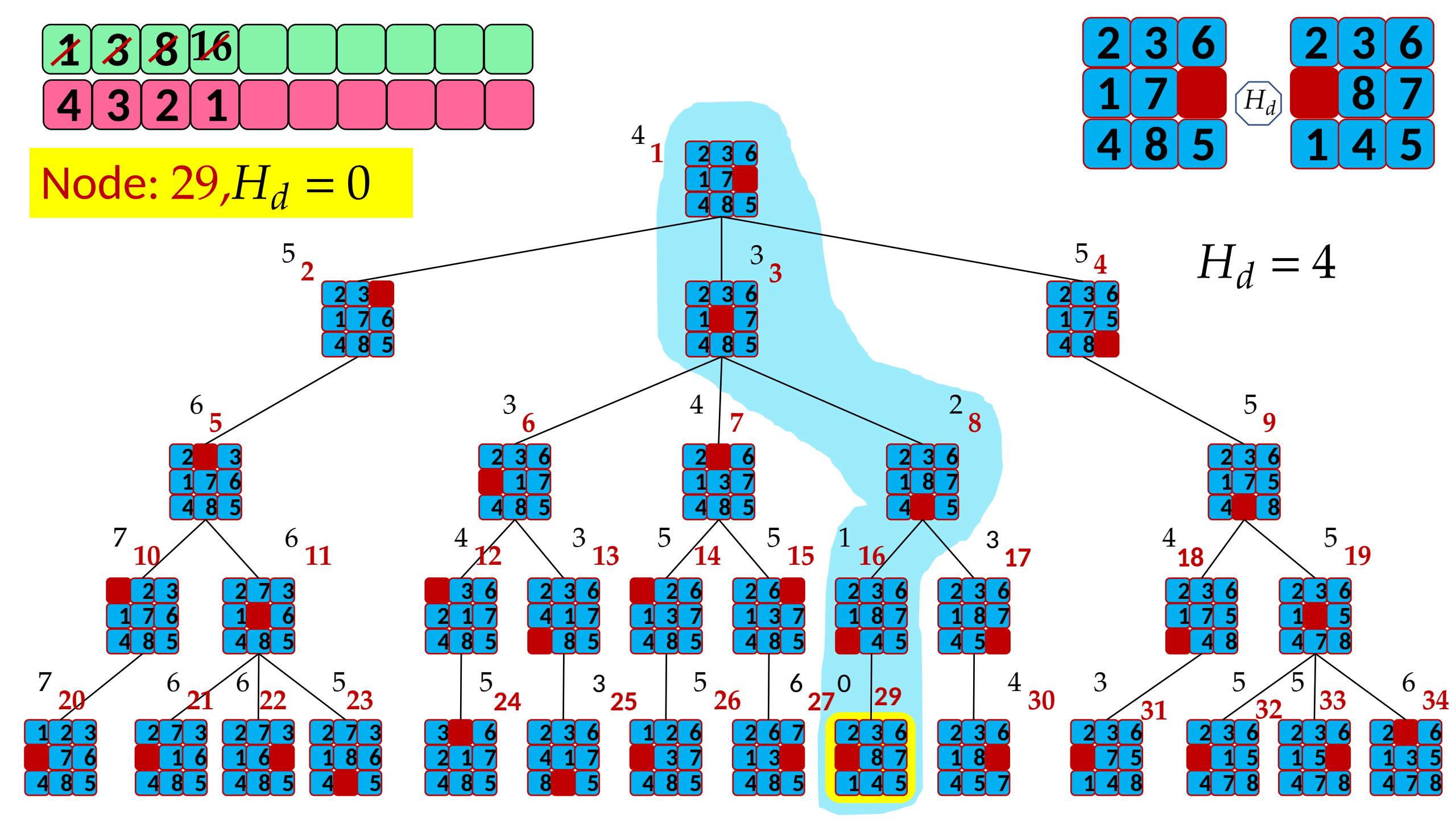
$H_d = 4$


1 3 8 16
4 3 2 1

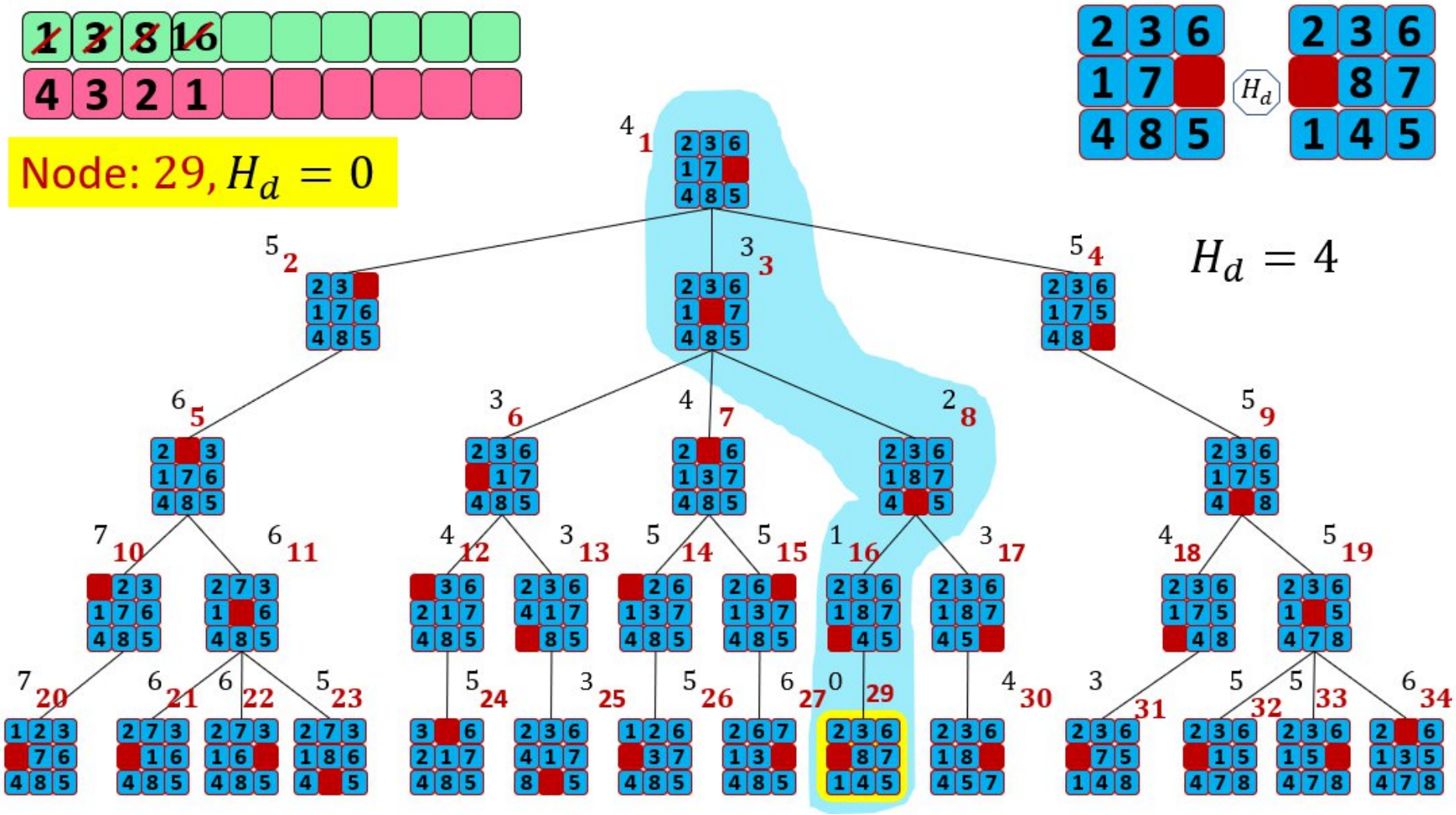

H_d

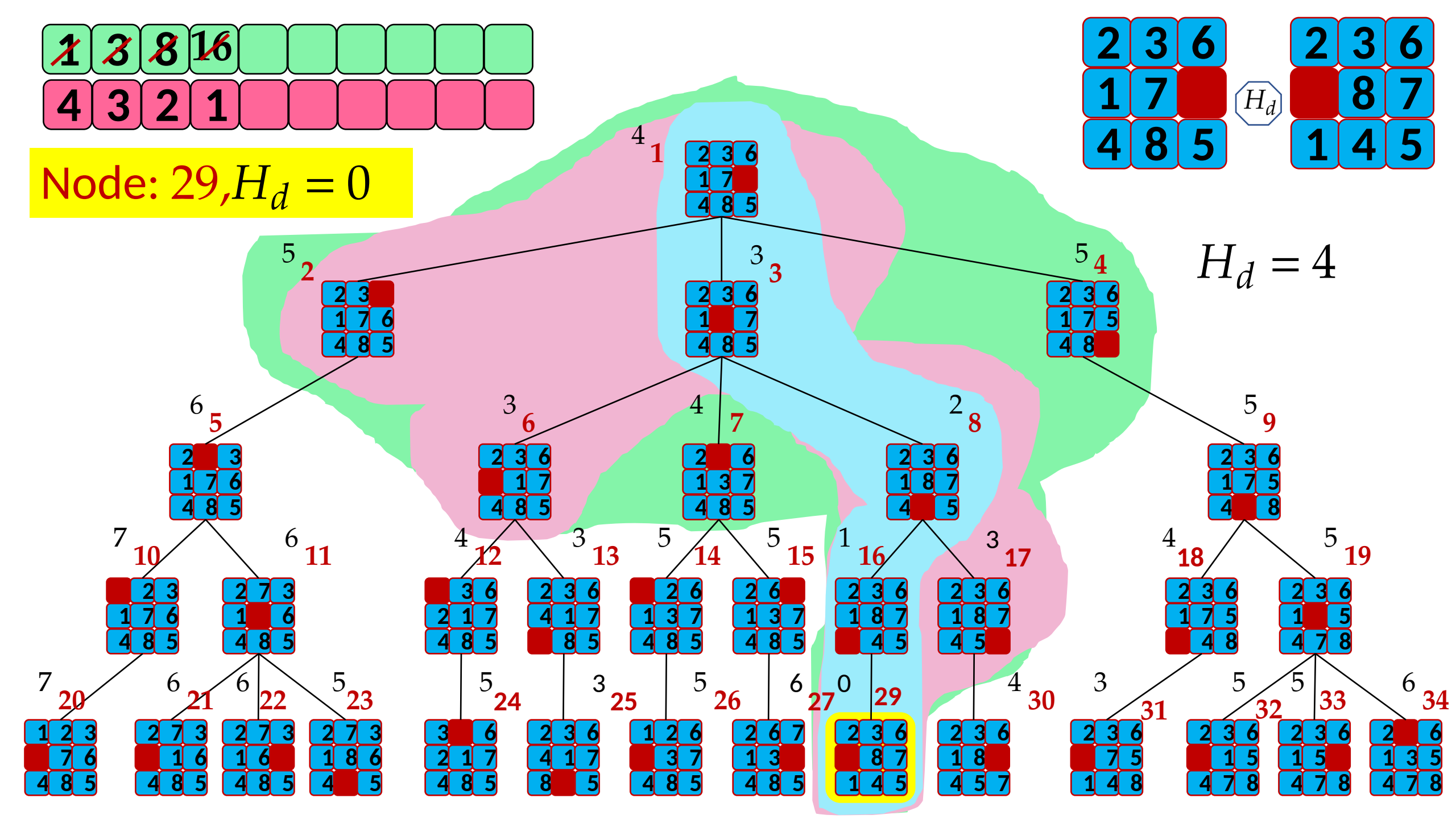
$$H_d = 4$$


4 1
5 2
3 3
5 4
6 5
3 6
4 7
2 8
5 9
7 10
6 11
4 12
3 13
5 14
5 15
1 16
3 17
4 18
5 19
7 20
6 21
6 22
5 23
5 24
3 25
5 26
6 27
0 29
4 30
3 31
5 32
5 33
6 34


1 3 8 16
4 3 2 1
Node: 29, $H_d = 0$
H_d
$H_d = 4$

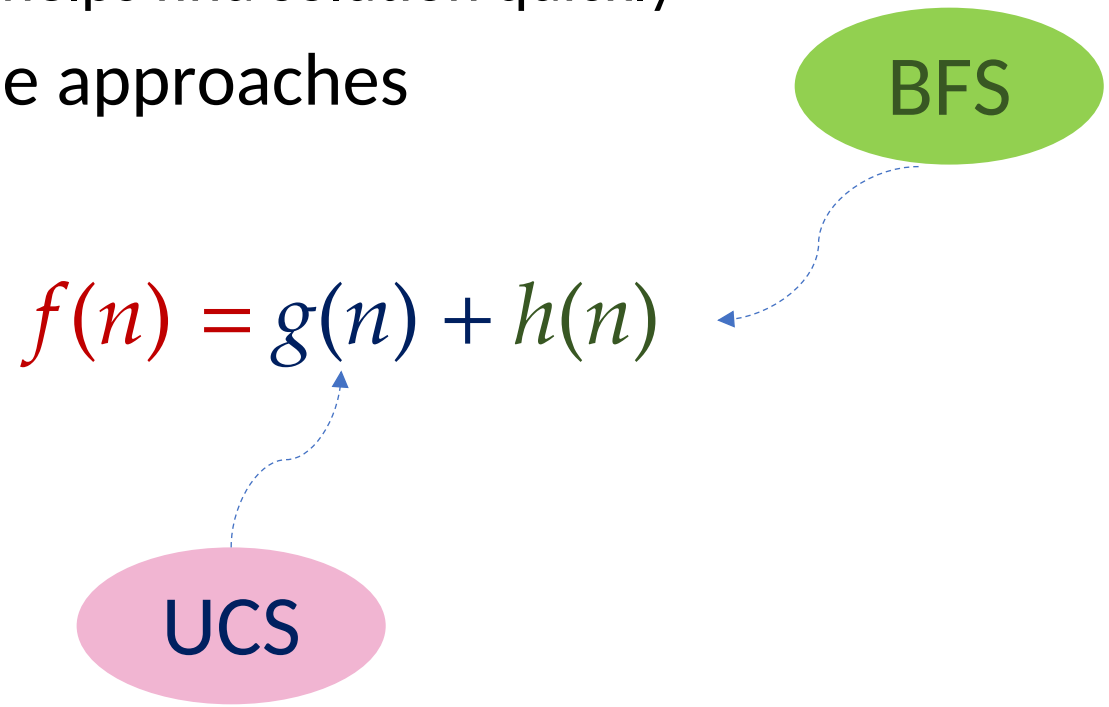
Hill Climbing

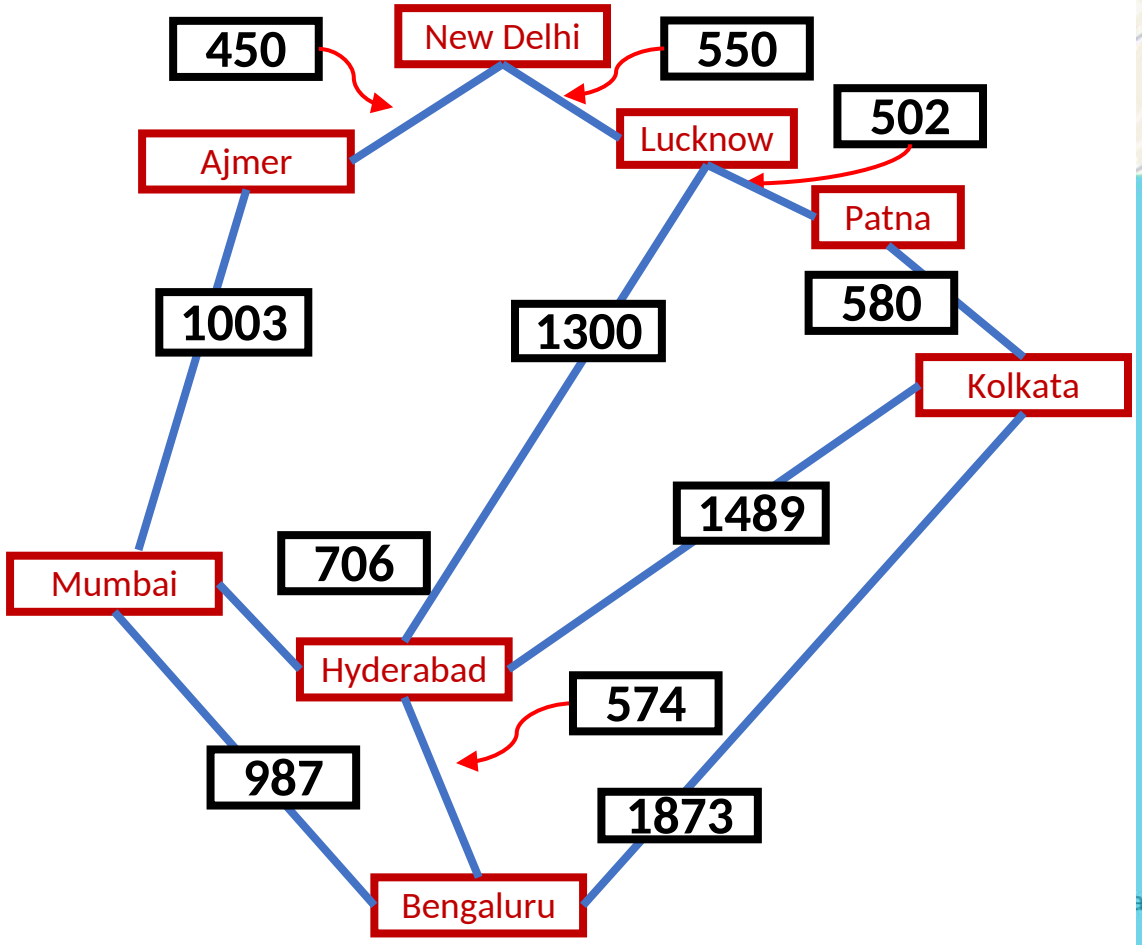


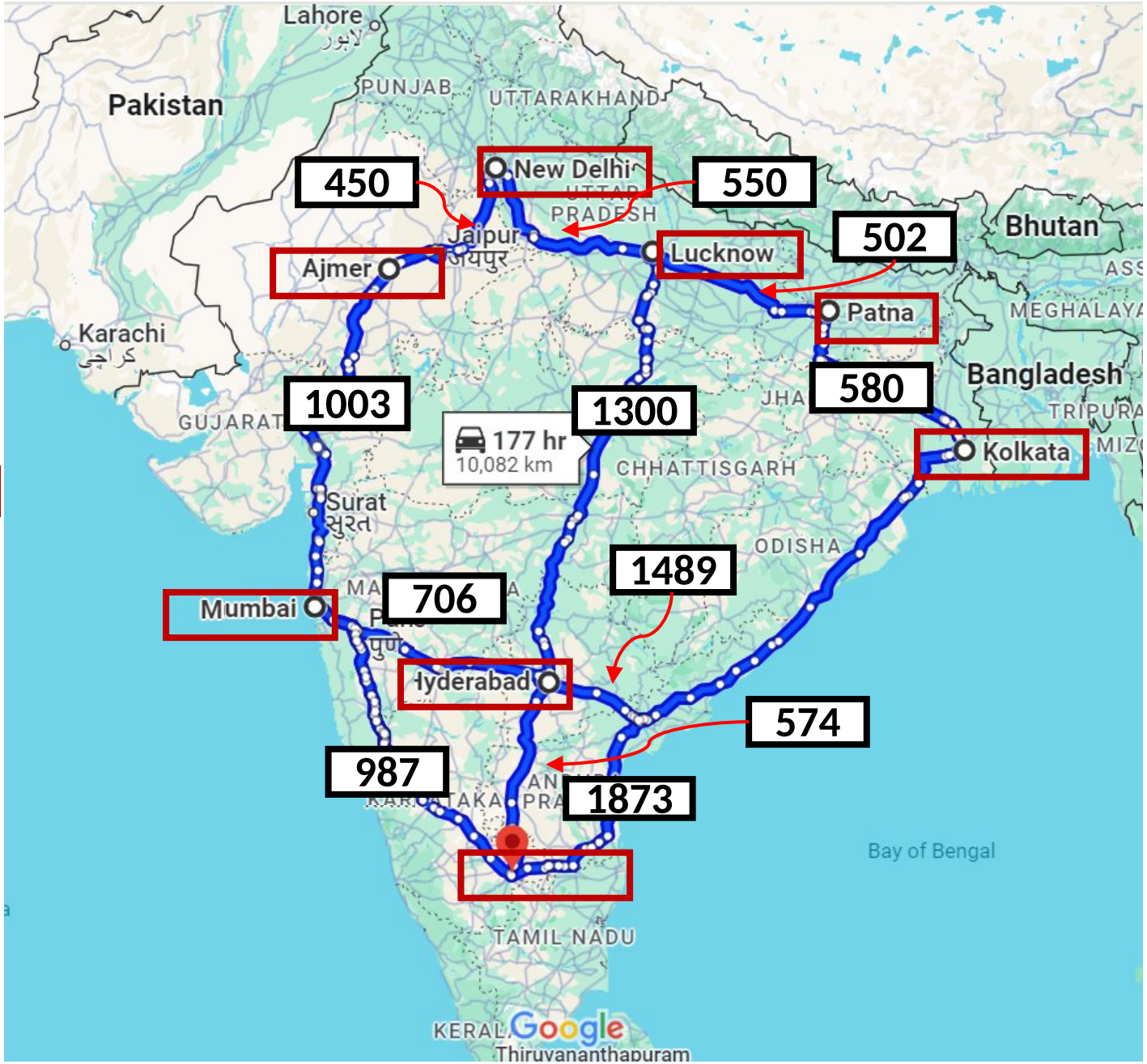

1 3 8 16
4 3 2 1
Node: 29, $H_d = 0$
H_d
$H_d = 4$

A^*

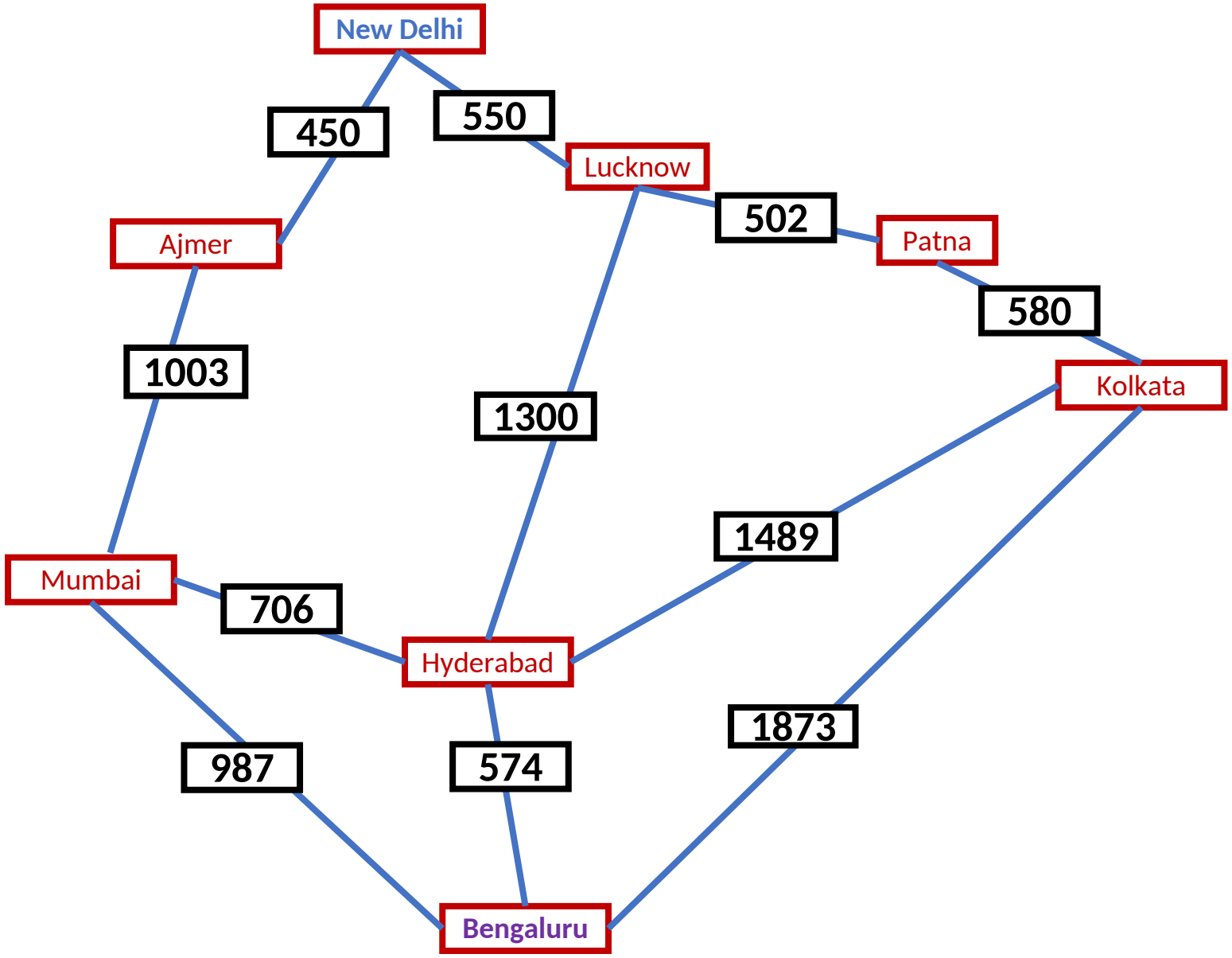
- Note that UCS and Best-first Search both improve search
  - UCS keeps solution cost low
  - Best-first Search helps find solution quickly
- A^* combines these approaches




450
New Delhi
550
Ajmer
Lucknow
502
Patna
1003
1300
580
Kolkata
1489
706
Mumbai
Hyderabad
574
987
1873
Bengaluru

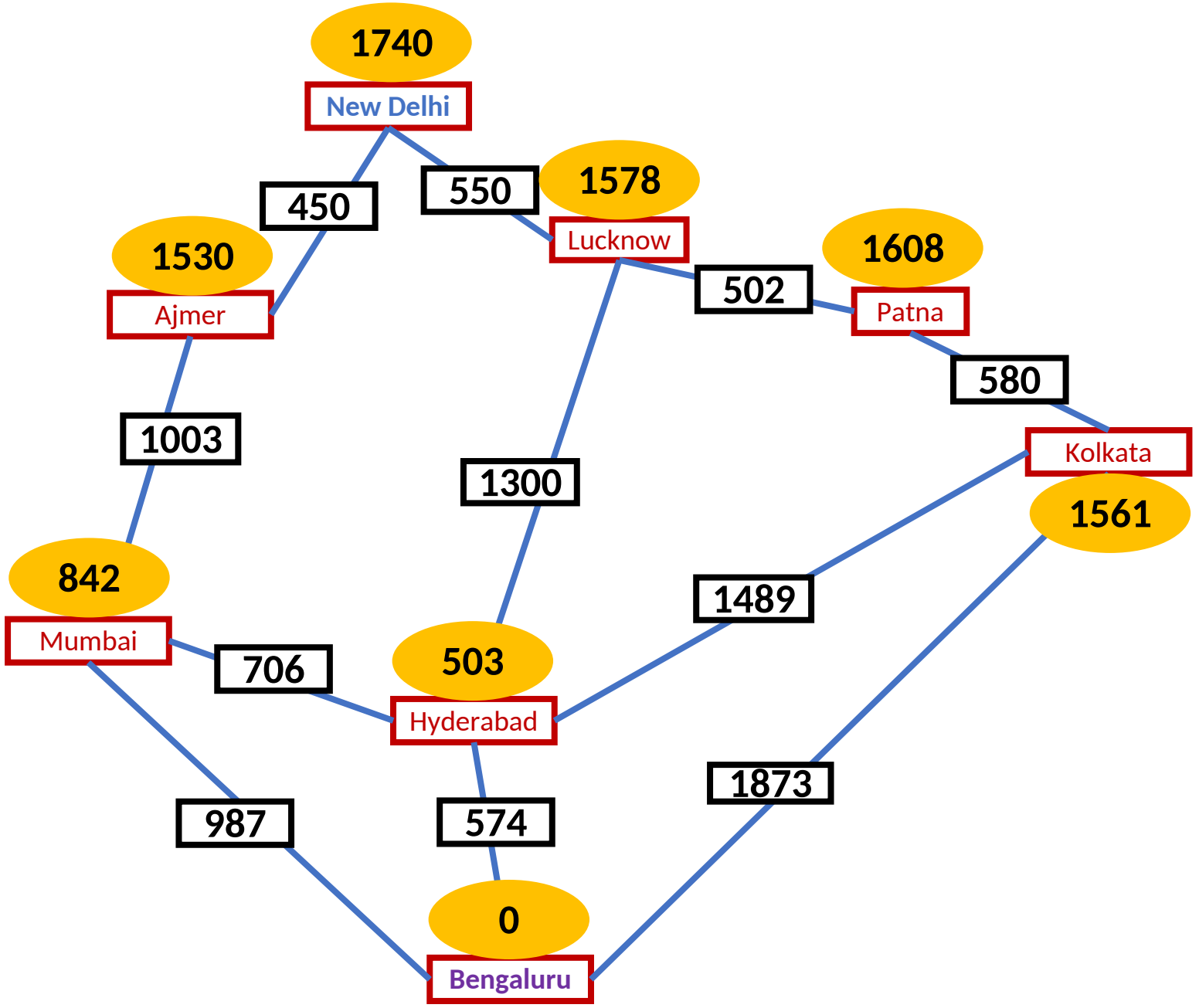

Lahore
Pakistan
PUNJAB
UTTARAKHAND
450
New Delhi
550
Jaipur
Ajmer
Lucknow
502
Bhutan
Patna
Karachi
Bangladesh
580
1003
1300
177 hr
10,082 km
Kolkata
GUJARAT
CHHATTISGARH
Surat
ODISHA
1489
706
Mumbai
Hyderabad
574
987
1873
Bay of Bengal
TAMIL NADU
Google
Thiruvananthapuram

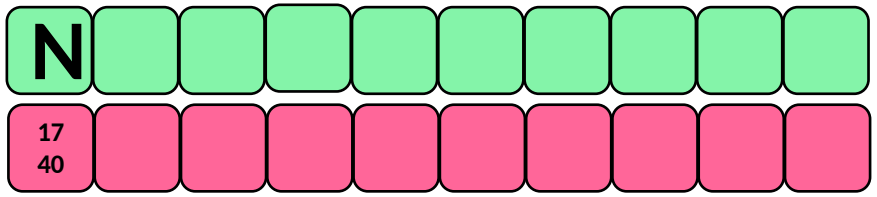
Shortest path from New Delhi to Bengaluru

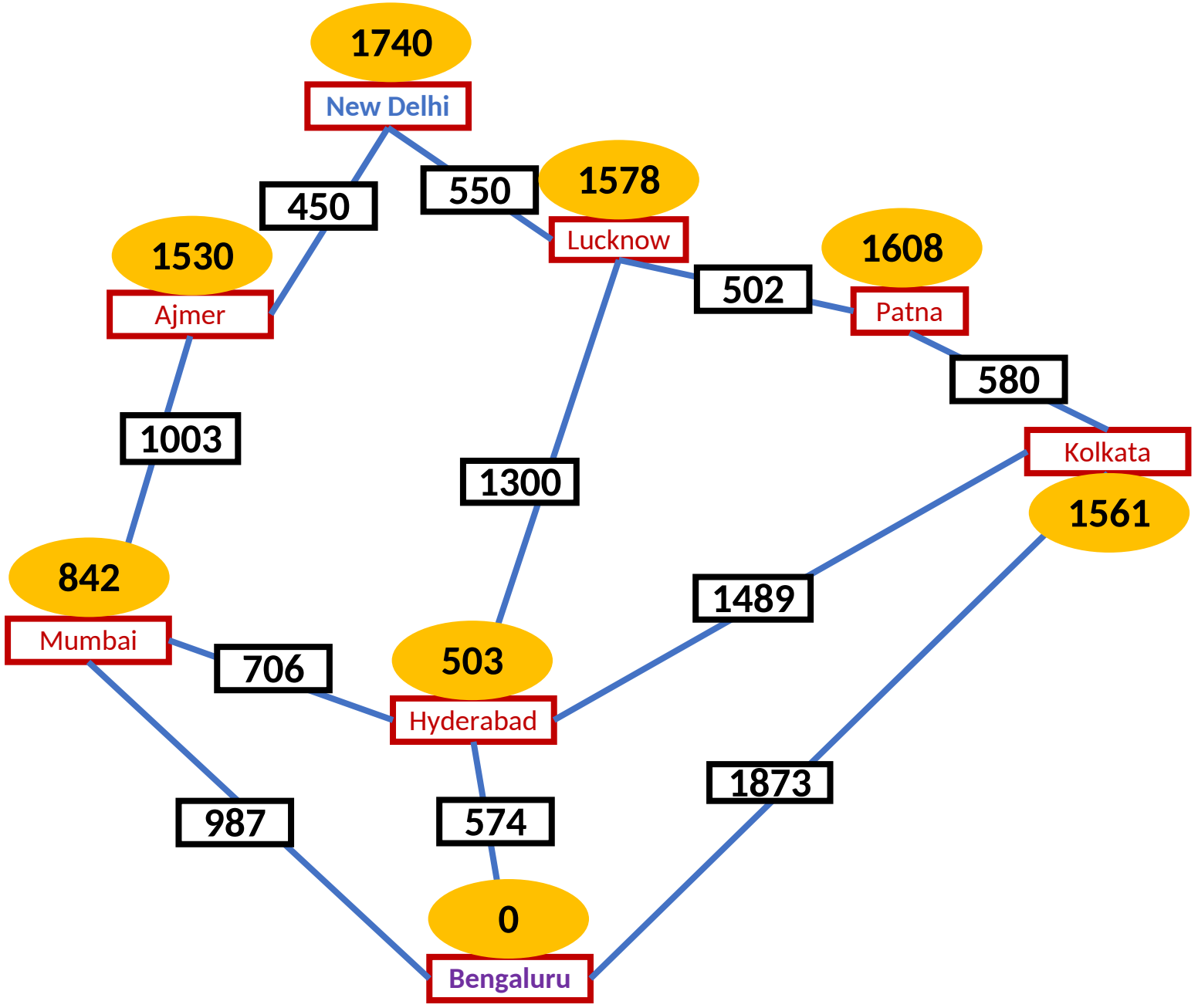


Shortest path from New Delhi to Bengaluru

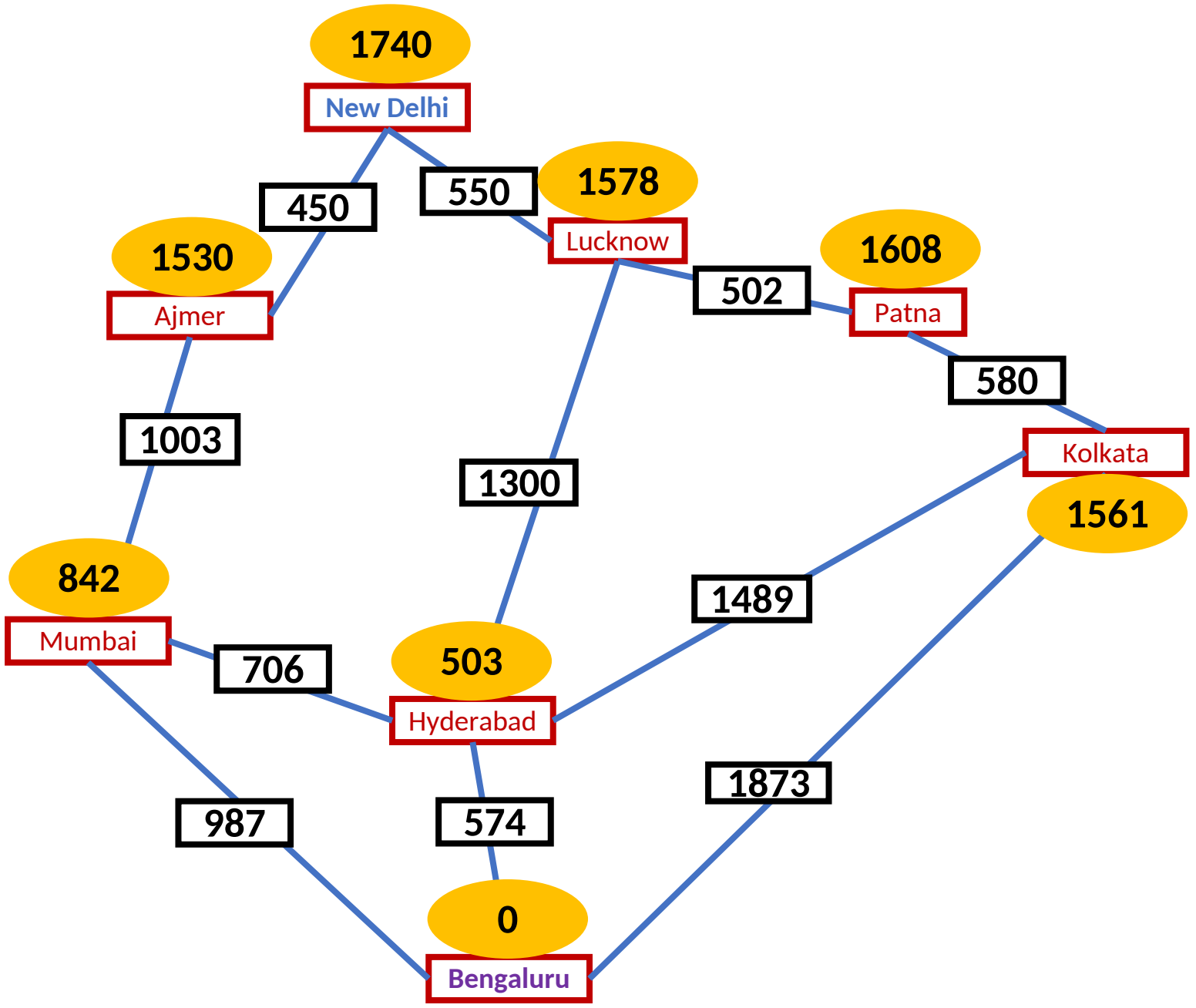
Heuristic: Aerial distance from a node to the destination



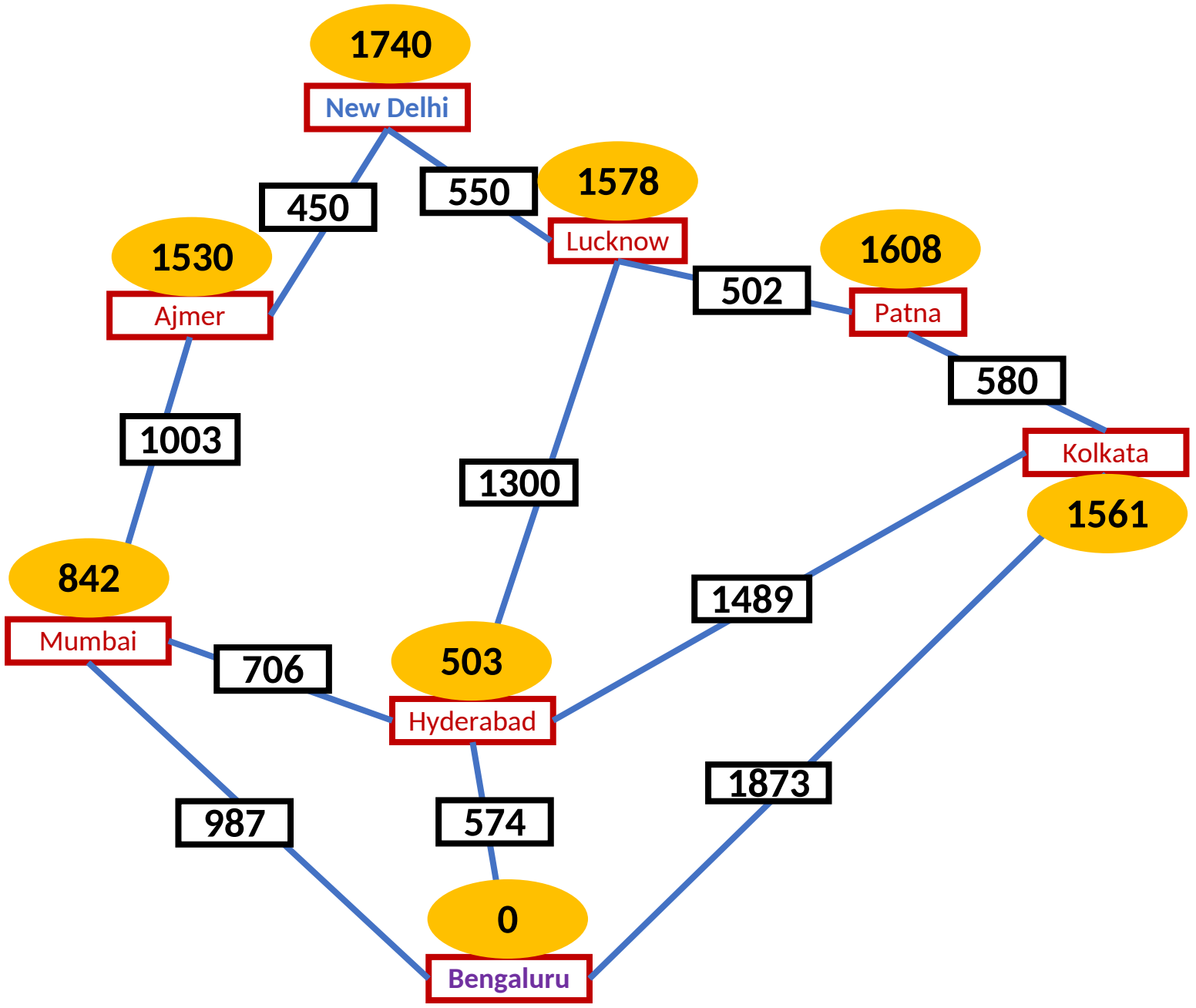

N
17
40


1740
New Delhi
450
550
1578
Lucknow
1530
Ajmer
502
1608
Patna
580
1003
1300
Kolkata
1561
842
Mumbai
706
503
Hyderabad
1489
987
574
1873
0
Bengaluru

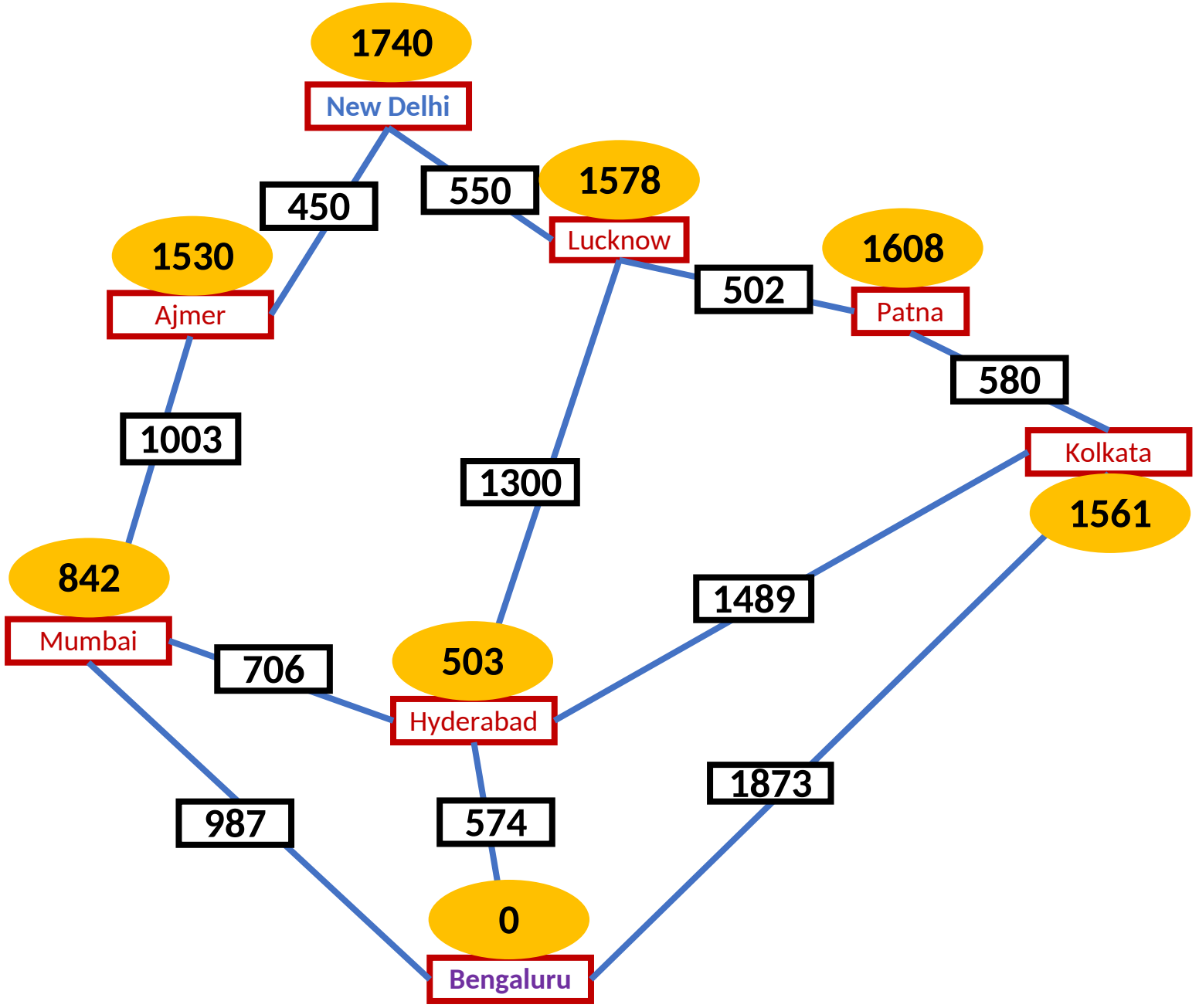
N	A	L							
17 40	15 30	15 78							



N	A	L	M						
17 40	15 30	15 78	84 2						



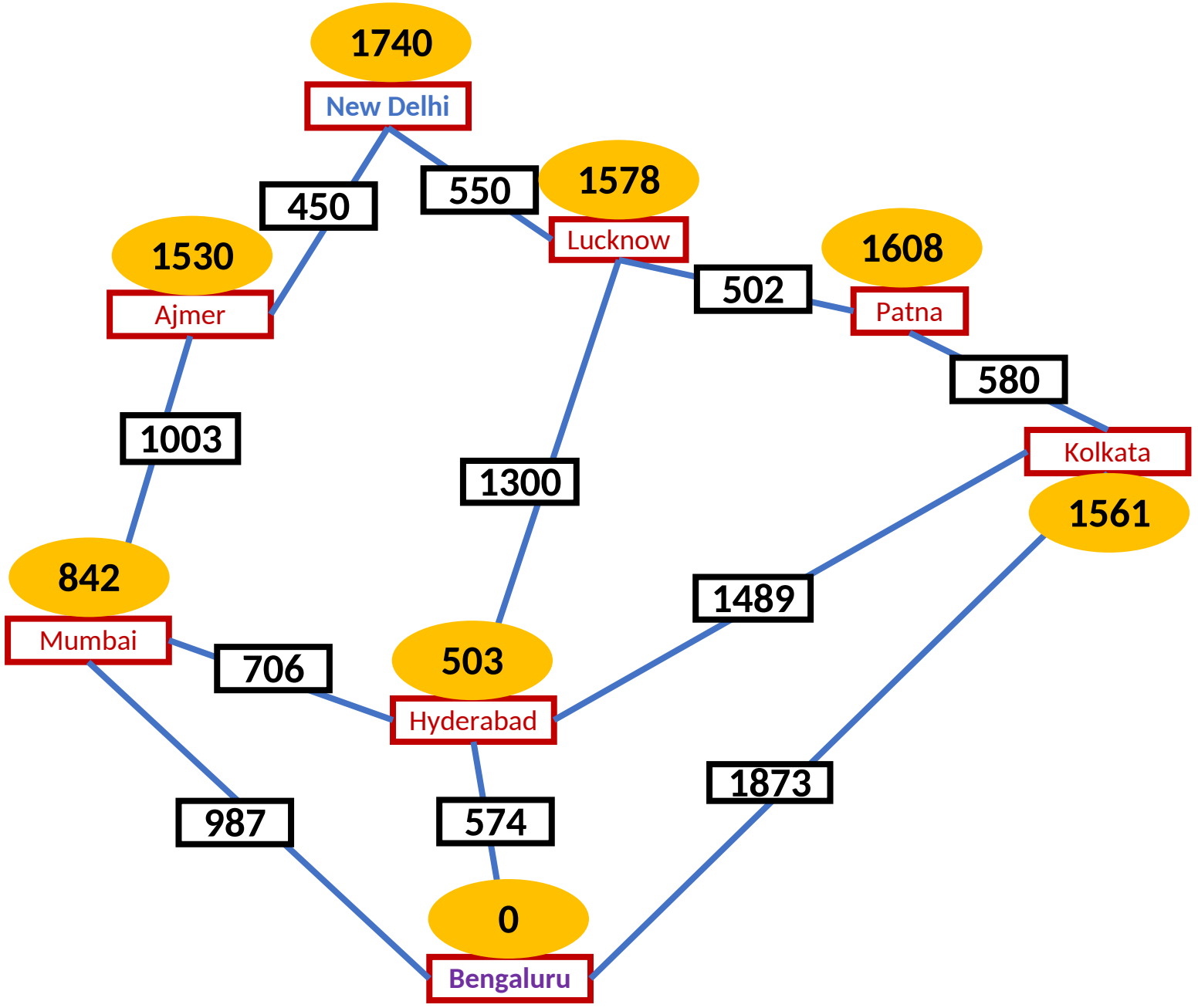
N	A	M	L						
17 40	15 30	84 2	15 78						



~~N~~	~~A~~	M	L						
17 40	15 30	84 2	15 78						

Node: B, $H = 0$

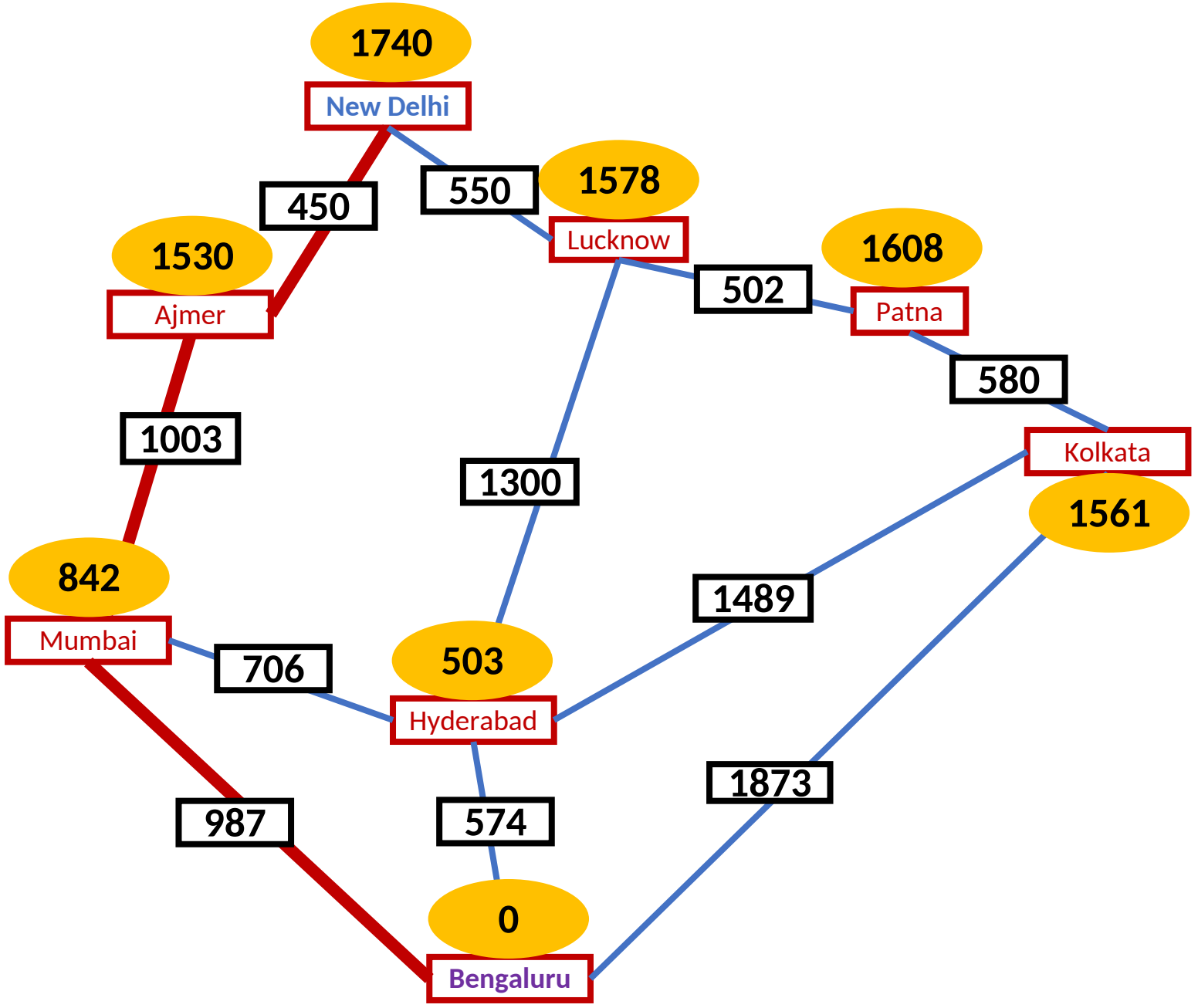
N → A → M → B (2440)



~~N~~	~~A~~	M	L						
17 40	15 30	84 2	15 78						

Node: B, $H = 0$

N → A → M → B (2440)

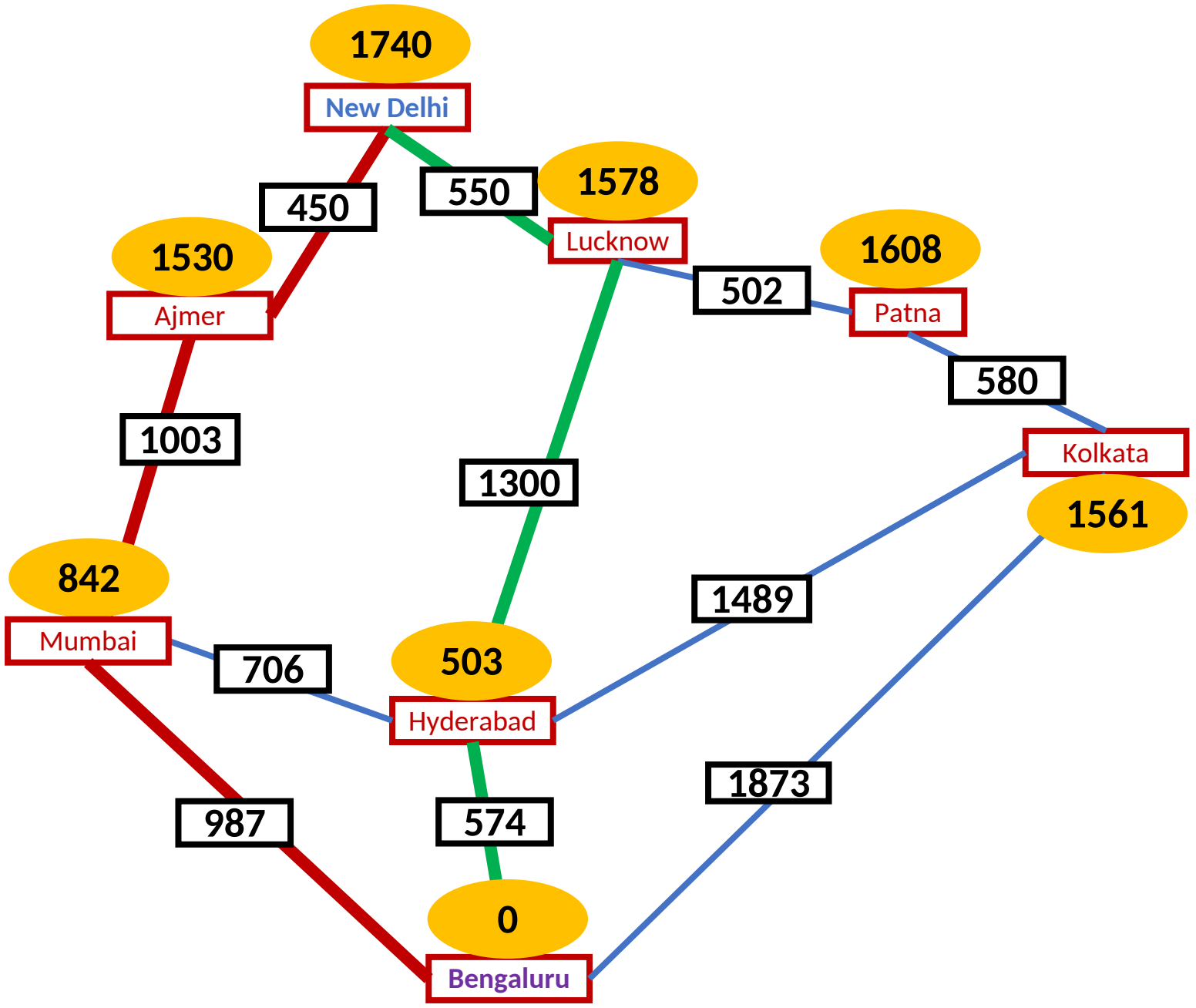


~~N~~	~~A~~	M	L						
17 40	15 30	84 2	15 78						

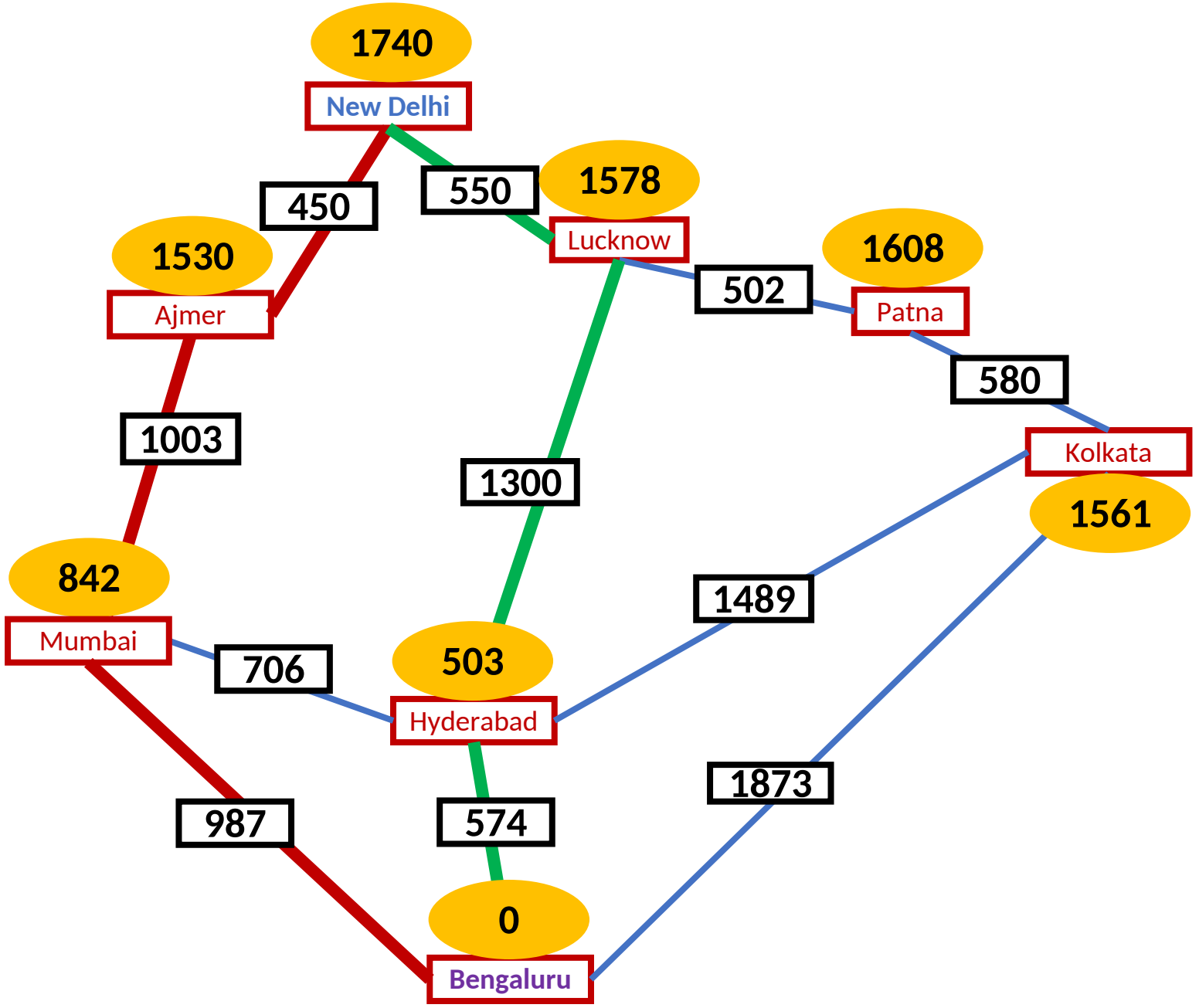
Node: B, $H = 0$

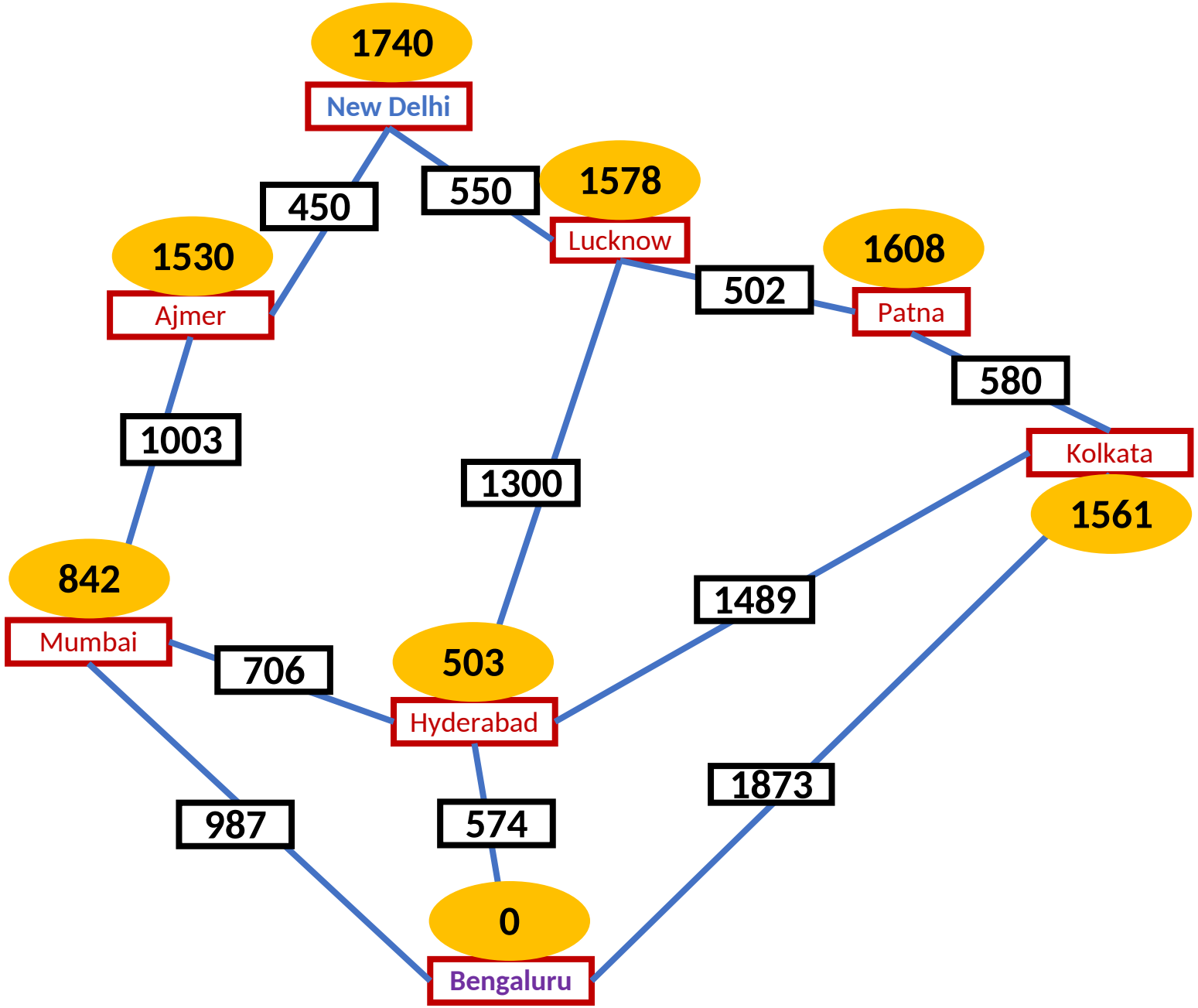
N → A → M → B (2440)

N → L → H → B (2424)



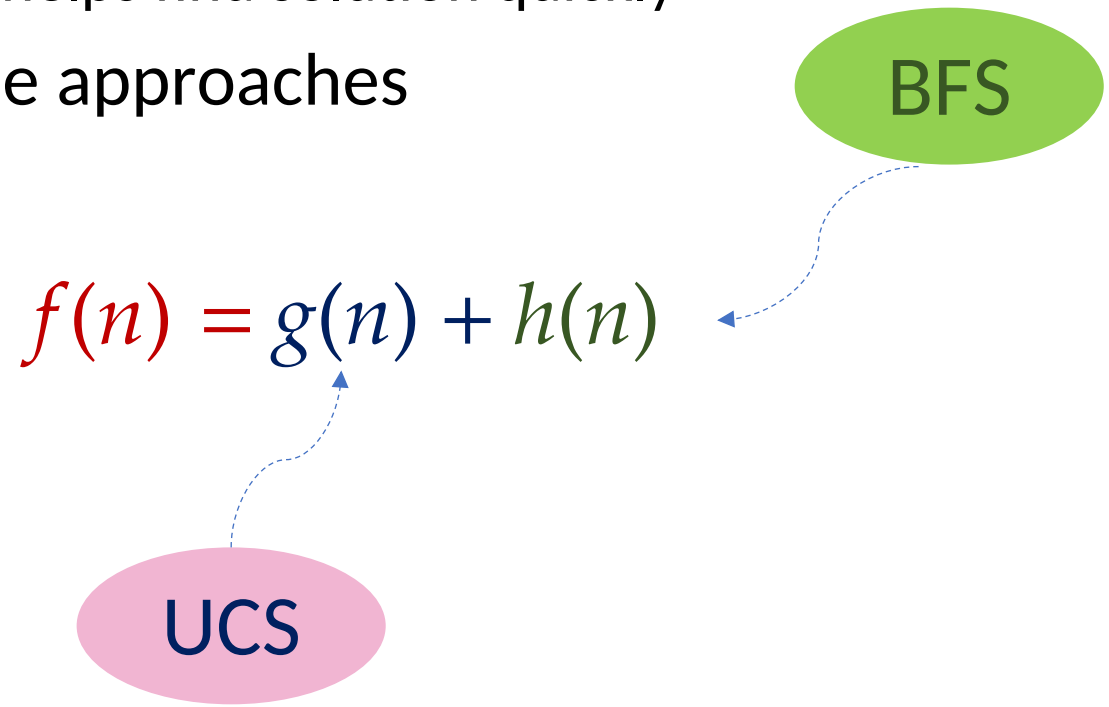
Any better way to find shortest path?

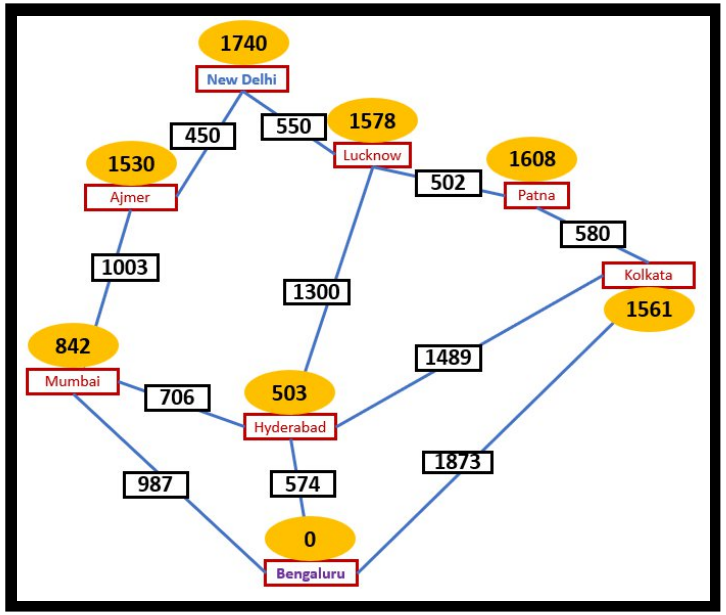



1740
New Delhi
450
550
1578
Lucknow
1530
Ajmer
502
1608
Patna
580
1003
1300
Kolkata
1561
842
Mumbai
706
503
Hyderabad
1489
987
574
1873
0
Bengaluru

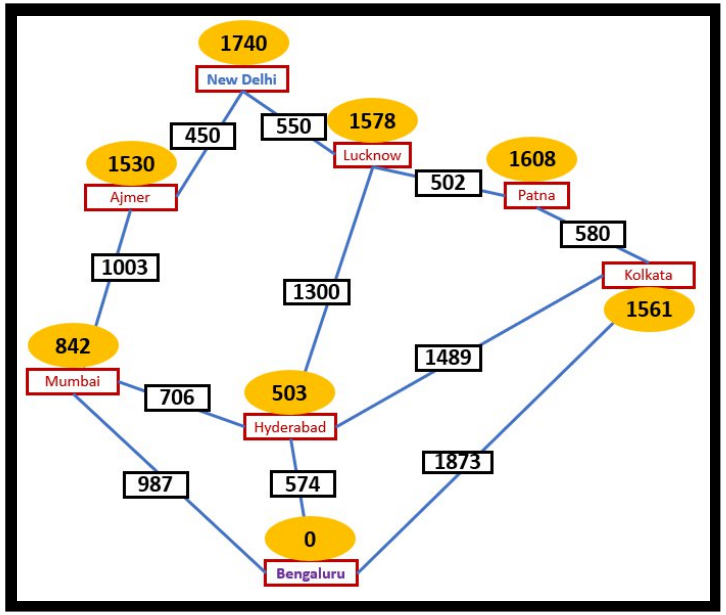
A^*

- Note that UCS and Best-first Search both improve search
  - UCS keeps solution cost low
  - Best-first Search helps find solution quickly
- A^* combines these approaches




1740
New Delhi
450
550
1578
Lucknow
1530
Ajmer
502
1608
Patna
580
1003
1300
Kolkata
1561
842
Mumbai
706
503
Hyderabad
1489
1873
987
574
0
Bengaluru

New Delhi
450
550
550+1578=2128
450+1530=1980
Lucknow
Ajmer
502
Patna
580
1003
1300
Kolkata
1489
Mumbai
706
Hyderabad
1873
987
574
Bengaluru



450+1530=1980

550+1578=2128

New Delhi

450

550

Ajmer

Lucknow

502

Patna

580

Kolkata

1003

1300

1561

842

Mumbai

706

503

Hyderabad

1489

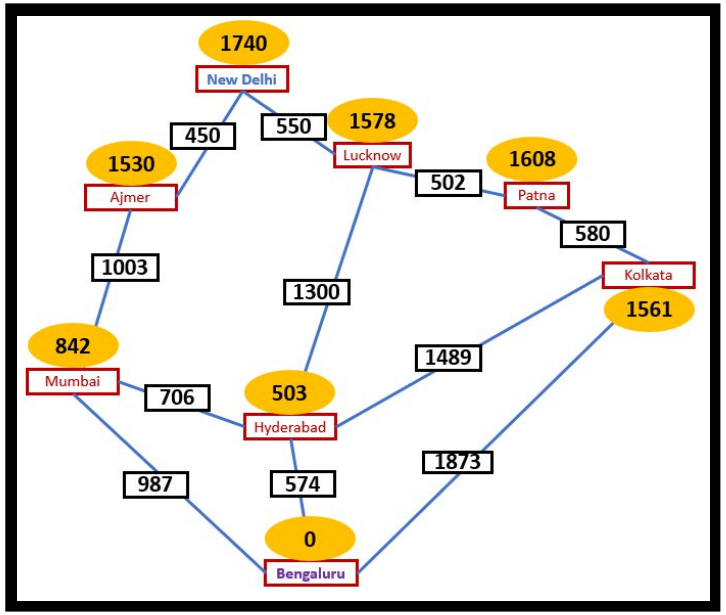
1873

987

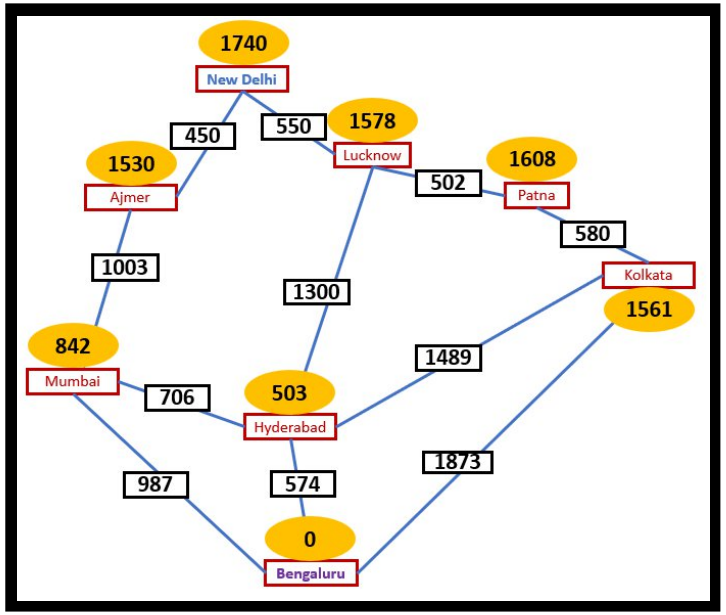
574

0

Bengaluru


1740
New Delhi
450
550
1578
Lucknow
1530
Ajmer
502
1608
Patna
580
1003
1300
Kolkata
1561
842
Mumbai
706
503
Hyderabad
1489
987
574
1873
0
Bengaluru

New Delhi
450
550
550+1578=2128
450+1530=1980
Lucknow
Ajmer
502
Patna
580
1003
1300
Kolkata
1453+842=2295
1489
Mumbai
706
Hyderabad
1873
987
574
Bengaluru



New Delhi

450

550

550+1578=2128

450+1530=1980

Lucknow

Ajmer

502

Patna

580

1003

1300

Kolkata

1453+842=2295

1489

Mumbai

706

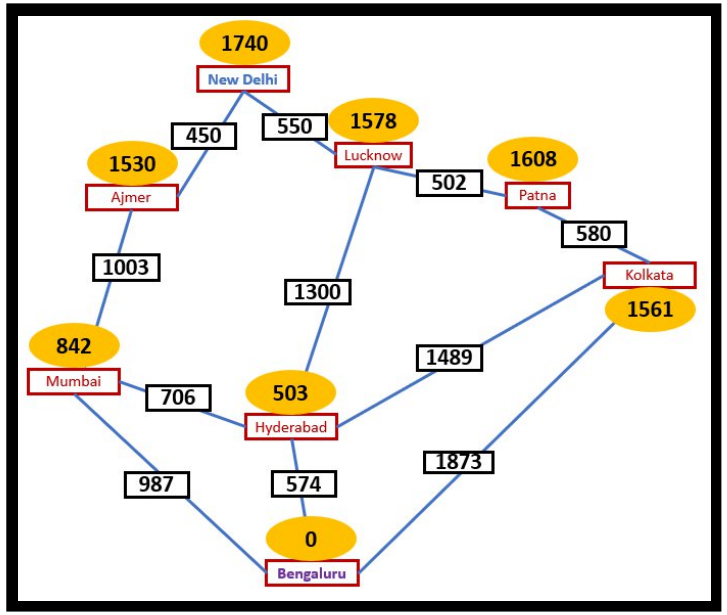
Hyderabad

1873

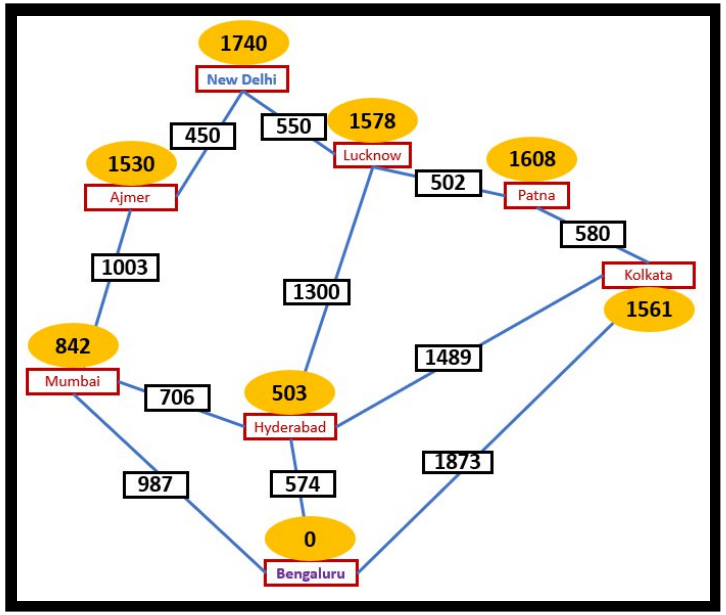
987

574

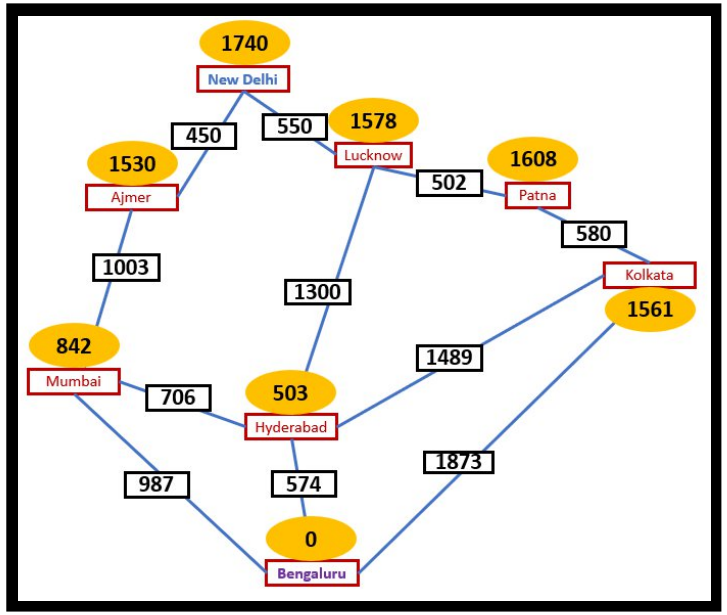
Bengaluru


1740
New Delhi
450
550
1578
Lucknow
1530
Ajmer
502
1608
Patna
580
1003
1300
Kolkata
1561
842
Mumbai
706
503
Hyderabad
1489
1873
987
574
0
Bengaluru

New Delhi
450
550
550+1578=2128
450+1530=1980
Lucknow
1052+1608=2660
Ajmer
502
Patna
580
1003
1300
Kolkata
1453+842=2295
1489
Mumbai
706
1850+503=2353
Hyderabad
1873
987
574
Bengaluru


1740
New Delhi
450
550
1578
Lucknow
1530
Ajmer
502
1608
Patna
580
1003
1300
Kolkata
1561
842
Mumbai
706
503
Hyderabad
1489
1873
987
574
0
Bengaluru

New Delhi
450
550
550+1578=2128
450+1530=1980
Lucknow
1052+1608=2660
Ajmer
502
Patna
580
1003
1300
Kolkata
1453+842=2295
1489
Mumbai
706
1850+503=2353
Hyderabad
1873
987
574
Bengaluru



New Delhi

450

550

550+1578=2128

450+1530=1980

Ajmer

Lucknow

1052+1608=2660

502

Patna

580

1003

1300

Kolkata

1453+842=2295

1489

Mumbai

706

1850+503=2353

Hyderabad

2159+503=2652

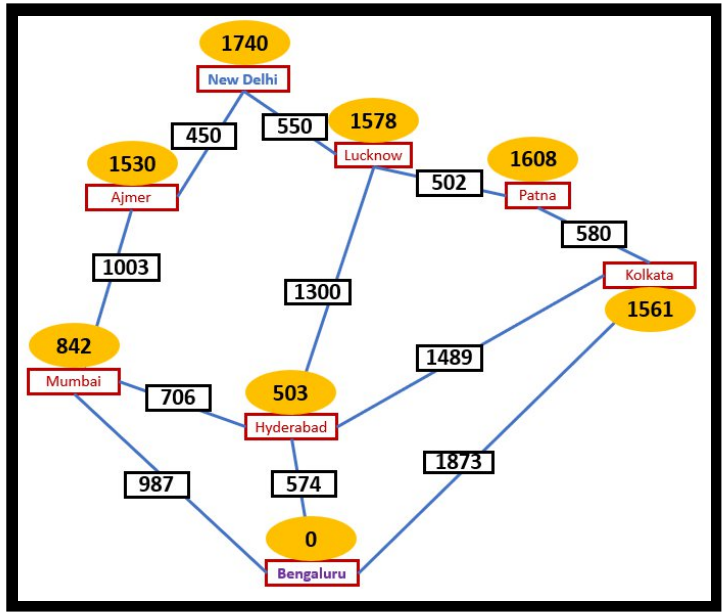
987

574

1873

2440+0=2440

Bengaluru



New Delhi

450

550

550+1578=2128

450+1530=1980

Lucknow

1052+1608=2660

Ajmer

502

Patna

580

1003

1300

Kolkata

1453+842=2295

1489

Mumbai

706

1850+503=2353

Hyderabad

2159+503=2652

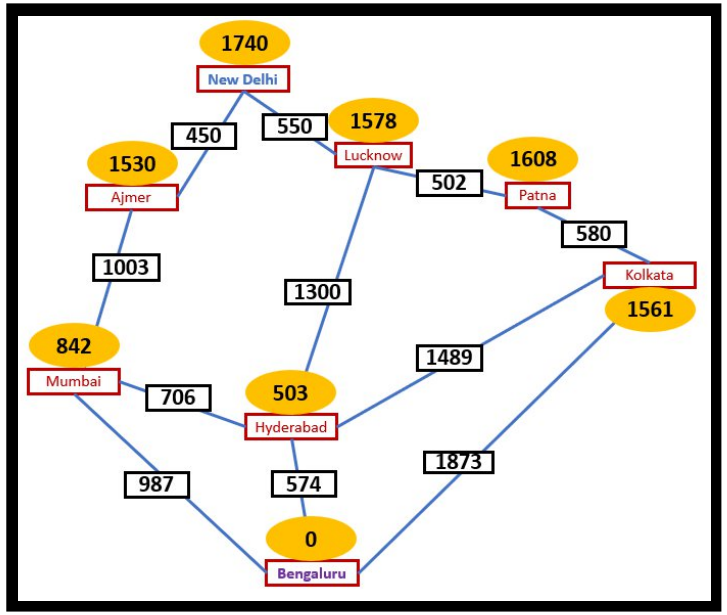
987

574

1873

2440+0=2440

Bengaluru



New Delhi

450

550

550+1578=2128

450+1530=1980

Ajmer

Lucknow

1052+1608=2660

502

Patna

580

1003

Kolkata

1300

3339+1561=4900

1453+842=2295

1489

Mumbai

706

1850+503=2353

Hyderabad

2159+503=2652

987

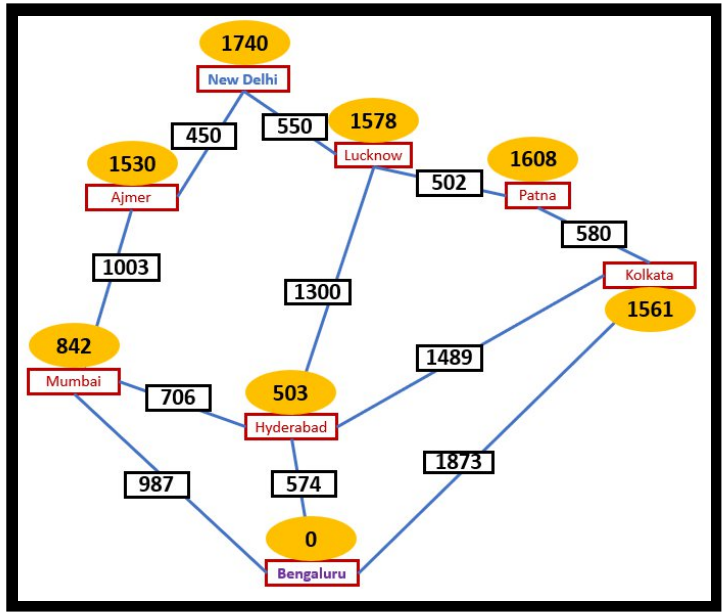
574

1873

2440+0=2440

Bengaluru

2424+0=2424



New Delhi

450

550

550+1578=2128

450+1530=1980

Ajmer

Lucknow

502

1052+1608=2660

Patna

580

1003

1300

Kolkata

3339+1561=4900

1453+842=2295

Mumbai

706

1850+503=2353

1489

Hyderabad

2159+503=2652

987

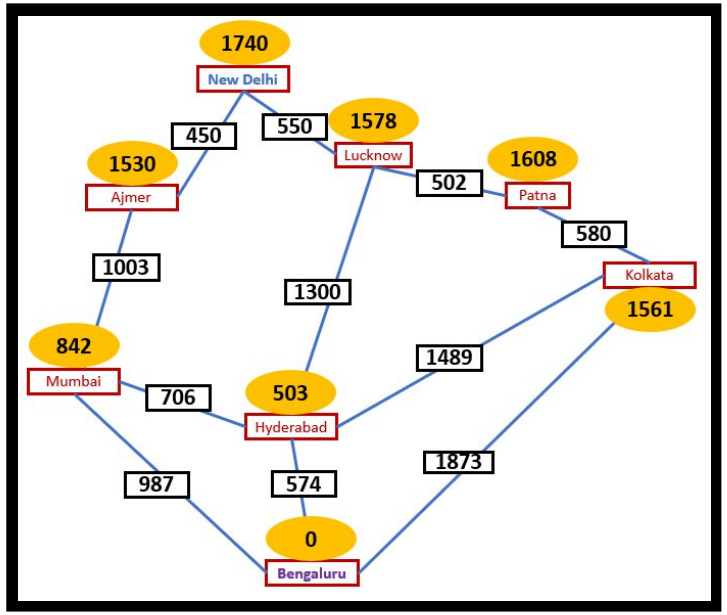
574

1873

2440+0=2440

Bengaluru

2424+0=2424



New Delhi

450

550

550+1578=2128

450+1530=1980

Ajmer

Lucknow

502

1052+1608=2660

Patna

580

1003

1300

Kolkata

3339+1561=4900

1453+842=2295

Mumbai

706

1850+503=2353

1489

Hyderabad

2159+503=2652

987

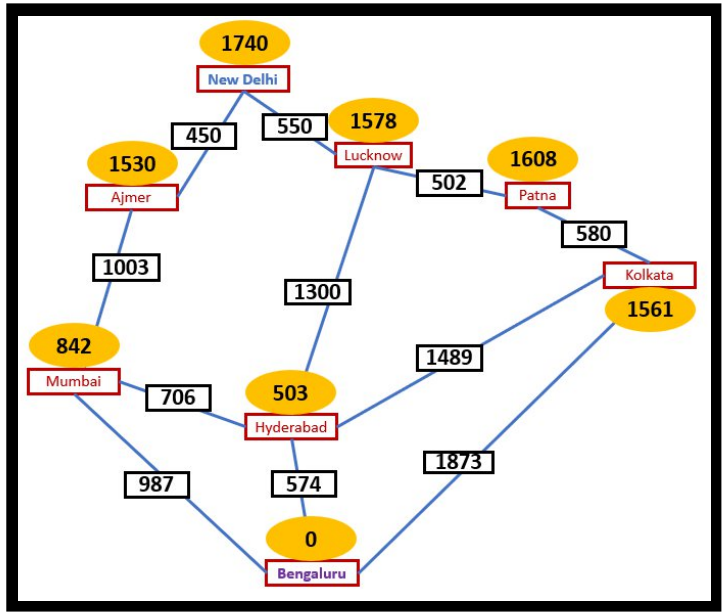
574

1873

2440+0=2440

Bengaluru

2424+0=2424



New Delhi

450

550

550+1578=2128

450+1530=1980

Ajmer

Lucknow

502

1052+1608=2660

Patna

580

1003

1300

Kolkata

3339+1561=4900

1453+842=2295

Mumbai

706

1850+503=2353

1489

Hyderabad

2159+503=2652

987

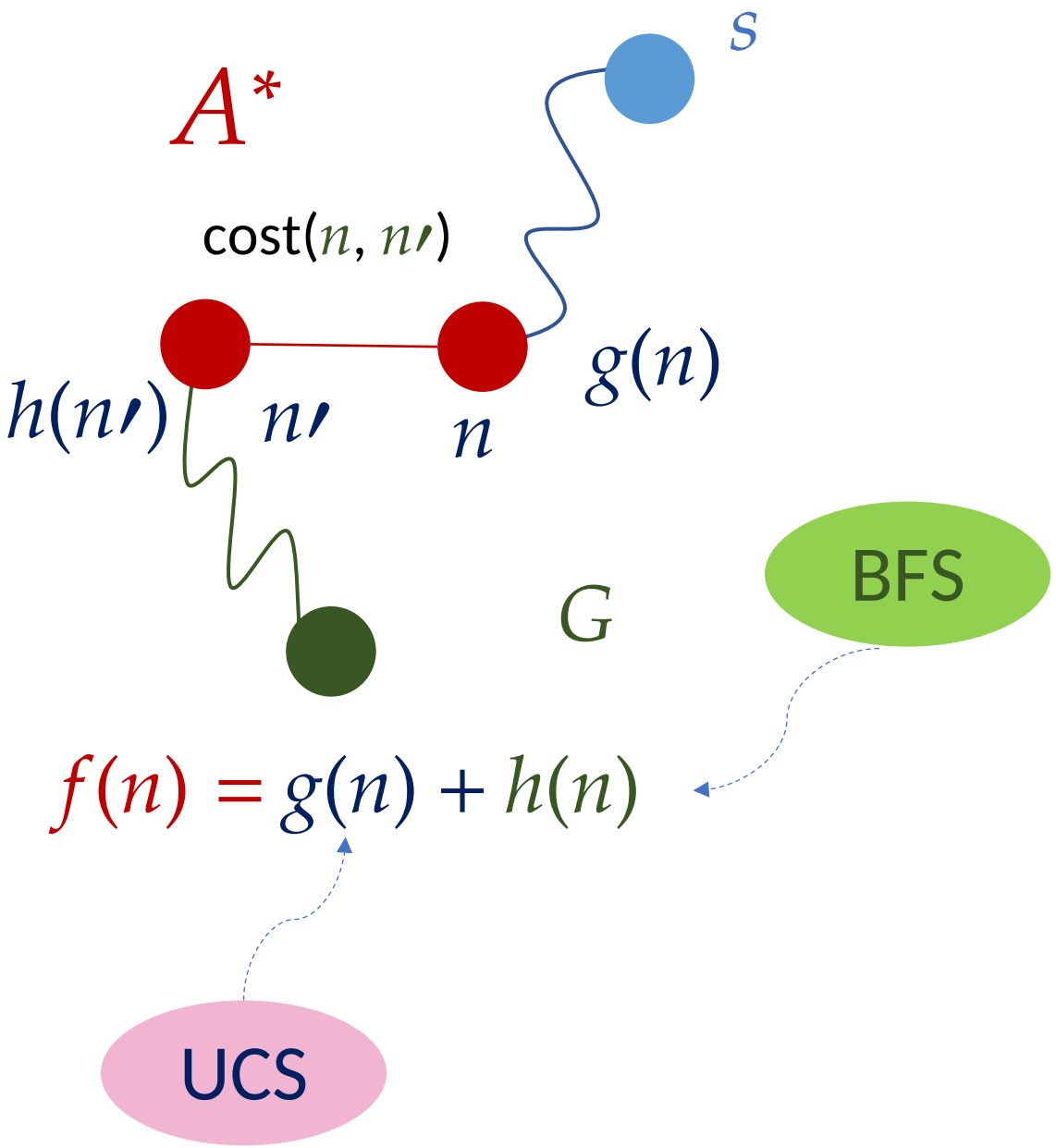
574

1873

2440+0=2440

Bengaluru

2424+0=2424



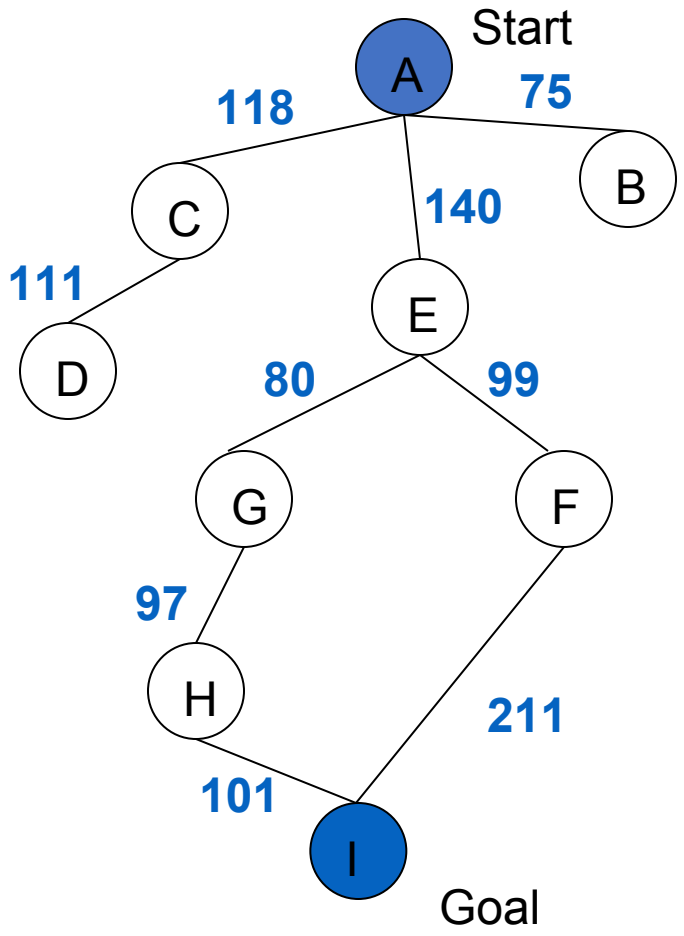
Search queue Q is empty.

2. Place the start state s in Q with f value $h(s)$.
3. If Q is empty, return failure.
4. Take node n from Q with lowest f value.
   (Keep Q sorted by f values and pick the first element).
5. If n is a goal node, stop and return solution.
6. Generate successors of node n.
7. For each successor n' of n do:
   a) Compute $f(n') = g(n) +$ cost$(n, n') + h(n')$.
   b) If n' is new (never generated before), add n' to Q.
   c) If node n' is already in Q with a higher f value, replace it with current $f(n')$ and place it in sorted order in Q.
   End for
8. Go back to step 3.

A^* with $f(n)$ not Admissible

$h(n)$ overestimates the cost to reach the goal state

A^* Search: h not admissible !



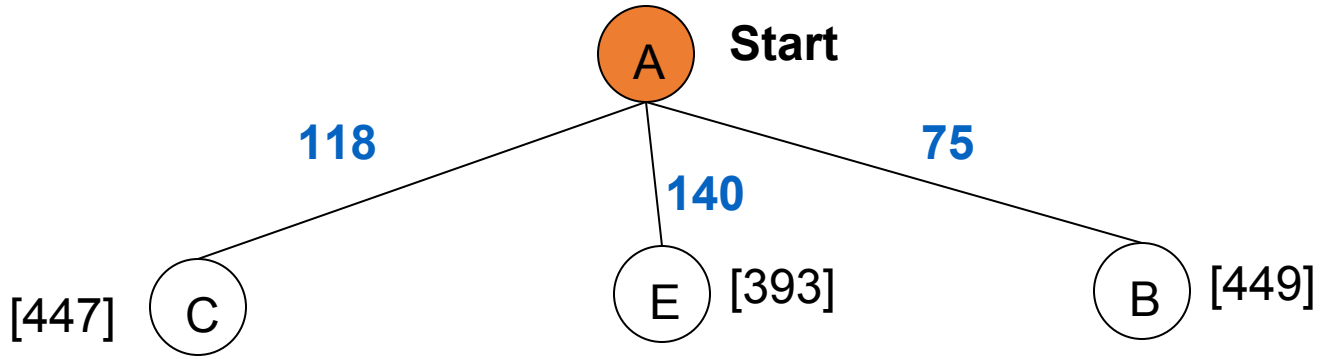
State	Heuristic: $h(n)$
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	**138**
I	0

$f(n) = g(n) + h(n)$ – **(H-I) Overestimated**

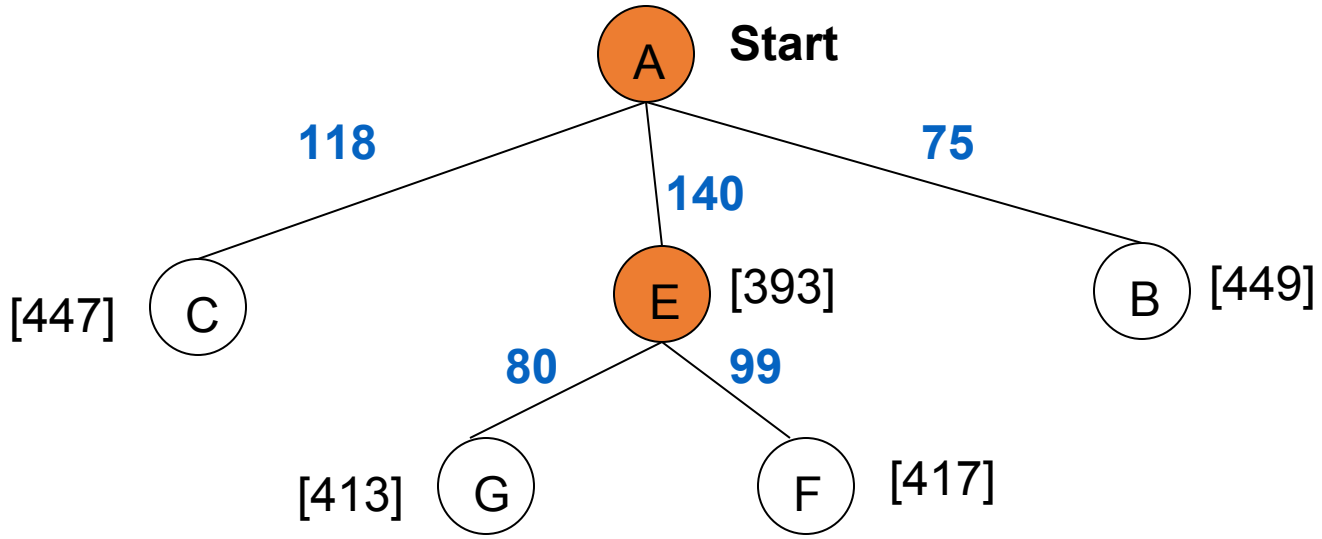
$g(n)$ is the exact cost to reach node n from the initial state.

A^* Search: Tree Search

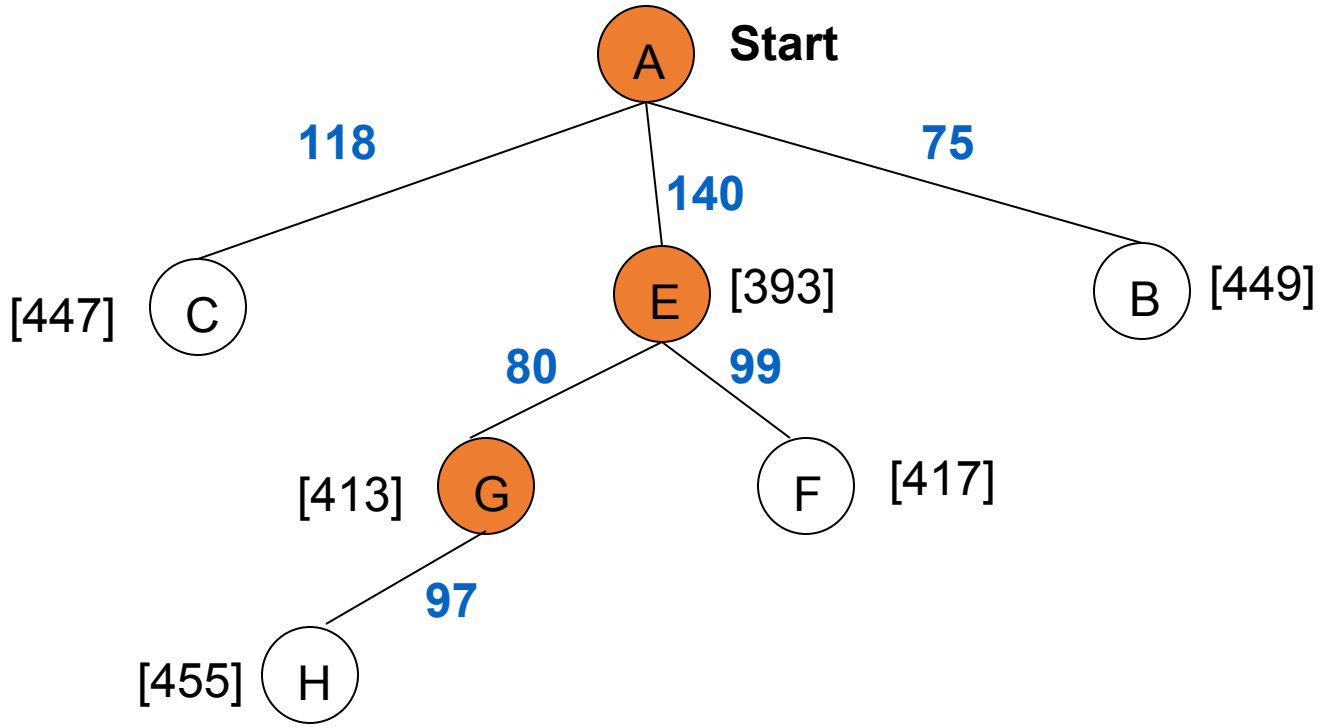
A^* Search: Tree Search



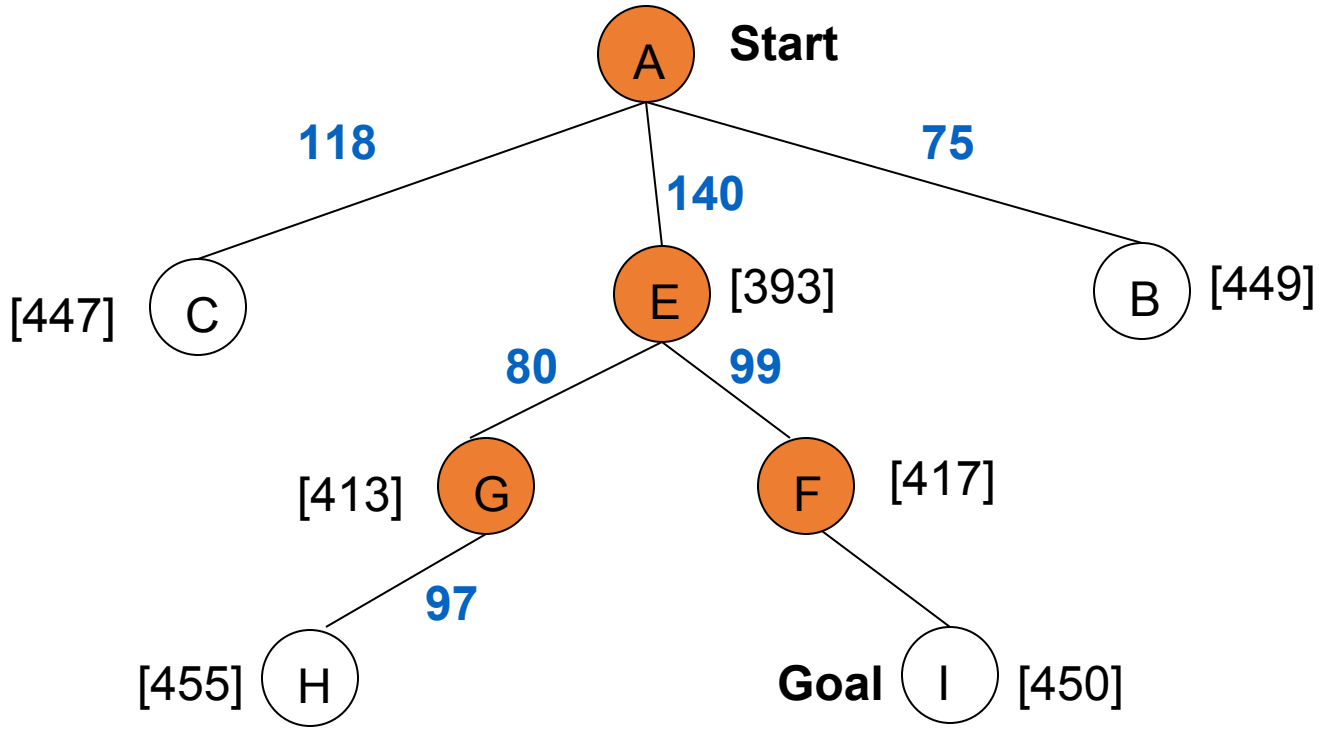
A^* Search: Tree Search



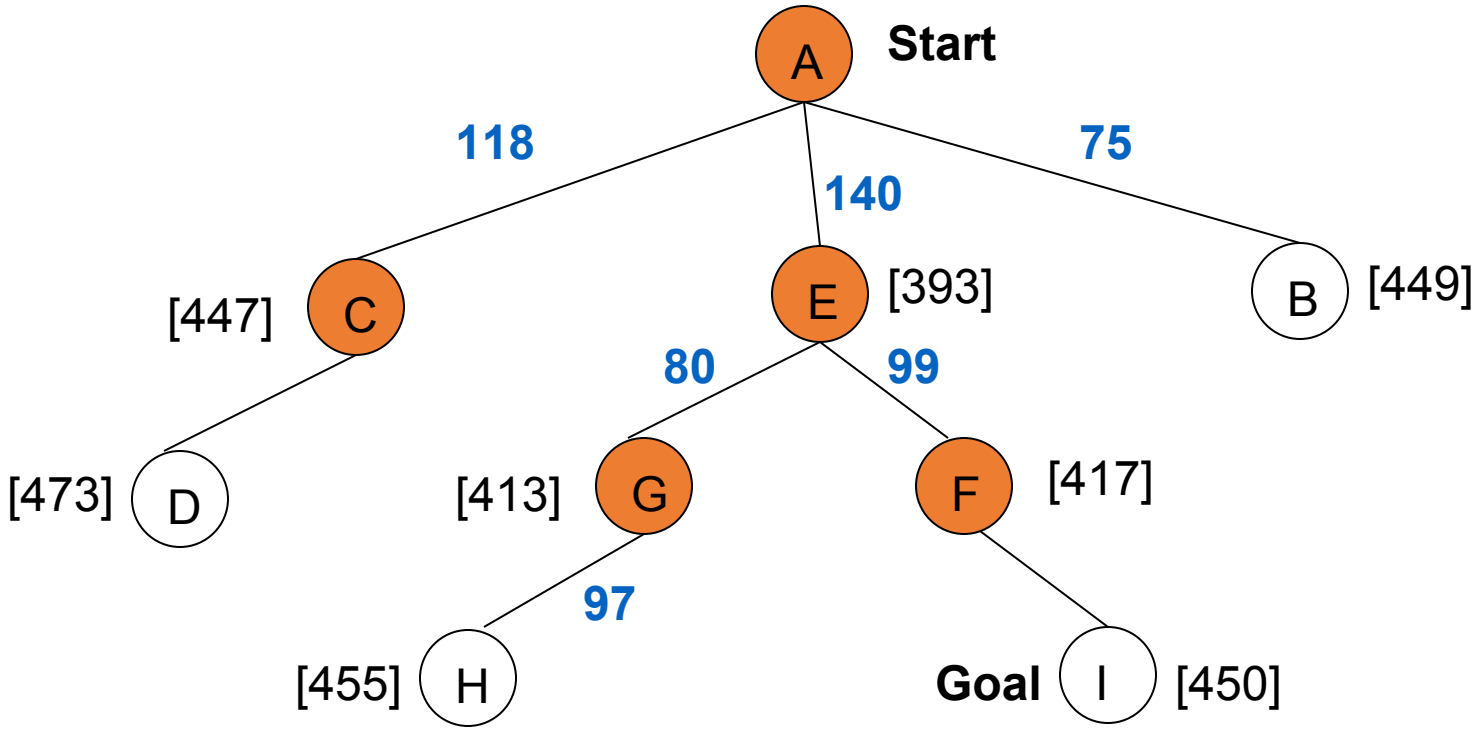
A^* Search: Tree Search



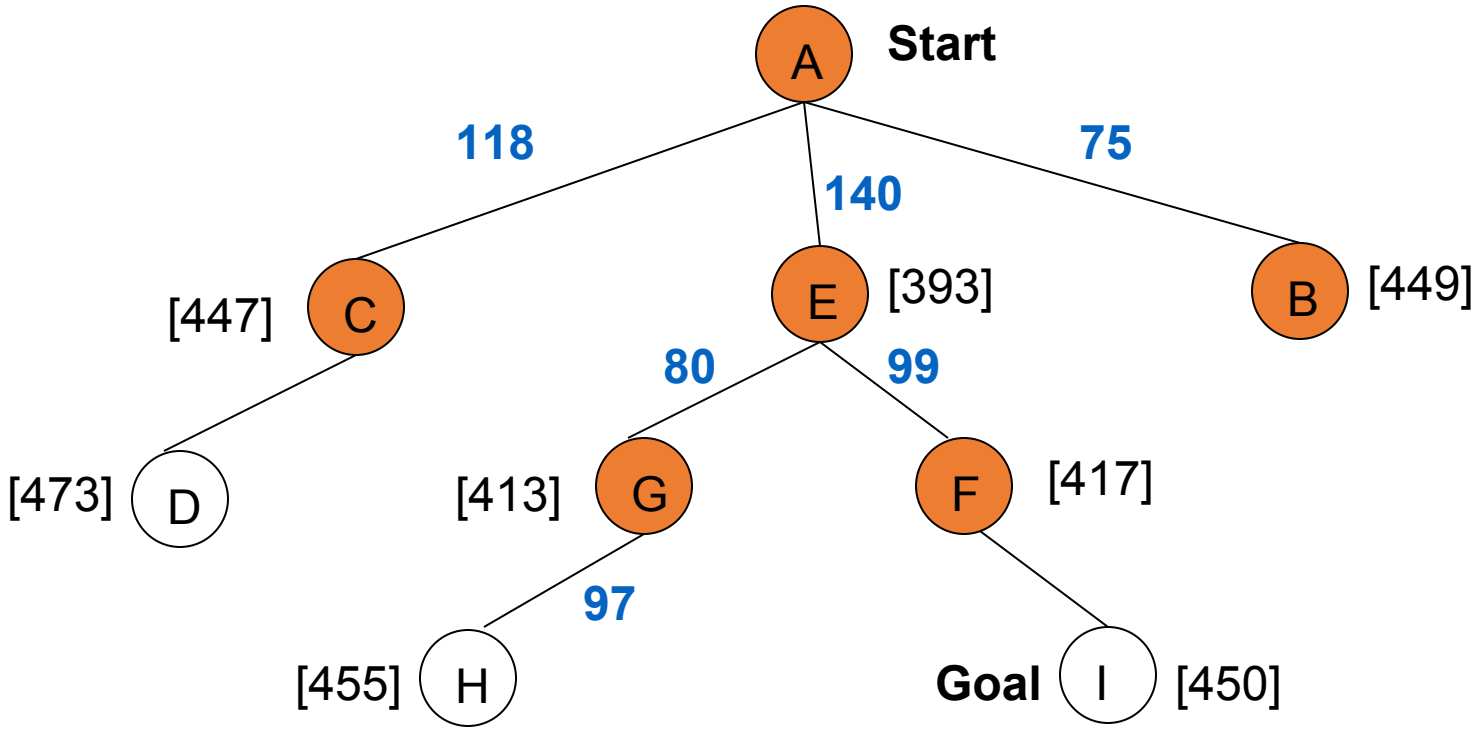
A^* Search: Tree Search



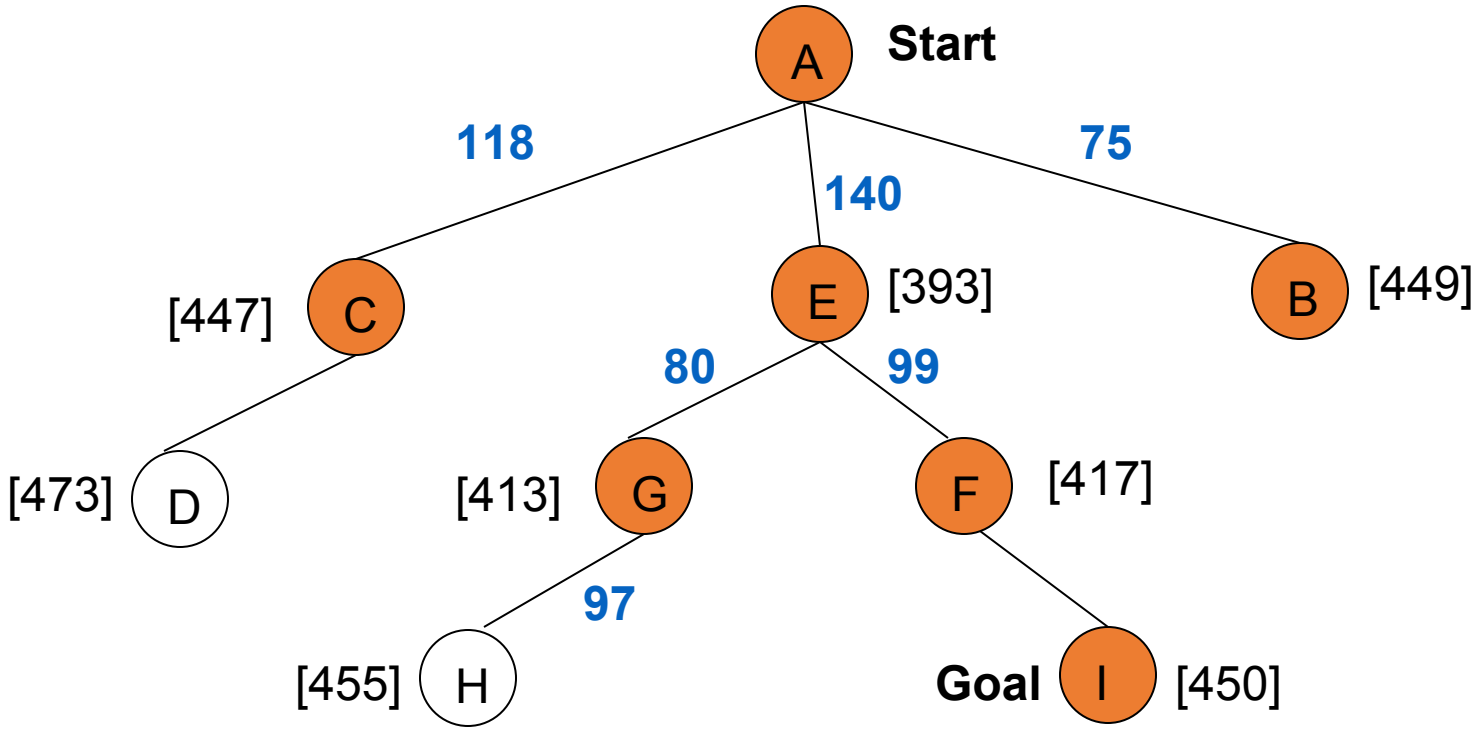
A^* Search: Tree Search



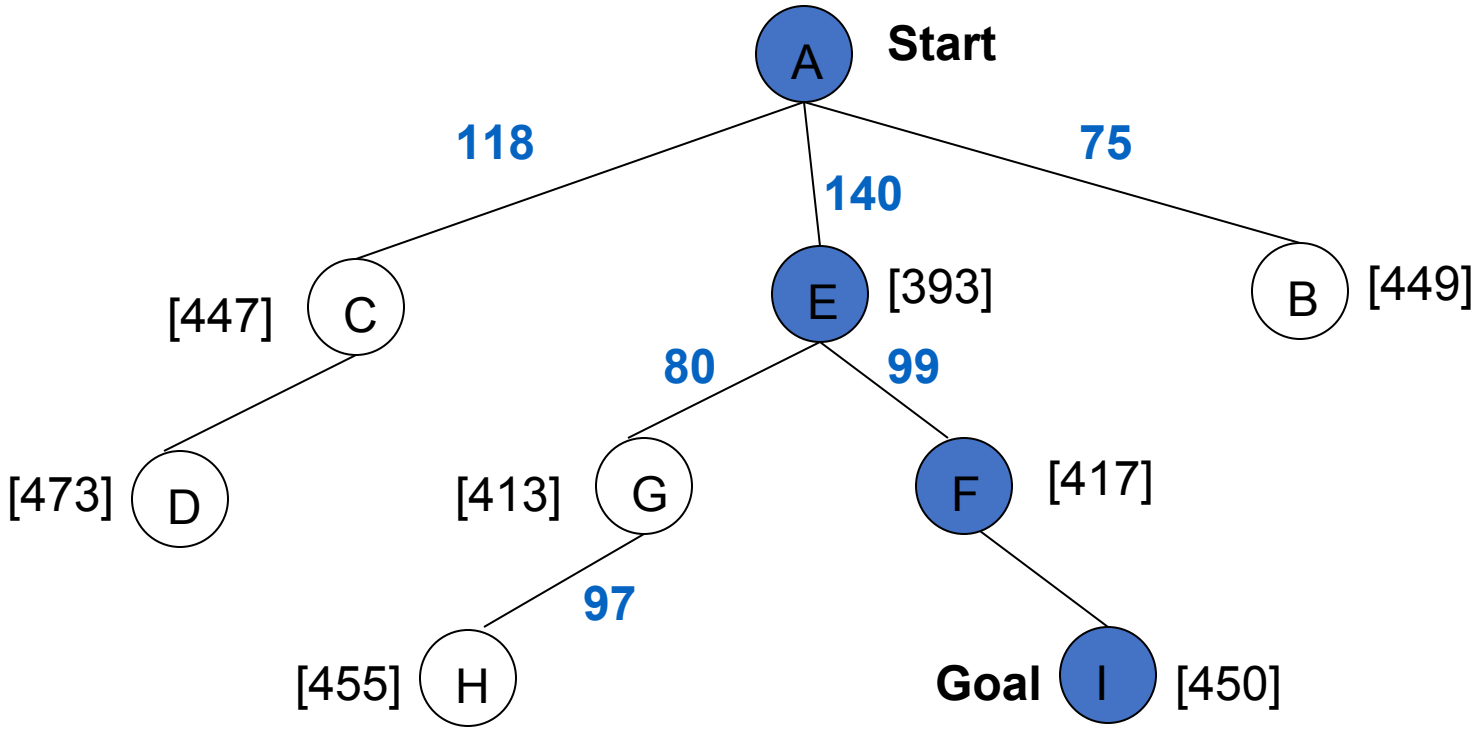
A^* Search: Tree Search



A^* Search: Tree Search



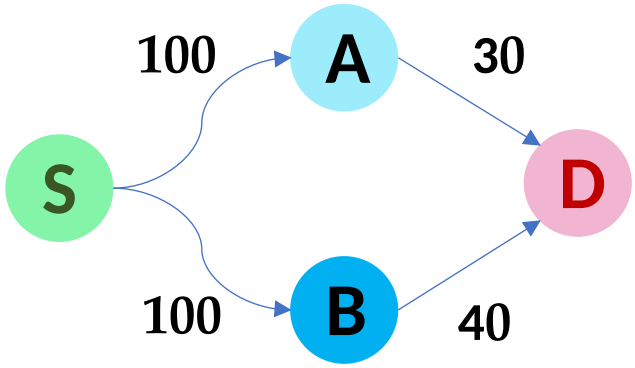
A^* Search: Tree Search



A^* not optimal !!!

Power of f

- If heuristic function is wrong it either
  - overestimates (guesses too high)
  - underestimates (guesses too low)
- Overestimating is worse than underestimating
- A^* returns optimal solution if $h(n)$ is admissible
  - heuristic function is admissible if never overestimates true cost to nearest goal
  - if search finds optimal solution using admissible heuristic, the search is admissible



$h(B) < h(A) > C_{BD}$

Overestimating

- $h(B) \ll C_{BD}, C_{AD}, h(A)$
- $f_{BD}(D) = g(B) + C_{BD} = g(D)$
- $f_{AD}(A) = g(A) + h(A)$
- $f_{AD}(A) \leq g(A) + C_{AD}$
- $f_{AD}(A) \leq g(B) + C_{BD}$
- $f_{AD}(A) \leq f_{BD}(D)$

- $h(A) = 35 \rightarrow 30$ √
- $h(B) = 45 \rightarrow 40$

- $h(A) = 55 \rightarrow 30$ *
- $h(B) = 47 \rightarrow 40$

- $h(A) = 20 \rightarrow 30$ √
- $h(B) = 35 \rightarrow 40$

- $h(A) = 25 \rightarrow 30$ √
- $h(B) = 15 \rightarrow 40$

Memory-Bounded Heuristic Search

- Iterative Deepening A^* (IDA^*)
  - Similar to Iterative Deepening Search, but cut off at $g(n) + h(n)$ instead of depth
  - At each iteration, cutoff is the first $f(n)$-cost that exceeds the cost of the node at the previous iteration

- Recursive BFS

- Simple Memory Bounded A^* (SMA^*)
  - Set max to some memory bound
  - If the memory is full, to add a node drop the worst $g(n) + h(n)$ node that is already stored
  - Expands newest best leaf, deletes oldest worst leaf

Iterative Deepening A^* : IDA^*

- Use $f(n) = g(n) + h(n)$ with admissible and consistent h.

- Each iteration is depth-first Search with cutoff on the value of f of expanded nodes.

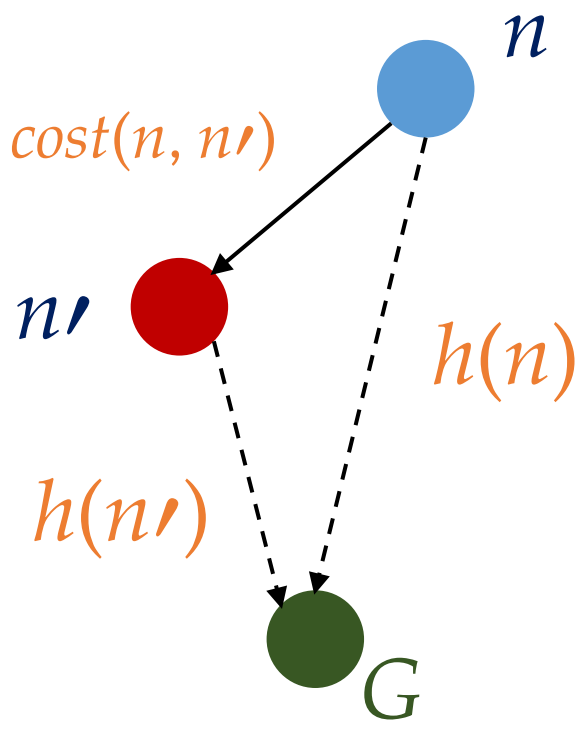
Consistent Heuristic

- The admissible heuristic h is consistent (or satisfies the monotone restriction) if for every node n and every successor n' of n:

$$h(n) \leq cost(n,n') + h(n')$$

(triangular inequality)

- A consistent heuristic is admissible.



IDA^* Algorithm

1. In the first iteration, we determine a $f_{cost\ limit}$ – **cut-off value**
   $f(n_0) = g(n_0) + h(n_0) = h(n_0)$, where n_0 is the start node.
2. We expand nodes using the **Depth-First Search** and backtrack whenever $f(n)$ for an expanded node n exceeds the cut-off value.
3. If this search does not succeed, determine the **lowest f-value** among the nodes that were visited but not expanded.
4. Use this f-value as the **new $f_{cost\ limit}$ – cut-off value** and do another **Depth-First Search**.
5. Repeat this procedure until a goal node is found.

8-Puzzle

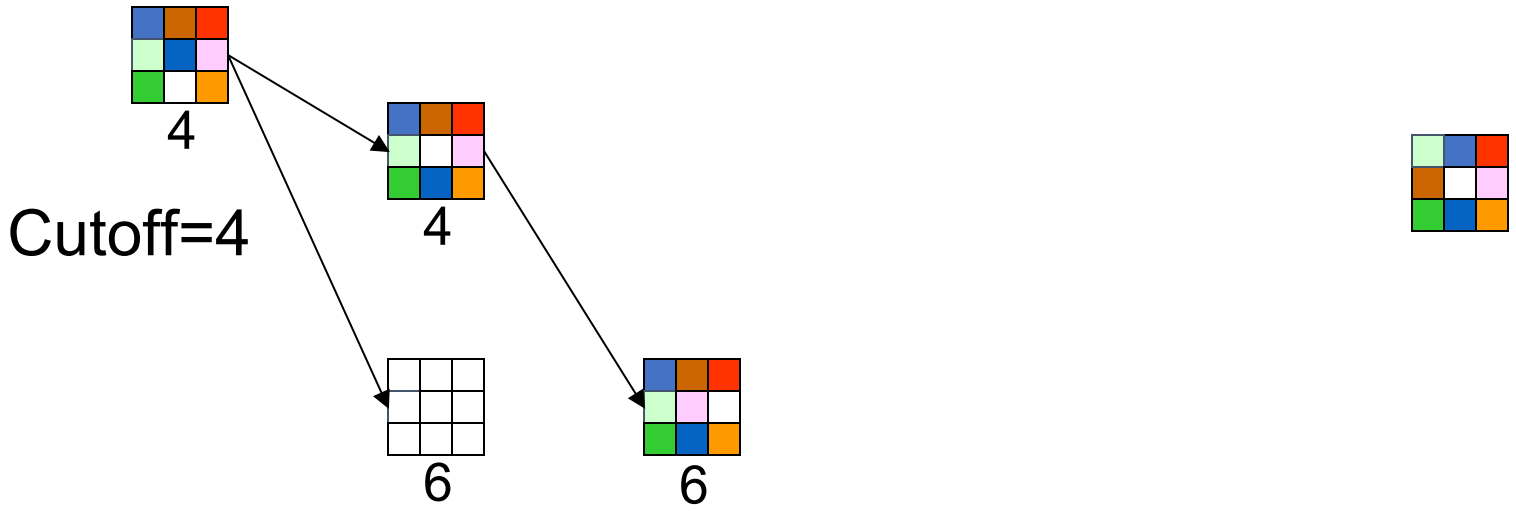
$$f(n) = g(n) + h(n)$$

with $h(n)$ = number of misplaced tiles

8-Puzzle

$$f(n) = g(n) + h(n)$$

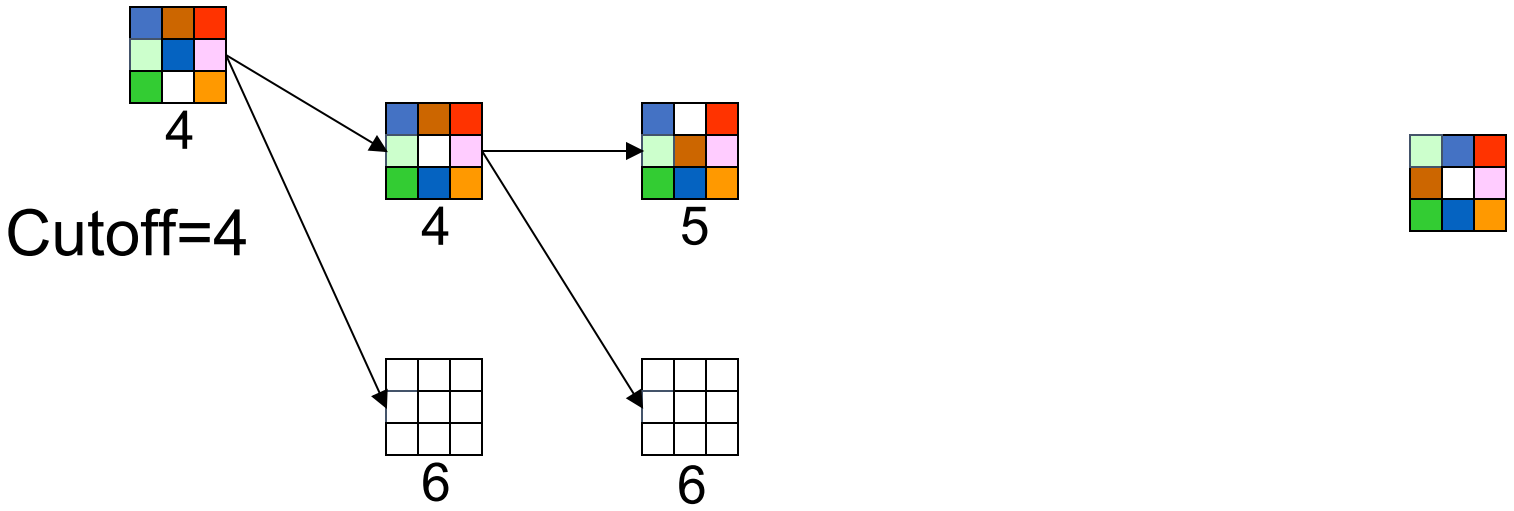
with $h(n)$ = number of misplaced tiles



8-Puzzle

$$f(n) = g(n) + h(n)$$

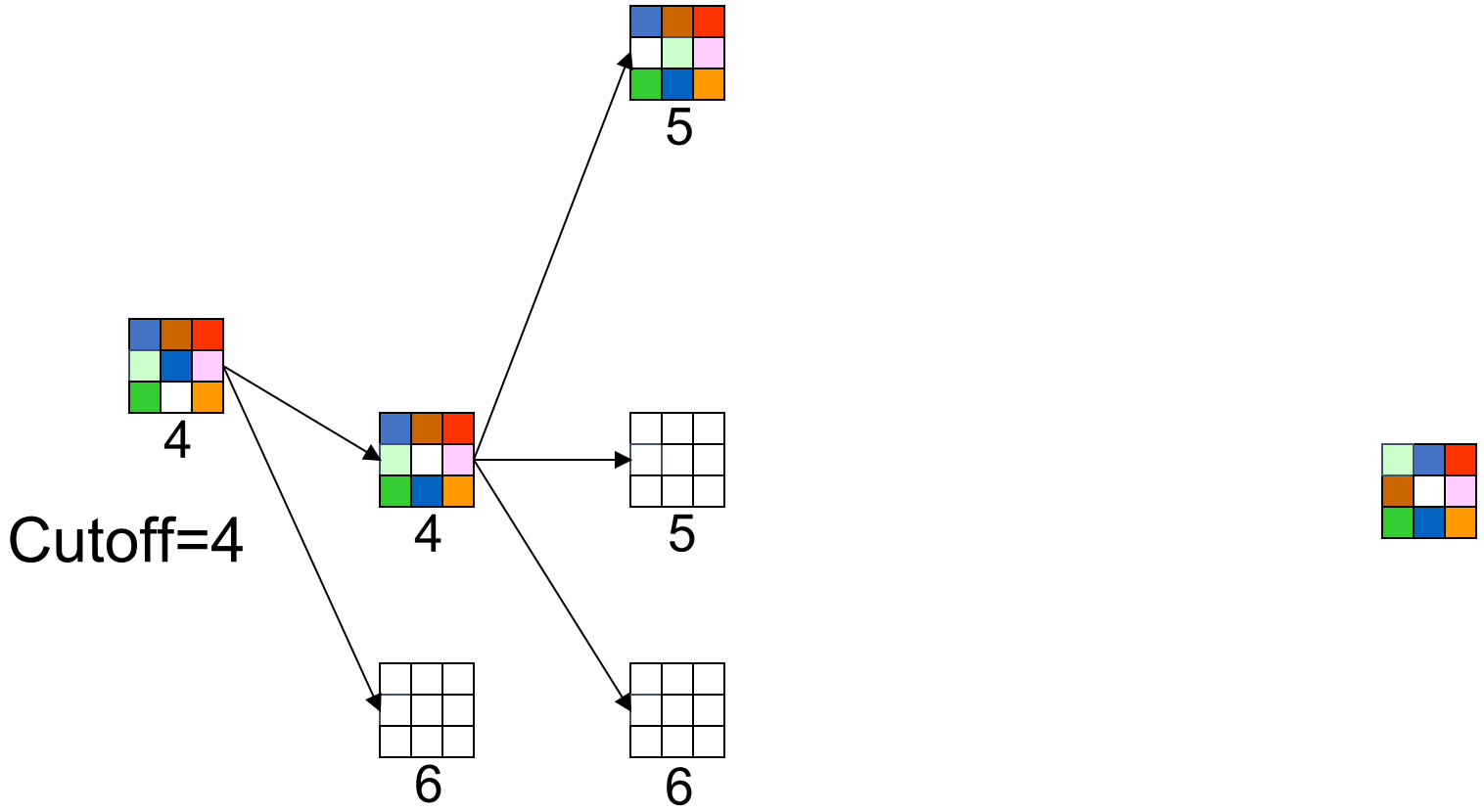
with $h(n)$ = number of misplaced tiles



8-Puzzle

$$f(n) = g(n) + h(n)$$

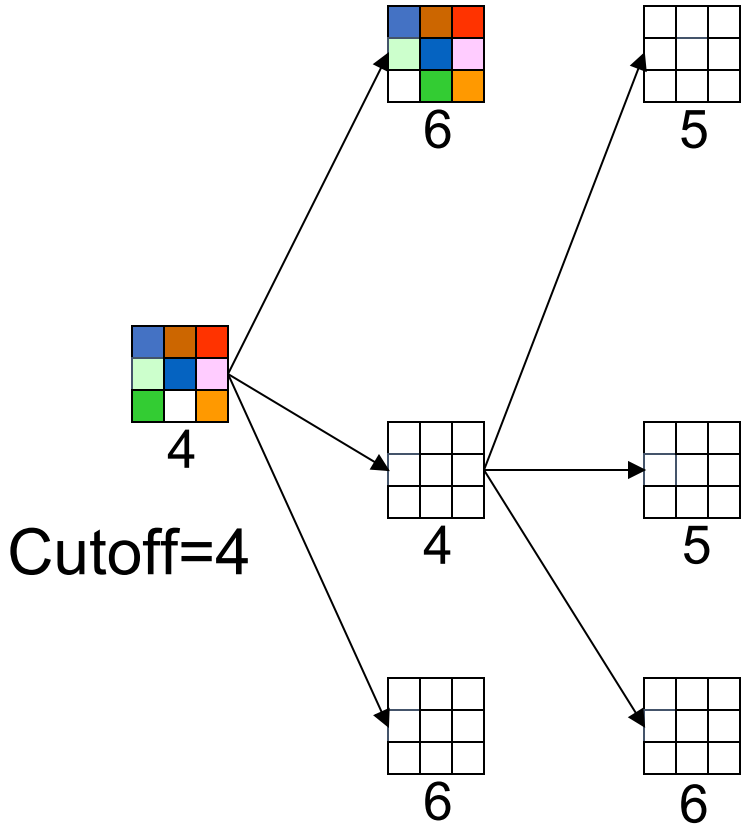
with $h(n)$ = number of misplaced tiles



8-Puzzle

$$f(n) = g(n) + h(n)$$

with $h(n)$ = number of misplaced tiles



8-Puzzle

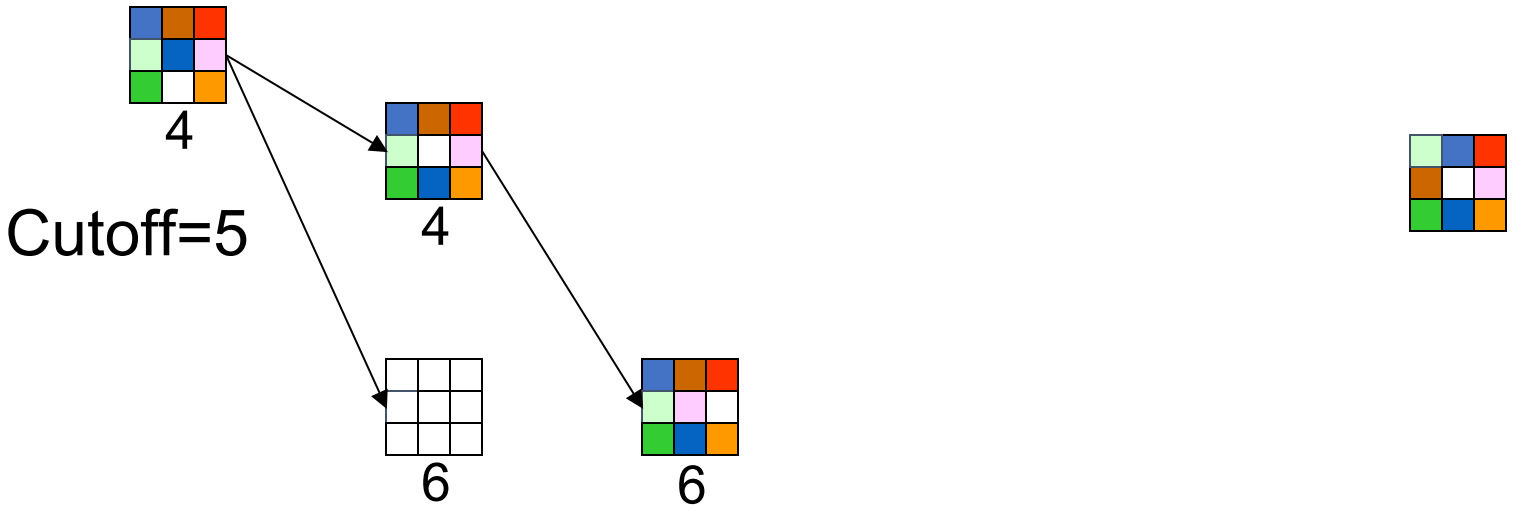
$$f(n) = g(n) + h(n)$$

with $h(n)$ = number of misplaced tiles

8-Puzzle

$$f(n) = g(n) + h(n)$$

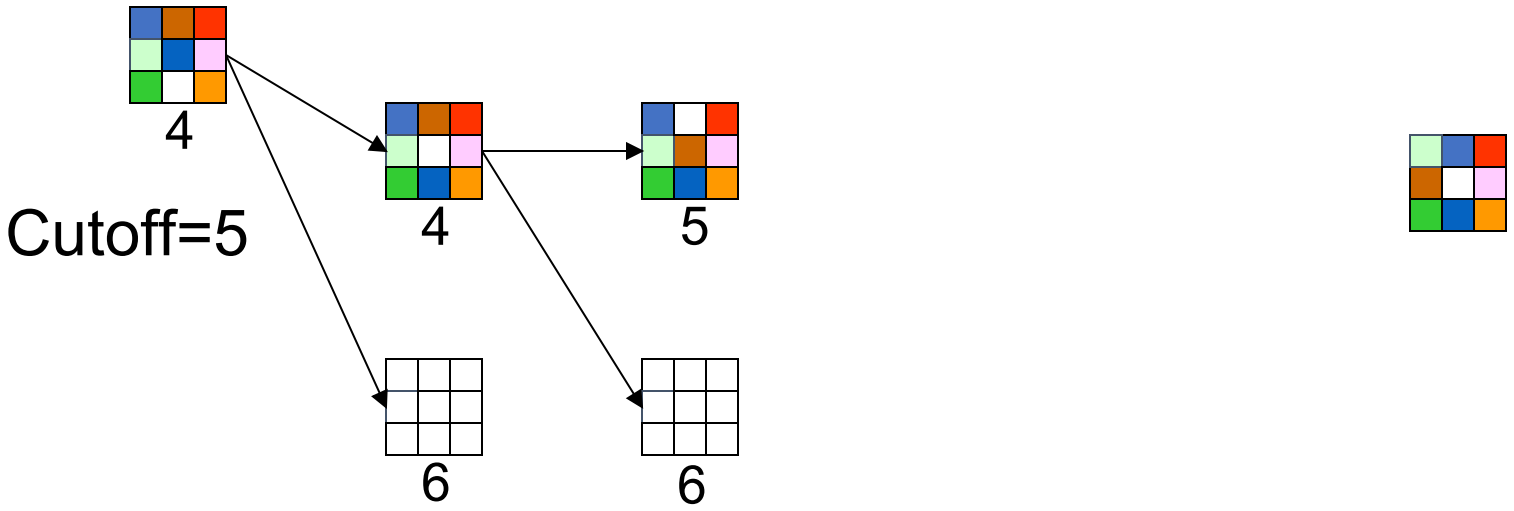
with $h(n)$ = number of misplaced tiles



8-Puzzle

$$f(n) = g(n) + h(n)$$

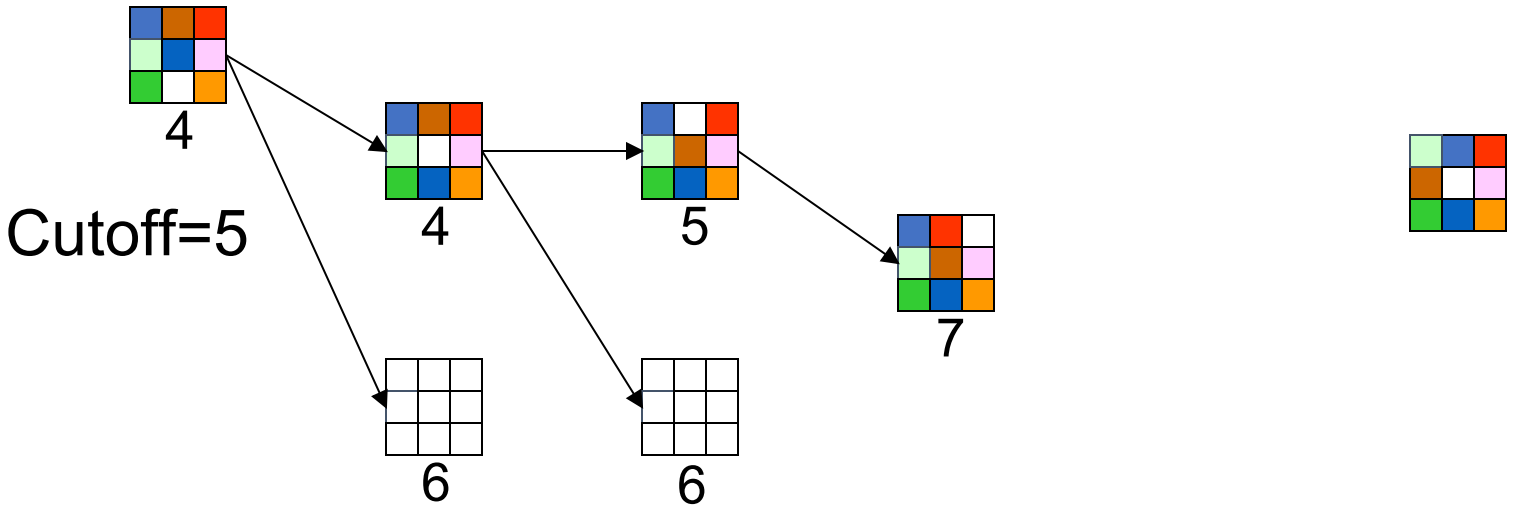
with $h(n)$ = number of misplaced tiles



8-Puzzle

$$f(n) = g(n) + h(n)$$

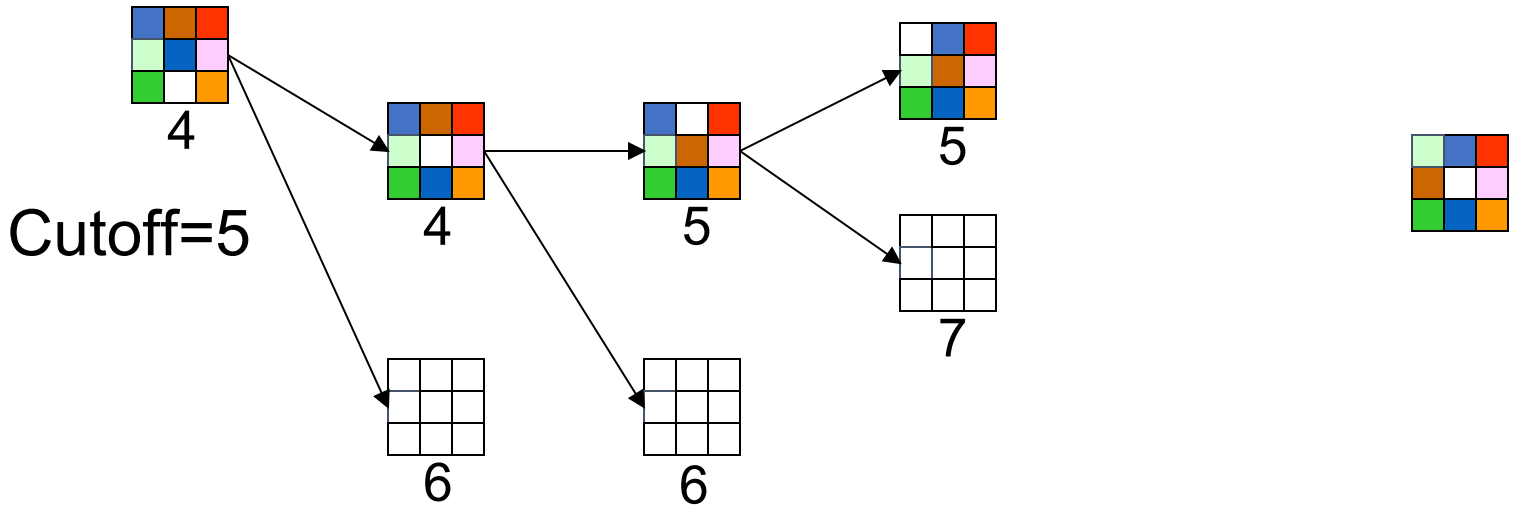
with $h(n)$ = number of misplaced tiles



8-Puzzle

$$f(n) = g(n) + h(n)$$

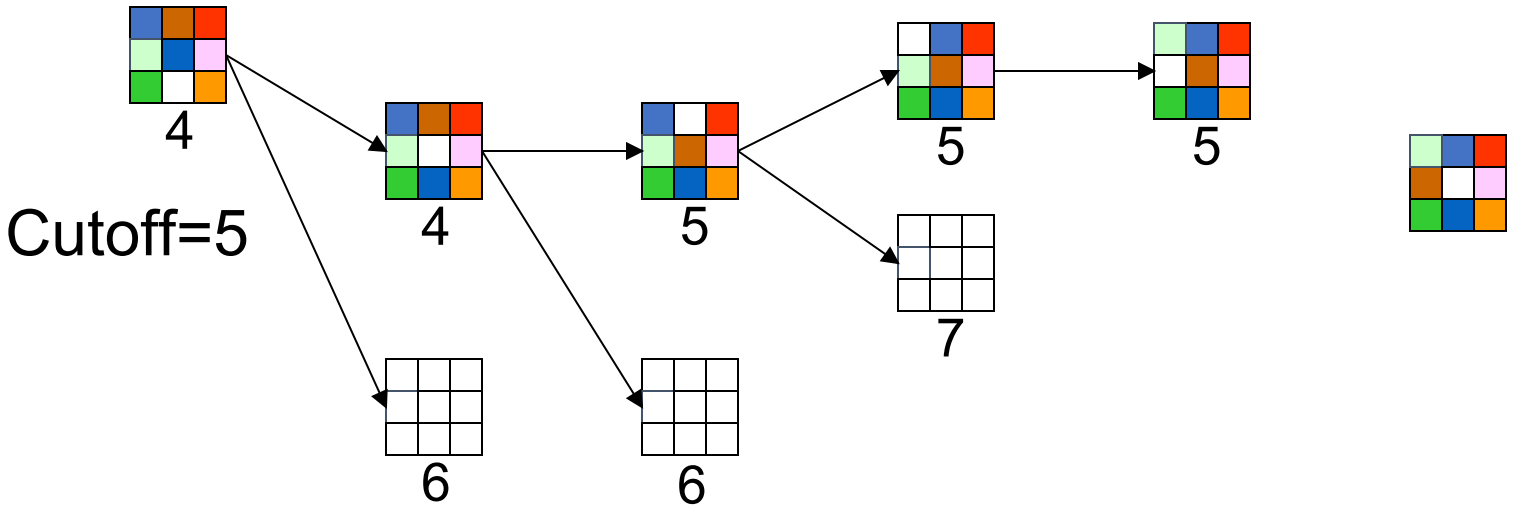
with $h(n)$ = number of misplaced tiles



8-Puzzle

$$f(n) = g(n) + h(n)$$

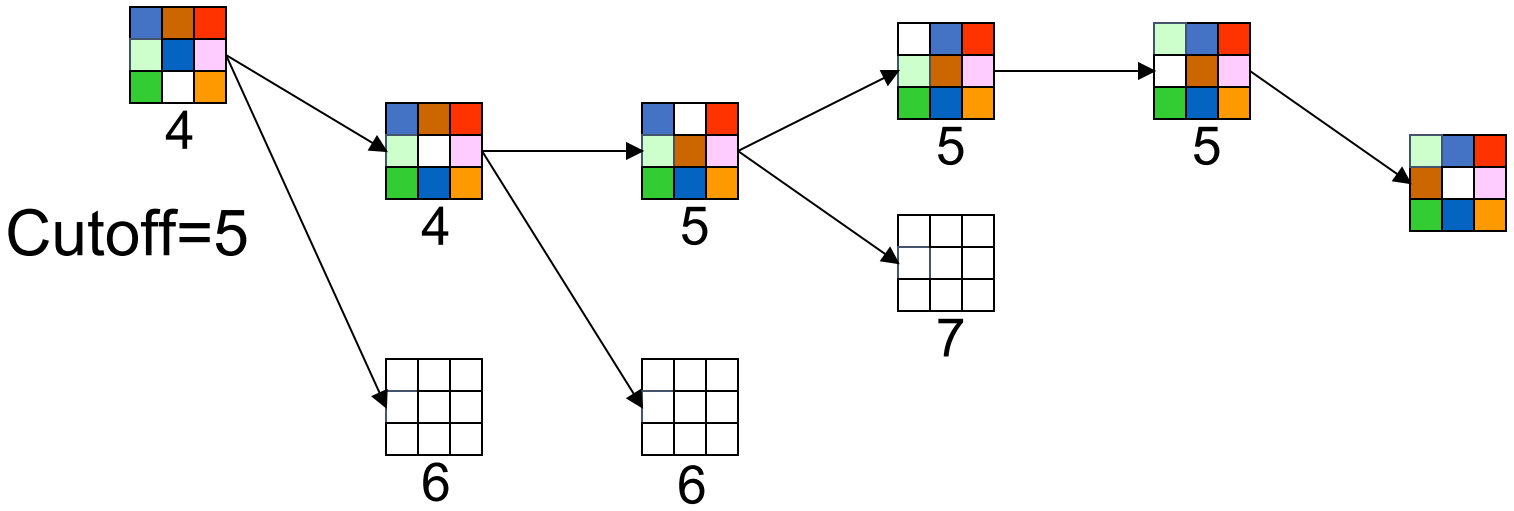
with $h(n)$ = number of misplaced tiles



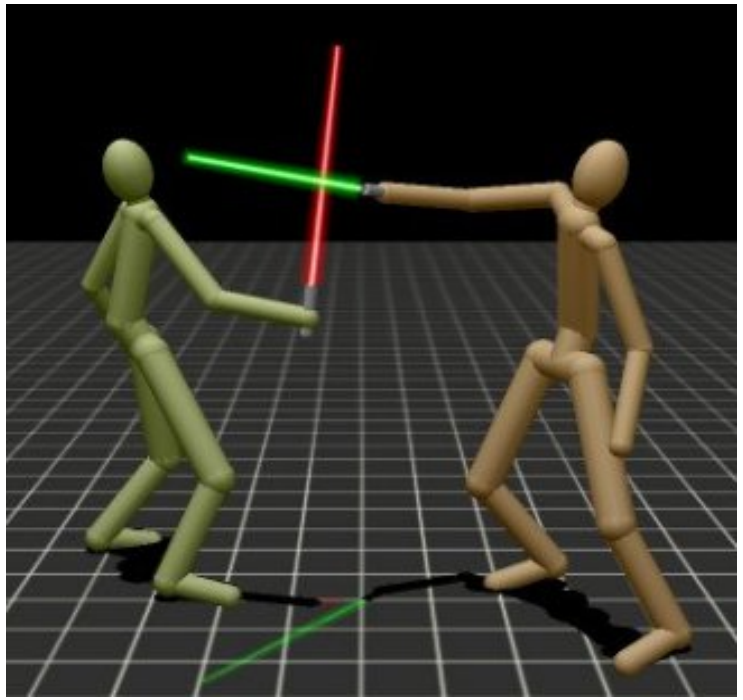
8-Puzzle

$$f(n) = g(n) + h(n)$$

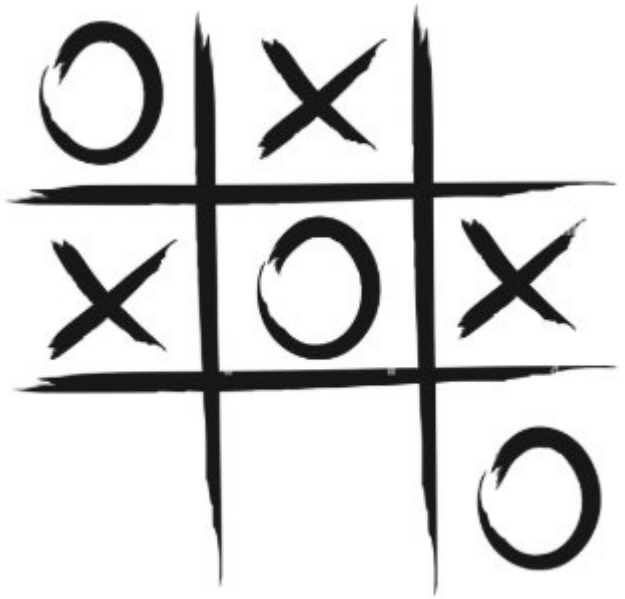
with $h(n)$ = number of misplaced tiles



Adversarial Search



Zero-Sum Games



- Focus primarily on “adversarial games”
- Two-player, zero-sum games

As Player 1 gains strength →

Player 2 loses strength ←

and vice versa

The sum of the two strengths is always 0.

Search Applied to Adversarial Games

- Initial state
  - Current board position (description of current game state)
- Operators
  - Legal moves a player can make
- Terminal nodes
  - Leaf nodes in the tree
  - Indicate the game is over
- Utility function
  - Payoff function
  - Value of the outcome of a game
  - Example: tic tac toe, utility is -1, 0, or 1

Using Search

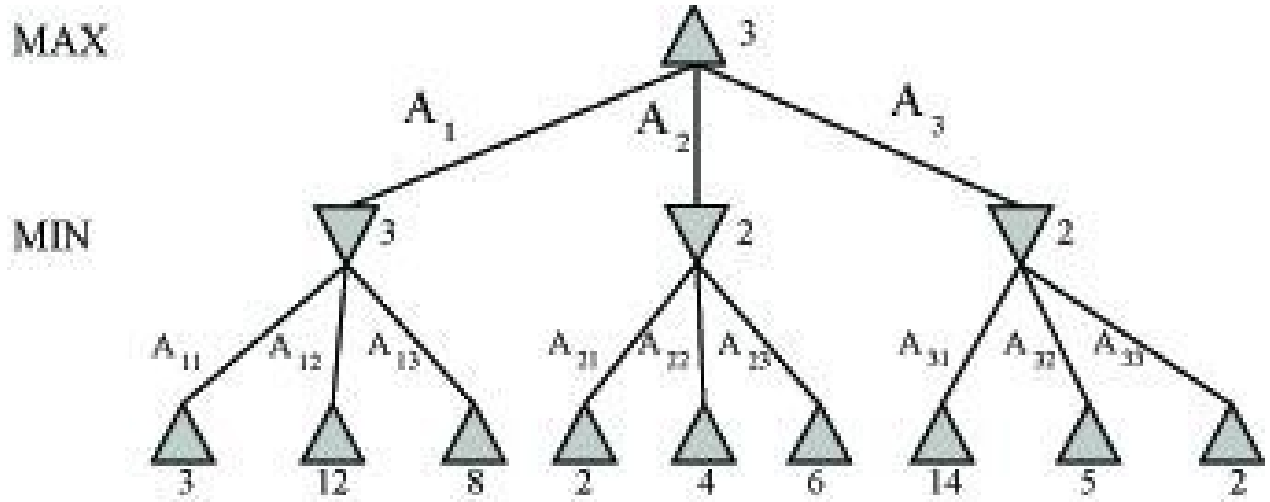
- Search could be used to find a perfect sequence of moves except the following problems arise:
  - There exists an adversary who is trying to minimize your chance of winning every other move
    - ❖ You cannot control his/her move

Tic Tac Toe (Game Trees)

Tic Tac Toe
Two players, MAX and MIN
Moves (and levels) alternate between two players

Minimax Algorithm

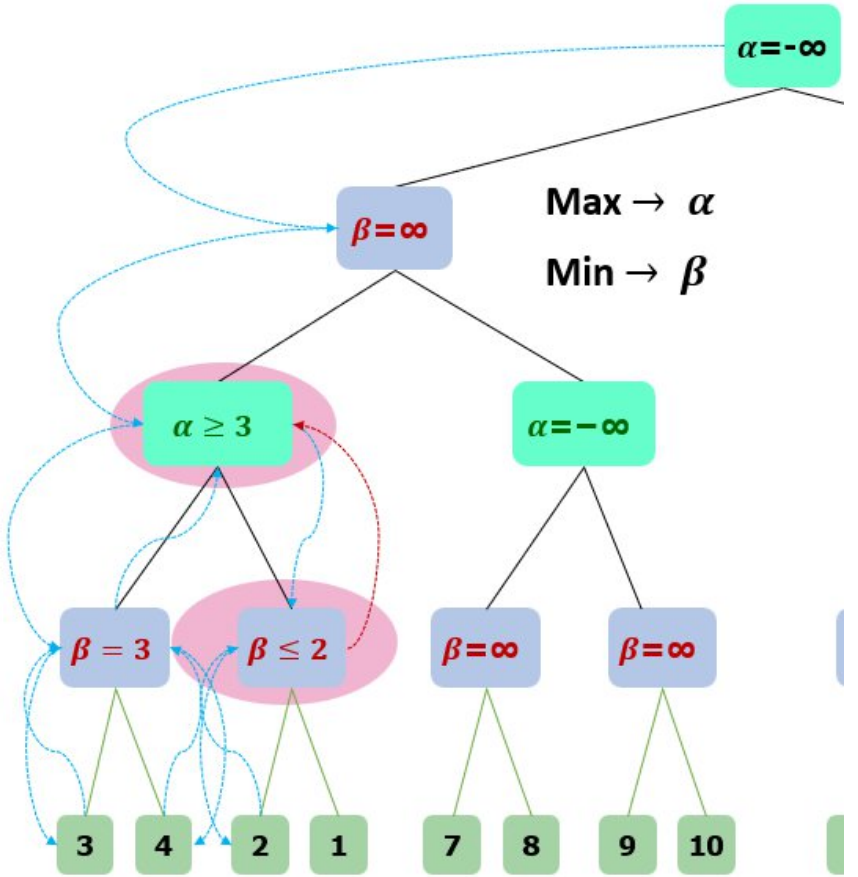
- Search the tree to the end
- Assign utility values to terminal nodes
- Find the best move for MAX (on MAX's turn), assuming:
  - MAX will make the move that maximizes MAX's utility
  - MIN will make the move that minimizes MAX's utility
- Here, MAX should make the leftmost move



Minimax Properties

- Complete if tree is finite
- Time complexity is $O(b^m)$
- Space complexity is $O(bm)$ (depth-first exploration)

Alpha-Beta (Pruning) Strategy

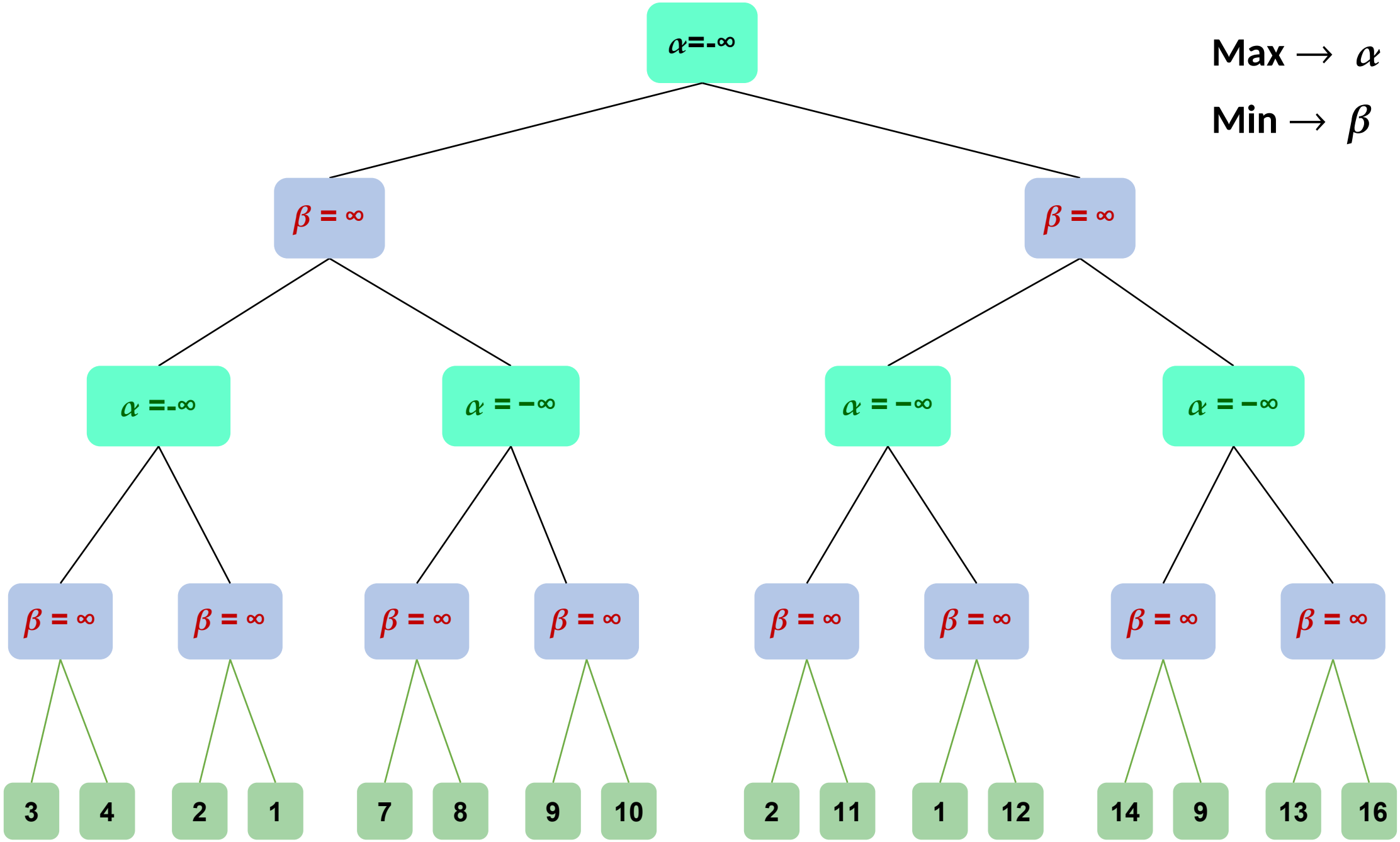


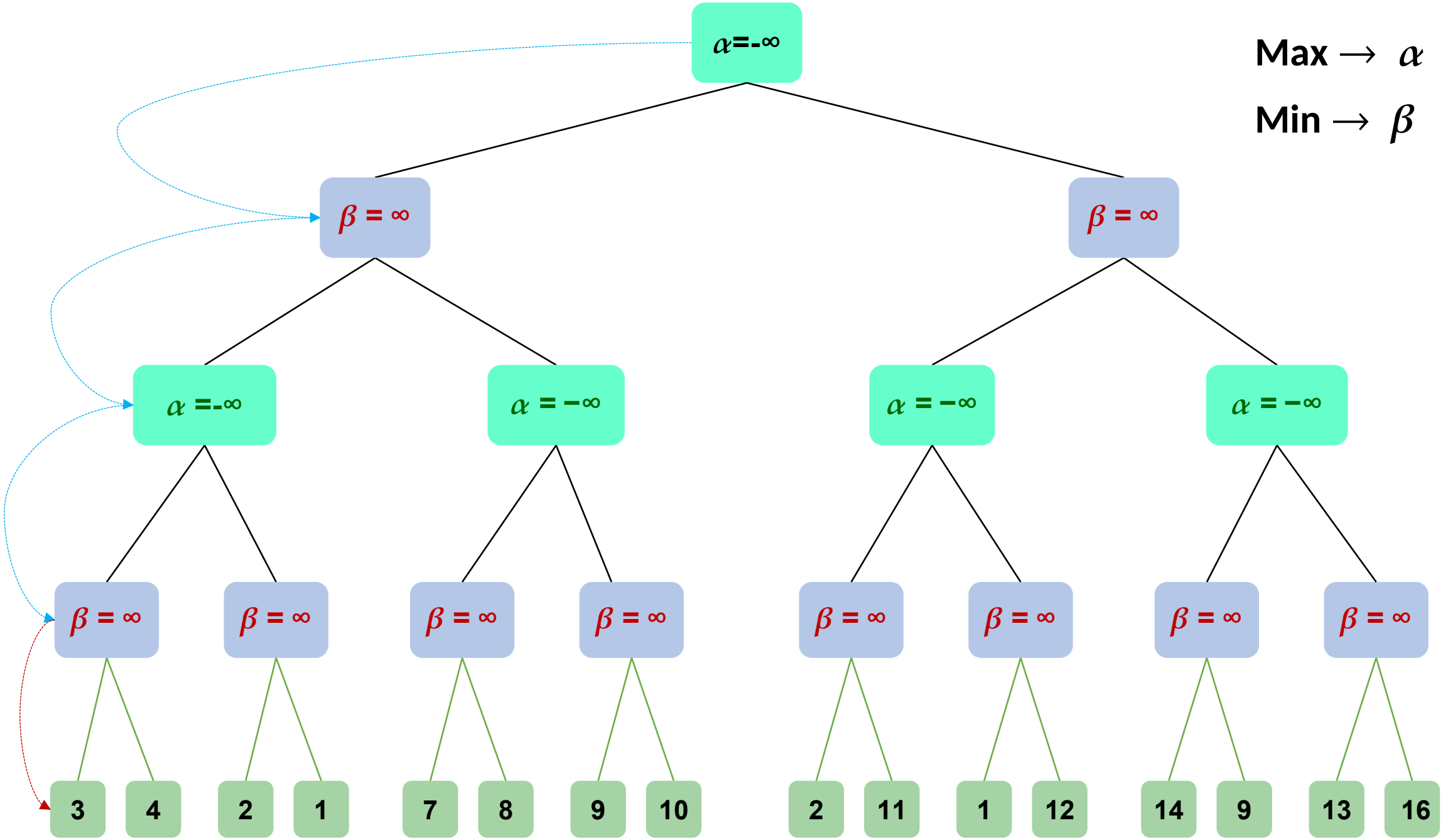
- Maintain two bounds:

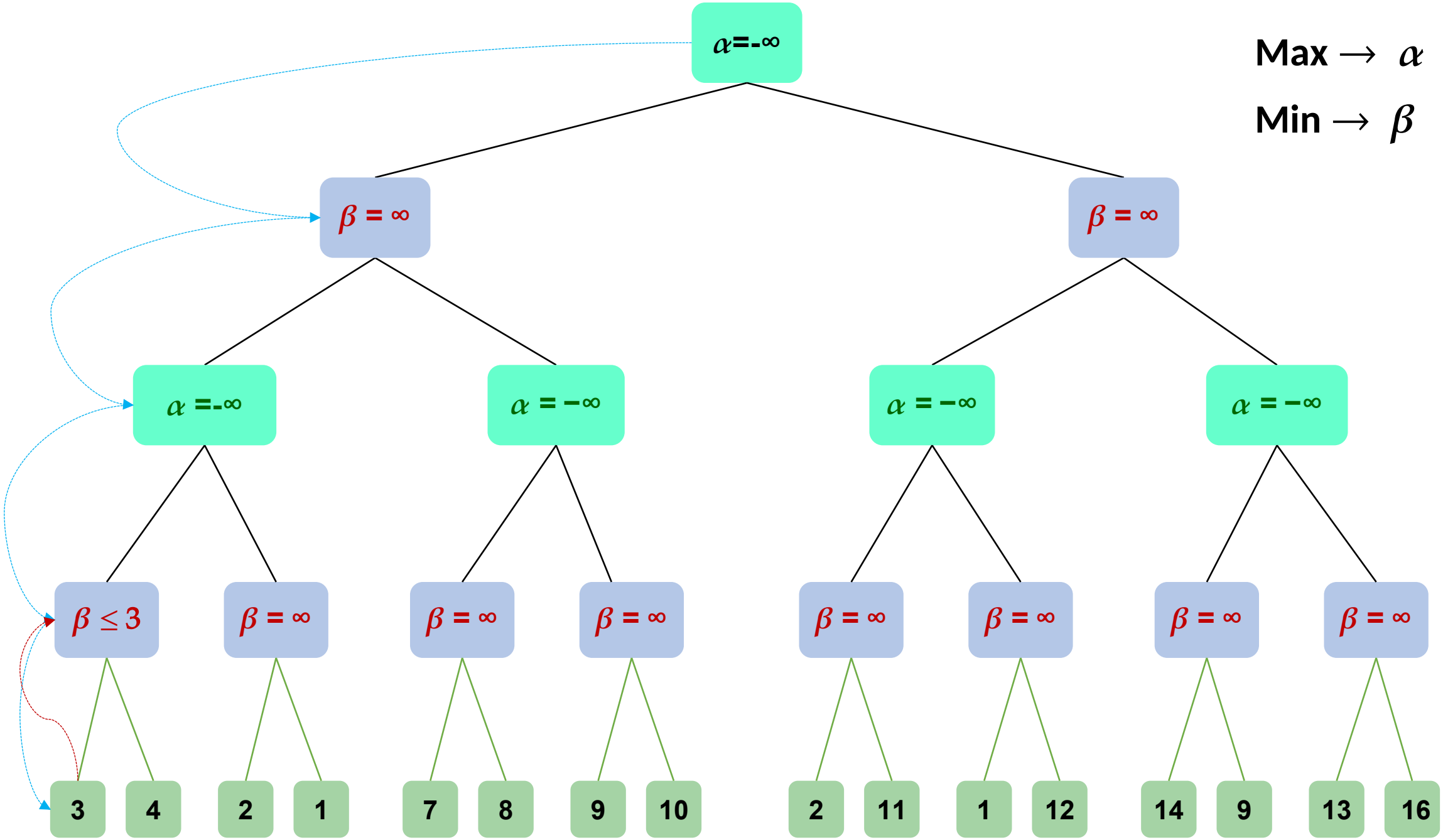
  Alpha (α): a lower bound on best that the player to move can achieve

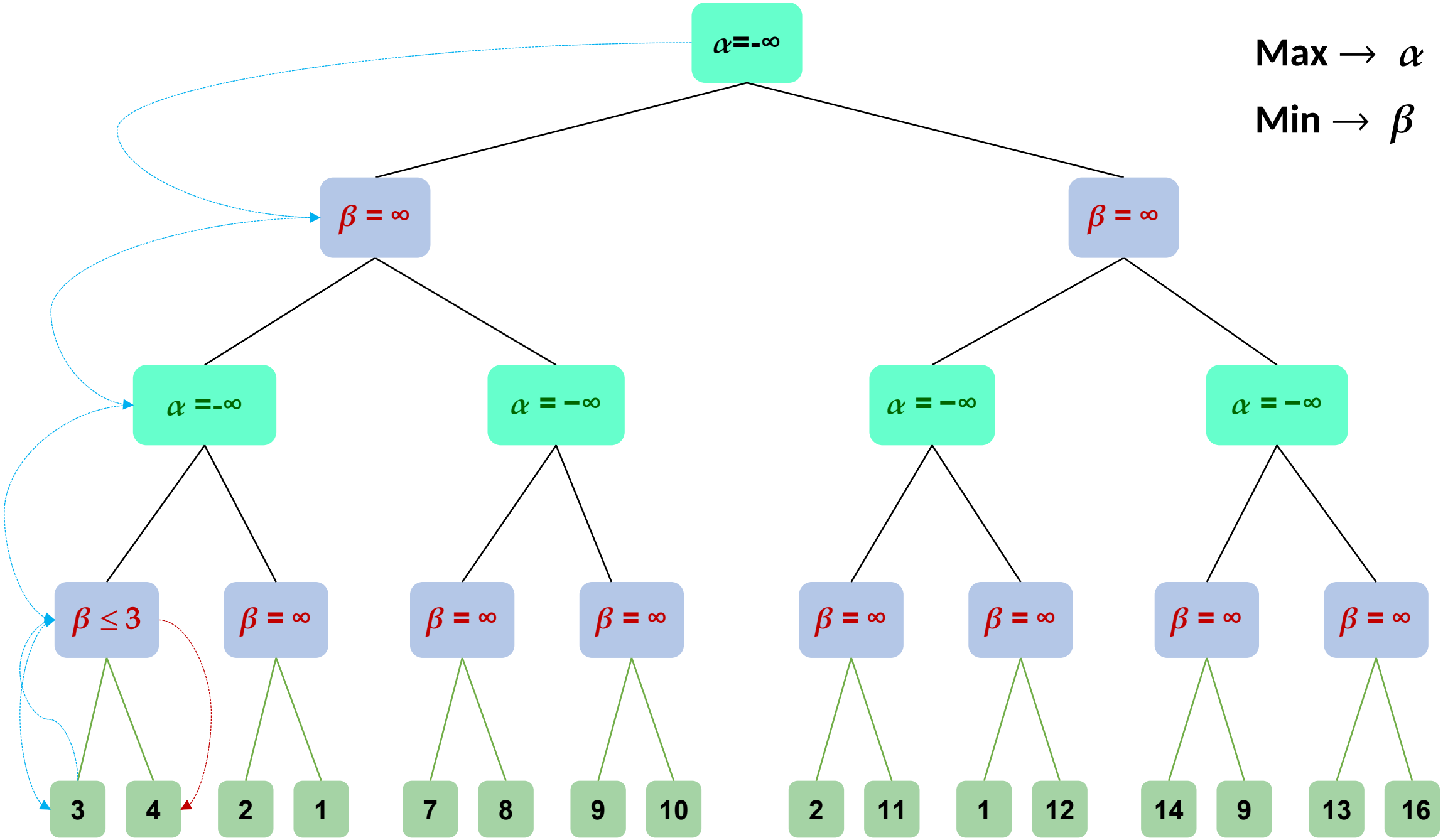
  Beta (β): an upper bound on what the opponent can achieve

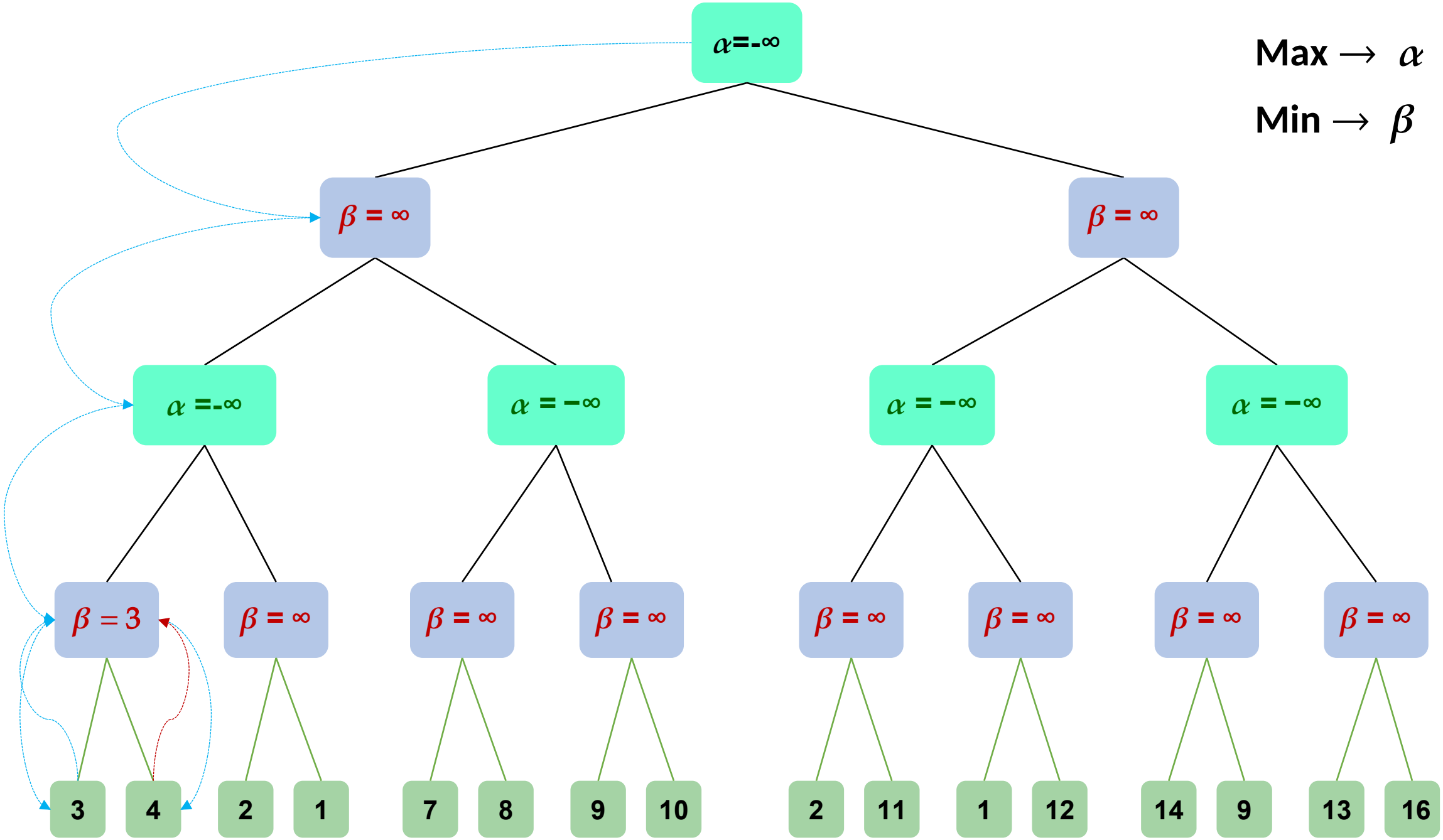
- Search, maintaining α and β
- Whenever $\alpha \geq \beta_{higher}$, or $\beta \leq \alpha_{higher}$ further search at this node is irrelevant

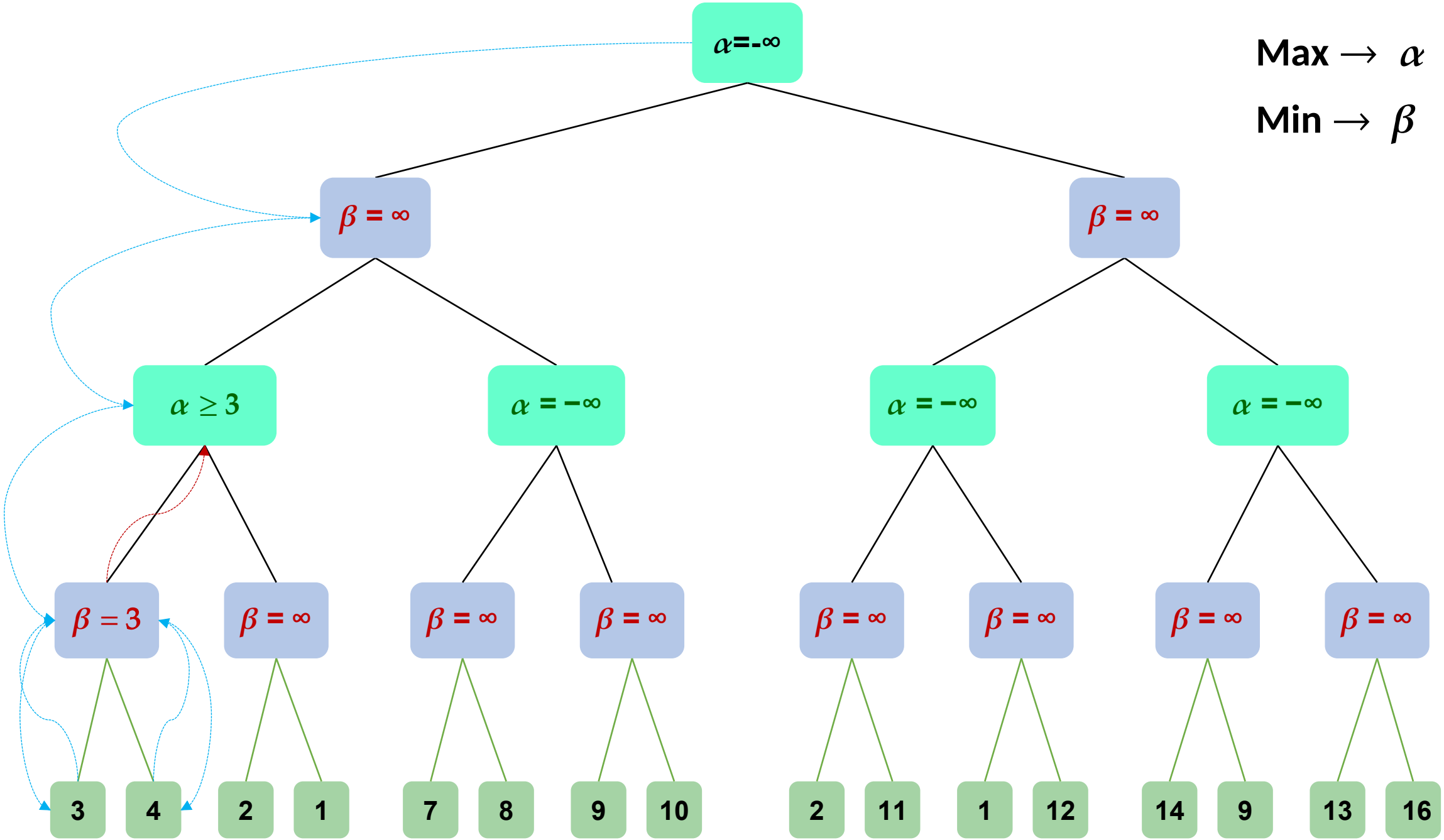

α=-∞
Max → α
Min → β
β = ∞
β = ∞
α =-∞
α = −∞
α = −∞
α = −∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

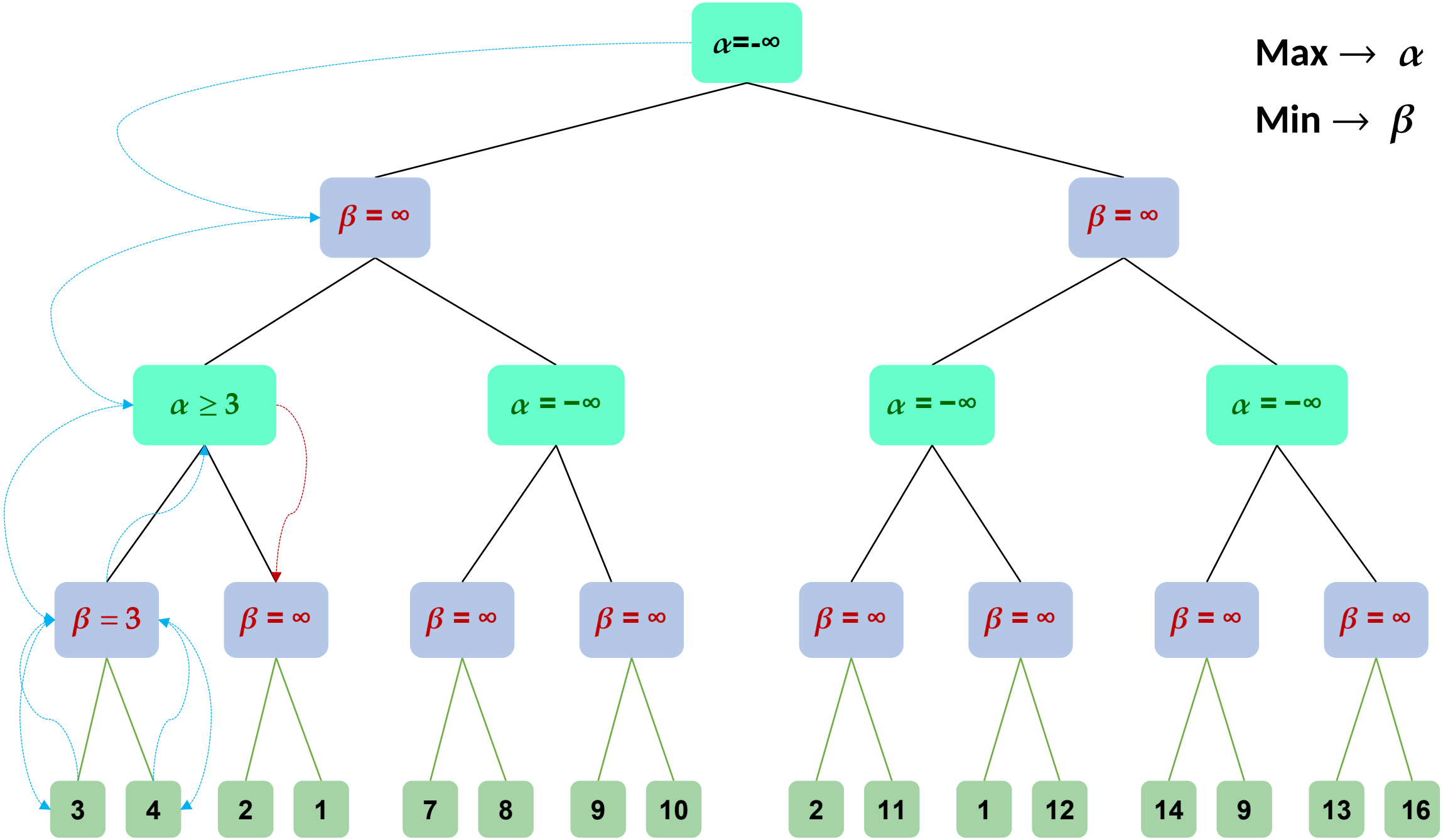

α=-∞
Max → α
Min → β
β = ∞
β = ∞
α =-∞
α = −∞
α = −∞
α = −∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

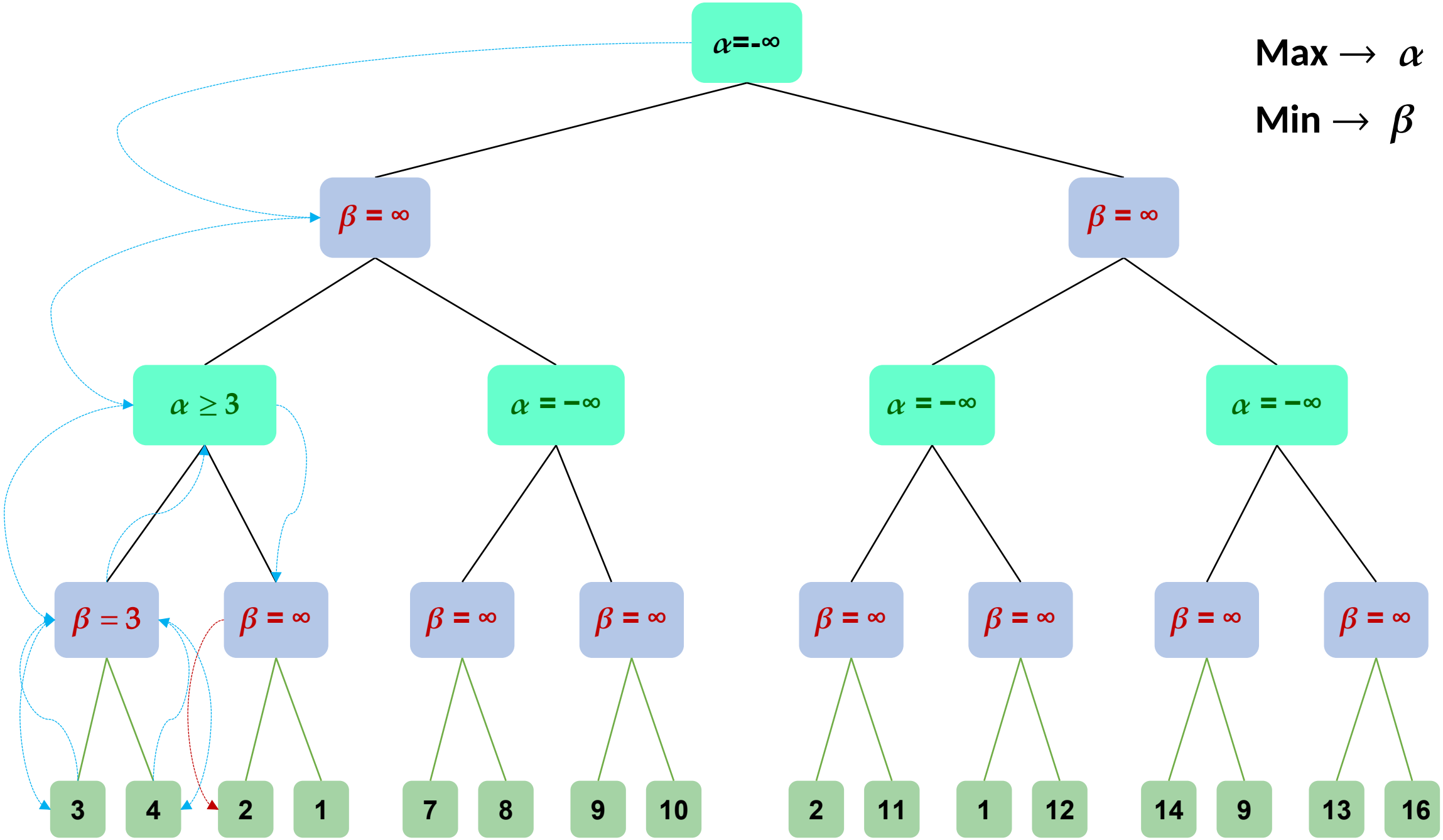

α=-∞
Max → α
Min → β
β = ∞
β = ∞
α =-∞
α = −∞
α = −∞
α = −∞
β ≤ 3
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

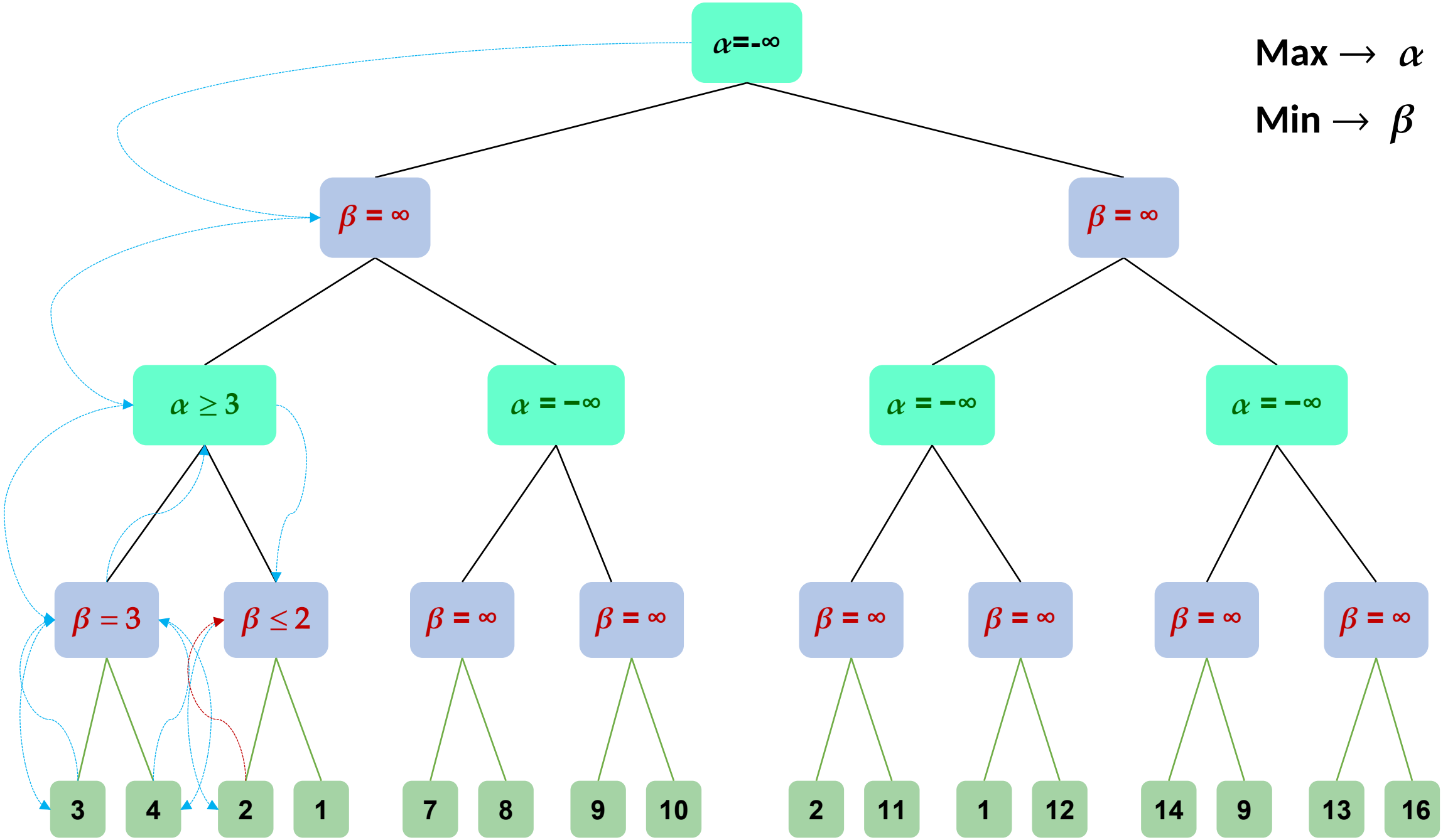

α=-∞
Max → α
Min → β
β = ∞
β = ∞
α =-∞
α = −∞
α = −∞
α = −∞
β ≤ 3
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

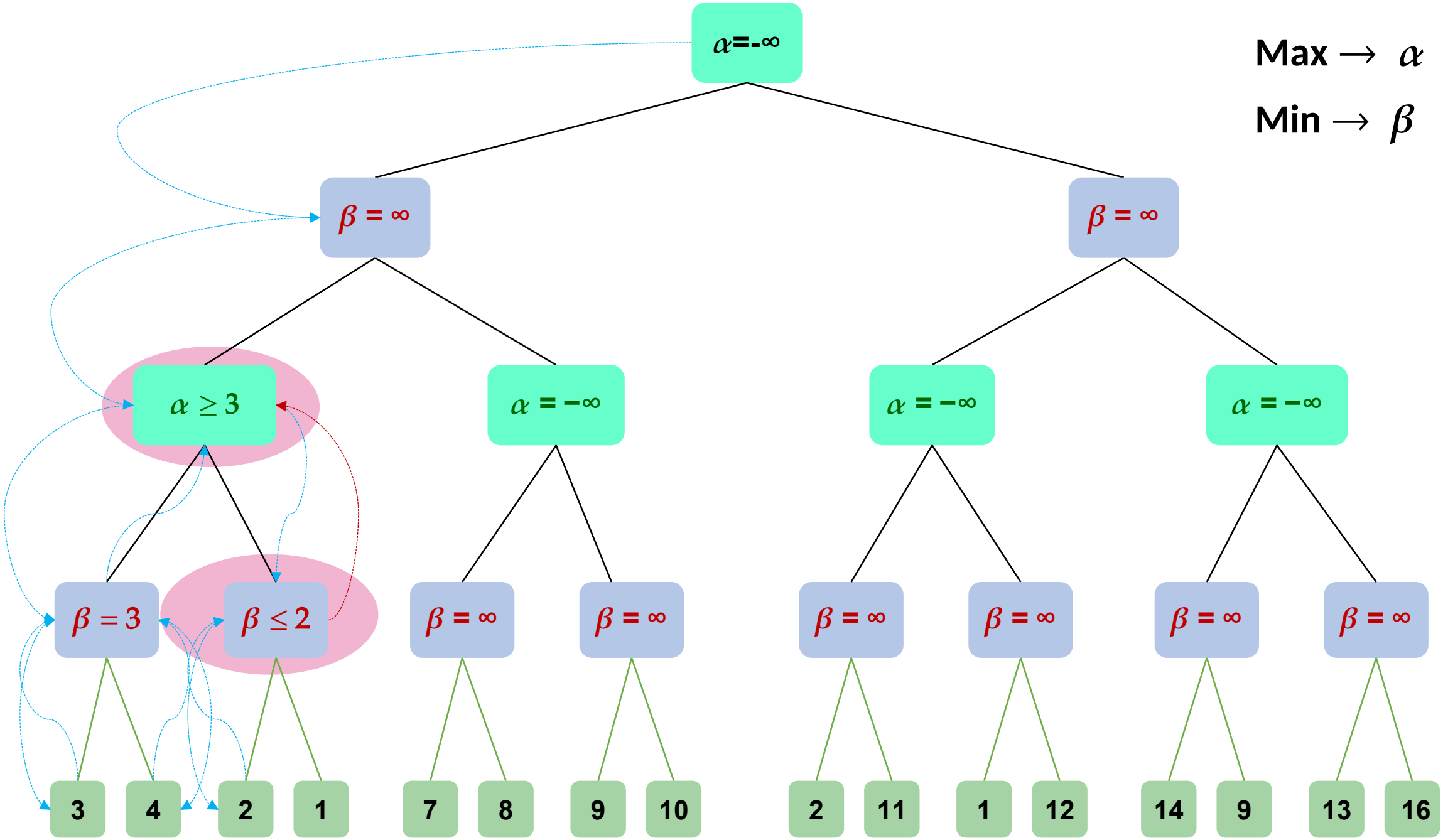

α=-∞
Max → α
Min → β
β = ∞
β = ∞
α =-∞
α = −∞
α = −∞
α = −∞
β = 3
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

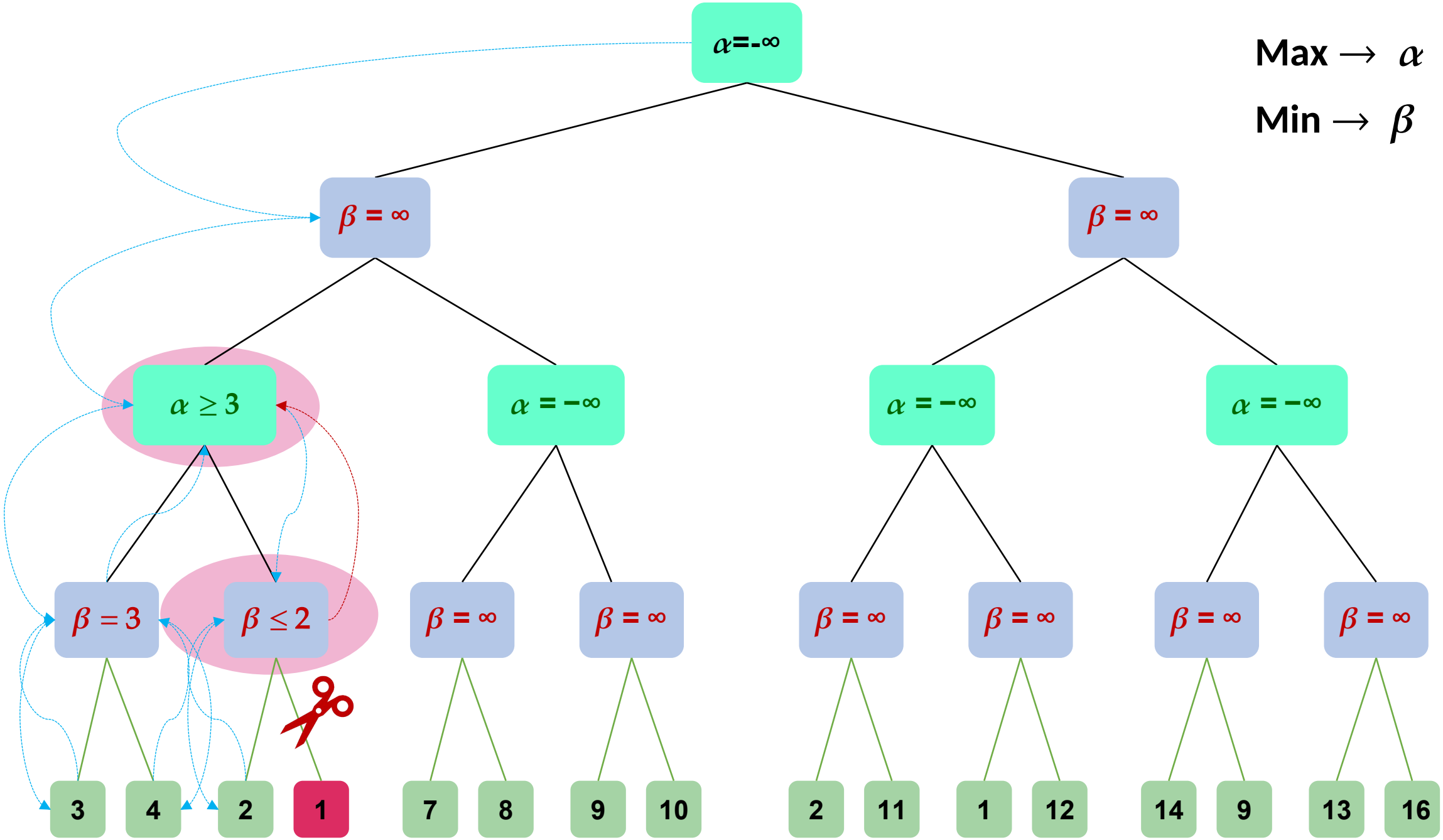

α=-∞
Max → α
Min → β
β = ∞
β = ∞
α ≥ 3
α = −∞
α = −∞
α = −∞
β = 3
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

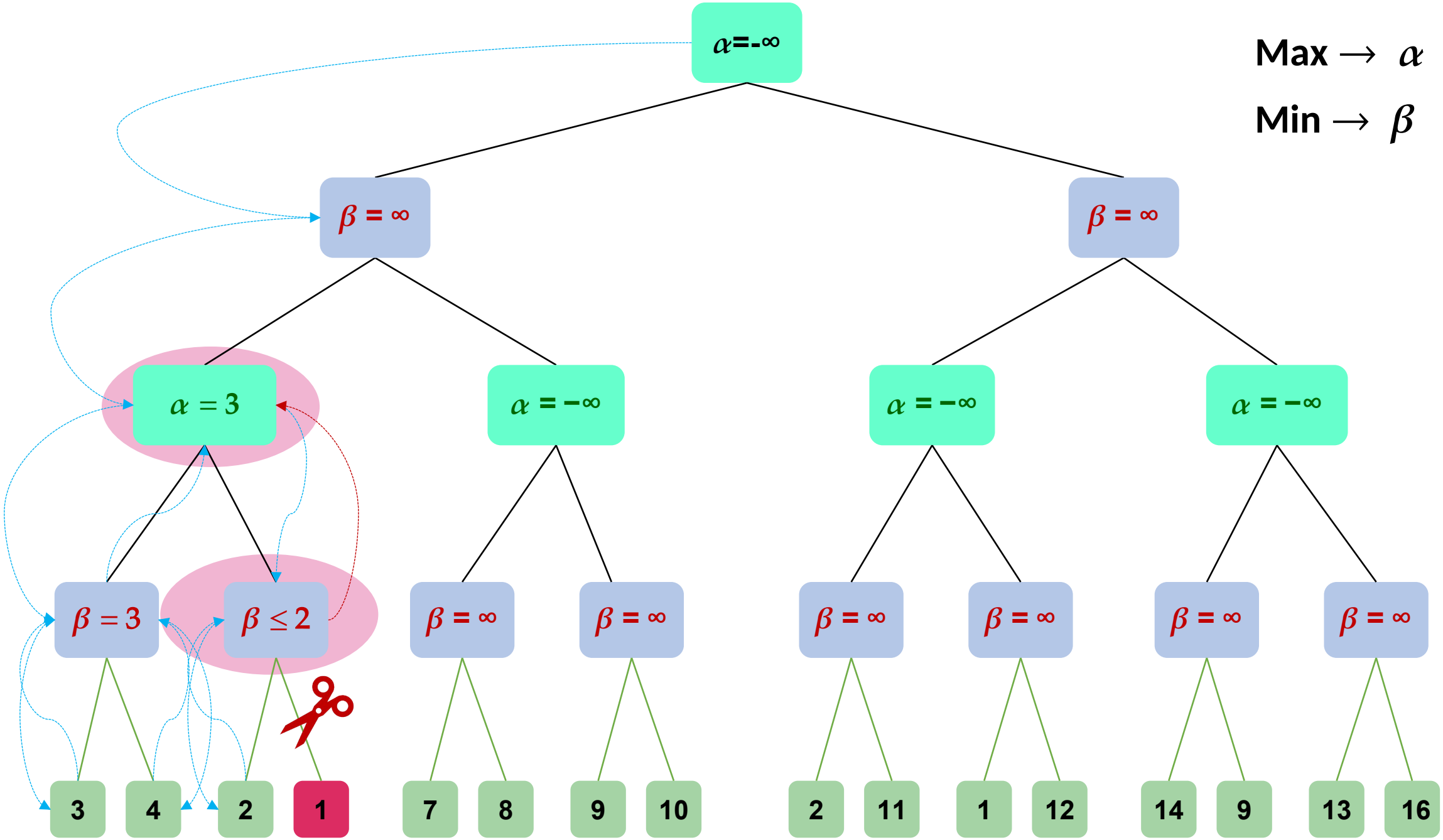

α=-∞
Max → α
Min → β
β = ∞
β = ∞
α ≥ 3
α = −∞
α = −∞
α = −∞
β = 3
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

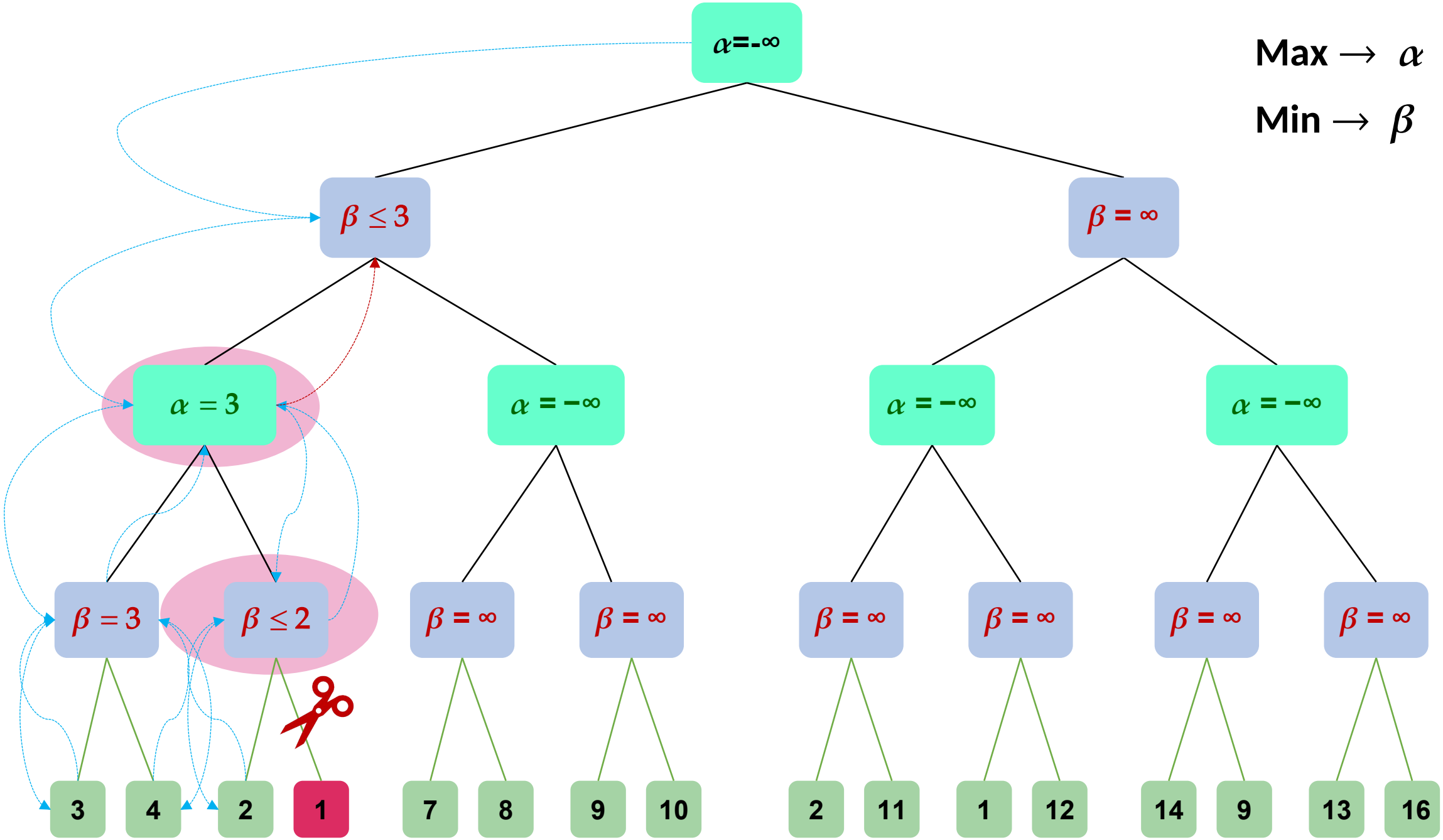

α=-∞
Max → α
Min → β
β = ∞
β = ∞
α ≥ 3
α = −∞
α = −∞
α = −∞
β = 3
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

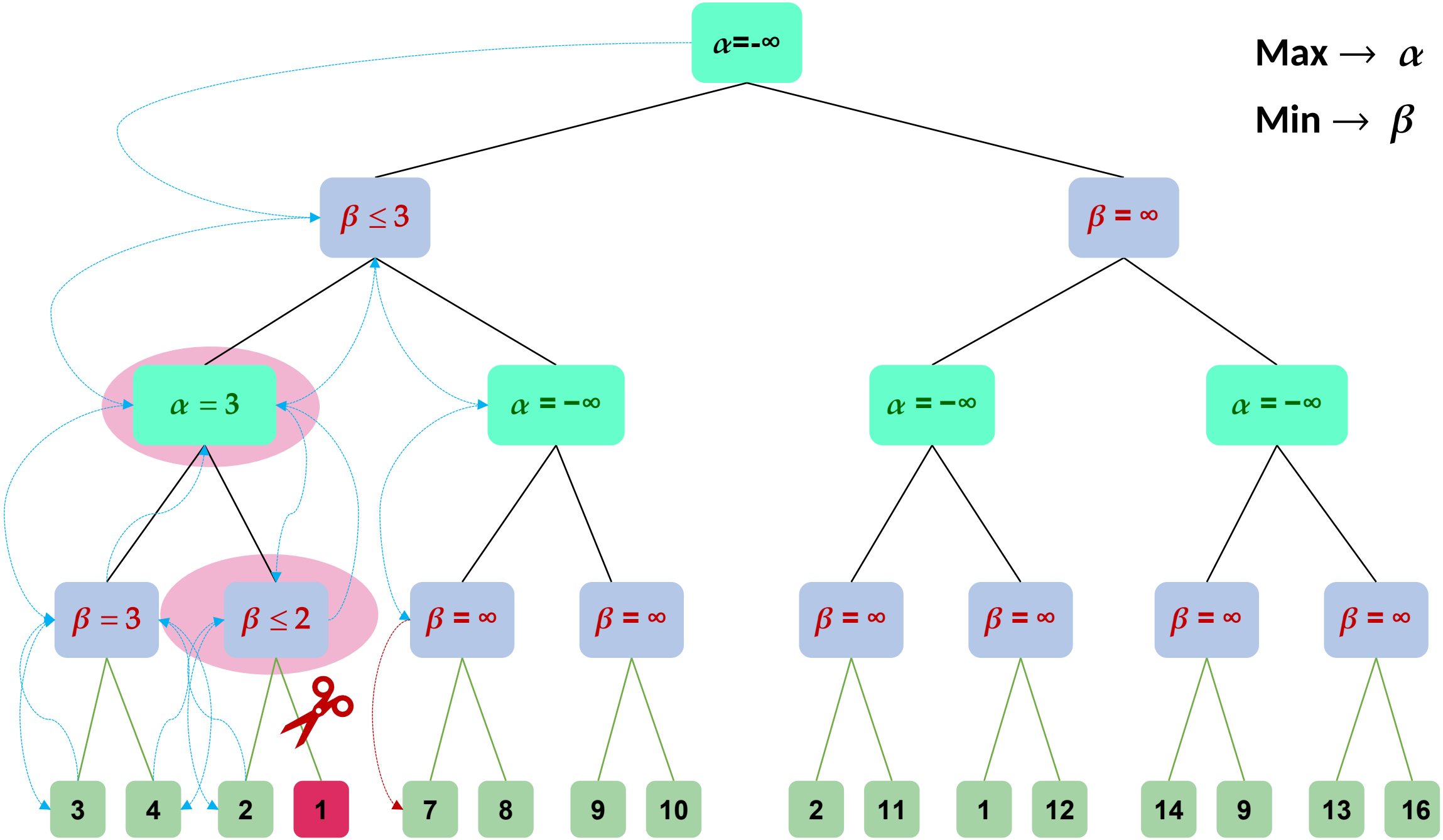

α=-∞
Max → α
Min → β
β = ∞
β = ∞
α ≥ 3
α = −∞
α = −∞
α = −∞
β = 3
β ≤ 2
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

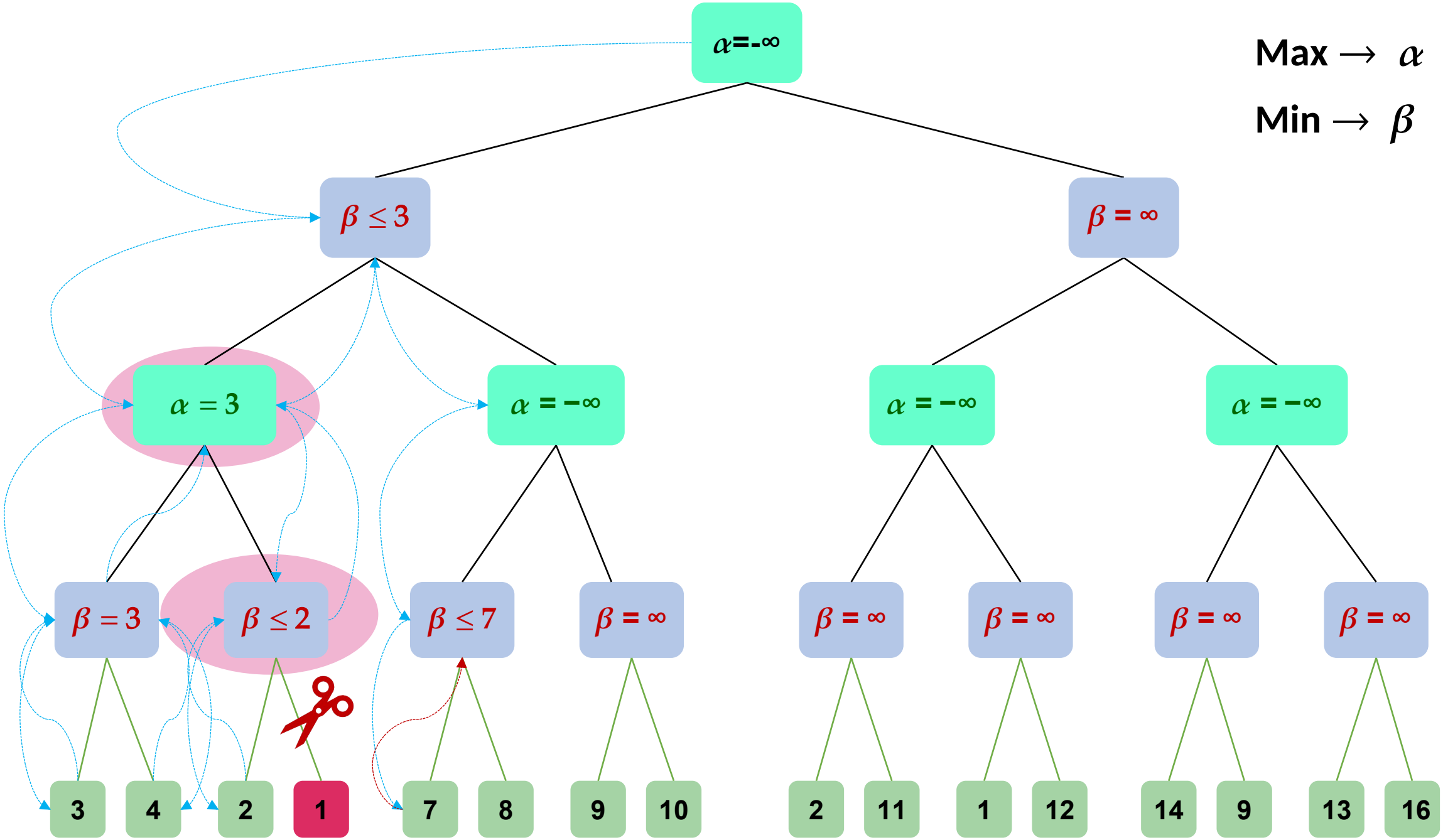

α=-∞
β = ∞
β = ∞
α ≥ 3
α = −∞
α = −∞
α = −∞
β = 3
β ≤ 2
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16
Max → α
Min → β

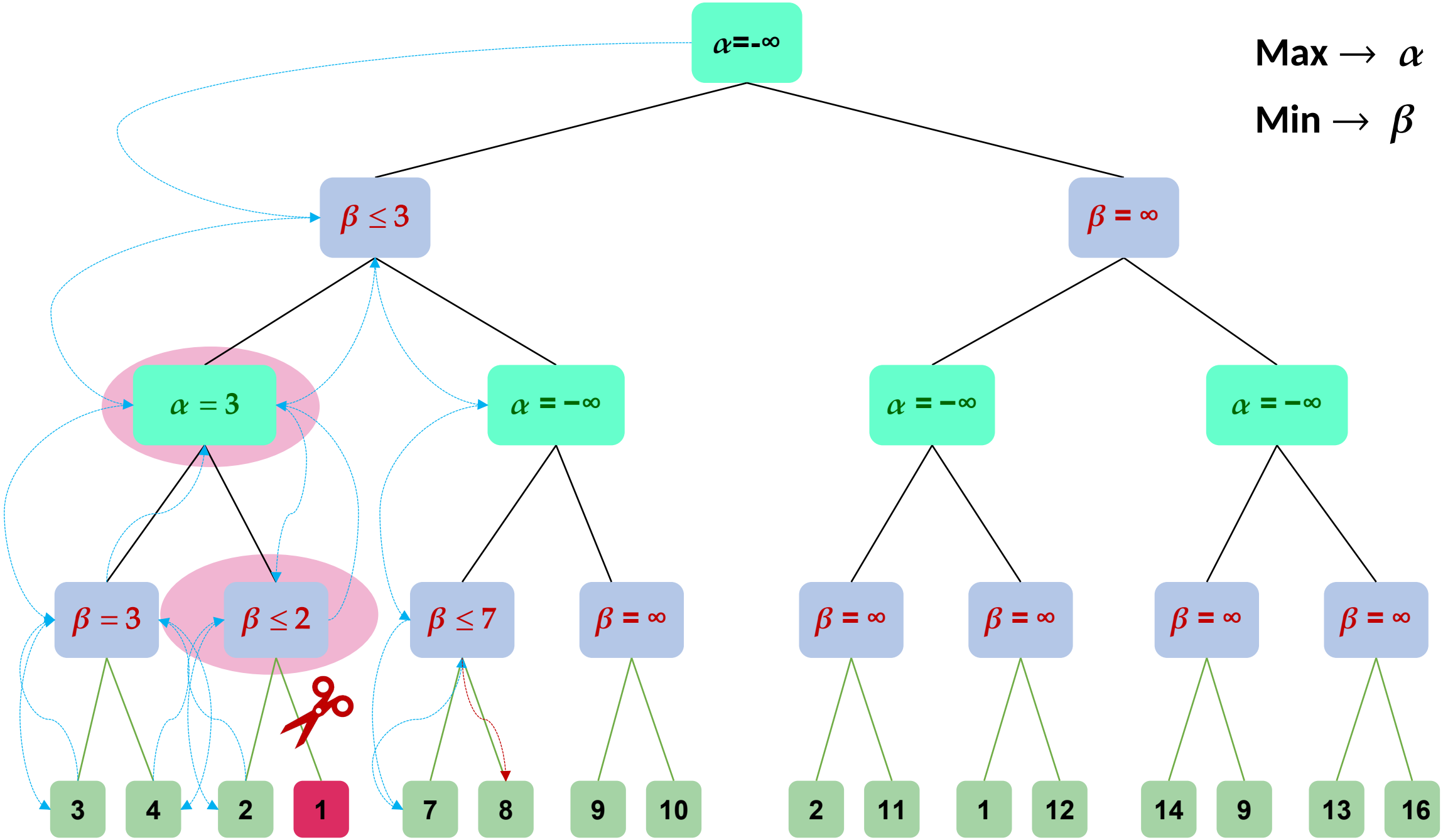

α=-∞
Max → α
Min → β
β = ∞
β = ∞
α ≥ 3
α = −∞
α = −∞
α = −∞
β = 3
β ≤ 2
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

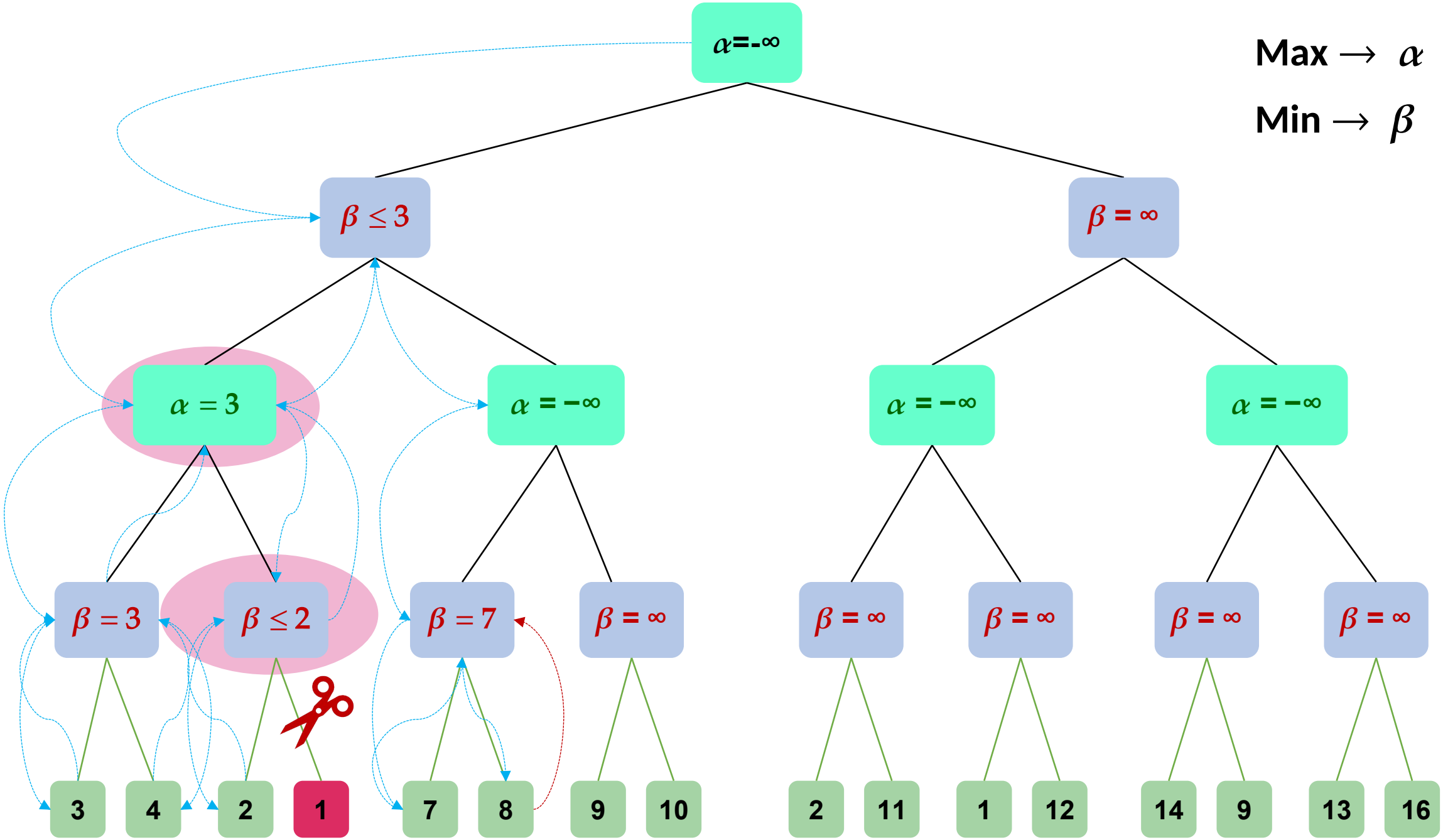

α=-∞
Max → α
Min → β
β = ∞
β = ∞
α = 3
α = −∞
α = −∞
α = −∞
β = 3
β ≤ 2
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

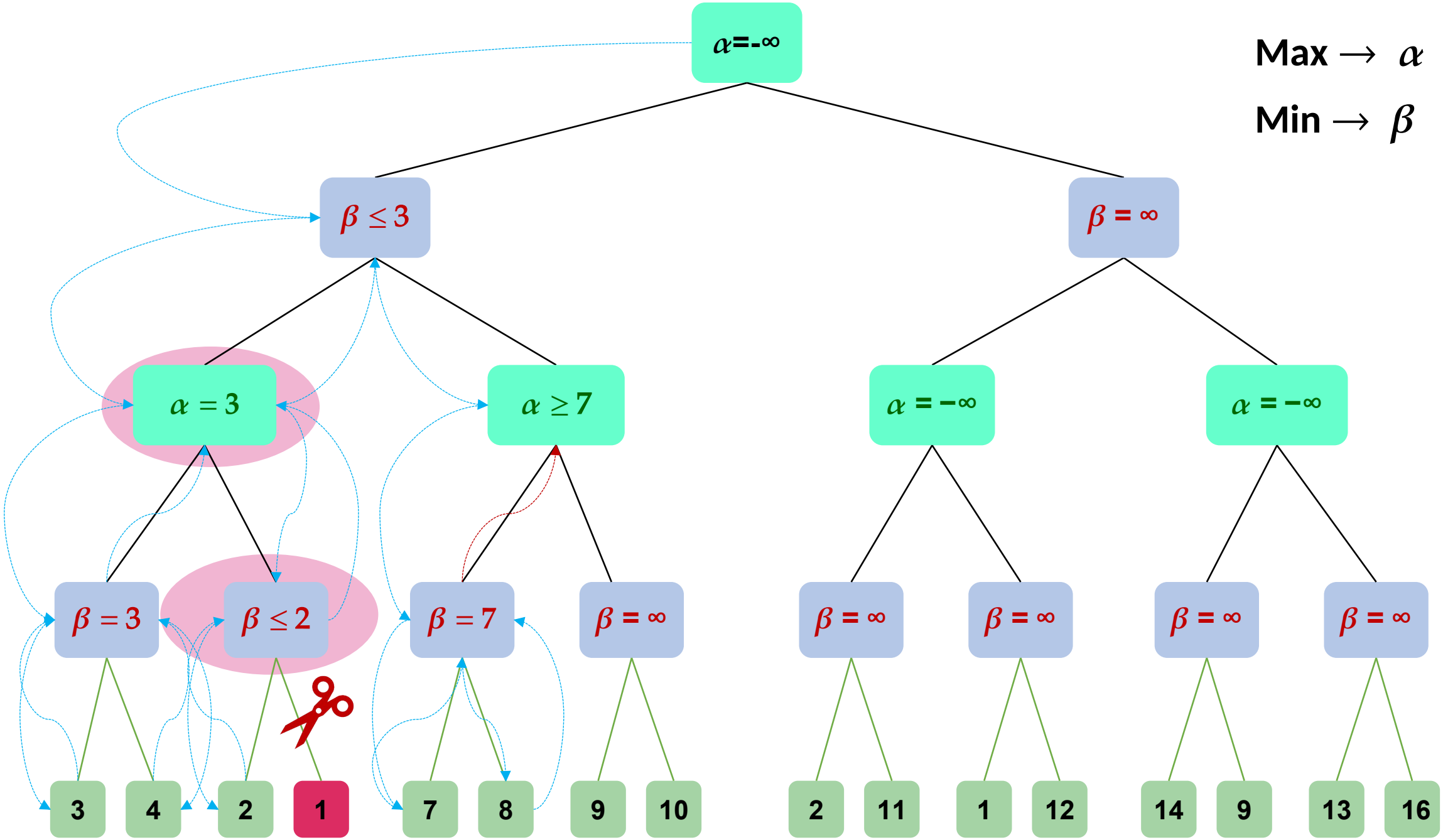

α=-∞
Max → α
Min → β
β ≤ 3
β = ∞
α = 3
α = −∞
α = −∞
α = −∞
β = 3
β ≤ 2
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

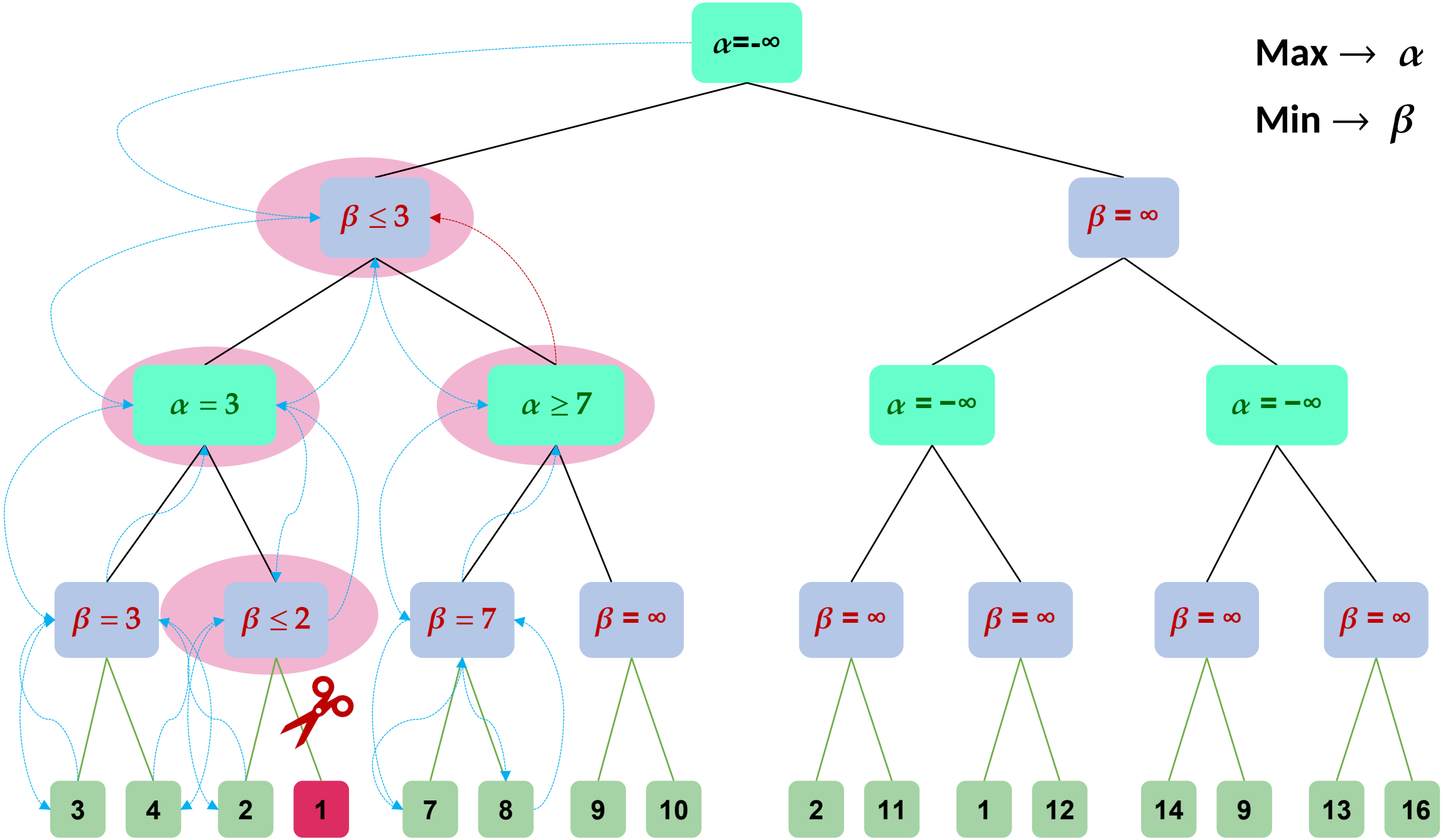

α=-∞
Max → α
Min → β
β ≤ 3
β = ∞
α = 3
α = −∞
α = −∞
α = −∞
β = 3
β ≤ 2
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

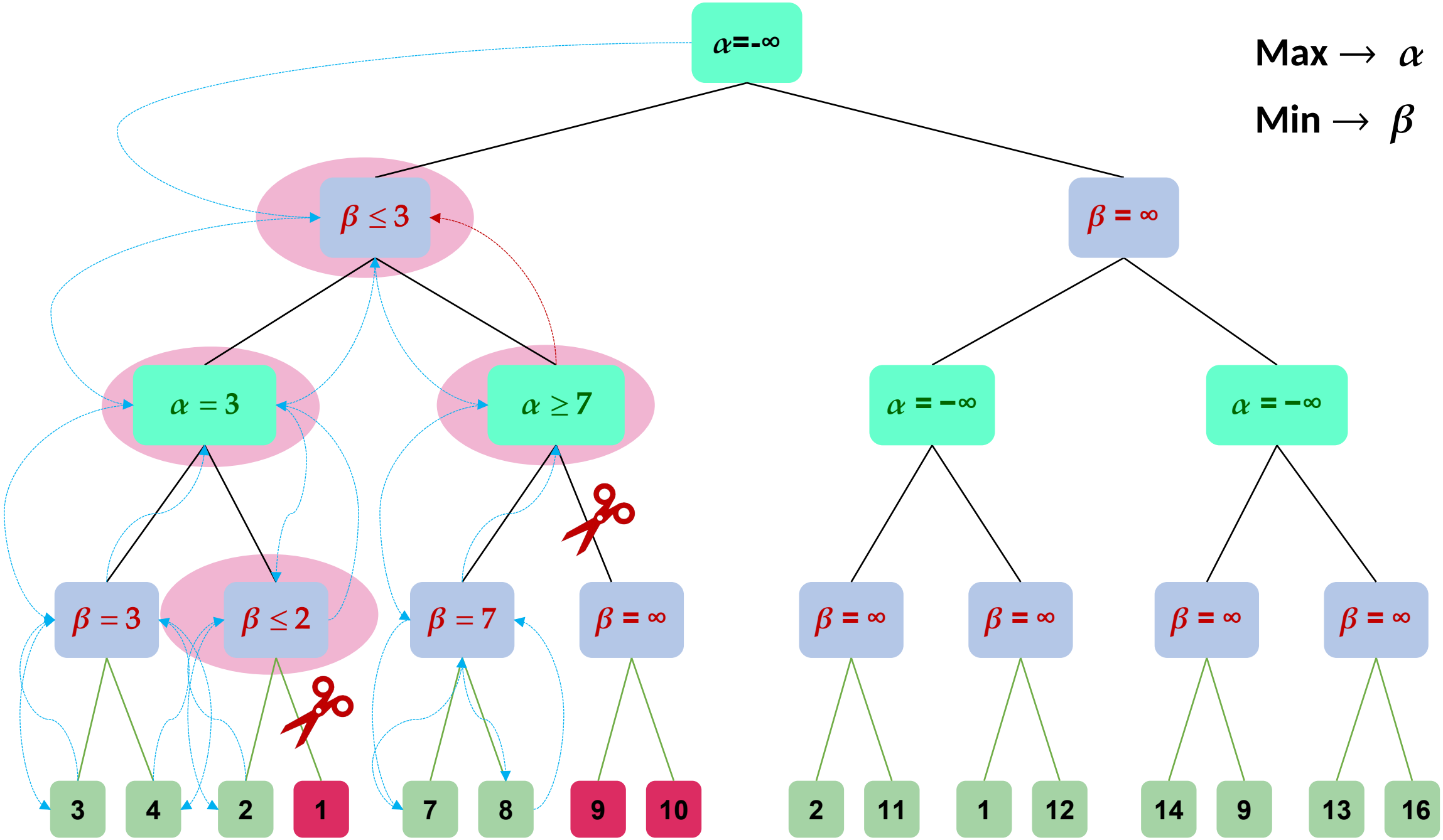

α=-∞
Max → α
Min → β
β ≤ 3
β = ∞
α = 3
α = −∞
α = −∞
α = −∞
β = 3
β ≤ 2
β ≤ 7
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

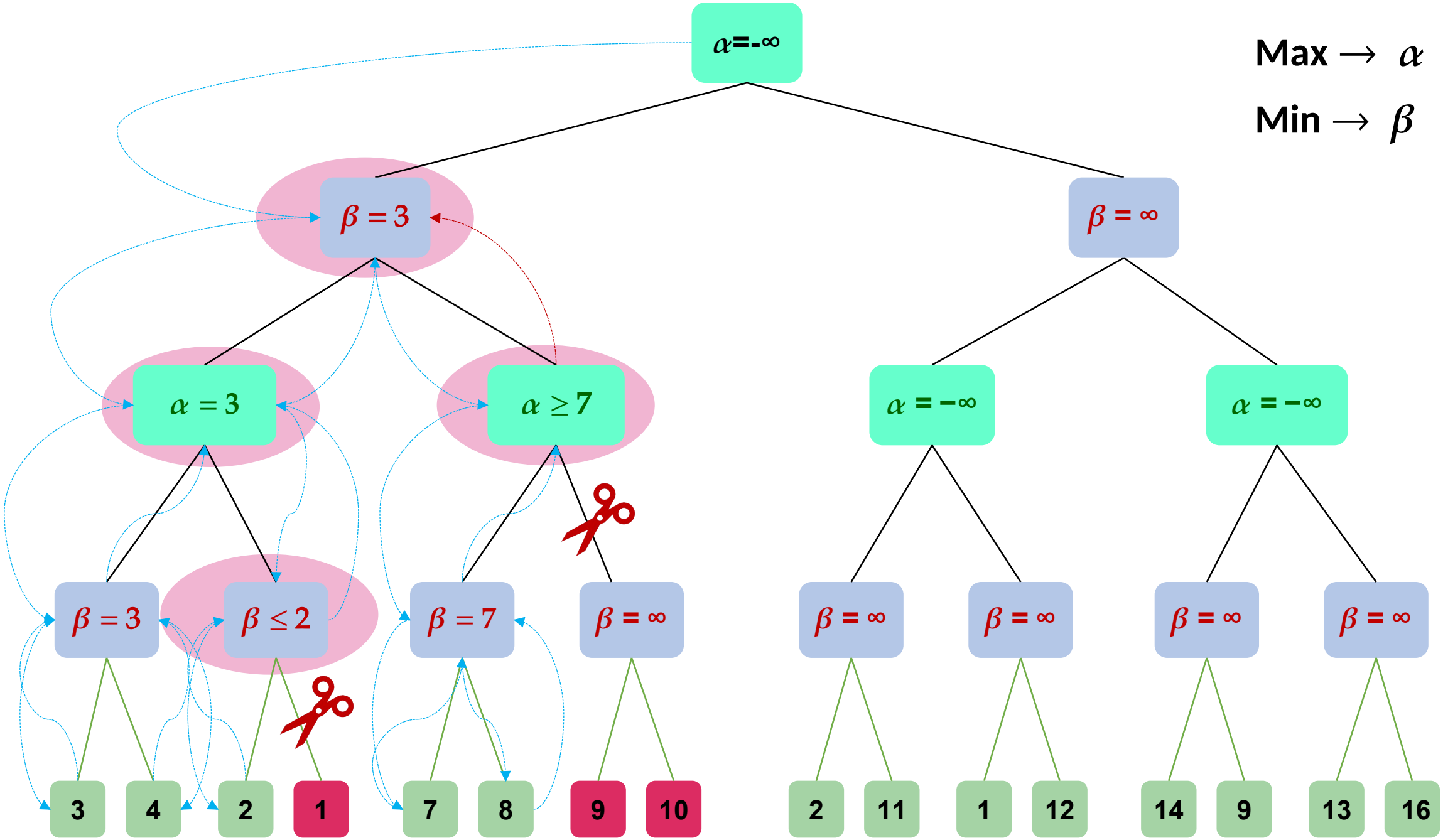

α=-∞
Max → α
Min → β
β ≤ 3
β = ∞
α = 3
α = −∞
α = −∞
α = −∞
β = 3
β ≤ 2
β ≤ 7
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

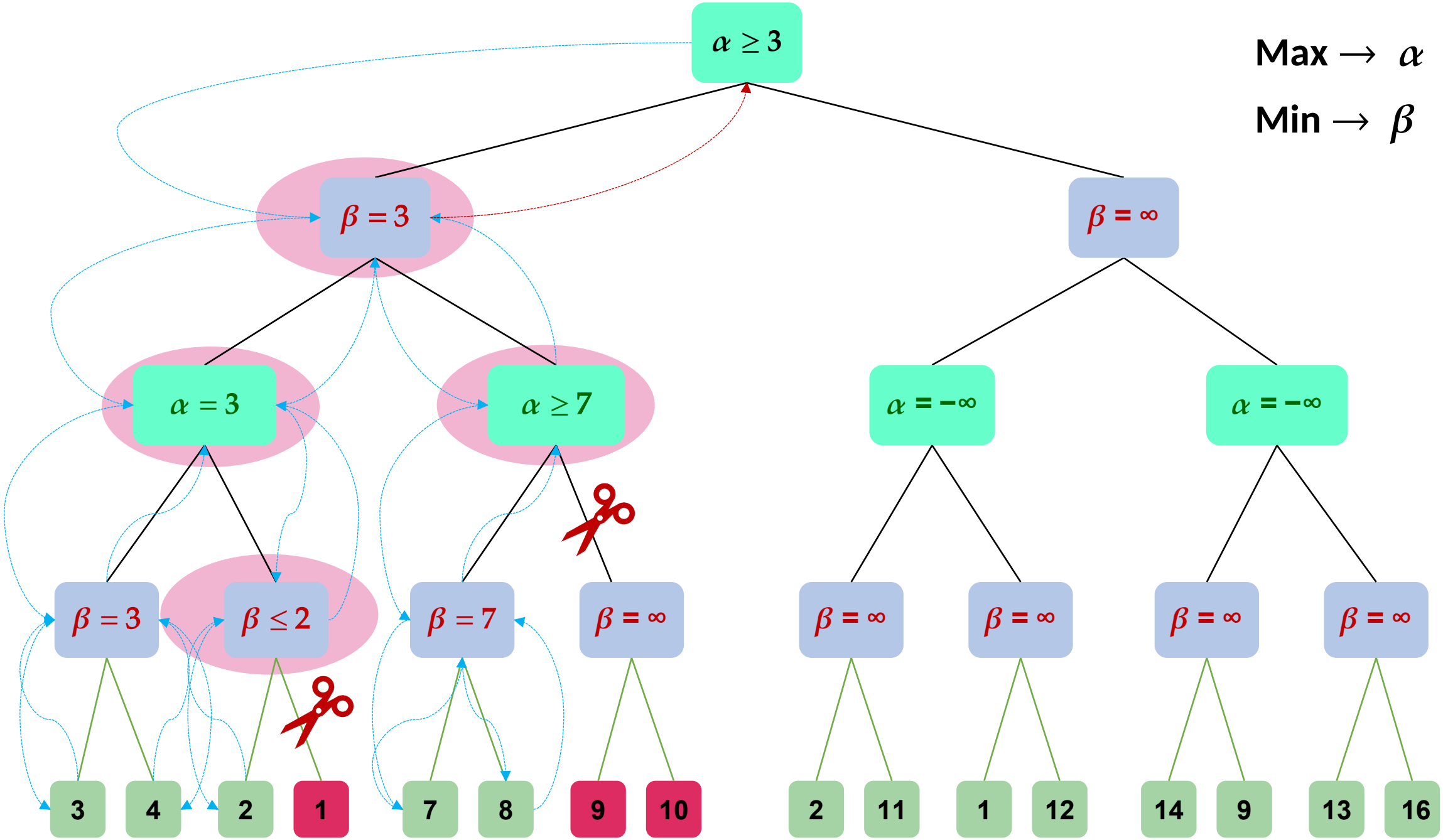

α=-∞
Max → α
Min → β
β ≤ 3
β = ∞
α = 3
α = −∞
α = −∞
α = −∞
β = 3
β ≤ 2
β = 7
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

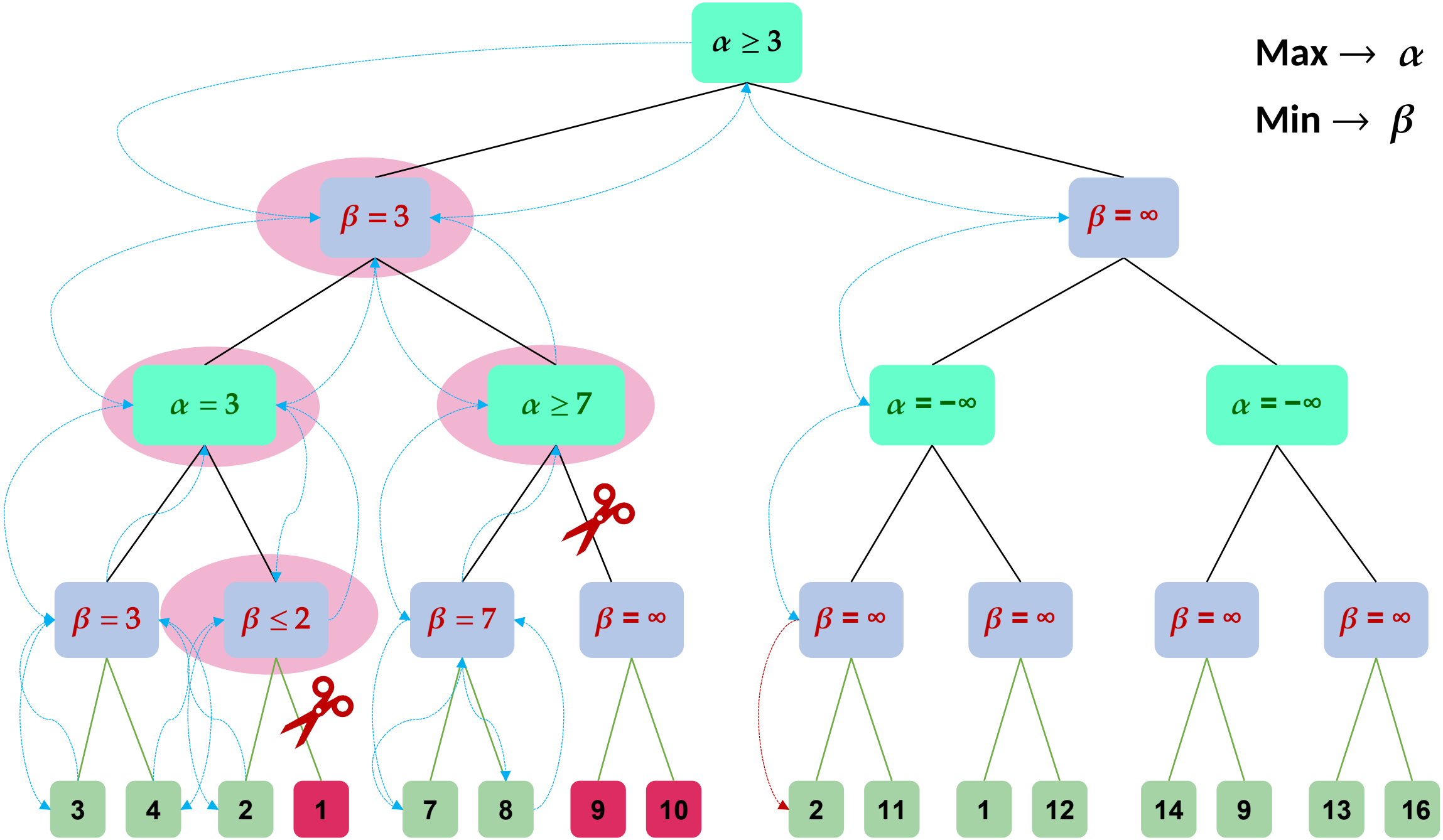

α=-∞
Max → α
Min → β
β ≤ 3
β = ∞
α = 3
α ≥ 7
α = −∞
α = −∞
β = 3
β ≤ 2
β = 7
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

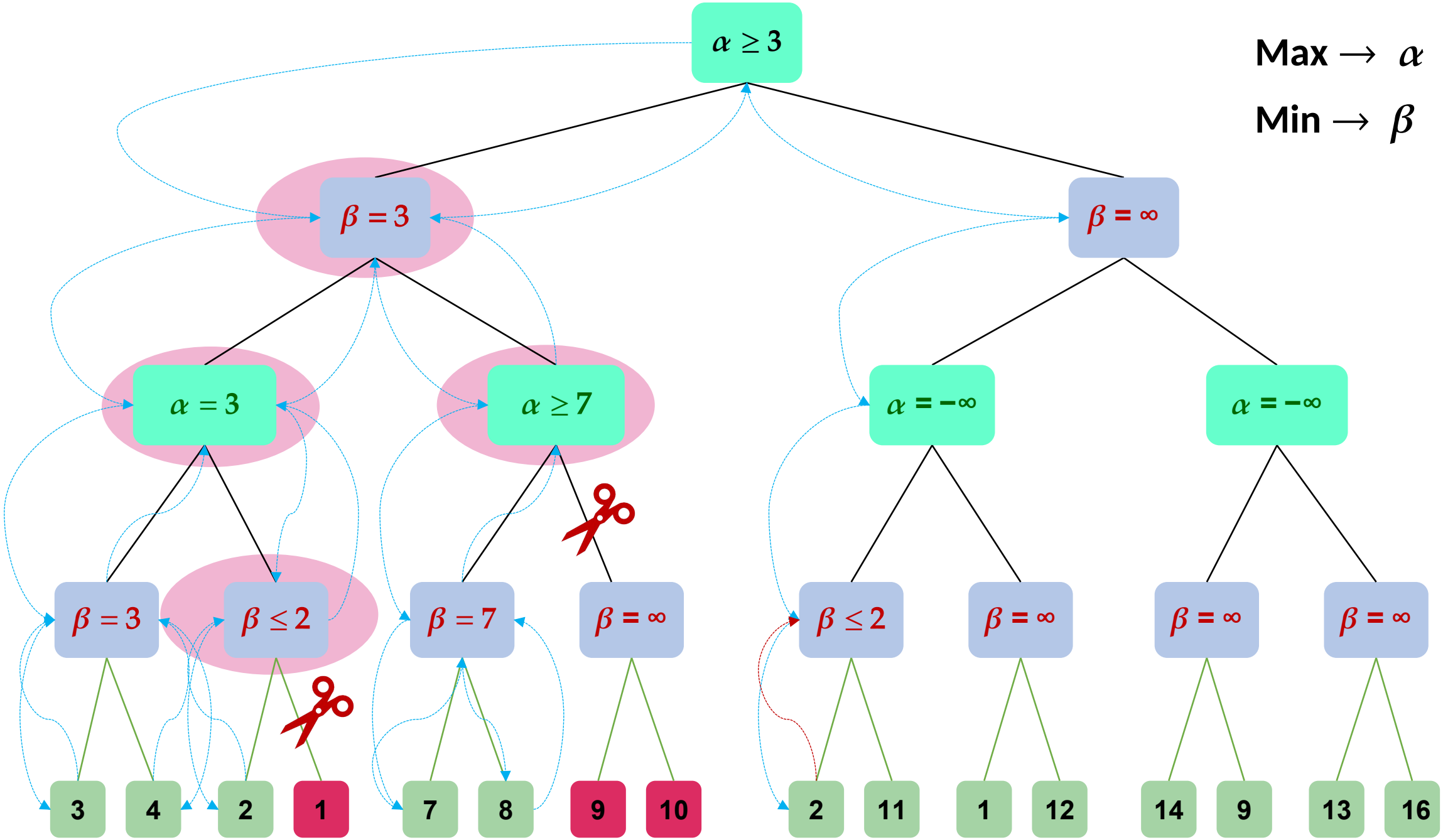

α=-∞
Max → α
Min → β
β ≤ 3
β = ∞
α = 3
α ≥ 7
α = −∞
α = −∞
β = 3
β ≤ 2
β = 7
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

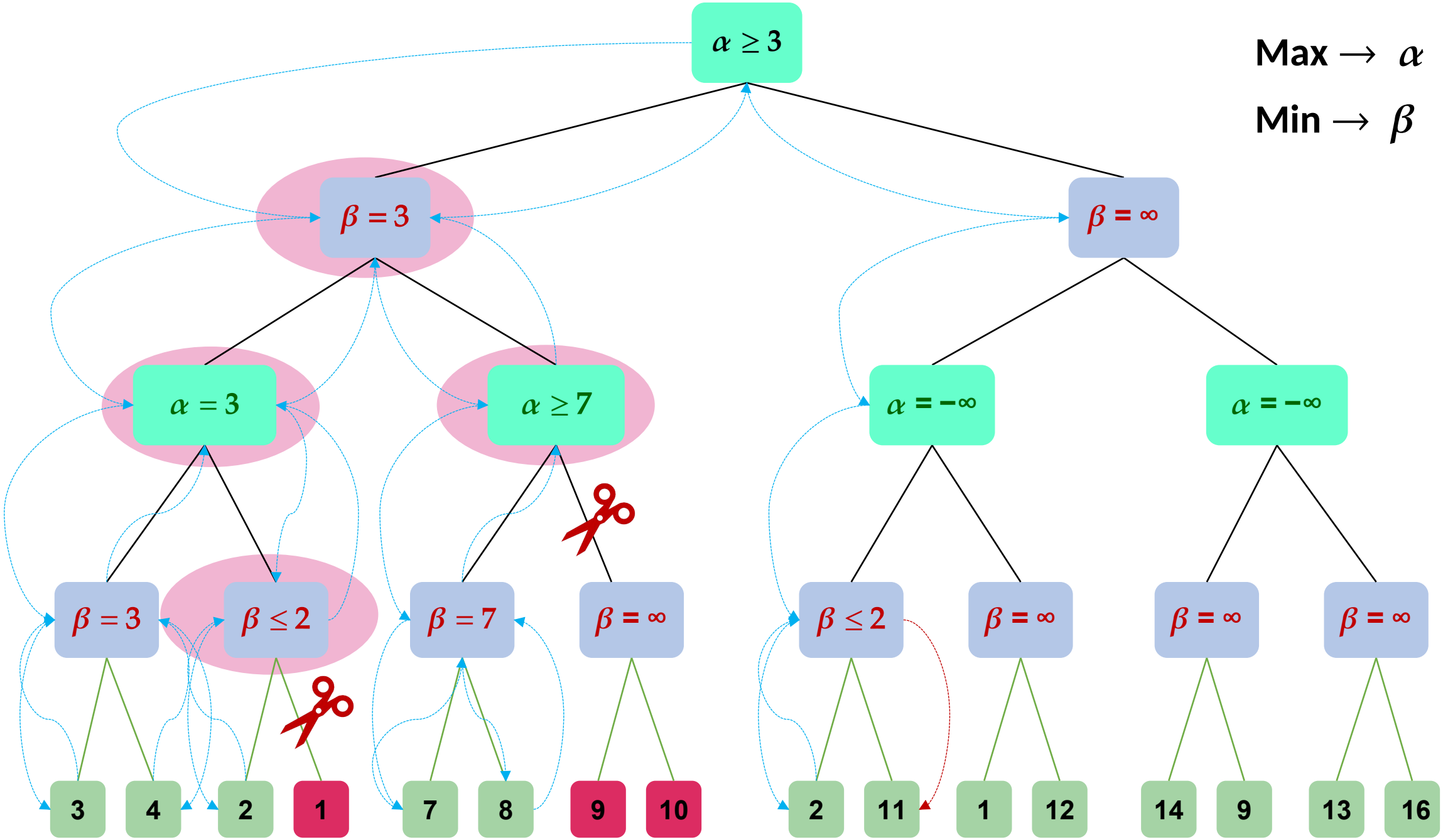

α=-∞
Max → α
Min → β
β ≤ 3
β = ∞
α = 3
α ≥ 7
α = −∞
α = −∞
β = 3
β ≤ 2
β = 7
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

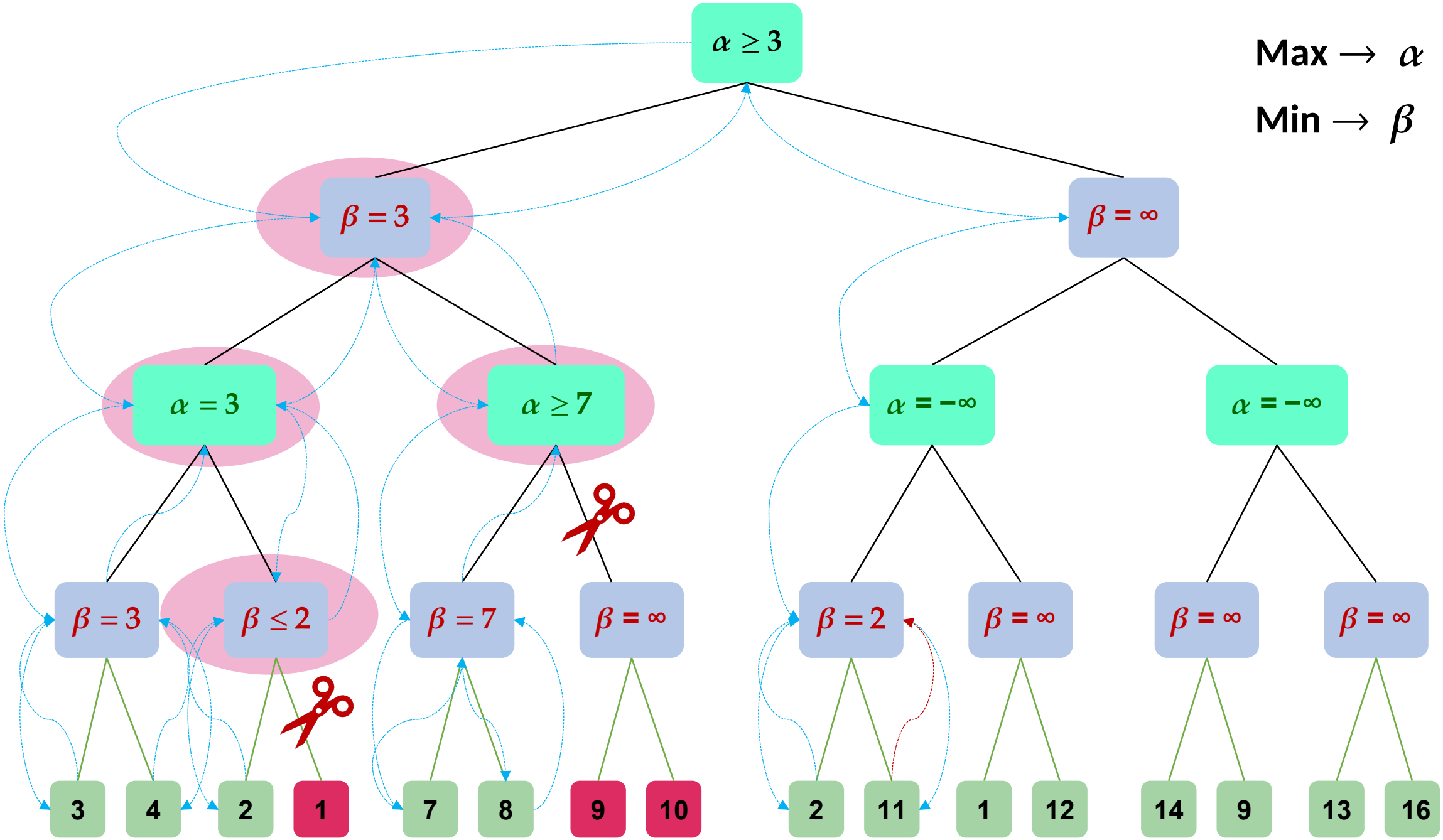

α=-∞
Max → α
Min → β
β = 3
β = ∞
α = 3
α ≥ 7
α = −∞
α = −∞
β = 3
β ≤ 2
β = 7
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

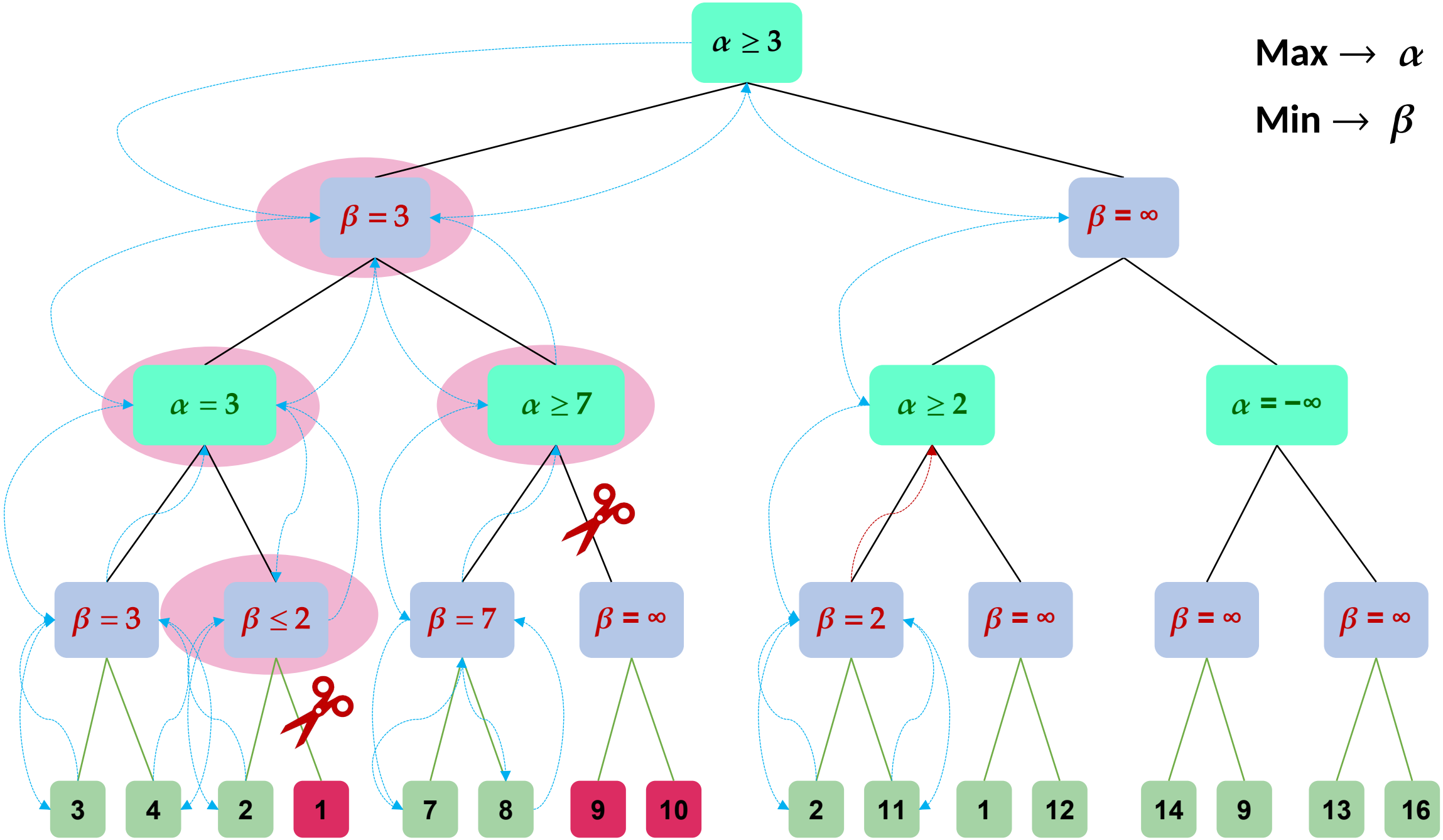

α ≥ 3
Max → α
Min → β
β = 3
β = ∞
α = 3
α ≥ 7
α = −∞
α = −∞
β = 3
β ≤ 2
β = 7
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

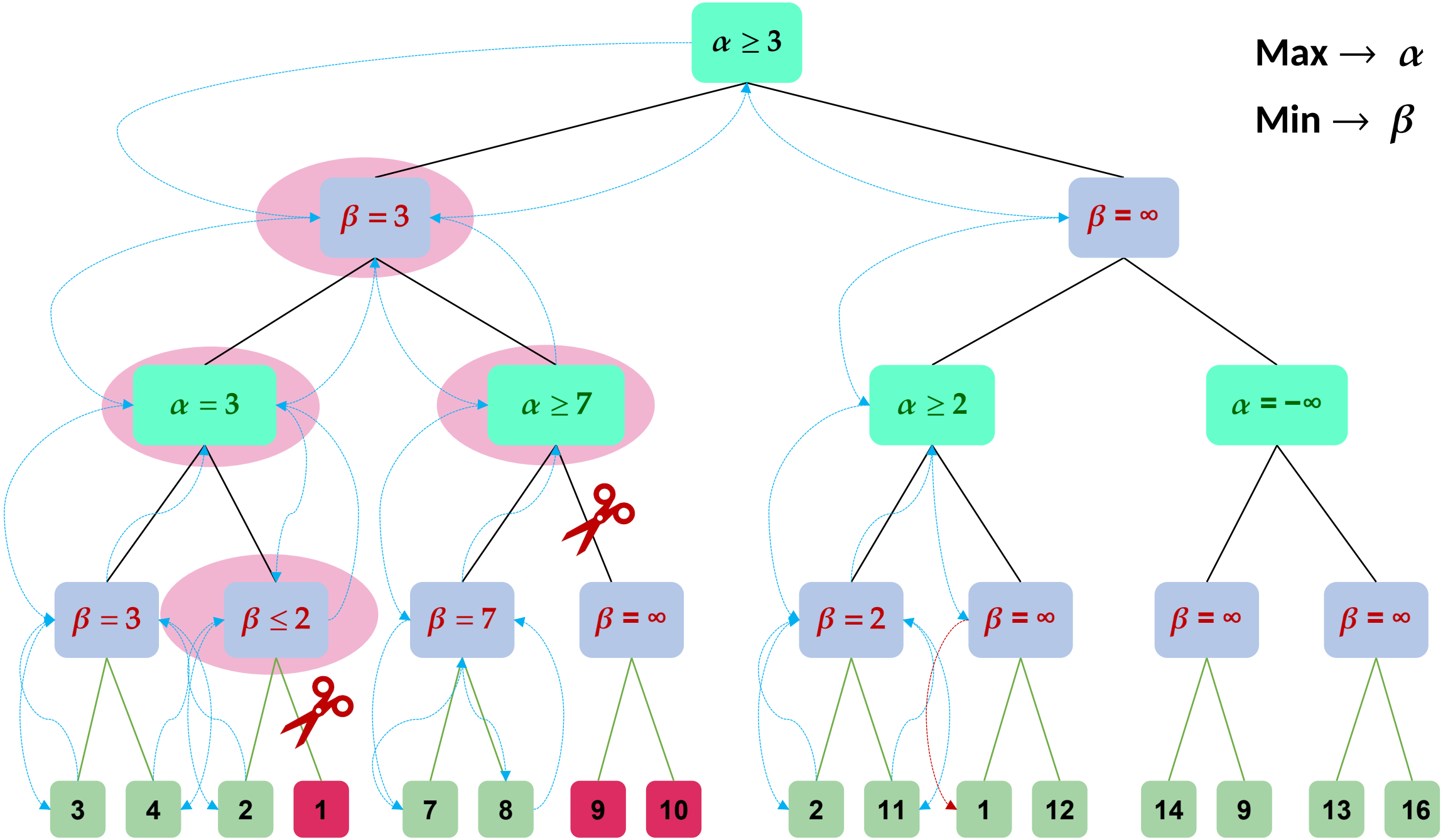

α ≥ 3
Max → α
Min → β
β = 3
β = ∞
α = 3
α ≥ 7
α = −∞
α = −∞
β = 3
β ≤ 2
β = 7
β = ∞
β = ∞
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

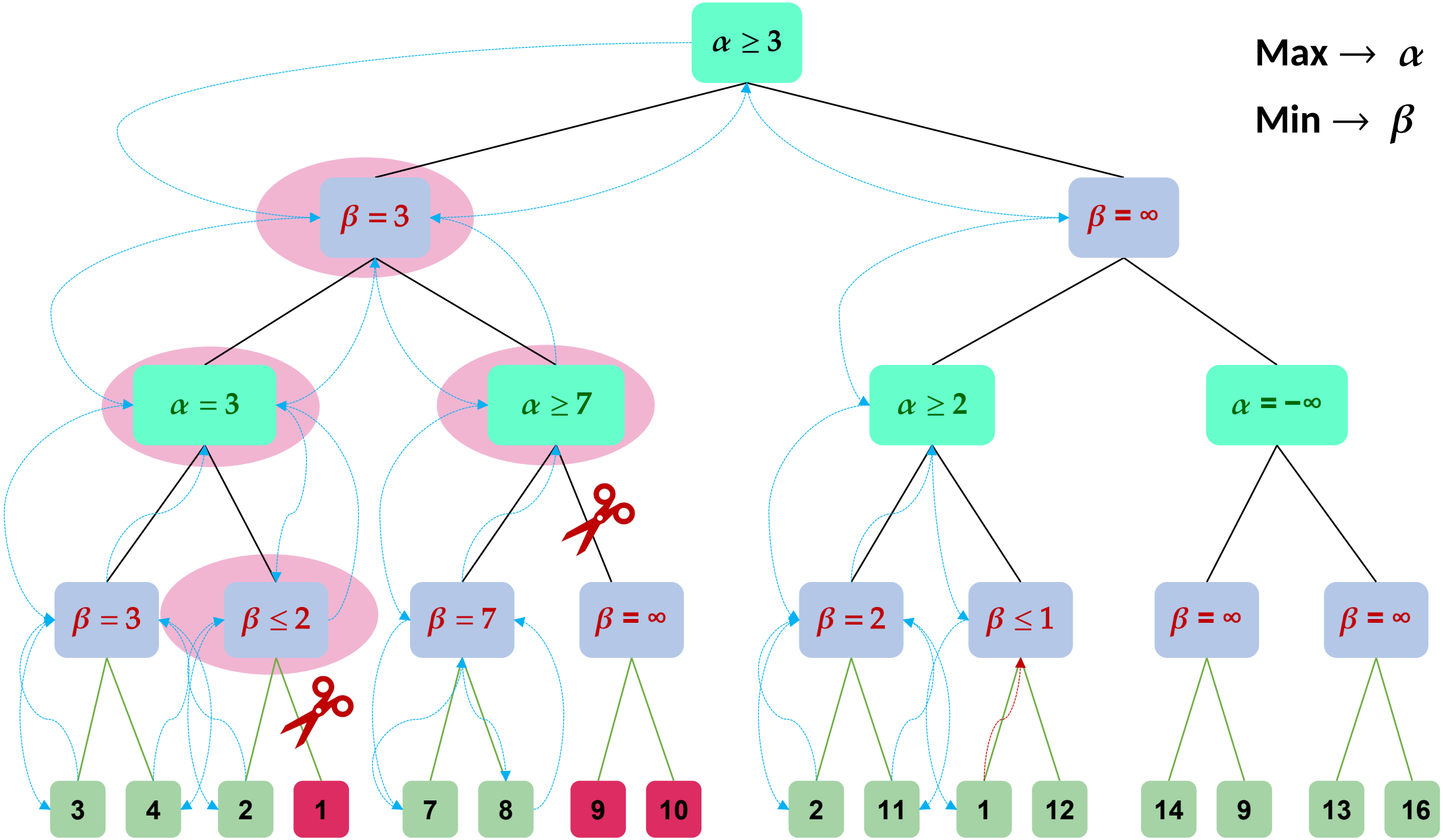

α ≥ 3
Max → α
Min → β
β = 3
β = ∞
α = 3
α ≥ 7
α = −∞
α = −∞
β = 3
β ≤ 2
β = 7
β = ∞
β ≤ 2
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

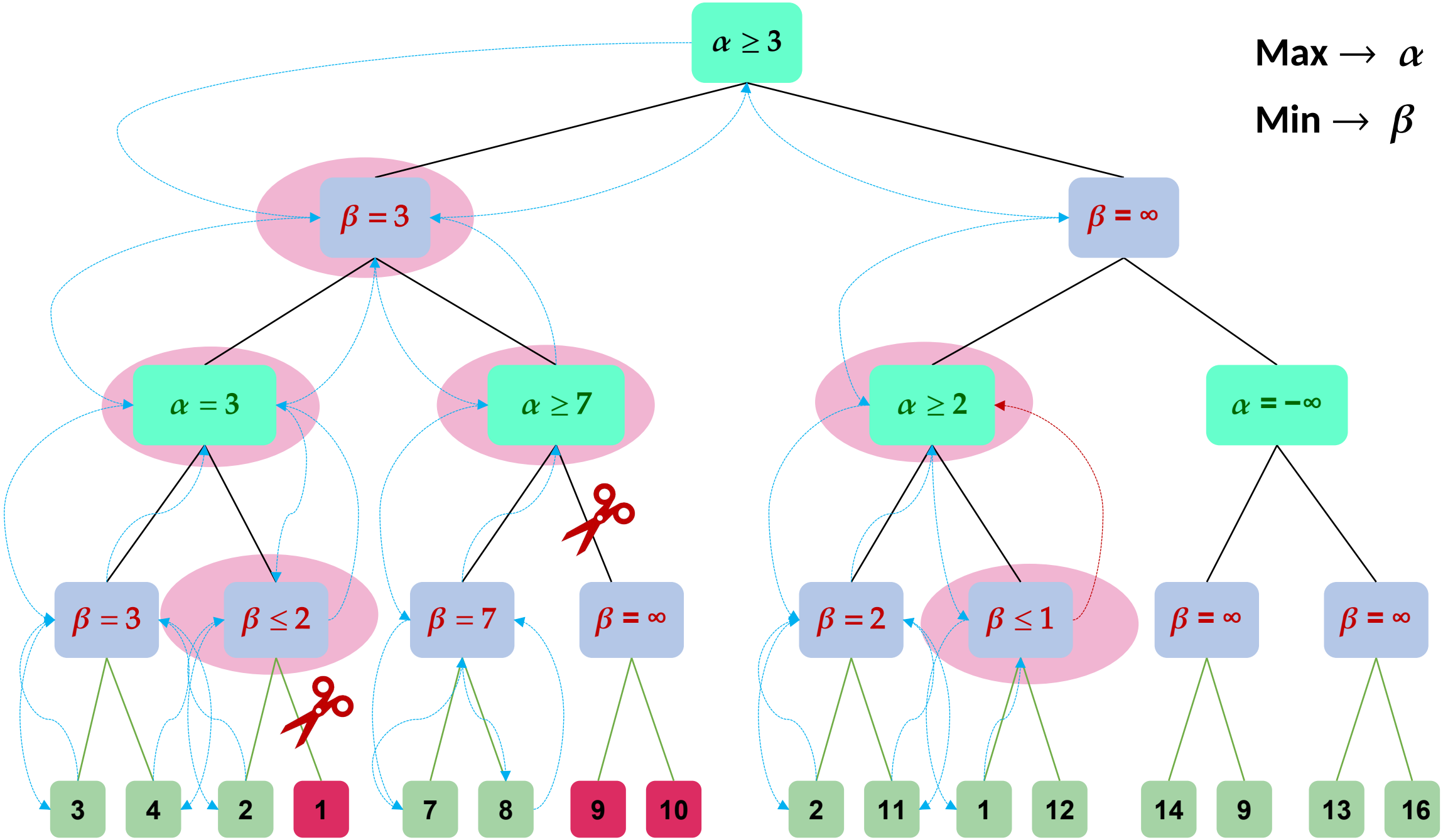

α ≥ 3
Max → α
Min → β
β = 3
β = ∞
α = 3
α ≥ 7
α = −∞
α = −∞
β = 3
β ≤ 2
β = 7
β = ∞
β ≤ 2
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

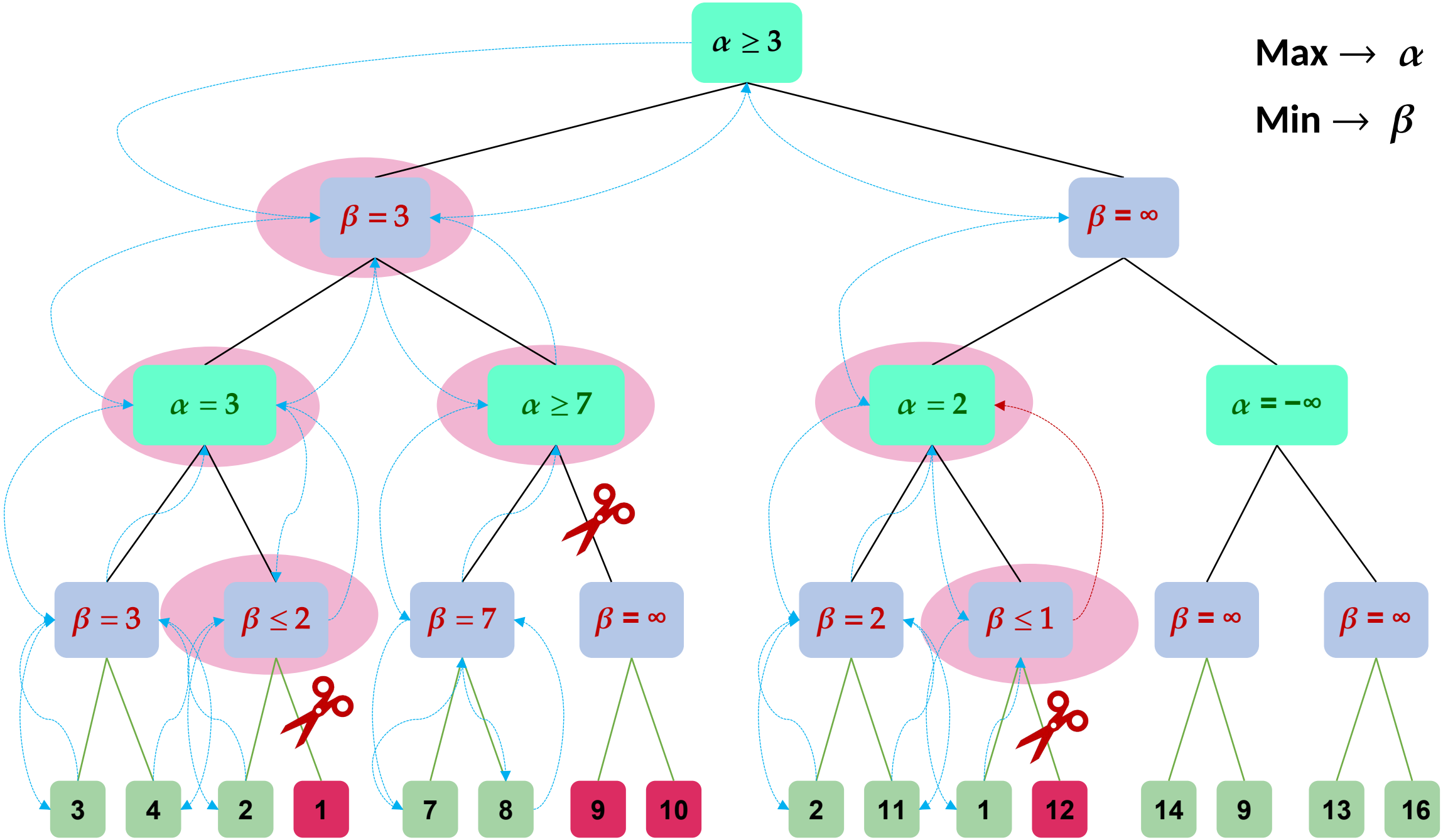

α ≥ 3
Max → α
Min → β
β = 3
β = ∞
α = 3
α ≥ 7
α = −∞
α = −∞
β = 3
β ≤ 2
β = 7
β = ∞
β = 2
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

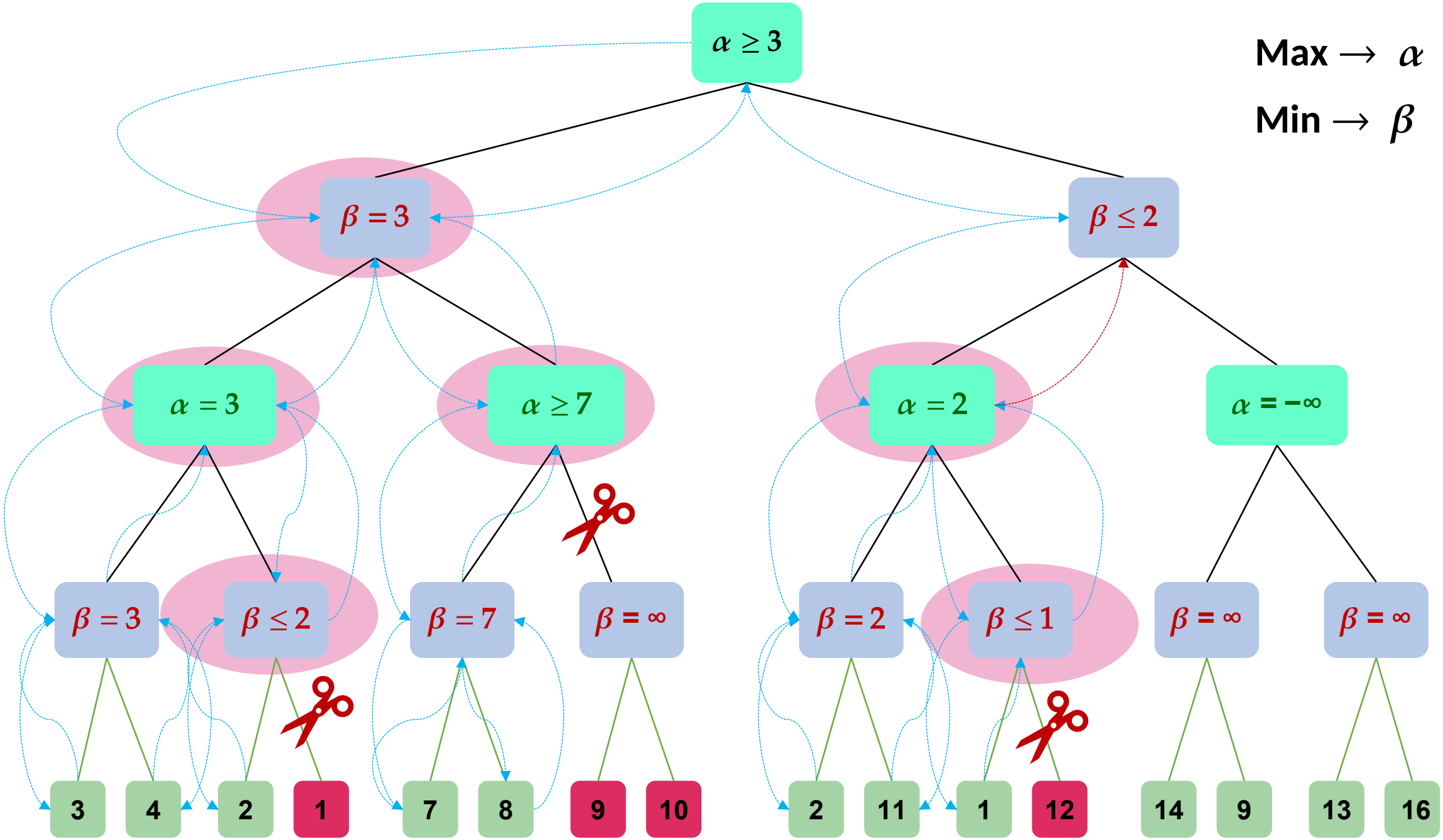

α ≥ 3
Max → α
Min → β
β = 3
β = ∞
α = 3
α ≥ 7
α ≥ 2
α = −∞
β = 3
β ≤ 2
β = 7
β = ∞
β = 2
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

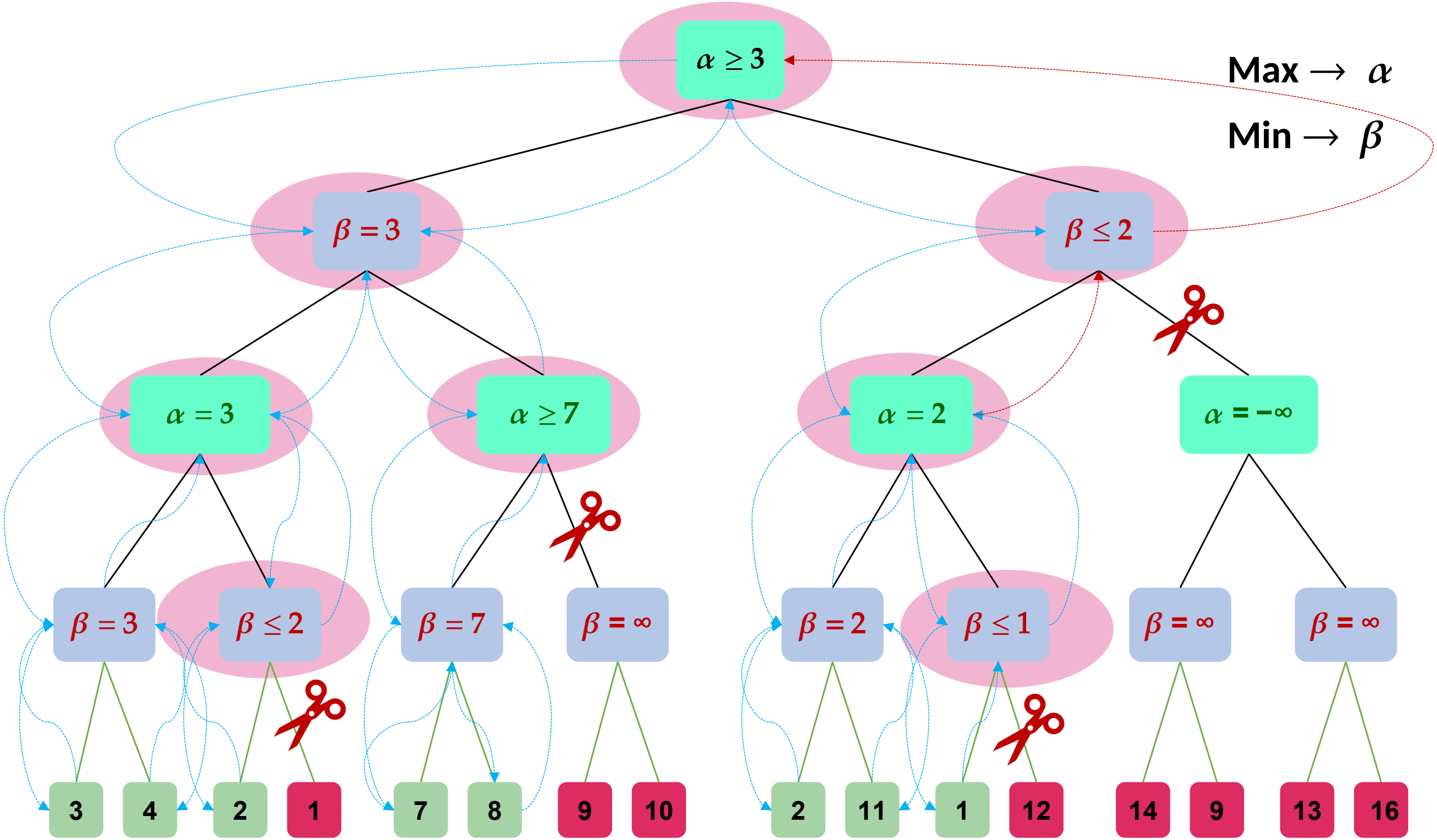

α ≥ 3
Max → α
Min → β
β = 3
β = ∞
α = 3
α ≥ 7
α ≥ 2
α = −∞
β = 3
β ≤ 2
β = 7
β = ∞
β = 2
β = ∞
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

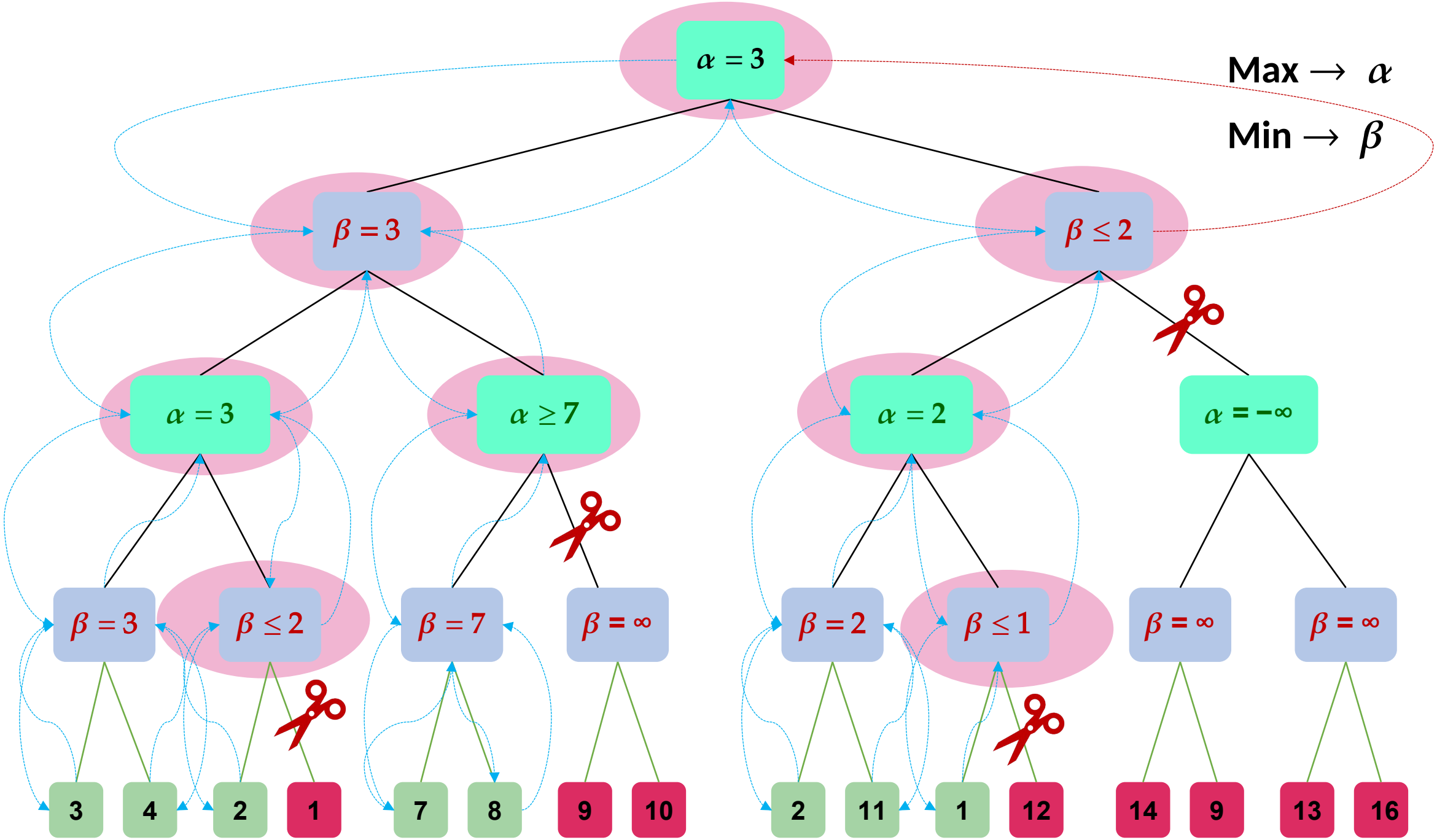

α ≥ 3
Max → α
Min → β
β = 3
β = ∞
α = 3
α ≥ 7
α ≥ 2
α = −∞
β = 3
β ≤ 2
β = 7
β = ∞
β = 2
β ≤ 1
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16


α ≥ 3
Max → α
Min → β
β = 3
β = ∞
α = 3
α ≥ 7
α ≥ 2
α = −∞
β = 3
β ≤ 2
β = 7
β = ∞
β = 2
β ≤ 1
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16


α ≥ 3
Max → α
Min → β
β = 3
β = ∞
α = 3
α ≥ 7
α = 2
α = −∞
β = 3
β ≤ 2
β = 7
β = ∞
β = 2
β ≤ 1
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16


α ≥ 3
Max → α
Min → β
β = 3
β ≤ 2
α = 3
α ≥ 7
α = 2
α = −∞
β = 3
β ≤ 2
β = 7
β = ∞
β = 2
β ≤ 1
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16


α ≥ 3
Max → α
Min → β
β = 3
β ≤ 2
α = 3
α ≥ 7
α = 2
α = −∞
β = 3
β ≤ 2
β = 7
β = ∞
β = 2
β ≤ 1
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16


α = 3
Max → α
Min → β
β = 3
β ≤ 2
α = 3
α ≥ 7
α = 2
α = −∞
β = 3
β ≤ 2
β = 7
β = ∞
β = 2
β ≤ 1
β = ∞
β = ∞
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

Alpha Beta Properties

- Pruning does not affect final result
- Good move ordering improves effectiveness of pruning
- With perfect ordering, time complexity is $O(b^{m/2})$